# ESTRUCTURAS DE DATOS



UNIVERSIDAD DE SALAMANCA

Escuela politécnica superior de Zamora

Apuntes de 2º de G.I.I.S.I.

# Estructuras de datos

1 Introducción....7
1.1 Introducción....7
1.2 Conceptos generales....7
2 Búsqueda y ordenación....9
2.1Búsqueda y ordenación interna....9
2.1.1Definición....9
2.1.2Operaciones Básicas....10
Algoritmos de Búsqueda....10
Algoritmo de búsqueda lineal....11
Algoritmo de Búsqueda Binaria....12
Algoritmos de Ordenación....13
Algoritmo de Ordenación por Inserción Directa....14
Algoritmo de Ordenación por Inserción Binaria....15
Algoritmo de Ordenación por Selección Directa....16
Algoritmo de Ordenación por Intercambio Directo....17
Algoritmo de Ordenación por Sacudida....20
Algoritmo de Ordenación Shell....24
Algoritmo de Ordenación "QuickSort"....25
Algoritmo de Ordenación "MergeSort"....27
2.2Búsqueda y ordenación externa....29
2.2.1Definición....29
2.2.2Operaciones básicas....30
Operaciones con Ficheros....33
Algoritmos de Clasificación....33
Clasificación por Mezcla Directa....34
Clasificación por Mezcla Natural....40
Clasificación de Raíz....46
Algoritmos de Búsqueda....50
Algoritmos de Búsqueda en ficheros Desordenados....51
Algoritmos de Búsqueda en ficheros Ordenados....53
3 ESTRUCTURAS DE DATOS LINEALES....55
3.1Listas....55
3.1.1Definición....55
3.1.2Operaciones Básicas con Listas....55
3.1.3Representación de Listas....56
Representación Vectorial de Listas....56
Representación mediante el uso de asignación de Memoria Dinámica....56
3.1.4Operaciones con listas....59
Operaciones mediante representación vectorial....59
Creación de la lista mediante representación vectorial....59
Inserción de un nuevo elemento en la lista....60
Eliminación de un elemento de la lista....63
Eliminación de un elemento de la lista por su posición....63
Eliminación de un elemento de la lista por su contenido....65
Comprobación de lista vacía....66
Comprobación de lista llena....67
Recorrido de la lista....67
Acceso a un elemento de la lista....68
Ordenación de los elementos de la lista....68
Búsqueda de un elemento en la lista....68
Operaciones mediante representación con memoria dinámica....68

Lista Lineal Simplemente Enlazada....68
Creación de la Lista Lineal Simplemente Enlazada....69
Inserción de un elemento en una Lista Lineal Simplemente Enlazada....69
Eliminación de un elemento en una Lista Lineal Simplemente Enlazada....76
Eliminación de un elemento en una Lista Lineal Simplemente Enlazada (por su posición)....76
Eliminación de un elemento en una Lista Lineal Simplemente Enlazada (por su contenido)....82
Comprobación de Lista Lineal Simplemente Enlazada vacía....83
Recorrido de una Lista Lineal Simplemente Enlazada....83
Acceso a un elemento de una Lista Lineal Simplemente Enlazada....85
Búsqueda de un elemento en una Lista Lineal Simplemente Enlazada....87
Vaciado de una Lista Lineal Simplemente Enlazada....91
Lista Lineal Doblemente Enlazada....93
Creación de la Lista Lineal Doblemente Enlazada....94
Inserción de un elemento en una Lista Lineal Doblemente Enlazada....95
Eliminación de un elemento en una Lista Lineal Doblemente Enlazada....101
Eliminación de un elemento en una Lista Lineal Doblemente Enlazada (por su posición)....101
Eliminación de un elemento en una Lista Lineal Doblemente Enlazada (por su contenido)....106
Lista Circular Simplemente Enlazada....106
Creación de la Lista Circular Simplemente Enlazada....106
Inserción de un elemento en una Lista Circular Simplemente Enlazada....107
Eliminación de un elemento en una Lista Circular Simplemente Enlazada....112
Eliminación de un elemento en una Lista Circular Simplemente Enlazada (por su posición)....112
Eliminación de un elemento en una Lista Circular Simplemente Enlazada (por su contenido)....120
Comprobación de Lista Circular Simplemente Enlazada vacía....120
Recorrido de una Lista Circular Simplemente Enlazada....120
Acceso a un elemento de una Lista Circular Simplemente Enlazada....123
Búsqueda de un elemento en una Lista Circular Simplemente Enlazada....123
Vaciado de una Lista Circular Simplemente Enlazada....126
Lista Circular Doblemente Enlazada....127
Creación de la Lista Circular Doblemente Enlazada....127
Inserción de un elemento en una Lista Circular Doblemente Enlazada....128
Eliminación de un elemento en una Lista Circular Doblemente Enlazada....132
Eliminación de un elemento en una Lista Circular Doblemente Enlazada (por su posición)....132
Eliminación de un elemento en una Lista Circular Doblemente Enlazada (por su contenido)....136
Listas Ordenadas....136
3.2Pilas....147
3.2.1Definición....147
3.2.2Operaciones Básicas con Pilas....148
3.2.3Representación de Pilas....149
Representación Vectorial de Pilas....149
Representación de Pilas mediante el uso de asignación de Memoria Dinámica....149
3.2.4Operaciones con Pilas....150
Operaciones mediante representación vectorial....150
Creación de la pila mediante representación vectorial....150
Comprobación de pila vacía....151
Comprobación de pila Llena....151

Operación de Apilado....152
Operación de desapilado....153
Comprobación del valor de la cima de la pila....155
Operaciones mediante representación de Memoria Dinámica....156
Creación de la pila mediante representación con Memoria Dinámica....156
Comprobación de pila vacía....157
Operación de Apilado....157
Operación de Desapilado....160
Comprobación del valor de la cima de la pila....163
Operación de eliminación de la pila....163
3.3Colas....165
3.3.1Definición....165
3.3.2Operaciones Básicas con Colas....166
3.3.3Representación de Colas....166
Representación Vectorial de Colas....166
Representación mediante el uso de asignación de Memoria Dinámica....168
3.3.4Operaciones con Colas....169
Operaciones mediante representación vectorial....169
Creación de la cola mediante representación vectorial....169
Siguiente elemento de un vector circular....170
Comprobación de Cola vacía....170
Comprobación de Cola llena....170
Operación de Encolado....171
Operación de Desencolado....173
Acceso al elemento situado en la salida de la cola....175
Operaciones mediante representación de Memoria Dinámica....175
Implementación con Cola Lineal....175
Creación de la Cola Lineal mediante representación con Memoria Dinámica....176
Comprobación de Cola Lineal vacía....176
Operación de Encolado sobre Cola Lineal....177
Operación de Desencolado sobre Cola Lineal....181
Acceso al elemento situado en la salida de la Cola Lineal....186
Operación de vaciado de la Cola Lineal....187
Implementación con Cola Circular....188
Creación de la Cola Circular mediante representación con Memoria Dinámica....188
Comprobación de Cola Circular vacía....189
Operación de Encolado sobre Cola Circular....189
Operación de Desencolado sobre Cola Circular....195
Acceso al elemento situado en la salida de la Cola Circular....198
Operación de vaciado de la Cola Circular....199
4Estructuras de datos no lineales....201
4.1Grafos....201
4.1.1Definición....201
4.1.2 Representación de Grafos....202
Representación mediante Matriz de Adyacencia....202
Representación mediante Listas de Adyacencia....203
4.1.3Operaciones en la representación mediante Matriz de Adyacencia....205
Creación....205
Añadir un Arco....205
Añadir una Arista....206
Comprobación de la existencia de un Vértice....206
Comprobación de la existencia de un Arco....207
Comprobación de la existencia de una Arista....207

Eliminar un Arco....207
Eliminar una Arista....208
4.1.4Operaciones en la representación mediante Listas de Adyacencia....208
Creación....209
Comprobación de la existencia de un Vértice....209
Comprobación de la existencia de un Arco....210
Comprobación de la existencia de una Arista....211
Añadir un Vértice....211
Añadir un Arco....212
Añadir una Arista....214
Eliminar un Vértice....215
Eliminar un Arco....216
Eliminar una Arista....217
Eliminar las Listas de Adyacencia....218
4.1.5Recorrido de un Grafo....218
Recorrido en Anchura....219
Recorrido en Profundidad....220
4.2 Árboles....222
4.2.1Definición - Árboles binarios....222
4.2.2 Operaciones básicas....224
4.2.3Representación de Árboles binarios....224
Representación vectorial de árboles....225
Árboles Binarios llenos y completos....226
Representación basada en el uso de memoria dinámica....228
4.2.4 Operaciones con Árboles....230
Operaciones en la representación vectorial....230
Creación de un árbol vacío....230
Comprobación de árbol vacío....230
Comprobación de árbol lleno....230
Inserción de un nodo....231
Eliminación de nodos....232
Modificación de un nodo del árbol....235
Operaciones en la representación basada en el uso de memoria dinámica....235
Creación de un árbol vacío....235
Comprobación de árbol vacío....235
Inserción de un nodo....236
Eliminación de nodos....237
Modificación de un nodo del árbol....238
4.2.5 Recorrido del árbol....239
Recorrido de un árbol representado mediante un vector....240
Recorrido de un árbol representado usando memoria dinámica....242
4.2.6Árboles Binarios de Búsqueda....244
Construcción de un Árbol Binario de Búsqueda....245
Búsquedas en un Árbol Binario de Búsqueda....251
4.2.7Montículos binarios (heaps)....253
Construcción de un montículo....254
Algoritmo Heapsort....257

# 1 INTRODUCCIÓN

## 1.1 Introducción

Esta herramienta presenta una descripción de las estructuras de datos fundamentales y sus algoritmos de manipulación.

Los contenidos se han organizado en 4 módulos:

- Presentación de la herramienta,
- Operaciones de búsqueda y ordenación, tanto en las estructuras de datos internas como en las externas,
- Estructuras de datos lineales,
- Estructuras de datos no lineales,

Los algoritmos se describen utilizando pseudocódigo y se acompañan de representaciones gráficas que facilitan su comprensión.

## 1.2 Conceptos generales

El pseudocódigo utilizado resulta muy sencillo de entender, por lo que no requiere una descripción pormenorizada.

Para representar las estructuras de datos que vamos a estudiar se utilizan dos métodos básicos: vectores o memoria dinámica. Dado que la segunda representación utiliza algunas operaciones predefinidas en los lenguajes de programación, introduciremos aquí la notación en pseudocódigo para estas operaciones.

Trabajo con memoria dinámica

Para trabajar con memoria dinámica es necesario manejar ***punteros***. Un puntero es una variable que almacena una dirección de memoria. Se suele decir que el puntero 'apunta' a la zona de memoria correspondiente. Para definir un puntero, se indica su nombre y el tipo asociado, que determina el tipo de los elementos a los que podrá apuntar. Por ejemplo, para declarar un puntero a enteros escribiremos:

**P puntero a enteros**

La variable P podrá apuntar a variables enteras o a una zona de memoria reservada dinámicamente para almacenar un entero. Es decir, un puntero podrá apuntar a variables estáticas o a ***variables dinámicas*** del tipo que tiene asociado. En el caso de una variable estática, se solicita espacio de memoria para la misma en tiempo de compilación; sin embargo, para las variables dinámicas se solicita en tiempo de ejecución.

Existe un valor especial, que representaremos por **NULO**, que se asigna a un puntero para indicar que no apunta a ninguna dirección de memoria válida.

**P <- NULO**

Una variable dinámica no tiene nombre y sólo se puede hacer referencia a ella mediante un puntero que apunte a la zona de memoria que se le ha asignado dinámicamente.

Para crear una variable dinámica, será necesario solicitar espacio para ella al sistema, mediante una llamada a una función predefinida del lenguaje, que llamaremos **RESERVAR_MEMORIA**. Dicha función recibe como parámetro el tamaño del bloque de memoria que se solicita al sistema, y devuelve la dirección de comienzo en memoria del bloque asignado (o NULO si la solicitud no tuvo éxito). El tamaño del bloque de memoria queda determinado por el tipo de variable dinámica que se desea crear. Supondremos que existe otra función predefinida, que denominaremos **tamaño**, que determina el espacio de memoria que requiere un dato del tipo especificado como argumento en la llamada. La representación en pseudocódigo de la reserva será:

**P <- RESERVAR_MEMORIA(tamaño(entero))**

Con la llamada anterior se solicita espacio para una variable de tipo entero. El valor retornado por la función se almacena en un puntero a enteros.

Antes de comenzar a trabajar sobre la zona de memoria reservada, es recomendable comprobar que la reserva tuvo éxito. Si el sistema no pudo asignar la memoria solicitada, cualquier intento de operación sobre ella provocará un error en el programa.

```
Si (P = NULO) entonces
        ESCRIBIR "error en la reserva de memoria"
        ACABAR   ** u otra operación
fin-si
```

Cuando una variable dinámica deje de ser necesaria, se podrá liberar la memoria que ocupa mediante una llamada a otra función predefinida, que llamaremos **LIBERAR**, y que retorna al sistema la memoria a la que apunta el puntero pasado como argumento:

**LIBERAR(P)**

# 2 BÚSQUEDA Y ORDENACIÓN

Este tema describe los algoritmos fundamentales para realizar búsquedas y ordenaciones en tablas y en ficheros.

## 2.1 Búsqueda y ordenación interna

### *2.1.1 Definición*

Una **tabla** o **array** está formada por un número fijo, finito y ordenado de elementos, todos del mismo tipo y bajo un nombre común.

Los elementos de una tabla (cada constante o variable) se denominan **componentes**.

El nombre de la tabla es el identificador que permite hacer referencia global a todos sus elementos. Para hacer referencia de modo individual a una componente se debe indicar el nombre de la tabla y uno o más **índices**, que especifican la posición relativa de la componente dentro de la tabla. Un índice puede estar expresado por un número entero, una variable numérica entera o una expresión entera.

La **dimensión** de una tabla es el número de índices que se necesitan para identificar una componente. Una tabla se puede estructurar en 1 o más dimensiones. Cuando se utiliza una sola dimensión se suele hablar de tablas **unidimensionales** o vectores, si se utilizan dos dimensiones hablaremos de tablas **bidimensionales** o matrices. Para 3 o más dimensiones se habla de tablas **multidimensionales** o poliedros. En algunos lenguajes el primer elemento de cada dimensión es el 0, mientras que en otros es el 1.

En este tema nos centraremos en el trabajo con tablas **unidimensionales** o **vectores**.

El **tamaño** de una tabla es el número de componentes que contiene. Si la tabla tiene varias dimensiones, existirá un tamaño para cada una de ellas.

Para definir una tabla, se debe indicar:

- **nombre**: permitirá hacer referencia a la tabla a lo largo del programa.
- **tamaño**: se indica normalmente entre paréntesis o entre corchetes. Para tablas multidimensionales, se suelen separar los tamaños de las distintas dimensiones mediante comas. El número de elementos de la tabla será igual al producto de los tamaños de todas las dimensiones.
- **tipo**: determina el tipo asociado a cada uno de sus elementos.

Esta operación reserva una zona de memoria con capacidad para almacenar todos los elementos de la tabla.

Para definir un vector, de n elementos de tipo T, escribiremos:

**nombre_vector(n) de tipo T**

Para definir una tabla bidimensional de n filas x m columnas, de elementos de tipo T:

**nombre_matriz(n, m) de tipo T**

Para más dimensiones el mecanismo sería idéntico, añadiendo tantos índices como dimensiones.

**V(3) de enteros**

| *1* | *2* | *3* | *índice* |
|---|---|---|---|
| | | | |

En general, una vez creada la tabla sus componentes tendrán valor indeterminado hasta que se les de un valor concreto en el programa, salvo que el lenguajes de programación utilizado asocie un valor por defecto a las variables de cada tipo.

Es posible dar valor a todas las componentes de la tabla en la misma operación, realizando lo que se denomina inicialización de la tabla. Esta operación consiste en dar valores a las componentes de la tabla en el momento en que ésta se declara.

**V(3) de enteros ← (100, 20, 35)**

| *1* | *2* | *3* | *índice* |
|---|---|---|---|
| **100** | **20** | **35** | |

Se puede dar valor a una componente de forma individual mediante una asignación:

**V(1) ← 48**

| *1* | *2* | *3* | *índice* |
|---|---|---|---|
| **48** | **20** | **35** | |

O mediante una lectura:

**LEER V( I )**

## *2.1.2 Operaciones Básicas*

Las operaciones que consideraremos sobre una tabla son:

- **Búsqueda:** consiste en determinar si un valor dado aparece en una tabla y la posición que ocupa, si es que se encuentra.
- **Ordenación:** consiste en reorganizar los elementos de la tabla según cierto criterio. Se pueden ordenar los valores de forma *ascendente* o *descendente*.

## *Algoritmos de Búsqueda*

Para describir los algoritmos de búsqueda consideraremos un vector V de tamaño N de algún tipo T, que puede contener valores repetidos. Para hacer la descripción más general, el número de valores almacenados en cualquier momento en V será menor o igual que N. Si el vector no contiene N elementos, las llamadas a los módulos de búsqueda que se describen en este apartado recibirán como segundo parámetro el

número de elementos realmente almacenados en el vector. En este caso supondremos que los elementos ocupan posiciones consecutivas del vector a partir de la primera.

Se analizarán dos técnicas de búsqueda: la búsqueda ***lineal*** y la búsqueda ***binaria*** o ***dicotómica***.

## Algoritmo de búsqueda lineal

Este método de búsqueda es válido tanto para vectores **desordenados** como **ordenados**.

Para buscar un valor X en un vector **no ordenado**, se recorre el vector de un extremo al opuesto (de izquierda a derecha o de derecha a izquierda) hasta encontrar una componente cuyo valor sea X, o hasta que se haya recorrido todo el vector, lo que primero ocurra. En el segundo caso el algoritmo debe indicar la no existencia del valor buscado.

Si el valor X aparece varias veces, el algoritmo anterior indicará el valor cuyo índice sea más próximo al extremo de comienzo del recorrido.

Si el vector está **ordenado** (de forma ascendente, en nuestro caso), la búsqueda puede concluir cuando se encuentre un valor mayor que el buscado, lo que significa que X no aparece en el vector. En este caso se sale del bucle de búsqueda bien porque se ha encontrado un valor mayor o igual que X, o bien porque se ha llegado al último elemento del vector. La búsqueda habrá tenido éxito si el valor del elemento en el que se detuvo es igual a X.

**Vector que puede estar desordenado**

```
MÓDULO busqueda_lineal

DATOS
        PARÁMETROS
                Recibe V() de tipo T
                Recibe N entera
                Recibe X de tipo T
        VARIABLES
                I entera
INICIO
        I ← 1

        Mientras ( (V(I) ≠ X)   y   (I < N) ) hacer
                I ← I+1
        fin-mientras

        Si ( V(I) = X ) entonces
                Escribir "Encontrado en posición", I
        sino
                Escribir "No encontrado"
        fin-si
FIN
```

**Vector ordenado**

```
MÓDULO busqueda_lineal_ordenado

DATOS
        PARÁMETROS
                Recibe V() de tipo T
                Recibe N entera
                Recibe X de tipo T
        VARIABLES
                I entera

INICIO
        I ← 1

        Mientras ( (V(I) < X)   y   (I < N) ) hacer
                I ← I+1
        fin-mientras

        Si ( V(I) = X ) entonces
                Escribir "Encontrado en posición", I
        sino
                Escribir "No encontrado"
        fin-si
FIN
```

## Algoritmo de Búsqueda Binaria

Este método sólo es aplicable a **vectores ordenados.**

Supongamos que los datos están ordenados de forma creciente. Se compara el valor buscado con la componente central del vector. Si coinciden, se ha encontrado el elemento buscado. En caso contrario, se reduce el intervalo de búsqueda a la mitad izquierda o derecha, según donde pueda encontrarse el valor buscado: si el elemento central es mayor que el valor buscado, la búsqueda debe continuar en la mitad izquierda; si es menor, debe continuar en la mitad derecha.

El algoritmo se repite sobre la mitad seleccionada, concluyendo cuando se encuentra el valor o cuando el intervalo de búsqueda se anula (lo que primero ocurra).

Si el vector contiene valores repetidos, este algoritmo obtendrá uno de ellos ya que necesariamente serán consecutivos.

```
MÓDULO busqueda_binaria
DATOS
        PARÁMETROS
                Recibe V() de tipo T
                Recibe N entera
                Recibe X de tipo T
```

```
        VARIABLES
                I, IZQ, DER, CEN enteras
INICIO
IZQ ← 1
DER ← N
CEN ← (N+1)\2        ** división entera

        Mientras ( (V(CEN) ≠ X)   y   (IZQ < DER) ) hacer
                Si ( V(CEN) > X ) entonces
                        DER ← CEN-1
                sino
                        IZQ ← CEN+1
                fin-si
                CEN ← (IZQ+DER)\2
        fin-mientras

        Si ( V(CEN) = X ) entonces
                Escribir "encontrado en la posición", CEN
        sino
                Escribir "no existe el valor buscado"
        fin-si
FIN
```

## Algoritmos de Ordenación

Podemos realizar una ordenación:

- **Ascendente** o **Creciente**: los valores mayores se sitúan al final del vector y los menores al principio.
- **Descendente** o **Decreciente**: los valores mayores se sitúan al principio del vector y los menores al final.

Si el vector incluye valores repetidos, éstos ocuparán posiciones consecutivas. Para valores alfanuméricos se utiliza el orden lexicográfico.

Si las componentes del vector almacenan datos compuestos (por ejemplo, varios datos relativos a personas: nombre, edad, DNI, dirección,...), la ordenación se realizará por alguno de tales datos.

Veremos varios métodos aplicados a vectores considerando, en todos ellos, que se desean ordenar los N elementos de un vector V de tipo T de forma ascendente.

### Algoritmo de Ordenación por Inserción Directa

Este método, también denominado **método de la baraja**, toma cada elemento del vector y busca su posición en orden con respecto a los elementos situados a su izquierda, que ya están ordenados entre sí.

Se toman los elementos del vector desde el segundo hasta el último y con cada uno de ellos se repiten las siguientes operaciones:

1. Se copia el valor de la componente actual en una variable auxiliar.
2. Desde el elemento anterior al que estamos tratando de situar y hasta el primero de los elementos del vector, desplazamos un lugar a la derecha todos los que sean mayores, para crear un hueco donde colocar el valor que estaba desordenado
3. Insertamos el valor que estaba almacenado en la variable auxiliar en el hueco creado en el paso anterior.

```
MÓDULO ordenacion_insercion_directa
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe N entera
    VARIABLES
        I, J enteras
        AUX de tipo T
INICIO
    Para I desde 2 hasta N
        AUX ← V(I)
        J ← I

        Mientras ( (V(J-1) > AUX)   y   (J > 1) ) hacer
            V(J) ← V(J-1)
            J ← J-1
        fin-mientras

        V(J) ← AUX
    fin-para
FIN
```

### Algoritmo de Ordenación por Inserción Binaria

Es una variante del método de inserción directa, en la que **se sustituye la búsqueda secuencial por una búsqueda binaria** (que se puede aplicar dado que la zona de búsqueda contiene elementos ordenados).

Para cada elemento $a_i$ comprendido entre las posiciones 2 y N:

1. Se aplica una búsqueda binaria entre las posiciones [1, I-1], para determinar la posición que debería ocupar el elemento a ordenar, respecto a los de dicho intervalo.
2. Todos los elementos del intervalo anterior que sean mayores que el que se pretende colocar, se desplazarán una posición a la derecha.
3. Se coloca el valor en su nueva posición, ya en orden.

```
MÓDULO ordenacion_insercion_binaria
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe N entera
    VARIABLES
        I, J enteras
        AUX de tipo T
INICIO
    Para I desde 2 hasta N
        AUX ← V(I)
        IZQ ← 1
        DER ← I-1

        Mientras ( IZQ ≤ DER ) hacer
            CEN ← (IZQ+DER)\ 2      ** división entera

            Si ( V(CEN) ≤ AUX ) entonces
                IZQ ← CEN+1
            sino
                DER ← CEN-1
            fin-si
        fin-mientras
```

```
                J ← I
                Mientras (J > IZQ) hacer
                        V(J) ← V(J-1)
                        J ← J-1
                fin-mientras

                V(IZQ) ← AUX
        fin-para
FIN
```

### Algoritmo de Ordenación por Selección Directa

Este método realiza sucesivas búsquedas del menor de los elementos que quedan por ordenar, colocándolo en su posición definitiva dentro de la parte del vector ordenado. Tras cada iteración I del algoritmo, los elementos situados a la izquierda del I-ésimo están ordenados.

Por tanto y para cada iteración I del algoritmo (de un total de N-1) los pasos a dar son:

1. Se selecciona la componente de menor valor, de entre las existentes entre la I-ésima posición y la última.
2. Se intercambia el valor seleccionado con el almacenado en la posición I.

```
MÓDULO ordenacion_seleccion_directa
DATOS
        PARÁMETROS
                Transforma V() de tipo T
                Recibe N entera
        VARIABLES
                I, J, K enteras
                AUX de tipo T
INICIO
        Para I desde 1 hasta N-1
                K ← I
                AUX ← V(I)

                Para J desde I+1 hasta N
```

```
                    Si ( V(J) < AUX ) entonces
                        K ← J
                        AUX ← V(J)
                    fin-si
                fin-para

                V(K) ← V(I)
                V(I) ← AUX
            fin-para
        FIN
```

**Algoritmo de Ordenación por Intercambio Directo**

Este método también se conoce como **método de la burbuja**. Tiene dos versiones basadas en la misma idea: recorrer sucesivamente el vector comparando los elementos consecutivos e intercambiándolos cuando estén descolocados. En una de las versiones el vector se recorre de izquierda a derecha, desplazando los valores mayores hacia su derecha, y en la otra se recorre de derecha a izquierda, desplazando los valores menores hacia su izquierda, ambos para la clasificación en orden ascendente.

Cada pasada sobre el vector lleva un nuevo elemento a su posición definitiva, y en orden, en el extremo de la zona de comparación considerada (extremo derecho en el caso de recorrido de izquierda a derecha y en el extremo izquierdo en el caso de recorrido de derecha a izquierda), por lo que en la siguiente pasada se excluye de las comparaciones.

El algoritmo consiste en aplicar los siguientes pasos, considerando inicialmente todas las posiciones del vector:

1. Se comparan los valores de las posiciones contiguas, intercambiándolos si están mal colocados.
2. Una vez realizadas todas las posibles comparaciones, se descarta la posición del extremo izquierdo (si se realiza un recorrido de derecha a izquierda) o del extremo derecho (si se realiza un recorrido de izquierda a derecha) del intervalo de posiciones consideradas en la iteración actual, repitiéndose el proceso hasta completar un total de N-1 iteraciones.

Podemos mejorar el método, deteniendo la ordenación, si se detecta que en una pasada no se ha producido ningún intercambio, lo que significa que los datos ya están ordenados. Es necesario realizar una pasada completa sin intercambios antes de poder forzar el fin del algoritmo. Con esta mejora el número de iteraciones del algoritmo estará condicionado por el grado de desorden inicial de los datos. (Ver algoritmo de Ordenación por intercambio directo con test)

```
MÓDULO ordenacion_intercambio_directo_izq_der
DATOS
        PARÁMETROS
                Transforma V() de tipo T
        Recibe N entera
        VARIABLES
                I, J enteras
                AUX de tipo T
INICIO
        Para J desde 1 hasta N-1
                Para I desde 1 hasta N-J
                        Si ( V(I) > V(I+1) ) entonces
                                AUX <- V(I)
                                V(I) <- V(I+1)
                                V(I+1) <- AUX
                        fin-si
                fin-para
        fin-para
FIN

MÓDULO ordenacion_intercambio_directo_der_izq
DATOS
        PARÁMETROS
                Transforma V() de tipo T
                Recibe N entera
        VARIABLES
                I, J enteras
                AUX de tipo T
INICIO
        Para J desde 1 hasta N-1
                Para I desde N hasta J+1 con incremento -1
                        Si ( V(I-1) > V(I) ) entonces
                                AUX <- V(I-1)
```

```
                    V(I-1) <- V(I)
                    V(I) <- AUX
                fin-si
            fin-para
        fin-para
FIN

MÓDULO ordenacion_intercambio_directo_test
DATOS
        PARÁMETROS
            Transforma V() de tipo T
            Recibe N entera
        VARIABLES
            I, J enteras
            SEGUIR lógica
            AUX de tipo T
INICIO
        J <- 1
        SEGUIR <- VERDADERO
        Mientras ( (SEGUIR = VERDADERO)   y   (J < N) ) hacer
            SEGUIR <- FALSO
            Para I desde 1 hasta N-J
                Si ( V(I) > V(I+1) ) entonces
                    AUX <- V(I)
                    V(I) <- V(I+1)
                    V(I+1) <- AUX
                    SEGUIR <- VERDADERO
                fin-si
            fin-para
            J <- J+1
        fin-mientras
FIN
```

### Algoritmo de Ordenación por Sacudida

Este método, también conocido como **método de ordenación por vibración**, es una mejora del método de la burbuja, que mezcla sus dos versiones alternativamente, realizando una pasada de izquierda a derecha y a continuación otra de derecha a izquierda, recortándose los elementos a tratar por ambos extremos del vector.

Igual que en el caso de la burbuja, se puede aumentar la efectividad del método de la sacudida deteniendo la ordenación si se comprueba que en una pasada, tanto de izquierda a derecha como de derecha a izquierda, todos los elementos están ordenados. (Ver algoritmo de Ordenación por Sacudida con test)

El método de la sacudida se puede modificar para descartar más posiciones en los extremos, en lugar de una sola, como ocurre en los algoritmos anteriores. Si al realizar una pasada de derecha a izquierda el último intercambio afecta a las posiciones X y X+1, significa que los valores comprendidos entre las posiciones 1 y X ya están ordenados, por lo que no será necesario evaluar dichas posiciones en la siguiente pasada. Análogamente, si al realizar una pasada de izquierda a derecha el último intercambio afecta a las posiciones X-1 y X, significa que los valores comprendidos entre las posiciones X y N ya están ordenados, por lo que no será necesario evaluar dichas posiciones en la siguiente pasada. (Ver algoritmo de Ordenación por Sacudida con salto).

```
MÓDULO ordenacion_sacudida
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe N entera
    VARIABLES
        I, IZQ, DER enteras
        AUX de tipo T
INICIO
    IZQ ← 1
    DER ← N

    Mientras ( IZQ ≠ DER)  hacer
        ** pasada de izquierda a derecha
        Para I desde IZQ hasta DER-1
            Si ( V(I) > V(I+1) ) entonces
                AUX ← V(I)
                V(I) ← V(I+1)
                V(I+1) ← AUX
```

```
                fin-si
            fin-para

            DER ← DER-1

            Si ( IZQ ≠ DER ) entonces
                ** pasada de derecha a izquierda
                Para I desde DER hasta IZQ+1 con incremento -1
                    Si ( V(I-1) > V(I) ) entonces
                        AUX ← V(I-1)
                        V(I-1) ← V(I)
                        V(I) ← AUX
                    fin-si
                fin-para

                IZQ ← IZQ+1
            fin-si
        fin-mientras
FIN
```

**Ordenación por sacudida con test de comprobación**

```
MÓDULO ordenacion_sacudida_test
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe N entera
    VARIABLES
        I, IZQ, DER, CEN enteras
        SEGUIR lógica
INICIO
    IZQ ← 1
    DER ← N
    SEGUIR ← VERDADERO
```

```
Mientras ( (IZQ ≠ DER)   y   (SEGUIR = VERDADERO) ) hacer
        SEGUIR ← FALSO

        Para I desde IZQ hasta DER-1
                Si ( V(I) > V(I+1) ) entonces
                        AUX ← V(I)
                        V(I) ← V(I+1)
                        V(I+1) ← AUX
                        SEGUIR ← VERDADERO
                fin-si
        fin-para

        DER ← DER-1

        Si ( (IZQ ≠ DER)   y   (SEGUIR =VERDADERO) ) entonces
                SEGUIR ← FALSO

                Para I desde DER hasta IZQ+1 con incremento -1
                        Si ( V(I-1) > V(I) ) entonces
                                AUX ← V(I-1)
                                V(I-1) ← V(I)
                                V(I) ← AUX
                                SEGUIR ← VERDADERO
                        fin-si
                fin-para

                IZQ ← IZQ+1
        fin-si
    fin-mientras
FIN
```

**Ordenación por sacudida con salto**

```
MÓDULO ordenacion_sacudida_salto
```

```
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe N entera
    VARIABLES
        I, IZQ, DER, K enteras
        AUX de tipo T
INICIO
    IZQ ← 1
    DER ← N
    K ← 1

    Mientras ( IZQ < DER ) hacer
        Para I desde IZQ hasta DER-1
            Si ( V(I) > V(I+1) ) entonces
                AUX ← V(I)
                V(I) ← V(I+1)
                V(I+1) ← AUX
                K ← I+1
            fin-si
        fin-para

        DER ← K-1

        Si ( IZQ < DER ) entonces
            Para I desde DER hasta IZQ+1 con incremento -1
                Si ( V(I-1) > V(I) ) entonces
                    AUX ← V(I-1)
                    V(I-1) ← V(I)
                    V(I) ← AUX
                    K ← I-1
                fin-si
            fin-para
```

```
                    IZQ ← K+1
                fin-si
            fin-mientras
        FIN
```

### Algoritmo de Ordenación Shell

Este método, también conocido como **método de ordenación por inserción con incrementos decrecientes**, es una mejora del método de inserción directa. Consiste en ordenar por inserción los elementos que difieren $S_1$ posiciones. A continuación se ordenan los que difieren $S_2$ posiciones, siendo $S_2 < S_1$. Se repite el mismo proceso sucesivamente, considerando valores $S_i$ decrecientes, hasta alcanzar el valor 1, ordenando como último paso los elementos situados a distancia 1. Los valores $S_1$, $S_2$, ... suelen denominarse ***incrementos***. En el peor de los casos, toda la ordenación se realizará al considerar la última distancia, que garantiza la ordenación.

Las operaciones del algoritmo se podrían esquematizar como sigue:

1. Tomar una distancia de comparación inicial.
2. Ordenar entre sí los elementos situados a esa distancia.
3. Reducir la distancia de comparación. Si es mayor o igual que 1, volver al paso 2.

Se pueden elegir diferentes secuencias de incrementos o distancias. En nuestro caso la distancia inicial es igual a la mitad del número de elementos del vector. En cada nueva iteración se reduce la distancia previa a la mitad, tomando en la última iteración distancia 1.

```
MÓDULO ordenacion_shell
DATOS
        PARÁMETROS
                Transforma V() de tipo T
                Recibe N entera
        VARIABLES
                I, J, DISTANCIA, AUX enteras
INICIO
        DISTANCIA ← N

        Mientras ( DISTANCIA ≠ 1 ) hacer
                DISTANCIA ← DISTANCIA\2      ** división entera
```

```
        Para I desde DISTANCIA+1 hasta N
            AUX ← V(I)
            J ← I

            Mientras (  (J-DISTANCIA) ≥ 1)  y
                        (V(J-DISTANCIA) > AUX)  ) hacer
                V(J) ← V(J-DISTANCIA)
                J ← J-DISTANCIA
            fin-mientras

            V(J) ← AUX
        fin-para
    fin-mientras
FIN
```

**Algoritmo de Ordenación "QuickSort"**

Este método aplica la técnica **"divide y vencerás"**, subdividiendo el vector en vectores más pequeños y ordenando éstos. Para hacer esta división, se toma un valor cualquiera del vector, que se suele denominar ***pivote***, y se realiza el desplazamiento de todos los elementos menores que él a la izquierda de éste. De la misma forma, los elementos mayores que él se desplazarán a su derecha. Con esto se garantiza que los elementos colocados a la izquierda del pivote son menores o iguales que éste, mientras los situados a su derecha serán mayores o iguales a él. A continuación se aplica el mismo método a cada una de las dos partes en las que queda dividido el vector. El procedimiento se repite de forma recursiva hasta que al final todas las particiones son de tamaño 1, con lo que el vector queda ordenado.

Por tanto, dado un vector V de N elementos, el método de ordenación aplicará los siguientes pasos básicos:

1. Se elige un elemento como pivote
2. Si el tamaño del vector es 1, está ordenado. En caso contrario se divide el vector en 2 vectores, V1 y V2, de forma que los elementos de V1 sean menores (o iguales, si hay valores repetidos) que los de V2
3. Se ordenan V1 y V2 aplicando QuickSort
4. Se devuelve la concatenación de V1 y V2, obteniendo el vector V ordenado

La elección del pivote no es algo trivial. Aunque en principio se podría elegir cualquier valor, una elección inadecuada reducirá considerablemente la eficiencia del método de

ordenación. En nuestro caso se ha optado por tomar, como elemento pivote, el valor de la posición central del vector, o del subvector que se está ordenando.

```
MÓDULO ordenacion_rapida
DATOS
    PARÁMETROS
        Transforma V() de tipo T
        Recibe IZQ entera
        Recibe DER entera
    VARIABLES
        I, J enteras
        PIVOTE, AUX de tipo T
INICIO
    I ← IZQ
    J ← DER
    PIVOTE ← V((IZQ+DER)\2)   ** división entera

    Mientras ( I ≤ J ) hacer
        Mientras ( V(I) < PIVOTE ) hacer
            I ← I+1
        fin-mientras

        Mientras ( V(J) > PIVOTE ) hacer
            J ← J-1
        fin-mientras

        Si ( I ≤ J ) entonces
            AUX ← V(I)
            V(I) ← V(J)
            V(J) ← AUX
            I ← I + 1
            J ← J - 1
        fin-si
    fin-mientras
```

```
        Si ( IZQ < J ) entonces
                ordenacion_rapida(V, IZQ, J)
        fin-si

        Si ( DER > I ) entonces
                ordenacion_rapida(V, I, DER)
        fin-si
    FIN
```

### Algoritmo de Ordenación "MergeSort"

Este método se basa en la técnica **"divide y vencerás"**. Los pasos que aplica son los siguientes:

1. Se divide el vector en dos mitades.
2. Se ordena cada una de las dos mitades aplicando MergeSort.
3. Se mezclan ambas mitades, obteniendo un vector ordenado.

La operación de división se aplica, recursivamente, a cada una de las dos mitades obtenidas en cada paso. Cuando la partición obtenida tenga un solo elemento se la considera ordenada. A continuación se mezclarán las particiones de un elemento, obteniendo particiones ordenadas de dos elementos. El proceso de mezcla continuará, obteniendose particiones ordenadas cada vez más grandes, hasta obtener todo el vector ordenado.

La operación de mezcla de dos vectores ordenados genera un nuevo vector que contiene los elementos de ambos y que conserva el orden. En este caso los dos vectores que se pretenden mezclar son realmente dos intervalos de posiciones del vector que estamos intentando ordenar. El primer subvector será el que comprende las posiciones IZQ a CEN del vector original, mientras que el segundo comprende las posiciones CEN+1 a DER.

Para realizar la mezcla, se comparan dos elementos de ambos subvectores, copiando en el destino el menor de ellos. A continuación se toma otro elemento del mismo origen del que se acaba de copiar el valor en el destino y prosigue la comparación. El proceso concluye cuando se hayan copiado, en el destino, todos los elementos de algún origen. En ese momento se copian, en el destino, todos los elementos del otro origen que aún no se hayan copiado.

Se utilizará un vector auxiliar, llamado TEMP, que permitirá almacenar, durante el proceso de mezcla, los valores que se están mezclando. Concluida la mezcla, se copian los elementos de este vector sobre el original, considerando sólo las posiciones del vector TEMP comprendidas entre IZQ y DER.

```
MÓDULO ordenacion_por_mezcla
DATOS
        PARÁMETROS
                Transforma V() de tipo T
                Recibe IZQ entera
                Recibe DER entera
        VARIABLES
                CEN entera
INICIO
        Si ( IZQ < DER ) entonces
                CEN <- (IZQ+DER)\2      ** división entera
                ordenacion_por_mezcla(V, IZQ, CEN)
                ordenacion_por_mezcla(V, CEN+1, DER)
                mezclar(V, IZQ, CEN, DER)
        fin-si
FIN


MÓDULO mezclar
DATOS
        PARÁMETROS
                Transforma V() de tipo T
                Recibe IZQ entera
                Recibe CEN entera
                Recibe DER entera
        VARIABLES
                TEMP() de tipo T
                I1, I2, I_TEMP entera
INICIO
        I1 <- IZQ
        I2 <- CEN+1
        I_TEMP <- IZQ
```

```
        Mientras( (I1 ≤ CEN)  y  (I2 ≤ DER) ) hacer
                Si ( V(I1) < V(I2) ) entonces
                        TEMP(I_TEMP) <- V(I1)
                        I1 <- I1 + 1
                sino
                        TEMP(I_TEMP) <- V(I2)
                        I2 <- I2 + 1
                fin-si

                I_TEMP <- I_TEMP + 1
        fin-mientras

        Mientras (I1 ≤ CEN) hacer
                TEMP(I_TEMP) <- V(I1)
                I1 <- I1 + 1
                I_TEMP <- I_TEMP + 1
        fin-mientras

        Mientras (I2 ≤ DER) hacer
                TEMP(I_TEMP) <- V(I2)
                I2 <- I2 + 1
                I_TEMP <- I_TEMP + 1
        fin-mientras

        Para I_TEMP desde IZQ hasta DER
                V(I_TEMP) <- TEMP(I_TEMP)
        fin-para
FIN
```

## 2.2 Búsqueda y ordenación externa

### *2.2.1 Definición*

Se denomina **archivo** o **fichero** a un conjunto de informaciones estructuradas en

unidades de acceso denominadas registros, todos del mismo tipo, que se almacenan en algún soporte de memoria secundaria.

Un **registro** es una estructura de datos formada por uno o más elementos, denominados **campos**, que pueden ser de diferentes tipos y que, a su vez, pueden estar compuestos por **subcampos**.

Los registros suelen tener un campo especial, denominado **clave** o **identificativo**, que permite identificar al registro o establecer un criterio para las operaciones. En algunos ficheros las claves son únicas, mientras que en otros pueden repetirse en diferentes registros. Pueden existir varios campos clave, en cuyo caso se habla de claves primaria, secundaria,...

En este tema trabajaremos sobre archivos con acceso secuencial. Se denomina **acceso** al procedimiento a seguir para posicionarse sobre un registro concreto, con la intención de realizar sobre el mismo una operación de lectura o escritura. En los **ficheros secuenciales** el acceso a los registros se realiza según el orden de almacenamiento de los mismos dentro del fichero, de uno en uno a partir del primero. Para acceder al registro n es necesario acceder a los n-1 anteriores.

### *2.2.2 Operaciones básicas*

Las operaciones básicas para el trabajo con un archivo secuencial son: **apertura**, **cierre**, **lectura**, **escritura** y**comprobación de fin de fichero**. Estas operaciones se realizarán mediante funciones predefinidas del lenguaje de programación utilizado. La primera operación que se realizará es la de apertura y la última la de cierre. Una vez abierto el fichero se podrán hacer lecturas o escrituras. Cuando el objetivo sea realizar lecturas, se deberá comprobar si se ha alcanzado el final del fichero antes de intentar leer, para evitar errores.

Para trabajar con un fichero desde un programa, se asocia al nombre del archivo físico (el que reside en disco) un nombre lógico, mediante una instrucción del lenguaje considerado. En general, esta asignación se realiza en la operación de apertura. Una vez obtenido el nombre lógico del fichero, todas las operaciones sobre el fichero se realizarán utilizando dicho nombre.

Supondremos que trabajamos con ficheros que contiene registros de tipo REGISTRO:

```
** estructura de los datos del fichero
REGISTRO es registro compuesto por
        ** campos de algún tipo
        ** . . .
fin-registro
```

Trabajaremos con un fichero físico denominado NOMBRE_F, al cual se asociará el fichero lógico F. El nombre físico del fichero puede incluir el path, considerándose en cualquier caso como una cadena de caracteres:

```
NOMBRE_F cadena              ** nombre físico del fichero
F es archivo de REGISTRO     ** nombre lógico del fichero
```

Se utilizará una variable para realizar las operaciones de lectura o escritura, que será de tipo REGISTRO y permitirá intercambiar registros entre el fichero y el programa:

**REG_BUFFER de tipo REGISTRO**

La operación de apertura reserva un archivo secuencial en exclusiva para el programa que lo ejecuta. Un archivo secuencial se puede abrir o para hacer lecturas o para hacer escrituras, lo que se indicará en el momento de la apertura. Si la apertura tiene éxito, se obtendrá un nombre lógico para el fichero, con el que se trabajará en el resto del programa para hacer cualquier referencia al fichero. Si la apertura no tiene éxito, no se podrá trabajar sobre el fichero. Para simplificar los algoritmos tratados en el tema, no se incluye una comprobación sobre el éxito de la operación de apertura, considerando que siempre ha tenido éxito. Esta comprobación debe tener en cuenta el retorno obtenido en la operación de apertura.

La apertura de un fichero indica el nombre físico del fichero y el **modo** en el que se abre. Existen 3 modos de apertura:

•**lectura**: si queremos leer el contenido del fichero. El puntero de lectura queda sobre el primer registro. El fichero indicado debe existir; de lo contrario se producirá un error en la apertura.

**F ← Abrir NOMBRE_FICHERO para lectura**

•**escritura**: si deseamos escribir en el fichero, eliminando los posibles datos que contenga. El puntero de escritura se coloca al comienzo del archivo. Si el archivo no existe, se crea; si ya existe, borra su contenido, salvo que el sistema disponga de protección para que esto último no ocurra.

**F ← Abrir NOMBRE_FICHERO para escritura**

•**añadir**: si deseamos escribir en el fichero y preservar el posible contenido del mismo, con lo que la escritura comienza a partir del final del fichero.

**F ← Abrir NOMBRE_FICHERO para añadir**

La operación de cierre libera el archivo del programa, quedando a disposición de cualquier programa que lo solicite:

**Cerrar F**

Si el archivo se había abierto para escritura, la operación de cierre coloca una marca especial de ***fin de archivo*** que será detectada en posteriores lecturas.

En un fichero abierto para escritura se puede solicitar la escritura de un nuevo dato, contenido en un registro buffer. El contenido de dicho registro se colocará en el lugar identificado por el puntero de escritura y éste avanza al siguiente registro del archivo:

**Escribir F, REG_BUFFER**

Para realizar una lectura sobre un fichero abierto para lectura, se indica la variable que almacenará el registro leído del fichero:

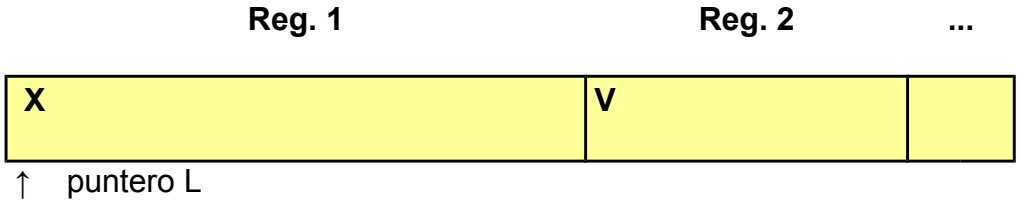
**Leer F, REG_BUFFER**

Si quedan datos por leer, se coloca el contenido del registro apuntado por el puntero de lectura sobre el registro buffer, avanzando dicho puntero al siguiente registro.

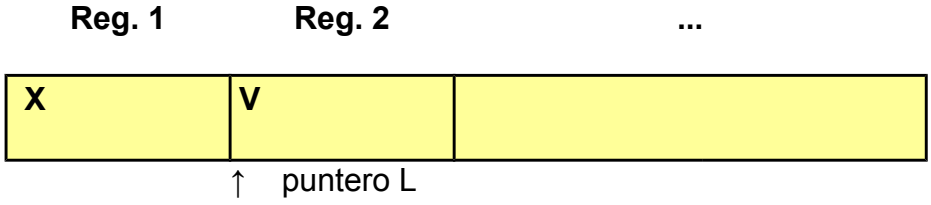
Antes de realizar una lectura es necesario comprobar que no se ha llegado al final del fichero (que quedan registros por leer), para evitar un error. La operación de comprobación de final de archivo es una función booleana que toma el valor **Verdadero** si el puntero de lectura señala la marca especial de fin de archivo, es decir, si no quedan registros por leer, y toma el valor **Falso** en caso contrario:

**FF( F )**

EJEMPLO: primera lectura realizada sobre un fichero

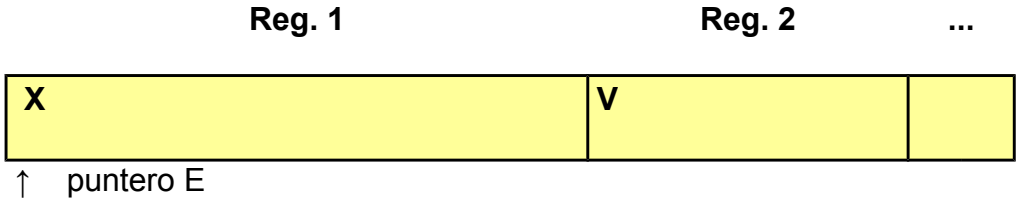



**Leer F, REG_BUFFER**



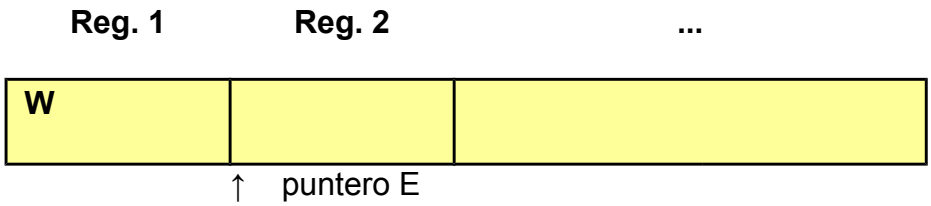

**REG_BUFFER** contiene el valor **X**

EJEMPLO: primera escritura realizada sobre un fichero abierto en modo escritura (nótese que el fichero abierto, de existir previamente, queda sobreescrito)

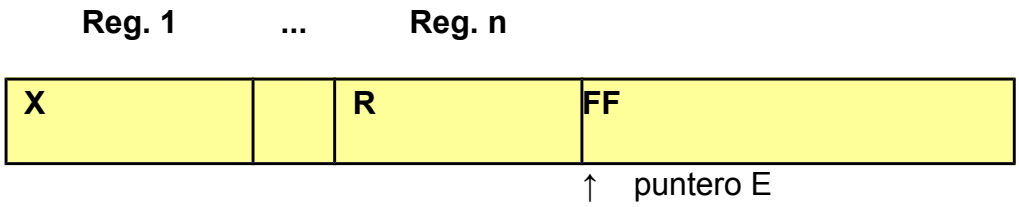



**REG_BUFFER ← W**

**Escribir F, REG_BUFFER**

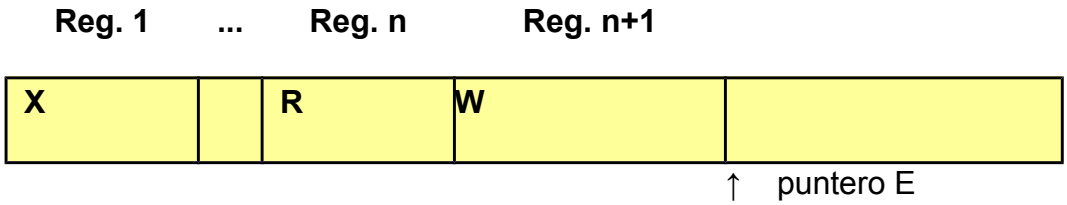


EJEMPLO: primera escritura realizada sobre un fichero abierto en modo añadir

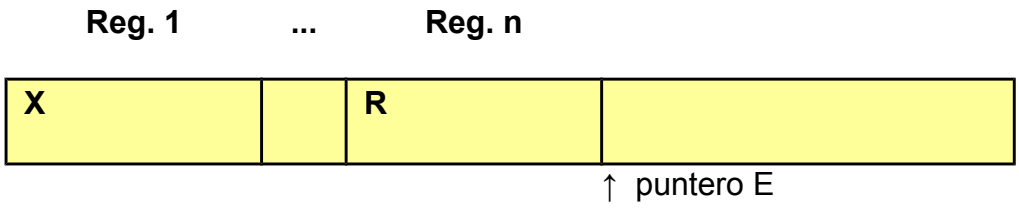



**REG_BUFFER ← W**

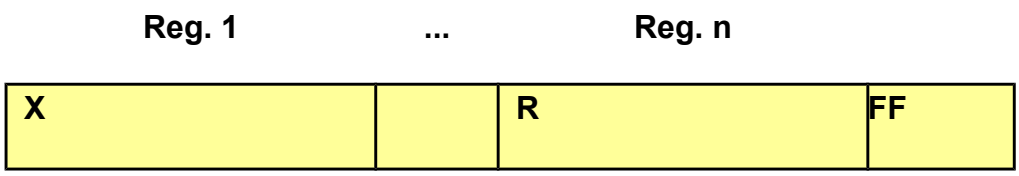
**Escribir F, REG_BUFFER**




EJEMPLO: cierre de un fichero abierto para escritura




**Cerrar F**




## *Operaciones con Ficheros*

Las operaciones que vamos a analizar, relacionadas con las estructuras de datos externas son:

- **Búsqueda:** consiste en intentar localizar cierta información en el fichero.
- **Clasificación:** reubicación de los registros de un fichero, de forma que queden ordenados según cierto criterio.

## *Algoritmos de Clasificación*

Un archivo está ordenado o clasificado de forma ascendente (o descendiente) si tiene todos sus registros en secuencia ascendente (o descendiente) respecto al valor de un campo que denominamos ***clave de ordenación***.

Si el tamaño del archivo a ordenar permite cargarlo en memoria principal, lo más adecuado es aplicarle algún algoritmo de ordenación interna y copiar los datos ordenados

en el archivo original. En caso contrario, será necesario aplicar algún algoritmo de ordenación externa.

Muchos algoritmos de clasificación externa se basan en la realización de sucesivas particiones y mezclas o fusiones del archivo a clasificar. Las particiones generan dos o más archivos a partir de otro; las mezclas combinan archivos ordenados o parcialmente ordenados, obteniendo un archivo que mantiene el orden. Ambos tipos de operaciones implicarán continuos accesos a la memoria secundaria, para leer o escribir registros en los ficheros, así como varios ficheros auxiliares. Cuanto mayor sea el número de accesos a disco, más lento será el método de ordenación.

Los métodos de ordenación que comentaremos trabajan sobre ficheros que contienen registros del tipo REGISTRO, que contiene al menos un campo, denominado CLAVE, que se utilizará para ordenar el fichero de forma ascendente.

```
REGISTRO es registro compuesto por
        CLAVE de tipo T
        ** otros campos de algún tipo

fin-registro
```

### Clasificación por Mezcla Directa

La ordenación por mezcla directa realiza sucesivas operaciones de partición y mezcla, que producen secuencias ordenadas de longitud cada vez mayor. La primera partición se hace en secuencias de longitud 1, y la fusión correspondiente produce sobre el archivo inicial secuencias ordenadas de longitud 2. En cada nueva partición y fusión se duplica la longitud de las secuencias ordenadas obtenidas. El proceso termina cuando la longitud de la secuencia ordenada es mayor o igual que la longitud del archivo a ordenar.

```
Módulo Clasificacion_mezcla_directa
DATOS
  PARÁMETROS
    Recibe NOMBRE_F cadena     ** fichero a ordenar
    Recibe NOMBRE_F1 cadena    ** primer fichero auxiliar
    Recibe NOMBRE_F2 cadena    ** segundo fichero auxiliar
  VARIABLES
    NUM_ELEM entero          ** num. de registros a ordenar
    TAM_SEC entero           ** tamaño de la secuencia actual
INICIO
  TAM_SEC ← 1
  Repetir
      Separar (NOMBRE_F, NOMBRE_F1, NOMBRE_F2,
```

```
                TAM_SEC, NUM_ELEM)
        Mezcla_directa (NOMBRE_F, NOMBRE_F1, NOMBRE_F2,
                        TAM_SEC, NUM_ELEM)

        TAM_SEC ← 2 * TAM_SEC
    Hasta que (TAM_SEC  ≥ NUM_ELEM)
FIN
```

**El siguiente módulo realiza la partición, copiando de forma alternativa en cada fichero auxiliar una secuencia de TAM_SEC registros consecutivos procedentes del fichero a ordenar. La última secuencia copiada puede tener menos de TAM_SEC registros:**

```
Módulo Separar
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena      ** fichero a ordenar
        Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
        Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
        Recibe TAM_SEC entero       ** tamaño de la secuencia
        Transforma ELEM_F entero    ** registros del fichero origen
    VARIABLES
        REG de tipo REGISTRO
        F, F1, F2 archivos de REGISTRO
        N entera      ** número de reg. escritos en un destino
INICIO
    F1 ← Abrir NOMBRE_F1 para Escritura
    F2 ← Abrir NOMBRE_F2 para Escritura
    F ← Abrir NOMBRE_F para Lectura

    ELEM_F ← 0

    Leer F, REG
    ELEM_F ← ELEM _F + 1

    Mientras (FF(F) = Falso) hacer
        N ← 0
        Mientras ( (FF(F) = Falso)   Y   (N < TAM_SEC) ) hacer
            Escribir F1, REG
            N ← N + 1

            Leer F, REG
            ELEM_F ← ELEM _F + 1
        fin-mientras
```

```
            N ← 0
            Mientras ( (FF(F) = Falso)   Y   (N < TAM_SEC) ) hacer
                Escribir F2, REG
                N ← N + 1
                Leer F, REG
                ELEM_F ← ELEM_F + 1
            fin-mientras
        fin-mientras

        Cerrar F
        Cerrar F1
        Cerrar F2
    FIN
```

**El siguiente módulo realiza la mezcla de los dos ficheros auxiliares, copiando los registros sobre el fichero a ordenar. Dicho fichero se construye mezclando secuencias ordenadas de tamaño TAM_SEC de los dos ficheros auxiliares. El parámetro ELEM_F indica el número de registros que contenía el fichero a ordenar antes de partirlo en dos. Por tanto, el fichero resultante de la mezcla debe tener tambiŕn ELEM_F registros. Cuando ya no queden registros por mezclar procedentes de uno de los dos ficheros auxiliares, se copiarán en el destino todos los registros restantes del otro fichero auxiliar (copia de cabos).**

```
    Módulo Mezcla_directa
    DATOS
        PARÁMETROS
            Recibe NOMBRE_F cadena      ** fichero a ordenar
            Recibe NOMBRE_F1 cadena     ** primer fichero auxiliar
            Recibe NOMBRE_F2 cadena     ** segundo fichero auxiliar
            Recibe TAM_SEC entera       ** tamaño de la secuencia
            Recibe ELEM_F entera         ** registros del fichero a
    ordenar
        VARIABLES
            REG1, REG2 de tipo REGISTRO
            RESTA_F1 entero    ** reg. de la sec. del fich. 1 por mezclar
            RESTA_F2 entero    ** reg. de la sec. del fich. 2 por mezclar
            RESTA enteros       ** reg. totales por mezclar
            F, F1, F2 archivos de REGISTRO
    INICIO
        F1 ← Abrir NOMBRE_F1 para Lectura
        F2 ← Abrir NOMBRE_F2 para Lectura
        F ← Abrir NOMBRE_F para Escritura

        RESTA ← ELEM_F     ** registros totales que faltan por mezclar
```

```
Repetir
    ** número de registros a mezclar del primer origen
    Si (RESTA ≥ TAM_SEC) entonces
        RESTA_F1 ← TAM_SEC
    sino
        RESTA_F1 ← RESTA
    fin-si

    RESTA ← RESTA - RESTA_F1

    ** número de registros a mezclar del segundo origen
    Si (RESTA ≥ TAM_SECUENCIA) entonces
        RESTA_F2 ← TAM_SEC
    sino
        RESTA_F2 ← RESTA
    fin-si

    RESTA ← RESTA - RESTA_F2

    Leer F1, REG1
    Leer F2, REG2

    Mientras ( (RESTA_F1 ≠ 0) Y (RESTA_F2 ≠ 0) ) hacer

        Si (REG1.CLAVE < REG2.CLAVE) entonces
            Escribir F, REG1
            RESTA_F1 ← RESTA_F1 - 1

            Si (RESTA_F1 ≠ 0) entonces
                Leer F1, REG1
            fin-si
        sino
            Escribir F, REG2
            RESTA_F2 ← RESTA_F2 - 1

            Si (RESTA_F2 ≠ 0) entonces
                Leer F2, REG2
            fin-si
        fin-si
    fin-mientras

    ** copia de cabos
    Mientras (RESTA_F2 ≠ 0) hacer
        Escribir F, REG2
        RESTA_F2 ← RESTA_F2 - 1
```

```
                Si (RESTA_F2 ≠ 0) entonces
                    Leer F2, REG2
                fin-si
            fin-mientras

            Mientras (RESTA_F1 ≠ 0) hacer
                Escribir F, REG1
                RESTA_F1 ← RESTA_F1 - 1

                Si (RESTA_F1 ≠ 0) entonces
                    Leer F1, REG1
                fin-si
            fin-mientras

        Hasta que (RESTA = 0)

        Cerrar F
        Cerrar F1
        Cerrar F2
FIN
```

**Ejemplo**

**Se ordena por mezcla directa el fichero F, cuyos valores del campo clave son los que figuran a continuación.**

| F | 15 | 18 | 7 | 75 | 14 | 13 | 43 | 40 | 51 | 93 | 75 | 26 | 64 | 27 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**Entenderemos que los valores que aparecen representan el contenido de los campos claves y "no son explícitamente su contenido". Por lo tanto la representación numérica que utilizamos pudiera ser la representación de un DNI, el conjunto de Apellidos+Nombre, num. de la Seguridad Social, num. de matrícula... o cualquier otro conjunto de datos que pudiera ser un campo clave. Así mismo, al ordenar la representación de los campos claves se entiende que estamos ordenando, a la vez, el resto del registro al que representan, por lo que cuando en la representación movamos el campo clave, realmente estaremos moviendo todo el registro.**

**Se utilizan dos archivos auxiliares, F1 y F2, con la misma estructura que F, para realizar las sucesivas particiones y fusiones. Para mayor claridad se ha empleado un color para el contenido de cada uno de los archivos.**

| F | 15 | 18 | 7 | 75 | 14 | 13 | 43 | 40 | 51 | 93 | 75 | 26 | 64 | 27 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**Se realiza una partición en secuencias de longitud 1:**

F1

| 15 | 7 | 14 | 43 | 51 | 75 | 64 | 13 |
|---|---|---|---|---|---|---|---|

F2

| 18 | 75 | 13 | 40 | 93 | 26 | 27 |
|---|---|---|---|---|---|---|

**Se realiza una fusión de secuencias de longitud 1 obteniendo secuencias ordenadas de longitud 2:**

F

| 15 | 18 | 7 | 75 | 13 | 14 | 40 | 43 | 51 | 93 | 26 | 75 | 27 | 64 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**Partición en secuencias de longitud 2: (Coloreamos nuevamente las claves anteriores para un mejor entendimiento)**

F

| 15 | 18 | 7 | 75 | 13 | 14 | 40 | 43 | 51 | 93 | 26 | 75 | 27 | 64 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

F1

| 15 | 18 | 13 | 14 | 51 | 93 | 27 | 64 |
|---|---|---|---|---|---|---|---|

F2

| 7 | 75 | 40 | 43 | 26 | 75 | 13 |
|---|---|---|---|---|---|---|

**Fusión de secuencias de longitud 2. Escribimos, de forma ordenada, los cuatro valores tomados (dos de cada fichero):**

F

| 7 | 15 | 18 | 75 | 13 | 14 | 40 | 43 | 26 | 51 | 75 | 93 | 13 | 27 | 64 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

**Partición en secuencias de longitud 4. (Coloreamos nuevamente las claves anteriores para un mejor entendimiento)**

F

| 7 | 15 | 18 | 75 | 13 | 14 | 40 | 43 | 26 | 51 | 75 | 93 | 13 | 27 | 64 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

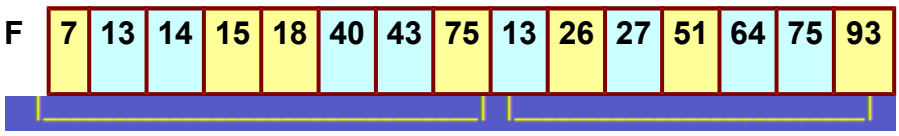
F1

| 7 | 15 | 18 | 75 | 26 | 51 | 75 | 93 |
|---|---|---|---|---|---|---|---|

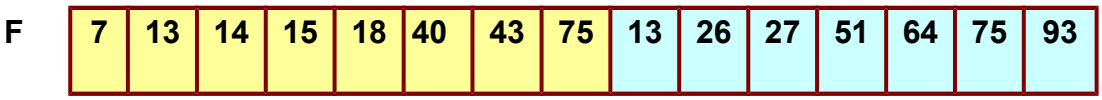
F2

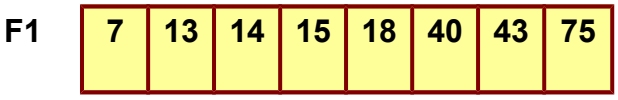
| 13 | 14 | 40 | 43 | 13 | 27 | 64 |
|---|---|---|---|---|---|---|

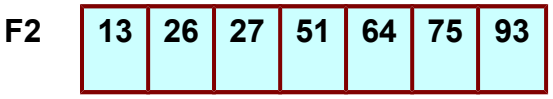
**Fusión de secuencias de longitud 4. Escribimos, de forma ordenada, los ocho valores tomados (cuatro de cada fichero):**



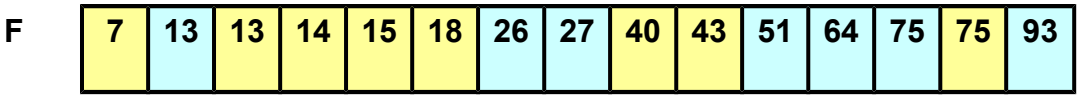

**Partición en secuencias de longitud 8. (Coloreamos nuevamente):**




**Fusión de secuencias de longitud 8:**




### Clasificación por Mezcla Natural

El método de ordenación por mezcla natural, al igual que el de clasificación por mezcla directa, realiza sucesivas particiones y mezclas, pero intenta aprovechar el orden existente en los datos, considerando secuencias de longitud variable.

En cada iteración, en primer lugar se separan los registros del fichero origen en otros dos ficheros, copiando en cada uno las secuencias de registros consecutivos que ya están ordenados entre sí en el origen. A continuación, se mezclan los dos ficheros auxiliares, obteniendo de cada dos tramos ordenados otro ordenado. El proceso de ordenación se repite hasta que se obtiene un solo tramo.

```
Módulo Clasificacion_mezcla_natural
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena    ** fichero a ordenar
        Recibe NOMBRE_F1 cadena   ** primer fichero auxiliar
        Recibe NOMBRE_F2 cadena   ** segundo fichero auxiliar
    VARIABLES
        L entera      ** num. de subsecuencias ordenadas mezcladas
INICIO
    Repetir
        Separar (NOMBRE_F, NOMBRE_F1, NOMBRE_F2)
        Mezcla_natural (NOMBRE_F, NOMBRE_F1, NOMBRE_F2, L)
    Hasta que (L = 1)
FIN
```

El módulo Separar copia en el primer fichero auxiliar la primera secuencia ordenada que encuentre en el fichero origen; a continuación, copia la segunda secuencia ordenada en el segundo fichero auxiliar. El proceso continúa, copiando alternativamente en cada uno de los dos ficheros auxiliares, hasta que se hayan procesado todos los registros del fichero a ordenar:

```
Módulo Separar
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena     ** fichero que se va a partir
        Recibe NOMBRE_F1 cadena    ** primer fichero auxiliar
        Recibe NOMBRE_F2 cadena    ** segundo fichero auxiliar
    VARIABLES
        REG de tipo REGISTRO
        F, F1, F2 archivos de REGISTRO
INICIO
    F1 ← Abrir NOMBRE_F1 para escritura
    F2 ← Abrir NOMBRE_F2 para escritura
    F ← Abrir NOMBRE_F para lectura

    Leer F, REG

    Repetir
        Copiar_secuencia (F, F1, REG)
        Si ( FF(F) = Falso) entonces
            Copiar_secuencia  (F, F2, REG)
        fin-si
    Hasta que (FF(F) = Verdadero)

    Cerrar F
    Cerrar F1
    Cerrar F2
FIN
```

El módulo Copiar_secuencia copia en un fichero destino la siguiente secuencia ordenada que se encuentre en el origen. El proceso concluye cuando se encuentra el primer valor que no está ordenado respecto al último copiado en el fichero destino (es decir, cuando el valor leído del fichero origen sea menor que el último escrito en el destino), o cuando ya no queden registros por procesar en el fichero origen, lo que primero ocurra.

```
Módulo Copiar_secuencia
DATOS
    PARÁMETROS
        Recibe F_O archivo de REGISTRO        ** fichero origen
        Transforma F_D archivo de REGISTRO  ** fichero destino
        Transforma REG de tipo REGISTRO       ** reg. leído del fichero
```

```
origen
    VARIABLES
        PARAR lógica
INICIO
    PARAR ← Falso

    Repetir
        Escribir F_D, REG
        Leer_y_chequear (F_O, REG, PARAR)
    Hasta (PARAR = Verdadero)
FIN
```

El módulo Leer_y_chequear comprueba si se ha llegado al final del fichero indicado, o si se ha llegado al final de la secuencia ordenada. En cualquiera de los dos casos, pone a Verdadero el último parámetro pasado en la llamada:

```
Módulo Leer_y_chequear
DATOS
    PARÁMETROS
        Recibe FICH archivo de REGISTRO  ** fichero abierto para
lectura
        Transforma REG de tipo REGISTRO  ** reg. leído del fichero
origen
        Transforma PARAR lógica
    VARIABLES
        REG_SIG de tipo REGISTRO
INICIO
    Si  (FF(FICH) = Falso) entonces
         Leer FICH, REG_SIG

         Si (REG.CLAVE > REG_SIG.CLAVE) entonces
            PARAR ← Verdadero
        fin-si

        REG ← REG_SIG
    sino
       PARAR ← Verdadero
    fin-si
FIN
```

El módulo Mezcla_natural obtiene un nuevo fichero mezclando el contenido de los dos ficheros auxiliares. La mezcla se realiza sobre cada par de secuencias ordenadas existentes en los ficheros auxiliares, que pueden ser de longitudes diferentes. Si sólo se ha mezclado una secuencia procedente de cada origen, el fichero obtenido ya estará ordenado:

```
Módulo Mezcla_natural
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena    ** fichero resultado
        Recibe NOMBRE_F1 cadena  ** primer fichero auxiliar
        Recibe NOMBRE_F2 cadena  ** segundo fichero auxiliar
        Transforma L entera             ** num. de secuenc. mezcladas
    VARIABLES
        R1, R2 de tipo REGISTRO
        FIN_ORDEN booleana
        F, F1, F2 archivo de REGISTRO
INICIO
    F1 ← Abrir NOMBRE_F1 para lectura
    F2 ← Abrir NOMBRE_F2 para lectura
    F ← Abrir NOMBRE_F para escritura

    L ← 0

    Leer F1, R1
    Leer F2, R2

    Repetir
      FIN_ORDEN ← Falso

      Mientras (FIN_ORDEN = Falso) hacer
        Si (R1.CLAVE < R2.CLAVE) entonces
            Escribir F, R1
            Leer_y_chequear(F1, R1, FIN_ORDEN)

            Si (FIN_ORDEN = Verdadero) entonces
                Copiar_secuencia(F2, F, R2)
            fin-si
        sino
            Escribir F, R2
            Leer_y_chequear (F2, R2, FIN_ORDEN)

            Si (FIN_ORDEN = Verdadero) entonces
                Copiar_secuencia(F1, F, R1)
            fin-si
        fin-si
      fin-mientras

      L ← L + 1
    Hasta que ( (FF(F1) = Verdadero)  O  (FF(F2) = Verdadero))
```

```
        ** copias de colas
        Mientras (FF(F1) = Falso) hacer
                Copiar_secuencia(F1, F, R1)
                L ← L+1
        fin-mientras

        Mientras (FF(F2) = Falso) hacer
                Copiar_secuencia(F2, F, R2)
                L ← L+1
        fin-mientras
    FIN
```

**Ejemplo**

Se ordena por mezcla natural el fichero F, cuyos valores del campo clave son los que figuran a continuación.

| F | 15 | 18 | 7 | 75 | 14 | 13 | 43 | 40 | 51 | 93 | 75 | 26 | 64 | 27 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

Entenderemos que los valores que aparecen representan el contenido de los campos claves y "no son explícitamente su contenido". Por lo tanto la representación numérica que utilizamos pudiera ser la representación de un DNI, el conjunto de Apellidos+Nombre, num. de la Seguridad Social, num. de matrícula... o cualquier otro conjunto de datos que pudiera ser un campo clave. Así mismo, al ordenar la representación de los campos claves se entiende que estamos ordenando, a la vez, el resto del registro al que representan, por lo que cuando en la representación movamos el campo clave, realmente estaremos moviendo todo el registro.

Se utilizan dos archivos auxiliares, F1 y F2, con la misma estructura que F, para realizar las sucesivas particiones y fusiones. Para mayor claridad se ha empleado un color para el contenido de cada uno de los archivos.

| F | 15 | 18 | 7 | 75 | 14 | 13 | 43 | 40 | 51 | 93 | 75 | 26 | 64 | 27 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

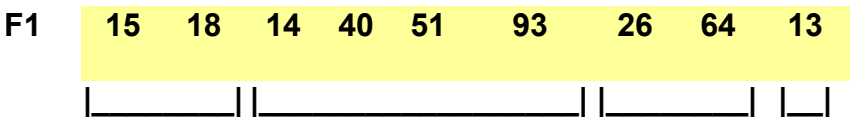
Se realiza la primera partición. Se incluye una línea que indica cada secuencia consecutiva copiada desde el fichero origen:

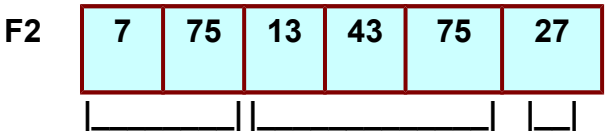
| F1 | 15 | 18 | 14 | 40 | 51 | 93 | 26 | 64 | 13 |
|---|---|---|---|---|---|---|---|---|---|
| | \|_____\| | \|__\| | \|_____\| | | \|_____\| | \|_____\| | \|__\| | |

| F2 | 7 | 75 | 13 | 43 | 75 | 27 |
|---|---|---|---|---|---|---|
| | \|_____\| | \|_____\| | \|__\| | | \|__\| | |

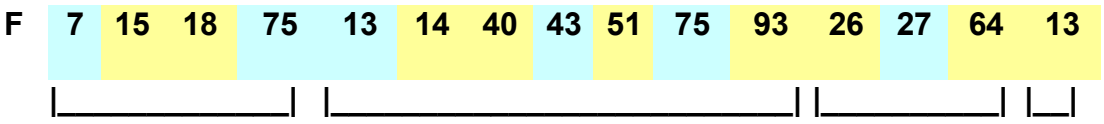
Se realiza la primera fusión, tomando subsecuencias ordenadas de los ficheros auxiliares. A continuación y marcadas con una línea amarilla, se pueden apreciar las subsecuencias ordenadas que se mezclarán en cada uno de los pasos. (En este caso las subsecuencias a tomar en la fusión no coinciden con

las de la partición, al generarse nuevas subsecuencias ya ordenadas).

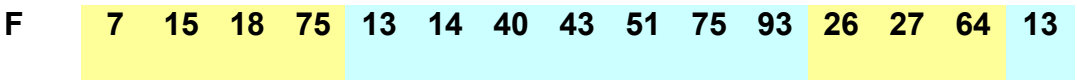


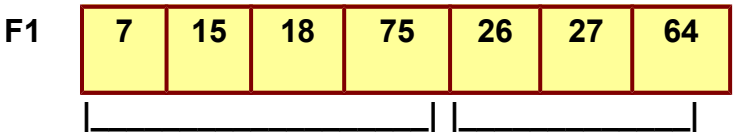

Se marcan las subsecuencias ordenadas obtenidas al tomar una subsecuencia de cada archivo:

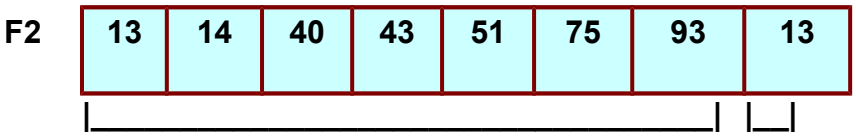



Coloreamos, de nuevo, las subsecuencias ordenadas que se han creado en el archivo a ordenar:

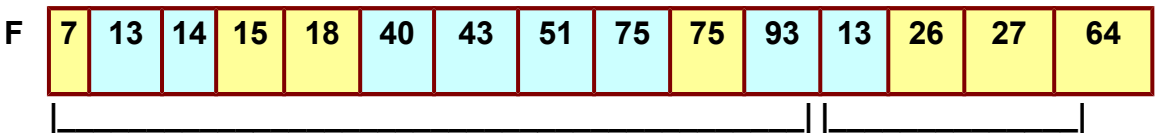



Se realiza una segunda partición. Se incluye un marco que indica cada secuencia consecutiva copiada desde el fichero origen

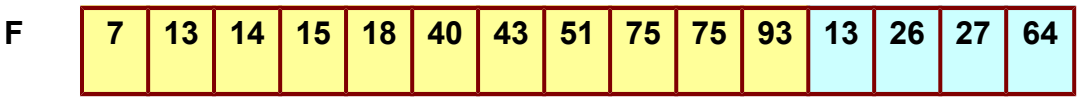


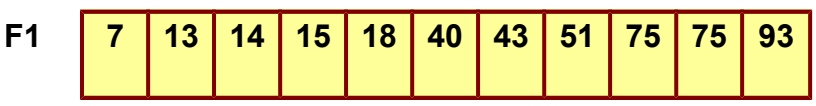

Se realiza la segunda fusión tomando una subsecuencia ordenada de cada archivo (en este caso coinciden con las de la partición):




Coloreamos, de nuevo, las subsecuencias ordenadas que se han creado en el archivo a ordenar:




La tercera partición copia una sola subsecuencia ordenada en cada fichero auxiliar:



F2

| 13 | 26 | 27 | 64 |
|---|---|---|---|

Tras la tercera fusión, el fichero queda ordenado

F

| 7 | 13 | 13 | 14 | 15 | 18 | 26 | 27 | 40 | 43 | 51 | 64 | 75 | 75 | 93 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

### Clasificación de Raíz

Esta técnica de ordenación se puede utilizar para tratar ficheros con *claves numéricas enteras*. La ordenación se realiza aplicando sucesivas iteraciones, en las que primero se realiza una partición y a continuación una concatenación. En la primera etapa se distribuyen los registros en archivos auxiliares en función de la cifra de la clave que corresponda con la iteración que se está realizando. En la segunda etapa se concatenan los 10 archivos sobre el archivo original. Los dígitos de la clave se toman de derecha a izquierda (empezando por las unidades), completándose la ordenación en tantas iteraciones como dígitos tenga la clave que incluya más cifras.

```
Módulo Clasificacion_de_raiz
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena                    ** fichero original
        Recibe NOMBRE_F_AUX(10) de cadenas  ** ficheros auxiliares
    VARIABLES
        SEGUIR lógica          ** seguir/no seguir con las iteraciones
        ITERACION entera      ** contador de iteraciones
INICIO
    SEGUIR ← Verdadero   ** seguir iterando
    ITERACION ← 1

    Repetir
        Separar (NOMBRE_F, NOMBRE_F_AUX, ITERACION, SEGUIR)
        Concatenar (NOMBRE_F, NOMBRE_F_AUX)
         ITERACION ← ITERACION +1
    Hasta que (SEGUIR = Falso)
FIN
```

El módulo Separar realiza una partición de un fichero origen en 10 ficheros destino. La partición se realiza copiando cada registro del fichero origen en el destino que le corresponda, en función de la cifra de la clave coincidente con la posición dada por ITERACION: si la cifra es 0, el registro se lleva al primer fichero destino, si es 1, se lleva al segundo,..., si es 8 se lleva al penúltimo y si es 9 se lleva al último. El parámetro SEGUIR tomará valor Falso si no deben proseguir las iteraciones (si se han procesado todas las cifras de la clave más larga).

```
Módulo Separar
DATOS
    PARÁMETROS
```

```
        Recibe NOMBRE_ORIGEN cadena              ** fichero a ordenar
        Recibe NOMBRE_DESTINO(10) de cadenas  ** ficheros auxiliares
        Recibe ITERACION entera                  ** iteración actual
        Transforma SEGUIR lógica           ** seguir/no seguir ordenando
    VARIABLES
        REG de tipo REGISTRO                ** registro de un fichero
        CIFRA entero                        ** una cifra de la clave
        F_O fichero de REGISTRO             ** fichero origen
        F_D(10) de ficheros de REGISTRO  ** ficheros destino
        I entera                            ** contador
INICIO
    SEGUIR ← Falso

    Para I desde 1 hasta 10
        F_D(I) ← Abrir NOMBRE_DESTINO(I) para escritura
    fin-para

    F_O ← Abrir NOMBRE_ORIGEN para lectura

    Mientras ( FF(F_O) = Falso) hacer
        Leer REG, F_O
        CIFRA ← Obtener_cifra(REG.CLAVE, ITERACION, SEGUIR)
        Escribir REG, F_D(CIFRA+1)
    fin-mientras

    Para I desde 1 hasta 10
        Cerrar F_D(I)
    fin-para

    Cerrar F_O
FIN
```

El módulo Obtener_cifra obtiene la cifra de la clave que corresponde a la iteración actual del algoritmo. Además, indica si el algoritmo de ordenación debe continuar con las iteraciones (si la clave tiene al menos otra cifra a la izquierda de la considerada):

```
MÓDULO Obtener_cifra  devuelve entero
DATOS
    PARÁMETROS
        Recibe CLAVE entero          ** clave de un registro
        Recibe ITERACION entero      ** iteración actual
        Transforma SEGUIR lógica     ** seguir/no seguir iterando
    VARIABLES
        I entero             ** contador
        CIFRA entero         ** una cifra de la clave
```

```
        N entero                ** copia de la clave
INICIO
    N ← CLAVE

    Para I desde 1 hasta ITERACION
        CIFRA ← N MOD 10        ** resto de la división
        N ← N DIV 10            ** división entera
    fin-para

    Si (N ≠ 0) entonces
        SEGUIR ← Verdadero
    fin-si

    DEVOLVER CIFRA
FIN
```

El módulo Concatenar copia el contenido de los 10 ficheros auxiliares sobre el fichero original, empezando por el fichero 1 y terminando por el 10:

```
Módulo Concatenar
DATOS
    PARÁMETROS
        Recibe NOMBRE_DESTINO cadena          ** fichero a ordenar
        Recibe NOMBRE_ORIGEN(10) de cadenas   ** ficheros auxiliares
    VARIABLES
        REG de tipo REGISTRO
        F_D fichero de REGISTRO
        F_O(10) de ficheros de REGISTRO
        I entera
INICIO
    Para I desde 1 hasta 10
        F_O(I) ← Abrir NOMBRE_ORIGEN(I) para lectura
    fin-para

    F_D ← Abrir NOMBRE_DESTINO para escritura

    Para I desde 1 hasta 10
        Mientras ( FF(F_O(I)) = Falso ) hacer
            Leer REG, F_O(I)
            Escribir REG, F_D
        fin-mientras
    fin-para

    Para I desde 1 hasta 10
        Cerrar F_O (I)
    fin-para
```

**Cerrar F_D**
**FIN**

## Ejemplo

Se ordena el fichero F, cuyos valores del campo clave son los que figuran a continuación:

| F | 15 | 18 | 7 | 75 | 14 | 13 | 43 | 40 | 51 | 93 | 75 | 26 | 64 | 27 | 13 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

siendo F0, F1,... F9 10 archivos auxiliares.

Entenderemos que los valores que aparecen representan el contenido de los campos claves y "no son explícitamente su contenido". Por lo tanto la representación numérica que utilizamos pudiera ser la representación de un DNI, número de la Seguridad Social, número de matrícula... o cualquier otro conjunto de datos numéricos enteros que pudieran ser un campo clave. Así mismo, al ordenar la representación de los campos claves se entiende que estamos ordenando, a la vez, el resto del registro al que representan, por lo que cuando en la representación movamos el campo clave, realmente estaremos moviendo todo el registro.

Como las claves de F contienen un máximo de 2 cifras, el proceso necesitará dos pasadas, consistentes cada una de ellas en una partición y una concatenación posterior. Para la clave con valor 7, que sólo incluye una cifra, se rellena con un cero a la izquierda (07).

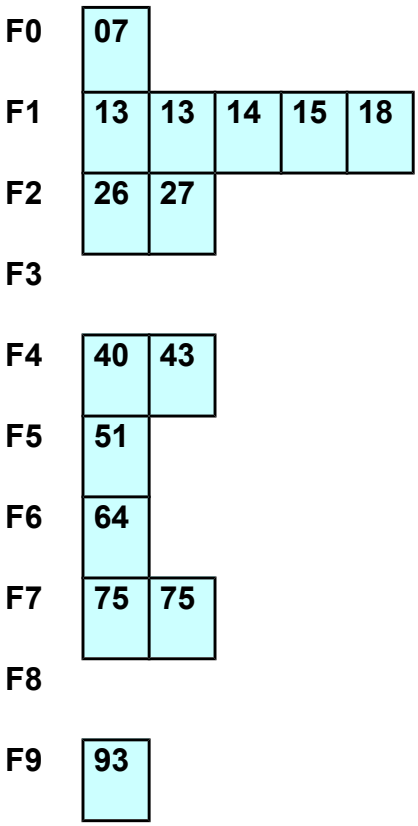
La primera partición tiene en cuenta la cifra de la clave correspondiente a las unidades para determinar el fichero destino en que se copian los registros de F:

| | | | | |
|---|---|---|---|---|
| F0 | 40 | | | |
| F1 | 51 | | | |
| F2 | | | | |
| F3 | 13 | 43 | 93 | 13 |
| F4 | 14 | 64 | | |
| F5 | 15 | 75 | 75 | |
| F6 | 26 | | | |
| F7 | 07 | 27 | | |
| F8 | 18 | | | |
| F9 | | | | |

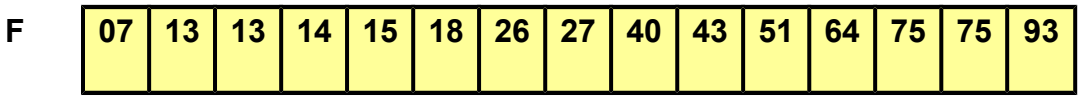
Se realiza la primera concatenación:

| F | 40 | 51 | 13 | 43 | 93 | 13 | 14 | 64 | 15 | 75 | 75 | 26 | 07 | 27 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

La segunda partición tiene en cuenta la cifra correspondiente a las decenas:




Se realiza la segunda concatenación, obteniendo el fichero ordenado:




## *Algoritmos de Búsqueda*

Esta operación consiste en localizar el o los registros del fichero que cumplan cierta condición. La búsqueda se suele realizar de forma que uno o varios campos del registro contengan unos valores predeterminados. Denominaremos al campo o campos por los que se realiza la búsqueda ***clave de búsqueda***. Si la búsqueda tiene éxito, se recupera el registro que cumple el criterio de búsqueda (o todos los registros que lo cumplan); si fracasa, se debe informar al usuario o al módulo que solicitó la operación de búsqueda.

Para hacer la descripción más general, consideraremos búsquedas que se realizan por un solo campo, pudiendo adaptarse fácilmente los algoritmos propuestos para hacer búsquedas por varios campos. Consideraremos ficheros formados por registros de tipo REGISTRO, que incluyen al menos un campo, que denominaremos CLAVE, y que se toma como clave de búsqueda.

```
REGISTRO es registro compuesto por
        CLAVE de tipo T
        ** otros campos de algún tipo

fin-registro
```

Si las búsquedas se realizan sobre archivos ordenados por el campo tomado como clave de búsqueda, se puede forzar el final de la búsqueda cuando se llegue a un registro cuya clave contenga un valor superior al buscado, en el caso de ordenación ascendente, o inferior al buscado, en caso de ordenación descendente.

Los algoritmos de búsqueda en archivos desordenados también se pueden aplicar a archivos ordenados por la clave de búsqueda.

Tipos de búsqueda:

- ARCHIVOS ORDENADOS
  - BUSCAR UNA COINCIDENCIA
  - BUSCAR TODAS LAS COINCIDENCIAS
- ARCHIVOS QUE PUEDEN ESTAR DESORDENADOS
  - BUSCAR UNA COINCIDENCIA
  - BUSCAR TODAS LAS COINCIDENCIAS

### Algoritmos de Búsqueda en ficheros Desordenados

En los siguientes algoritmo suponemos que el fichero a tratar no está necesariamente ordenado de forma ascendente por la clave de búsqueda.

#### Búsqueda de una coincidencia en ficheros desordenados

Se recorre el archivo de forma secuencial desde el primer registro. La búsqueda concluye cuando se encuentre el registro coincidente (con lo que la operación ha tenido éxito) o cuando se llegue al final del fichero (con lo que la búsqueda habrá fracasado), lo que primero ocurra. Si el fichero incluye varios registros que contienen la clave de búsqueda, se encontrará el más próximo al comienzo del fichero.

```
Módulo Buscar_uno  devuelve lógico
DATOS
        PARÁMETROS
                Recibe NOMBRE_F cadena     ** fichero a procesar
                Recibe X de tipo T         ** clave que se busca
                Transforma REG_BUSCADO de tipo REGISTRO
        VARIABLES
                ENCONTRADO lógico
                REG de tipo REGISTRO
                F archivo de REGISTRO
INICIO
        F ← Abrir NOMBRE_F para lectura
        ENCONTRADO ← Falso

        Mientras ( (ENCONTRADO = Falso)  Y  (FF(F) = Falso) ) hacer
```

```
            Leer F, REG
            Si (REG.CLAVE = X) entonces
                  REG_BUSCADO ← REG
                  ENCONTRADO ← Verdadero
            fin-si
      fin-mientras

      Cerrar F
      Devolver ENCONTRADO
FIN
```

**Búsqueda de todas las coincidencias en ficheros desordenados**

Se recorre el archivo de forma secuencial desde el primer registro hasta el último. Cada vez que se encuentre un registro con la clave de búsqueda, se le aplicará el tratamiento elegido (representado por la operación**TRATAR**). Se utilizará una variable (que llamaremos ENCONTRADO) que indicará si la búsqueda ha tenido éxito (si es Verdadero), o no lo ha tenido (si es Falso).

```
Módulo Buscar_todos
DATOS
      PARÁMETROS
            Recibe NOMBRE_F cadena    ** fichero a procesar
            Recibe X de tipo T        ** clave que se busca
      VARIABLES
            REG de tipo REGISTRO
            ENCONTRADO lógico
            F archivo de REGISTRO
INICIO
      F ← Abrir NOMBRE_F para lectura
      ENCONTRADO ← Falso      ** aún no se ha encontrado coincidencia

      Mientras ( FF(F) = Falso ) hacer
            Leer F, REG

            Si (REG.CLAVE = X) entonces
                  TRATAR REG
                  ENCONTRADO ← Verdadero  ** éxito en la búsqueda
            fin-si
      fin-mientras

      Si (ENCONTRADO = Falso) entonces
            Escribir "Valor no encontrado"
      fin-si

      Cerrar F
FIN
```

## Algoritmos de Búsqueda en ficheros Ordenados

En los siguientes algoritmo suponemos que el fichero a tratar ya está ordenado de forma ascendente por la clave de búsqueda.

### Búsqueda de una coincidencia en ficheros ordenados

Se recorre el archivo de forma secuencial desde el primer registro. La búsqueda concluye cuando se encuentre el registro coincidente, cuando se lea el primer registro con clave superior a la de búsqueda o cuando se llegue al final del fichero. En el primer caso la búsqueda ha tenido éxito, mientras que en los dos últimos habrá fracasado.

El siguiente módulo recibe como argumentos el fichero sobre el que se realizará la búsqueda, la clave del registro a buscar, y una variable donde se almacenará la información del registro que contiene la clave buscada, en caso de existir:

```
Módulo Buscar_uno_ordenado   devuelve lógico
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena          ** fichero a procesar
        Recibe X de tipo T              ** clave que se busca
        Transforma REG_BUSCADO de tipo REGISTRO  ** reg. coincidente
    VARIABLES
        F archivo de REGISTRO
        ACABAR lógico
        REG de tipo REGISTRO
INICIO
     F <- Abrir NOMBRE_F para lectura
    ACABAR <- Falso

    Mientras ( (ACABAR = Falso)  Y  (FF(F) = Falso) ) hacer
         Leer F, REG
         Si (REG.CLAVE <= X) entonces
              ACABAR <- Verdadero
         fin-si
    fin-mientras

    Si (ACABAR = Verdadero) entonces
         Si (REG.CLAVE = X) entonces
              REG_BUSCADO <- REG
         sino
              ACABAR <- Falso
         fin-si
    fin-si

    Cerrar F
    Devolver ACABAR
FIN
```

### Búsqueda de todas las coincidencias en ficheros ordenados

Se recorre el archivo de forma secuencial desde el primer registro hasta el último. Cada vez que se encuentre un registro con la clave buscada, se le aplicará el tratamiento elegido (representado por la operación **TRATAR**) . Se utilizará una variable (que llamaremos ENCONTRADO) que indicará si la búsqueda ha tenido éxito (si es Verdadero), o no lo ha tenido (si es Falso).

```
Módulo Buscar_todos_ordenado
DATOS
    PARÁMETROS
        Recibe NOMBRE_F cadena    ** fichero a procesar
        Recibe X de tipo T        ** clave buscada
    VARIABLES
        SALIR, EXISTE lógicos
        REG de tipo REGISTRO
        F archivo de REGISTRO
INICIO
    SALIR <-  Falso
    EXISTE <-  Falso

    F <-  Abrir NOMBRE_F para lectura

    Mientras ( (SALIR = Falso)  Y  (FF(F) = Falso) ) hacer
        Leer F, REG

        Si (REG.CLAVE > X) entonces
            SALIR <-  Verdadero
        sino
            Si (REG.CLAVE = X) entonces
                TRATAR REG
                EXISTE <-  Verdadero
            fin-si
        fin-si
    fin-mientras

    Si (EXISTE = Falso) entonces
        Escribir "No existen coincidencias"
    fin-si

    Cerrar F
FIN
```

# 3 ESTRUCTURAS DE DATOS LINEALES

Este tema describe las principales estructuras de datos lineales.

## 3.1 Listas

### *3.1.1 Definición*

Una lista es una secuencia de elementos que pueden ser accedidos, insertados o suprimidos en cualquier posición de la estructura, no existiendo restricción en cuanto al número de elementos de ésta.

Matemáticamente, una lista es una secuencia de cero o más elementos de algún tipo. Se puede representar como a(1), a(2), a(3), ... , a(m), siendo m (m ≥ 0) la **longitud** de la lista. Si m es cero, se dice que la **lista** está **vacía**; si es mayor que 0, se dice que a(1) es el **primer elemento** de la lista y que a(m) es el **último elemento**.

Los elementos de una lista pueden estar ordenados en forma lineal de acuerdo con sus posiciones en la lista. Se dice que a(i) **precede** a a(i+1), para i= 1,2, ..., m-1 y que, por la misma razón, a(i) **sucede** a a(i-1) para i= 2, 3, ..., m. También se dice que el elemento a(i) está en la posición i-ésima de la lista.

Las listas se podrán representar de forma estática (mediante **vectores**) o de forma dinámica (**listas enlazadas**).

### *3.1.2 Operaciones Básicas con Listas*

Las operaciones fundamentales que consideraremos al trabajar con listas son:

•**Crear la lista**: debe ser la primera operación que se realiza sobre la estructura. Una lista se crea vacía.

•**Insertar un elemento**: permite añadir un nuevo elemento a la lista en cualquier posición.

•**Eliminar un elemento**: permite eliminar de la lista cualquier elemento existente.

•**Acceder a un elemento**: se podrá acceder a cualquier elemento de la lista para obtener su valor o modificarlo.

•**Comprobar si la lista está vacía**: permite comprobar si la estructura no contiene ningún elemento. Esta comprobación es necesaria antes de realizar operaciones como la eliminación de un elemento o el recorrido de la estructura, para evitar así errores en el programa al intentar acceder a un elemento inexistente.

•**Buscar**: permite localizar en la lista un elemento que cumpla cierta condición.

•**Ordenar**: permite reorganizar los elementos de una lista, de modo que queden ordenados según el criterio que se elija.

•**Recorrer la lista**: permite visitar todos y cada uno de los elementos de la

estructura, para poder aplicarles algún procesamiento (modificar su contenido, visualizarlos,…).

### *3.1.3 Representación de Listas*

Básicamente podemos distinguir dos tipos de representaciones para las listas: la que usa una estructura estática (un vector), y la que usa memoria dinámica.

En la representación estática los elementos de la lista ocupan posiciones consecutivas de memoria, mientras que en la dinámica los elementos se pueden situar en cualquier posición de memoria.

### *Representación Vectorial de Listas*

Se puede representar una lista mediante un array unidimensional, lo que permite un acceso eficiente a cada uno de los elementos de la estructura.

Se definirá un vector de tamaño N y del tipo de los elementos de la lista, que reserva espacio global para toda la estructura. Cada posición del vector almacenará un elemento de la lista, hasta un máximo de N. En todo momento los elementos estarán almacenados en las posiciones iniciales del vector.

Para determinar la longitud de la lista en cada momento se utiliza una variable entera, **NUM_ELEMENTOS**, que tomará valores entre 0 y N. Cuando la lista está vacía la variable será 0; al añadir un elemento se incrementará su valor en 1, reduciéndose en 1 al eliminar un elemento. Esta variable también determina cuál es la última posición ocupada dentro del vector.

| **a(1)** | **a(2)** | … | **a(m)** | | |
|---|---|---|---|---|---|
| **1** | **2** | … | **m** | … | **N** |

**Esquema de una lista implementada usando un vector**

Aunque una lista es una estructura dinámica que puede crecer sin límite, al utilizar una representación estática su tamaño estará condicionado por la capacidad del vector que la aloja. Esta limitación obliga a incluir entre las operaciones básicas sobre la lista la de comprobación de lista llena.

### *Representación mediante el uso de asignación de Memoria Dinámica*

En este caso la lista estará formada por un conjunto de **nodos**, que almacenarán tanto el dato que forma parte de la lista como información adicional para organizar los nodos dentro de la estructura.

En primer lugar se definirá la estructura de los nodos de la lista y, a continuación, un puntero que dará acceso a la lista. Dicho puntero apuntará a un nodo de la lista, que a su vez apuntará a otro. Cada nodo apuntará a algún otro nodo (en general), definiendo así la estructura enlazada o **lista enlazada**.

Todo nodo de una lista enlazada contendrá como mínimo un campo de tipo puntero y otro

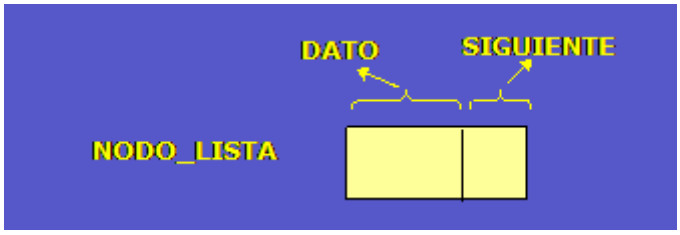
para almacenar la información de la lista propiamente dicha. Para hacer una representación más general, consideraremos un solo campo para representar el resto de los datos del nodo, de algún tipo T, que puede ser simple o compuesto.

Los nodos con los que vamos a trabajar podrán incluir uno o dos campos de enlace. Cuando una lista enlazada está formada por nodos que incluyen un único campo de enlace, hablaremos de ***listas dinámicas simplemente enlazadas*** o con ***enlace simple***; si los nodos contienen 2 enlaces, hablaremos de ***listas dinámicas doblemente enlazadas*** o con ***enlace doble***.

La estructura propuesta para un nodo con enlace simple es la siguiente:

**NODO_LISTA es registro compuesto por**
**DATO de tipo T**
**SIGUIENTE puntero a NODO_LISTA**
**fin-registro**

siendo **DATO** el campo donde se almacena la información de un elemento de la lista y **SIGUIENTE** un puntero que permite conectar un nodo de la lista con otro:

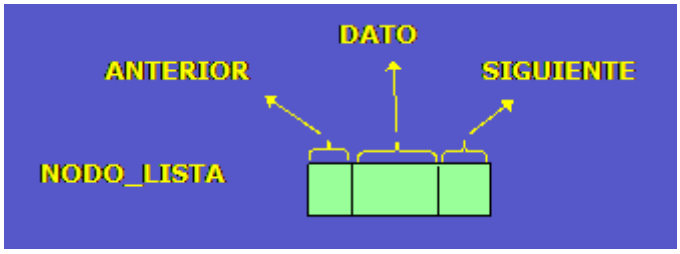



**Esquema de un nodo con un solo enlace**

La estructura propuesta para un nodo con enlace doble es la siguiente:

**NODO_LISTA es registro compuesto por**
**DATO de tipo T**
**SIGUIENTE puntero a NODO_LISTA**
**ANTERIOR puntero a NODO_LISTA**
**fin-registro**

que añade al caso anterior un puntero al predecesor del nodo:




**Esquema de un nodo con dos enlaces**

Podremos trabajar con listas dinámicas tanto **lineales** como **circulares**.

En una ***lista lineal*** el elemento situado en alguno de los extremos de la estructura no estará enlazado con ningún otro nodo. En el caso de una lista lineal con enlace simple **(Figura a)**, el nodo situado en uno de los extremos tendrá el campo **SIGUIENTE** igual a **NULO**; en una lista lineal con enlace doble **(Figura b)**, el nodo

del extremo izquierdo de la representación gráfica tendrá a **NULO** el enlace **ANTERIOR**, mientras que el nodo del extremo opuesto tendrá a **NULO** el enlace **SIGUIENTE**.

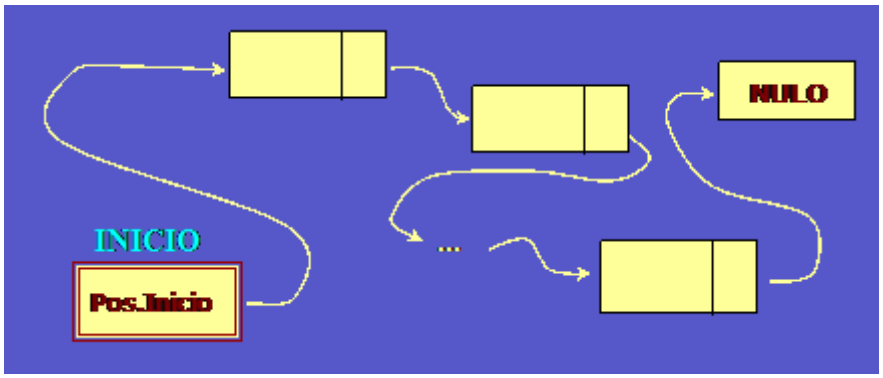



**Figura a: Representación gráfica de una lista lineal con enlace simple**

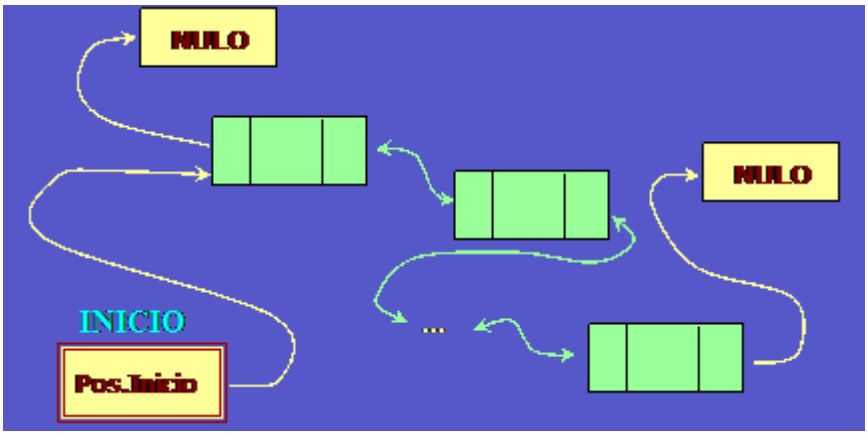



**Figura b: Representación gráfica de una lista lineal con enlace doble**

Las listas en las que cada elemento está enlazado con algún otro se denominan ***listas circulares*** **(Figuras c y d)**. En este caso ninguno de los campos de enlace toma valor nulo. La conexión entre los elementos es tal que se puede recorrer la lista completa pasando desde un nodo cualquiera a otro conectado a él.

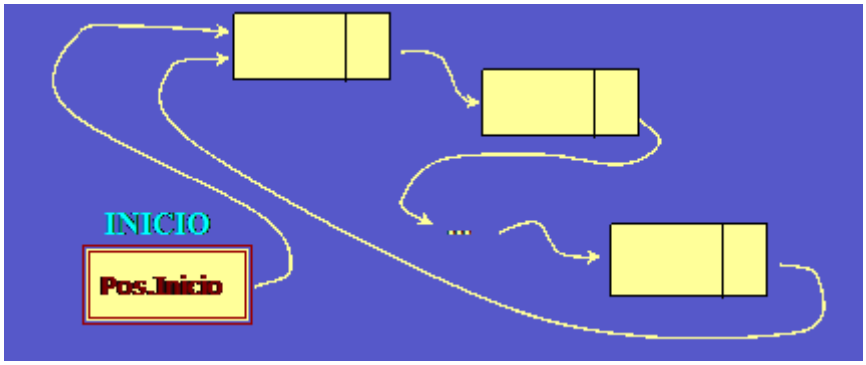



**Figura c: Representación gráfica de una lista circular con enlace simple**

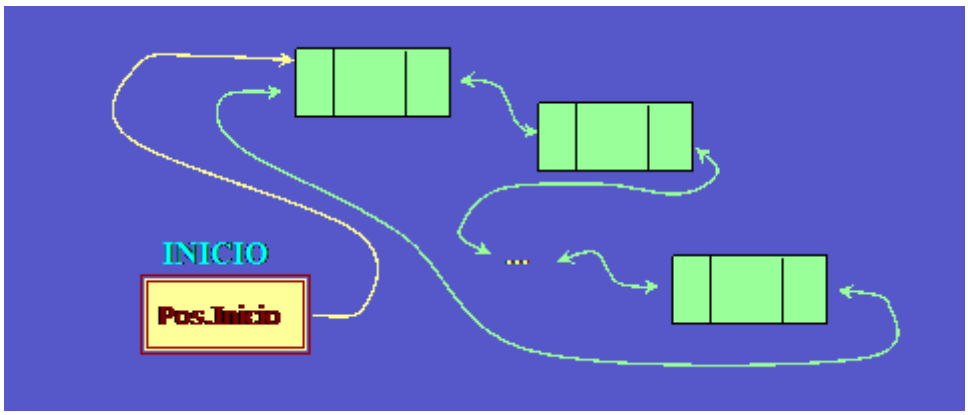



**Figura d: Representación gráfica de una lista circular con enlace doble**

Una vez definida la estructura de los nodos, se debe declarar una variable puntero que permita acceder a la lista. Tras la declaración del puntero, se le asocia el valor NULO, indicando que inicialmente la lista está vacía:

**INICIO puntero a NODO_LISTA**

**INICIO <- NULO**

Cuando la lista no está vacía, **INICIO** apuntará a un nodo de la misma a partir del cual se pueda acceder a todos los demás. En una lista lineal, apuntará al elemento situado en uno de los extremos de la estructura. En una lista con enlace simple, el extremo elegido será el que permita acceder a los nodos de la lista mediante el campo de enlace **(Figura a)**. En una lista con enlace doble, se puede elegir cualquiera de los dos extremos **(Figura b)**. En una lista circular, podrá apuntar a cualquiera de los nodos de la misma **(Figuras c y d)**.

En las listas lineales se suele decir que el nodo apuntado por el puntero **INICIO** es el ***primero*** de la estructura, mientras que el situado en el extremo opuesto será el ***último***. En una lista circular no existe primer ni último nodo, aunque a veces se utilizan estos conceptos para dar claridad a las explicaciones. Nosotros entenderemos por primer nodo de una lista circular el referenciado por el puntero que da acceso a la estructura, considerando como último nodo aquel cuyo enlace SIGUIENTE apunta al primer nodo. Hay que tener en cuenta que el hecho de considerar un nodo como primero o último de la lista es independiente del orden temporal en que se han introducido los elementos correspondientes en la estructura.

## *3.1.4 Operaciones con listas*

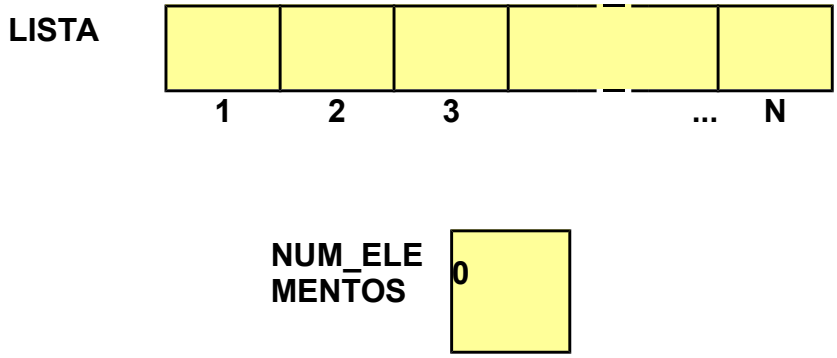
## *Operaciones mediante representación vectorial*

### Creación de la lista mediante representación vectorial

Para crear la lista se declara un vector de cierto tamaño, N, de elementos del tipo que deseemos almacenar en la lista. Además, se declarará un contador, que indicará en cada momento la longitud de la lista y que tomará valores entre 0 (lista vacía) y N (lista llena). Al crear la lista se iniciará dicho contador a 0, lo que representa lista vacía:

**LISTA(N) de tipo T**
**NUM_ELEMENTOS entera**

**NUM_ELEMENTOS <- 0**




**Esquema de una lista vacía implementada en un vector**

### Inserción de un nuevo elemento en la lista

Para insertar un elemento en cierta posición de la lista, se indica el valor del elemento y la posición que debe ocupar dentro de la lista.

Antes de añadir un nuevo elemento, se debe comprobar que la lista no está llena. En caso de estarlo, la operación de inserción se aborta.

Para mantener los valores de la lista entre las posiciones 1 y **NUM_ELEMENTOS** del vector, las posiciones de inserción válidas son todas las ocupadas y la primera libre que sigue a éstas (siempre que la lista no esté llena). Por ello, antes de realizar la inserción se comprobará si la posición indicada es válida. Si el nuevo elemento se debe colocar en una de las posiciones ya ocupadas, se deben desplazar una posición hacia el final del vector los elementos almacenados entre la posición de inserción y la última ocupada, dejando así un hueco para el nuevo elemento.

Para completar el proceso de inserción, se incrementa en 1 la longitud de la lista.

```
MÓDULO Insertar

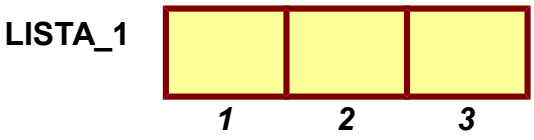
DATOS
    PARÁMETROS
        Recibe ELEMENTO de tipo T       ** valor a insertar
        Recibe POSICION entera          ** posición de inserción
        Transforma LISTA() de tipo T    ** vector lista
        Recibe N entera                 ** tamaño del vector
        Transforma NUM_ELEM entera      ** longitud de la lista
    VARIABLES
        I entera
INICIO
    Si (Lista_llena ( NUM_ELEM, N ) = Verdadero) entonces
        ESCRIBIR "Lista llena."
```

```
        sino
            Si ((POSICION > NUM_ELEM+1)  O  (POSICION < 1))  entonces
                ESCRIBIR "Posición de inserción no válida"
            sino
                Si (POSICION ≤ NUM_ELEM) entonces
                    Para I desde NUM_ELEM hasta POSICION con
incremento -1
                            LISTA(I+1) <- LISTA (I)
                    fin-para
                fin-si

                LISTA(POSICION) <- ELEMENTO
                NUM_ELEM <- NUM_ELEM +1
            fin-si
        fin-si
FIN
```

**Ejemplo de inserción de un nuevo elemento en una lista**

Supongamos una lista de enteros que se almacenará en un vector denominado LISTA_1, con capacidad para 3 elementos (N=3) y que inicialmente está vacía:

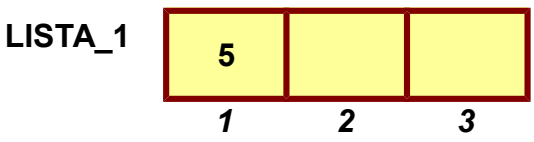



**NUM_ELEMENTOS 0**

Se inserta el primer elemento, con valor 5. Dicho elemento debe ocupar obligatoriamente la posición 1:

**Insertar(5, 1, LISTA_1, N, NUM_ELEMENTOS)**

Como resultado de la operación se almacena el valor indicado en la primera posición de la lista, cuya longitud pasa a ser 1:




**NUM_ELEMENTOS 1**

Si en este momento intentamos insertar un nuevo valor, las únicas posiciones válidas

serán la 1 ó la 2. Por ejemplo, al intentar añadir el valor 8 en la posición 3, la inserción no se completará, pues la posición indicada no es válida:

**Insertar(8, 3, LISTA_1, N, NUM_ELEMENTOS)**

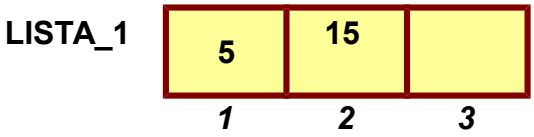
En este caso se presentará el siguiente mensaje:

**Posición de inserción no válida**

Se obtendrá el mismo resultado si se solicita la inserción en una posición no perteneciente al vector, por ejemplo, la posición 6.

Solicitamos que se inserte el valor 15 en la posición 2:

**Insertar(15, 2, LISTA_1, N, NUM_ELEMENTOS)**

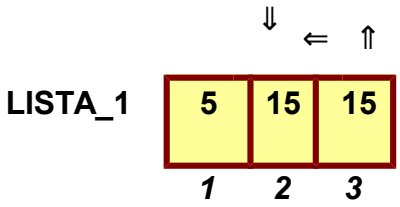


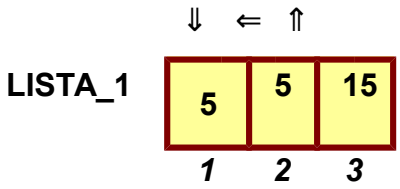

**NUM_ELEMENTOS 2**

Solicitamos que se inserte el valor 25 en la posición 1:

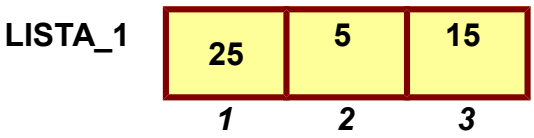
**Insertar(25, 1, LISTA_1, N, NUM_ELEMENTOS)**

Para completar la inserción, en primer lugar se deben desplazar los elementos de las posiciones 1 y 2 un lugar hacia el final del vector:




Por último, se almacena en la primera posición el nuevo valor y se incrementa en 1 la longitud de la lista:




**NUM_ELEMENTOS 3**

En este momento no se podrán realizar más inserciones, dado que la lista está llena. Si, por ejemplo, se solicita añadir el valor 4 en la posición 1:

**Insertar(4, 1, LISTA_1, N, NUM_ELEMENTOS)**

Se presentará el mensaje:

**Lista llena.**

### Eliminación de un elemento de la lista

Es posible eliminar cualquier elemento de una lista. Describiremos cómo eliminar un elemento en función de la posición que ocupa dentro de la lista, así como en función de su valor.

### Eliminación de un elemento de la lista por su posición

Se puede eliminar el elemento de la lista que ocupa la posición indicada.

Antes de intentar eliminar un elemento, se comprueba que la lista no esté vacía, pues en caso de estarlo la operación de eliminación es imposible. A continuación se comprueba que la posición del elemento a eliminar sea válida, es decir, que corresponda a una de las posiciones del vector que contienen elementos de la lista. Si la posición de eliminación es válida, se copian una posición hacia el comienzo del vector cada uno de los elementos comprendidos entre la posición siguiente a la de eliminación y la última ocupada. Si la posición del elemento a eliminar es la última ocupada, no será necesario desplazar ningún valor dentro del vector. En ambos casos, la última operación a realizar consiste en reducir en uno la longitud de la lista.

```
MÓDULO Eliminar

DATOS
    PARÁMETROS
        Recibe POSICION entera          ** posición de eliminación
        Transforma NUM_ELEM entera  ** longitud de la lista
        Transforma LISTA() de tipo T    ** vector lista
    VARIABLES
        I entera
INICIO
    Si (Lista_vacia ( NUM_ELEM ) = Verdadero) entonces
        ESCRIBIR "Lista vacía"
    sino
```

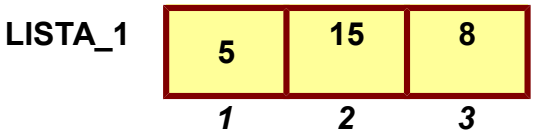
```
            Si ((POSICION>NUM_ELEM) O (POSICION<1)) entonces
                ESCRIBIR "Posición de eliminación no válida"
            sino
                I <- POSICION

                Mientras (I < NUM_ELEM) hacer
                    LISTA(I)<- LISTA(I+1)
                    I <- I+1
                fin-mientras

                NUM_ELEM <- NUM_ELEM - 1
            fin-si
        fin-si
    FIN
```

**Ejemplo de borrado de un elemento en una lista por su posición**

Consideremos una lista de enteros que puede almacenar 3 valores como máximo (N=3) y que actualmente está llena:

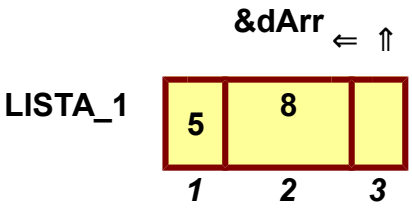



**NUM_ELEMENTOS 3**

Se solicita eliminar el elemento de la posición 2:

**Eliminar(2, NUM_ELEMENTOS, LISTA_1)**

Como resultado, el valor de la tercera posición pasa a ocupar la segunda y se reduce la longitud de la lista:




**NUM_ELEMENTOS 2**

Si se indica una posición de eliminación no válida:

**Eliminar(3, NUM_ELEMENTOS, LISTA_1)**

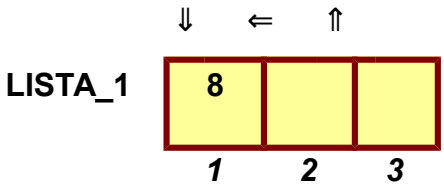
se presentará un mensaje indicando que no se puede realizar la operación:

**Posición de eliminación no válida**

Se solicita eliminar el elemento de la primera posición:

**Eliminar(1, NUM_ELEMENTOS, LISTA_1)**

Como resultado, el valor de la segunda posición se desplaza sobre la primera, quedando la lista con un único elemento:

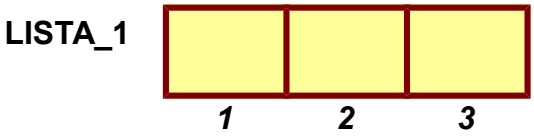



**NUM_ELEMENTOS 1**

En este momento sólo es posible eliminar el elemento de la primera posición:

**Eliminar(1, NUM_ELEMENTOS, LISTA_1)**

con lo que la lista queda vacía:




**NUM_ELEMENTOS 0**

Si se solicita una nueva eliminación (sobre cualquier posición):

**Eliminar(1, NUM_ELEMENTOS, LISTA_1)**

se mostrará un mensaje indicando que la lista está vacía:

**Lista vacía**

### Eliminación de un elemento de la lista por su contenido

Para eliminar de la lista el elemento cuyo valor coincida con uno especificado, en primer lugar habrá que comprobar si dicho valor aparece en la lista. En caso de aparecer, se procederá a su eliminación.

Por tanto, esta eliminación se puede realizar combinando las operaciones de *búsqueda* y *eliminación por posición* definidas para la lista. Se realizará una búsqueda lineal sobre el vector que contiene la lista. Si la búsqueda tiene éxito, se obtendrá un entero que representa la posición en la que se encontró el valor; en caso contrario, se obtendrá un valor menor que 1. En el primer caso el índice obtenido se pasa al módulo Eliminar, que eliminará el elemento de dicha posición.

Con las operaciones propuestas, si el valor aparece varias veces en la lista, se eliminará el que primero se encuentre de todos ellos. Si la búsqueda se realiza de izquierda a derecha, se eliminará el más próximo al inicio de la lista.

```
MÓDULO Eliminar_valor
DATOS
        PARÁMETROS
                Recibe VALOR de tipo T          ** valor a eliminar
                Transforma NUM_ELEM entera  ** longitud de la lista
                Transforma LISTA() de tipo T    ** vector lista
                Recibe N de tipo T              ** tamaño del vector
VARIABLES
                POSICION entera
INICIO
        POSICION <- Busqueda_lineal(LISTA, NUM_ELEM, VALOR)

        Si (POSICION < 1) entonces
                Escribir "El valor no forma parte de la lista"
        sino
                Eliminar (POSICION, NUM_ELEM, LISTA)
        fin-si
FIN
```

**Comprobación de lista vacía**

La lista está vacía si no contiene elementos.

```
MÓDULO Lista_vacia  devuelve lógico

DATOS
        PARÁMETROS
                Recibe NUM_ELEMENTOS entera
INICIO
        Si (NUM_ELEMENTOS = 0) entonces
                DEVOLVER Verdadero
        sino
                DEVOLVER Falso
        fin-si
FIN
```

### Comprobación de lista llena

La implementación mediante vectores obliga a comprobar si la lista está llena antes de intentar introducir un nuevo elemento. La lista está llena si contiene N elementos.

```
MÓDULO Lista_llena   devuelve lógico

DATOS
        PARÁMETROS
                Recibe NUM_ELEMENTOS entera
                Recibe N entera
INICIO
        Si (NUM_ELEMENTOS = N) entonces
                DEVOLVER Verdadero
        sino
                DEVOLVER Falso
        fin-si
FIN
```

### Recorrido de la lista

Para recorrer todos los elementos de una lista, se utiliza un bucle que recorre todas las posiciones ocupadas del vector que le da soporte.

Si la lista está vacía, se mostrará un mensaje de aviso. En caso contrario, se aplicará algún procesamiento (representado por la operación **TRATAR()**) a cada uno de los elementos de la misma. El tratamiento aplicado puede modificar la lista, por lo que se permite transformar el parámetro **LISTA**.

```
MÓDULO Recorrer_lista

DATOS
        PARÁMETROS
                Transforma LISTA() de tipo T
                Recibe NUM_ELEMENTOS entera
        VARIABLES
                I entera
INICIO
        Si (Lista_vacia(NUM_ELEMENTOS) = Verdadero) entonces
                ESCRIBIR "La lista está vacía"
        sino
                Para I desde 1 hasta NUM_ELEMENTOS
                        TRATAR (LISTA(I))
                fin-para
        fin-si
FIN
```

### Acceso a un elemento de la lista

Para acceder a un elemento cualquiera de la lista, I, se consulta la posición correspondiente dentro del vector, **LISTA(I)**.

Podremos modificar el valor del elemento I de la lista mediante una asignación:

**LISTA(I) <- NUEVO_VALOR**

o mediante una lectura:

**LEER LISTA(I)**

También se puede visualizar el valor del elemento:

**ESCRIBIR LISTA(I)**

### Ordenación de los elementos de la lista

Para ordenar una lista representada mediante un vector se pueden aplicar los algoritmos de ordenación tratados en el tema referente a arrays.

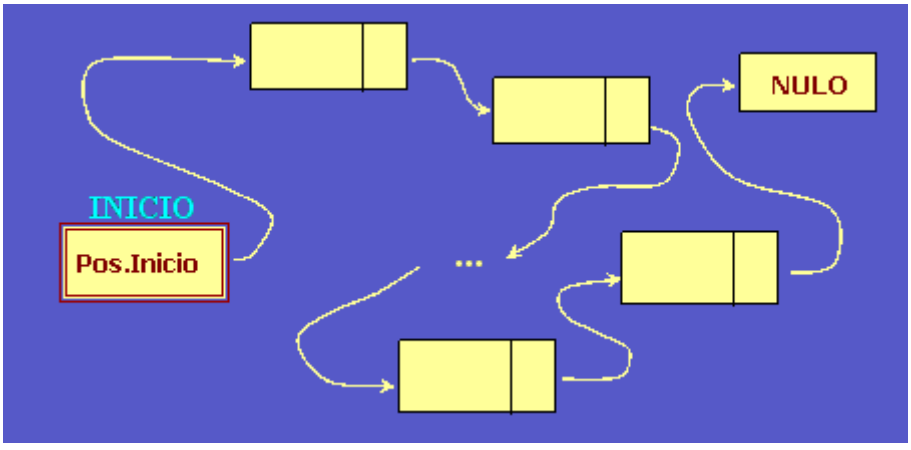
### Búsqueda de un elemento en la lista

Para buscar un valor en una lista representada mediante un vector se pueden aplicar los algoritmos de búsqueda tratados en el tema referente a arrays. Recordemos que si los datos no están ordenados, sólo se puede aplicar una búsqueda secuencial. Si los datos están ordenados, se puede aplicar además una búsqueda binaria.

### *Operaciones mediante representación con memoria dinámica*

## *Lista Lineal Simplemente Enlazada*

Una lista lineal con enlace simple está formada por un conjunto de nodos conectados mediante punteros. Cada nodo de la estructura conecta con otro nodo de la misma, salvo el nodo situado en uno de los extremos, cuyo campo de enlace será nulo. Existe un puntero externo que da acceso a la estructura, apuntando al nodo situado en el extremo opuesto al que tiene su campo de enlace nulo.



### Creación de la Lista Lineal Simplemente Enlazada

Para crear la lista, en primer lugar se define la estructura de sus nodos:

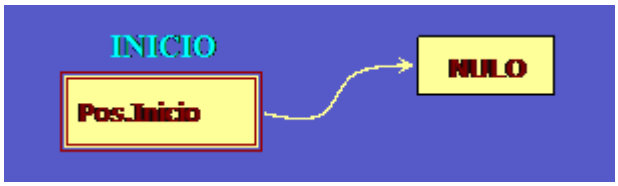
**NODO_LISTA es registro compuesto por**
**DATO de tipo T**
**SIGUIENTE puntero a NODO_LISTA**
**fin-registro**

A continuación, se define un puntero a nodos de la lista, que dará acceso a la estructura:

**INICIO puntero a NODO_LISTA**

Por último, se establece la condición de lista vacía, haciendo que el puntero a la lista sea nulo:

**INICIO <- NULO**

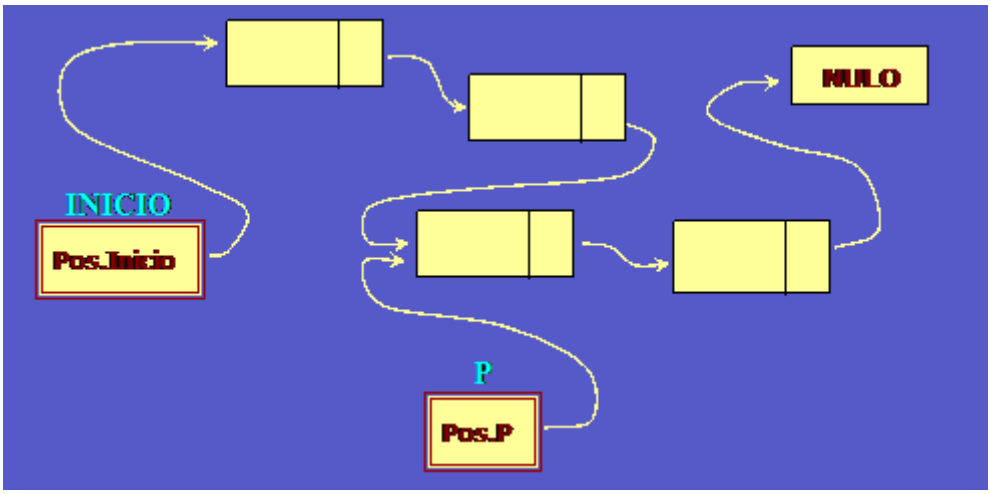



**Representación gráfica de lista lineal con enlace simple vacía**

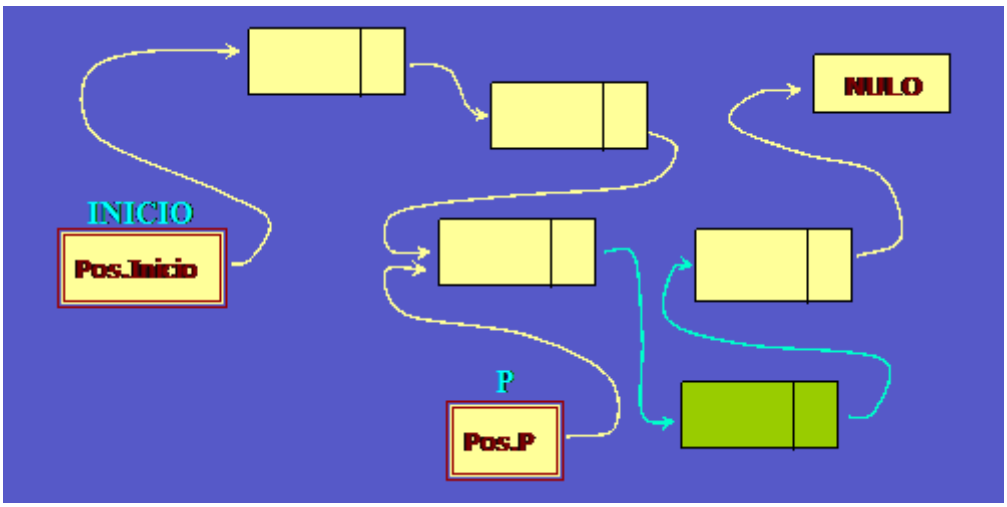
### Inserción de un elemento en una Lista Lineal Simplemente Enlazada

Para realizar la inserción, además del valor del elemento a insertar, indicaremos la posición que debe ocupar en la lista.

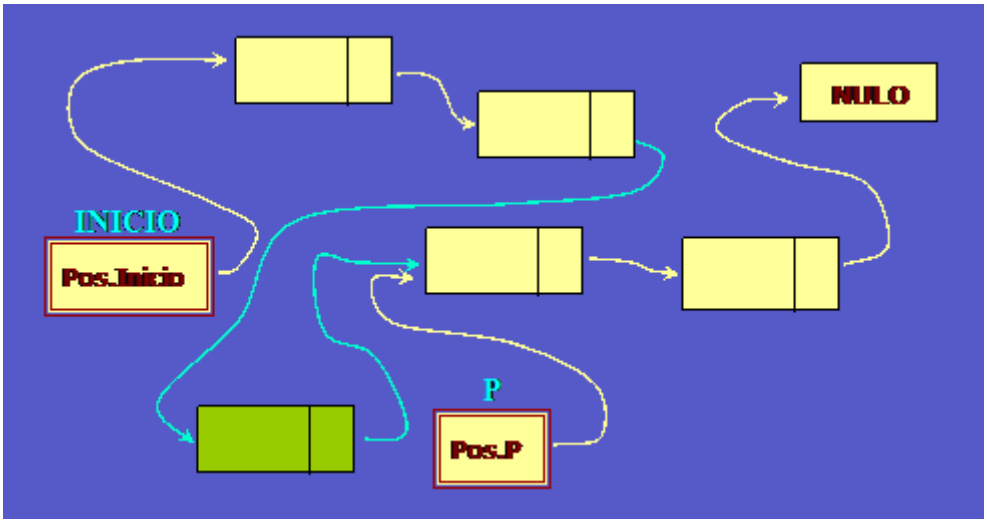
La inserción se podrá realizar en cualquier posición de la estructura: al principio de la lista, al final, o en cualquier posición intermedia. Para hacer una descripción más general, la posición de inserción vendrá determinada por un puntero que apuntará a un nodo ya existente en la lista. Dado un puntero **P** a un nodo de la lista, podemos realizar la inserción del nuevo nodo en dos posiciones: después del nodo apuntado por **P** o antes. Nosotros trataremos el caso de inserción después de un nodo dado.




**Posibles casos en la inserción de un nuevo nodo**




**Inserción de un nuevo nodo después del apuntado por P**




**Inserción de un nuevo nodo antes del apuntado por P**

El parámetro **P** debe ser una referencia válida a uno de los nodos de la lista, salvo en el caso en que la lista esté vacía, en el que no lo utilizamos, lo que posibilita la primera inserción. En este caso **P** puede hacerse NULO.

Una vez obtenida la memoria para el nuevo nodo, se almacena el valor del nuevo elemento en el campo de datos de dicho nodo. A continuación, se enlaza el nuevo nodo con el resto de la estructura de datos. Si la lista estaba vacía, el nuevo nodo será el único de la misma. Por tanto, su campo de enlace se hará nulo y, además, el puntero a la lista apuntará al nuevo elemento. Si la lista no estaba vacía, el nuevo nodo se debe colocar entre el referenciado por **P** y el referenciado por **P↑.SIGUIENTE**. Si el nuevo nodo se va a convertir en el último de la lista no tendrá nodo siguiente, por lo que su campo de enlace se hará nulo.

**MÓDULO Insertar_despues**

**DATOS**

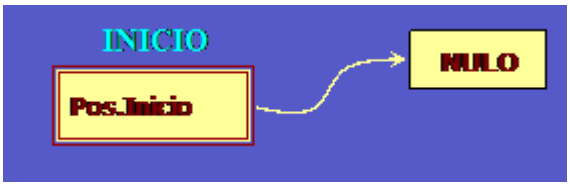
**PARÁMETROS**

**Transforma INICIO puntero a NODO_LISTA**

```
                Recibe P puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T          ** nuevo elemento
        VARIABLES
                NUEVO puntero a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))
        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO <- ELEMENTO
                Si (Lista_vacia(INICIO) = Verdadero) entonces
                        INICIO <- NUEVO
                        NUEVO↑.SIGUIENTE <- NULO
                sino
                        NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                        P↑.SIGUIENTE <- NUEVO
                fin-si
        fin-si
FIN
```

**CASO: INSERCIÓN EN LISTA VACÍA**

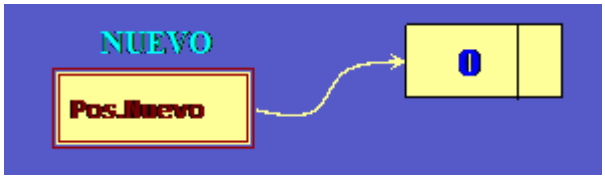
Supongamos una lista de enteros vacía:



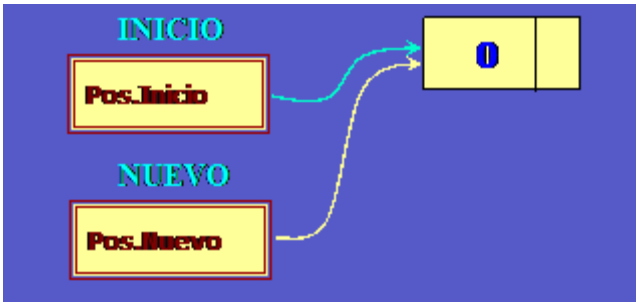

a la que intentamos añadir un primer elemento, de valor 0:

**Insertar_despues(INICIO, INICIO, 0)**

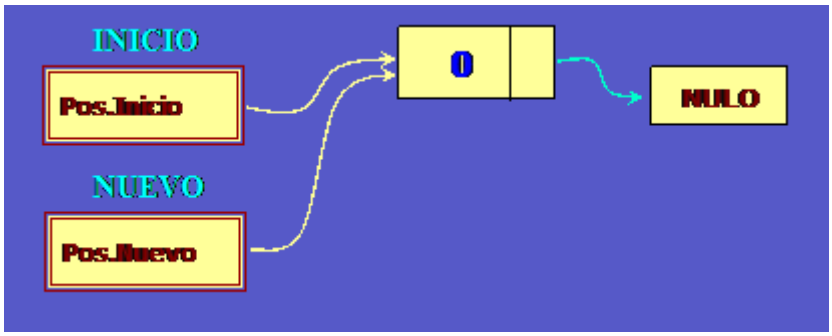
Se solicita memoria para el nuevo nodo y se almacena el dato:



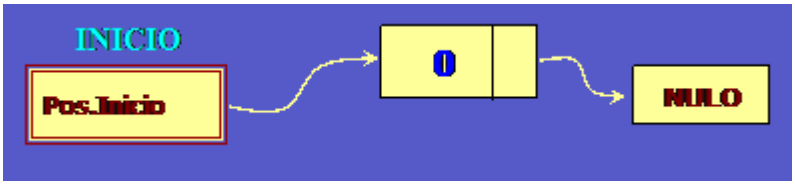

Como la lista está vacía, se hace que el puntero a la misma apunte al nuevo nodo:



y que el campo de enlace de dicho nodo se haga nulo:

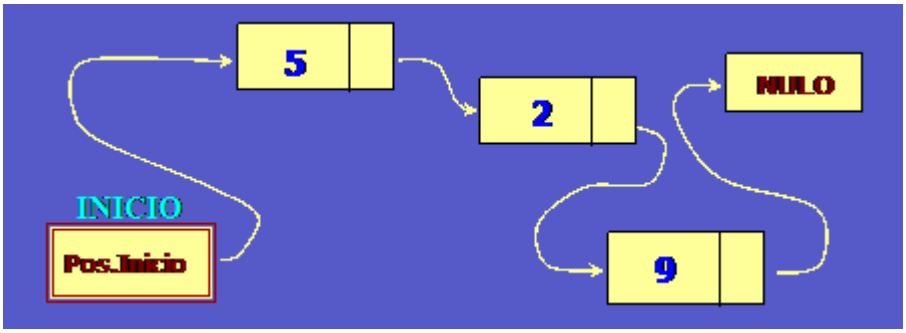



La lista quedará como sigue:

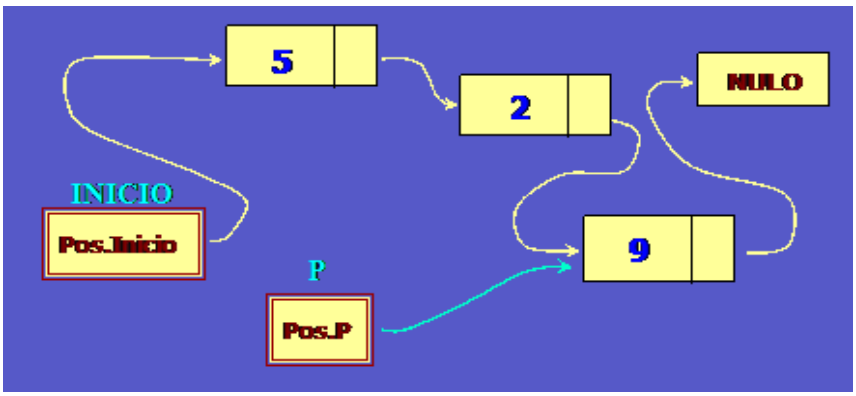



**CASO: INSERCIÓN AL FINAL DE LA LISTA**

Consideremos la lista de enteros que se muestra en la siguiente figura:

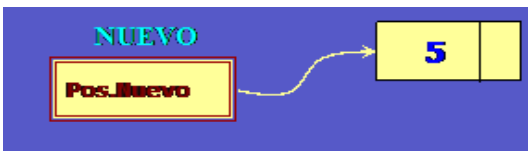



en la que deseamos insertar el dato 5 a continuación del último nodo de la lista. Por tanto, el puntero que identifica el punto de inserción debe apuntar al último nodo de la lista:

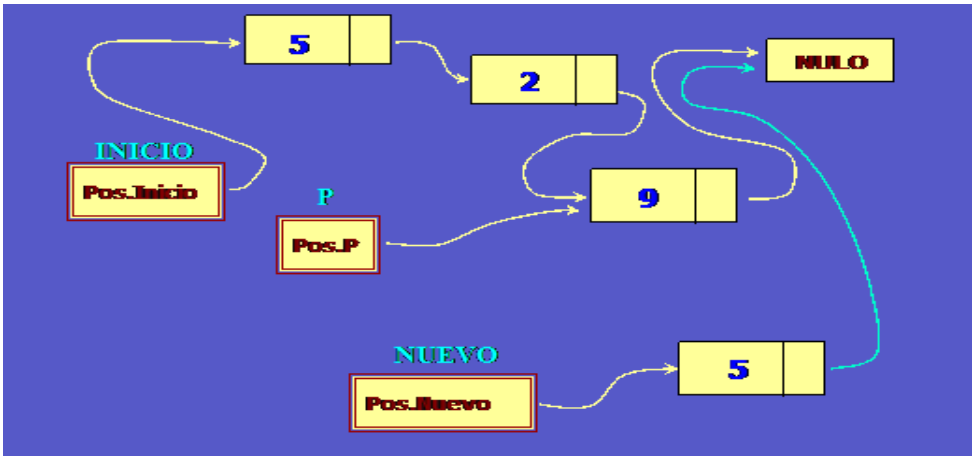



**Insertar_despues(INICIO, P, 5)**

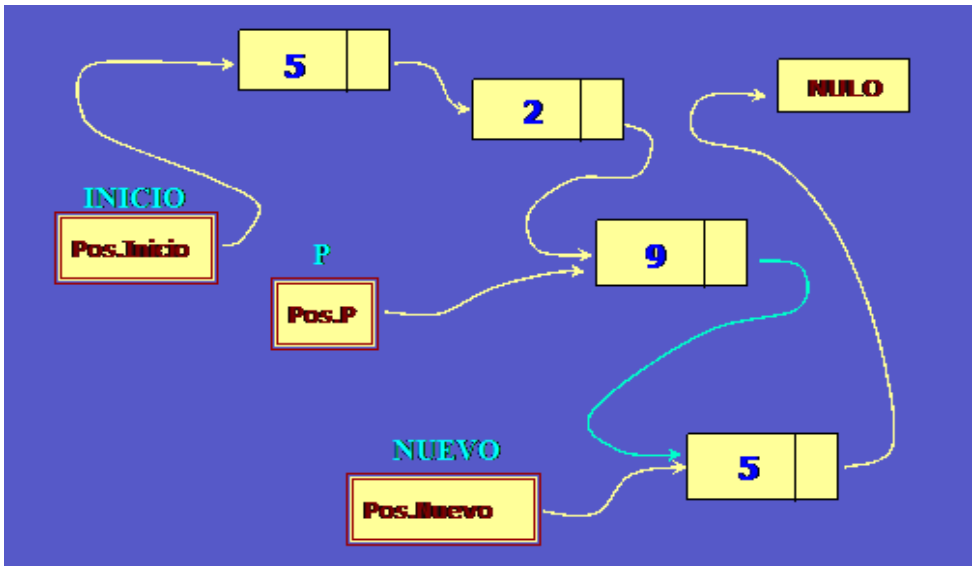
Se reserva memoria para el nuevo nodo y se almacena el dato:



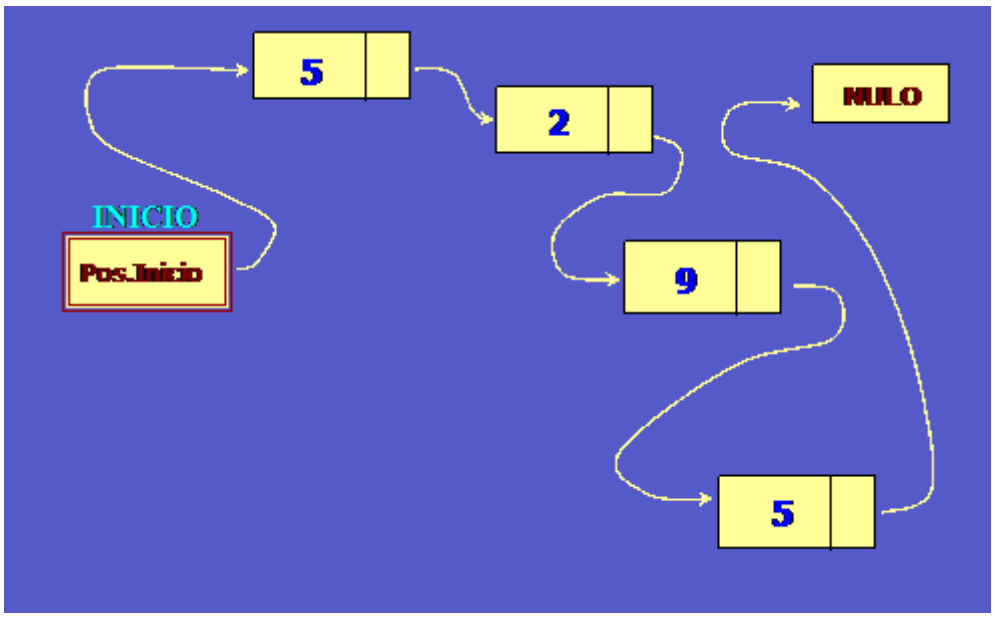
El campo de enlace del nuevo nodo tomará el mismo valor que el campo de enlace del nodo apuntado por P, es decir, NULO:




El campo de enlace del nodo apuntado por P pasará a apuntar al nuevo nodo:



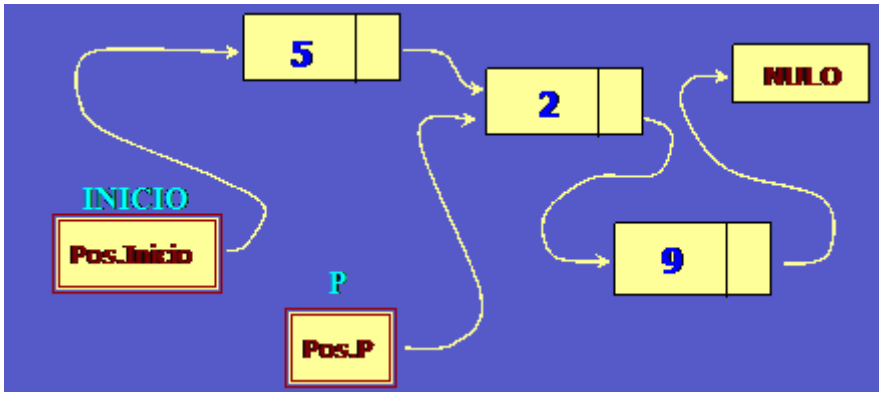

Tras la inserción del nuevo elemento, la lista quedará como sigue:



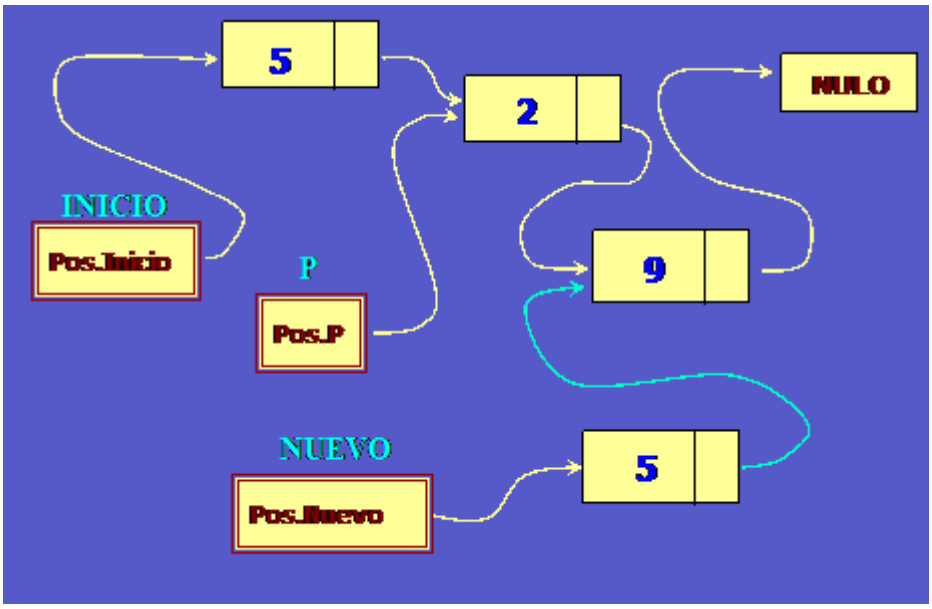
## CASO: INSERCIÓN INTERMEDIA

Supongamos ahora que, partiendo de la misma lista que en el ejemplo anterior, deseamos añadir el mismo valor, pero en este caso a continuación del segundo nodo:

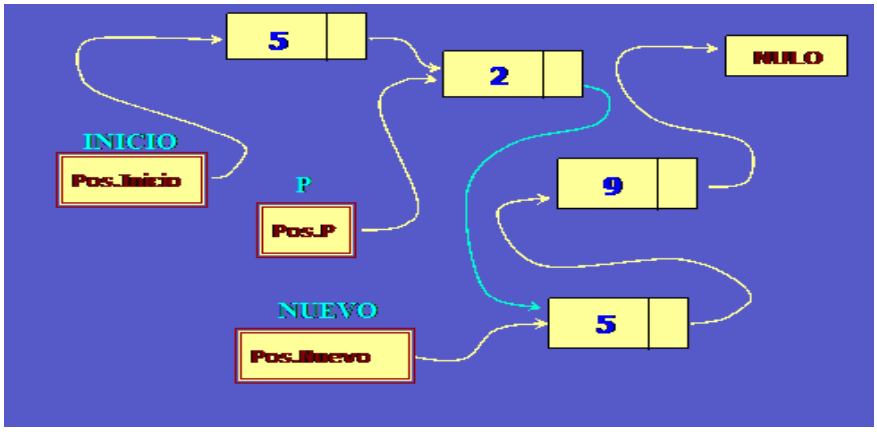
**Insertar_despues(INICIO, P, 5)**



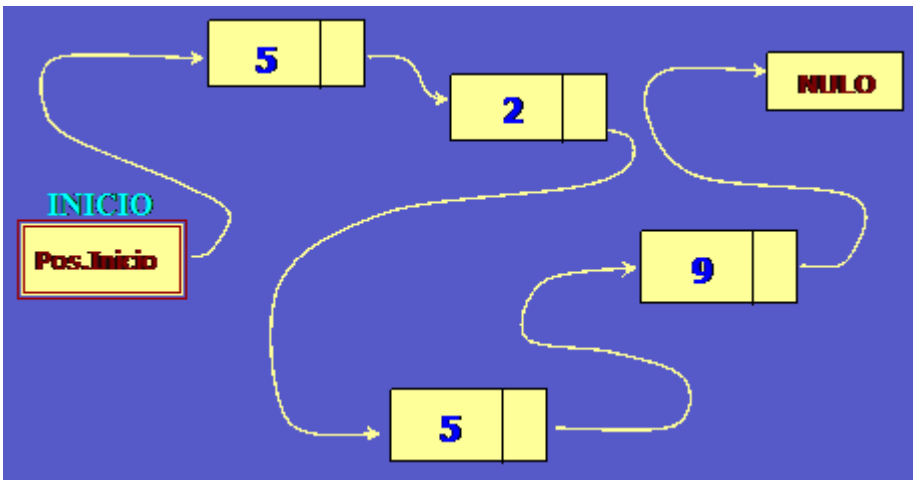

Se reserva memoria para el nuevo nodo y se almacena el dato, igual que en el ejemplo anterior. El campo de enlace del nuevo nodo apuntará al mismo nodo que el campo de enlace del nodo apuntado por P:




El campo de enlace del nodo apuntado por P pasará a apuntar al nuevo nodo:

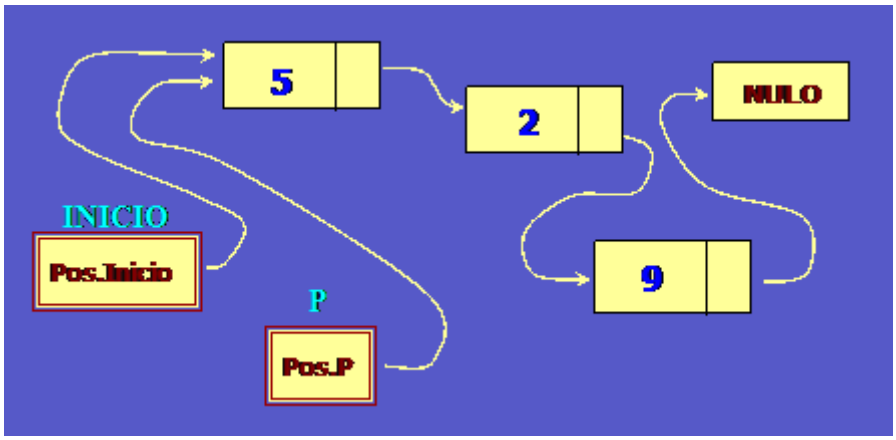



La lista quedará como sigue:



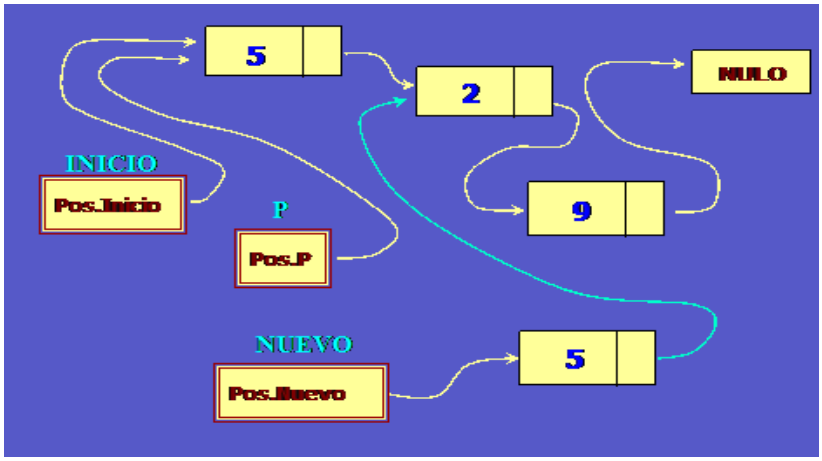

**CASO: INSERCIÓN AL PRINCIPIO**

Supongamos que en la lista inicial se desea insertar el valor 5 a continuación del primer nodo (el apuntado por el puntero INICIO):

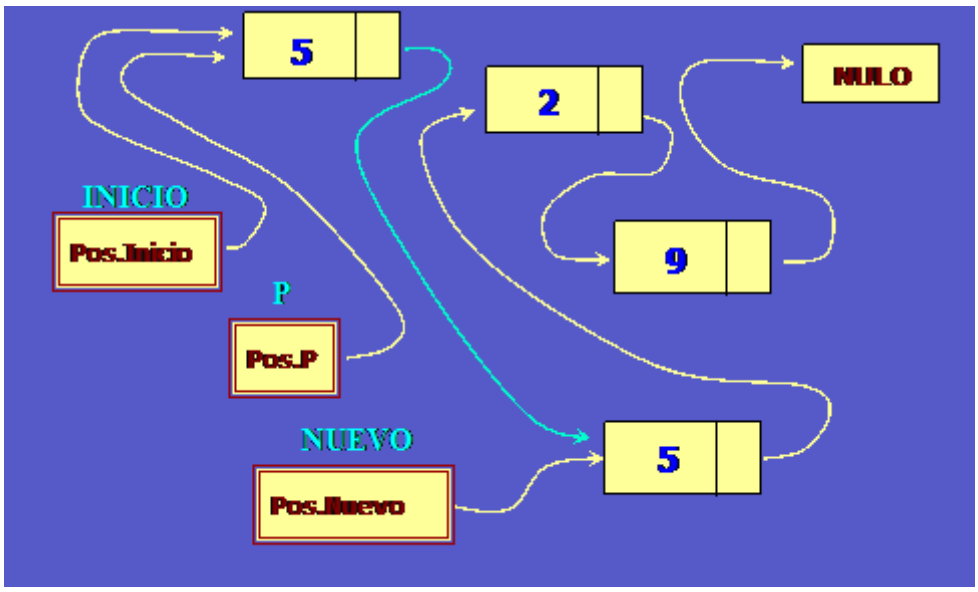



**Insertar_despues(INICIO, P, 5)**

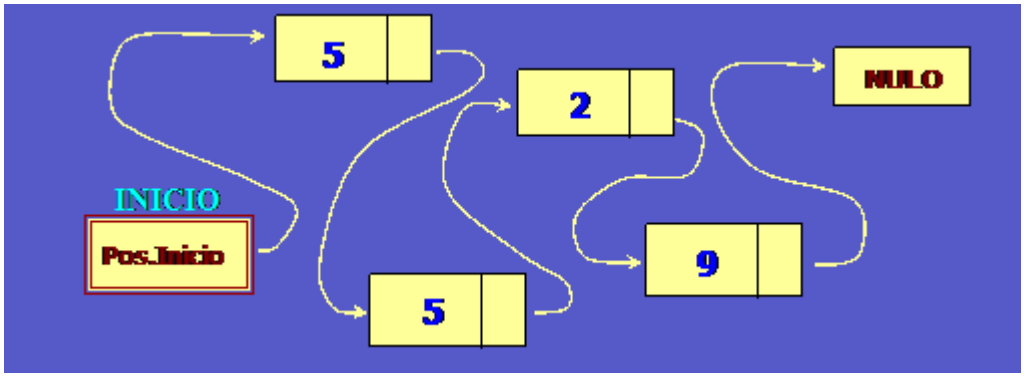
Si la solicitud de memoria para un nuevo nodo tiene éxito, se almacena el dato en el nuevo nodo. A continuación se hace que su campo de enlace apunte al mismo nodo que el campo de enlace del nodo de la lista que contiene el valor 5:




El nodo de la lista que contiene el valor 5 se enlaza con el nuevo:




La lista obtenida tras la inserción queda como sigue:




**Eliminación de un elemento en una Lista Lineal Simplemente Enlazada**

**Eliminación de un elemento en una Lista Lineal Simplemente Enlazada (por su posición)**
Supongamos que se desea eliminar un nodo de una lista enlazada, identificado mediante un puntero **P**. La operación de eliminación implica, además de liberar la memoria asociada al nodo elegido, reconectar adecuadamente los nodos que permanecen en la estructura, de forma que se garantice el correcto acceso a los mismos.

Si se debe eliminar el primer nodo de la lista, el puntero a la estructura debe pasar a apuntar al siguiente nodo, haciéndose nulo si la lista sólo contenía un nodo. Si el nodo a eliminar no es el primero, antes de eliminarlo se debe conectar su nodo previo con su nodo posterior, si existe éste, o hacer nulo el enlace del nodo previo en caso contrario. Para reconectar ambos nodos se debe recorrer la lista desde el nodo referenciado por el puntero a la estructura y hasta que se encuentre el nodo previo al referenciado por **P**.

**MÓDULO Eliminar**

```
DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
        VARIABLES

                PREVIO puntero a NODO_LISTA
INICIO
        Si (Lista_Vacia(INICIO) = Verdadero) entonces
                ESCRIBIR "Error: lista vacía"
        sino
                Si (P = INICIO) entonces
                        INICIO <- INICIO↑.SIGUIENTE
                sino
                        PREVIO <- INICIO

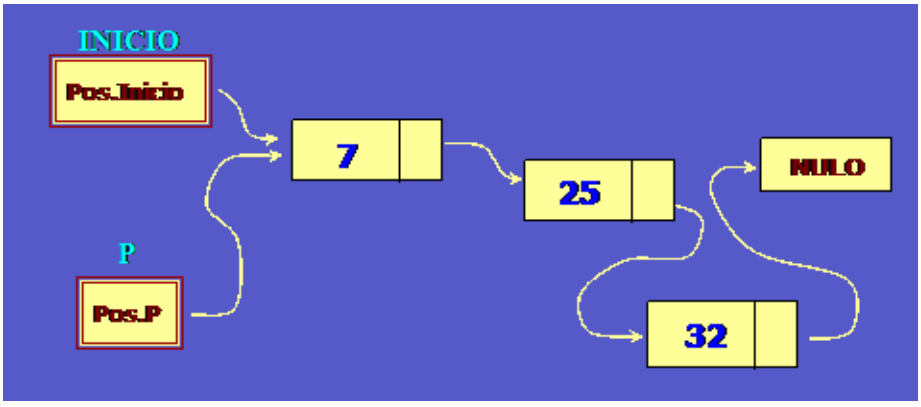
                        Mientras (PREVIO↑.SIGUIENTE ≠ P) hacer
                                PREVIO <- PREVIO↑.SIGUIENTE
                        fin-mientras

                        PREVIO↑.SIGUIENTE <- P↑.SIGUIENTE
                fin-si

                LIBERAR(P)
        fin-si
FIN
```

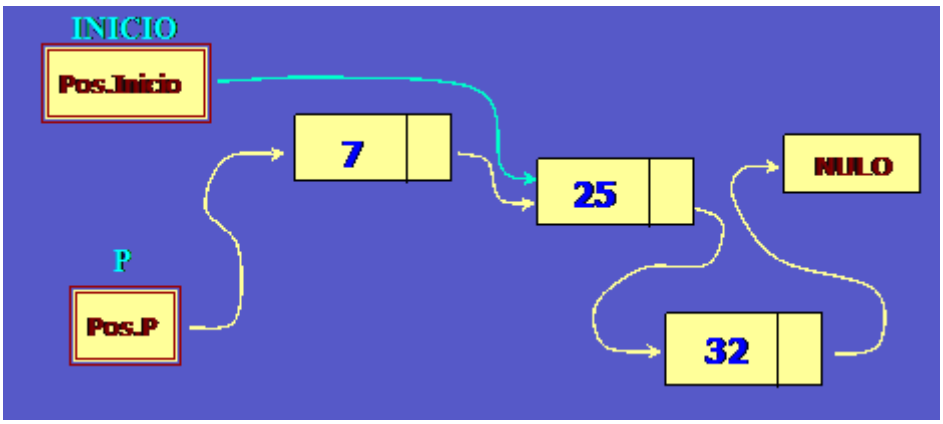
**CASO: ELIMINAR EL PRIMER NODO EN UNA LISTA QUE NO QUEDARÁ VACÍA**

Supongamos que en la lista de la figura, que almacena 3 enteros, se desea eliminar el nodo apuntado por P, es decir, el apuntado por el puntero que da acceso a la estructura:

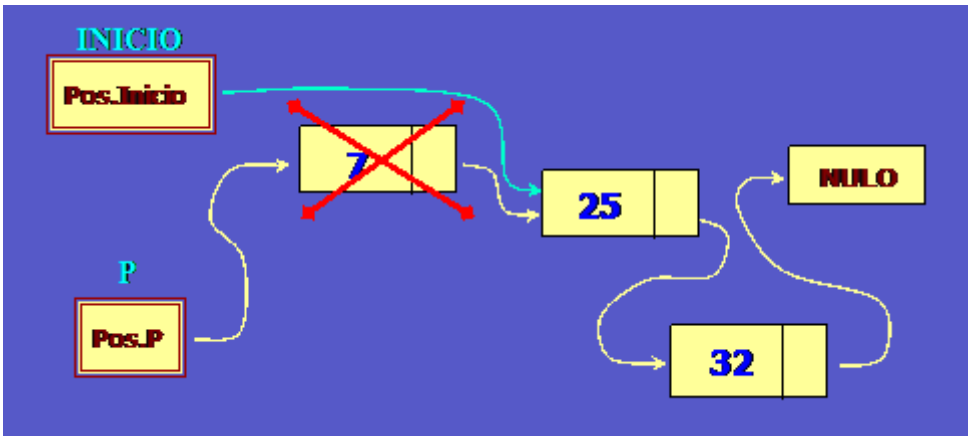



**Eliminar(INICIO, P)**

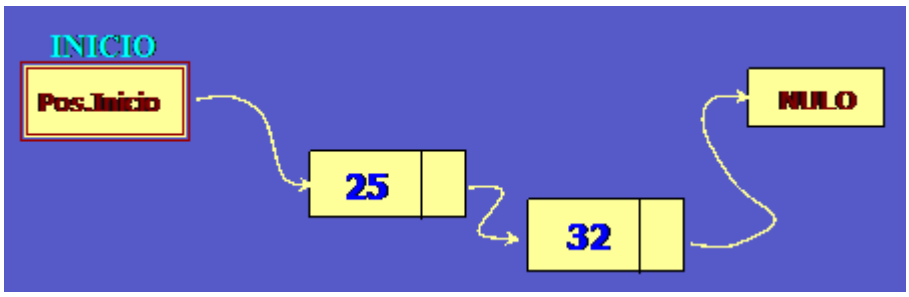
Para proceder a eliminar el nodo, en primer lugar se redefine el valor del puntero **INICIO**, haciendo que pase a apuntar al segundo nodo:




Por último, se libera la memoria asociada al primer nodo:

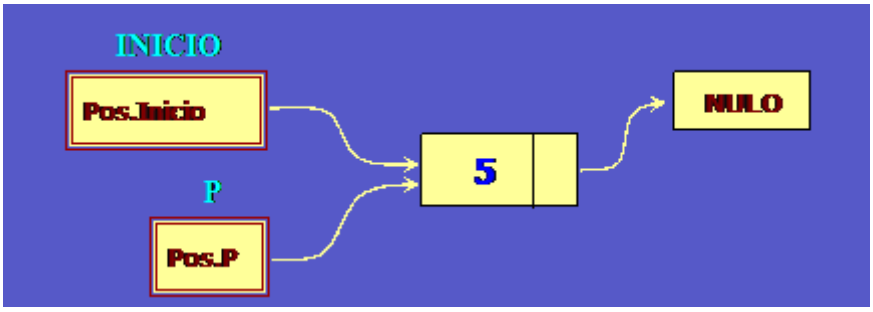



La lista quedará con 2 nodos:




**CASO: ELIMINAR EL PRIMER NODO EN UNA LISTA QUE SI QUEDARÁ VACÍA**

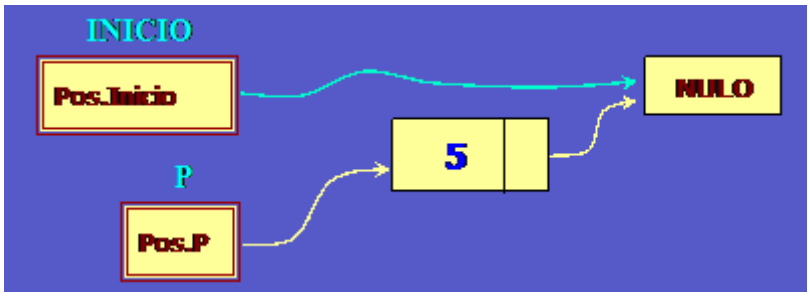
Supongamos que se desea eliminar el único nodo de una lista:



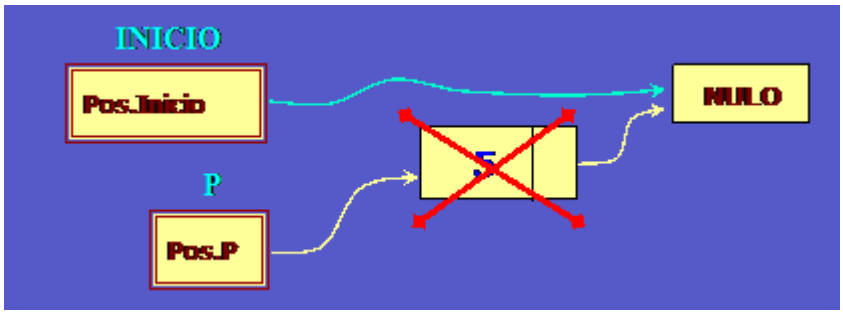

**Eliminar (INICIO, P)**

El puntero a la estructura tomará el valor del campo de enlace del nodo que se desea

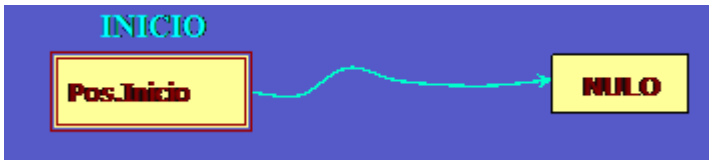
eliminar, es decir, NULO:




A continuación, se libera la memoria asociada al nodo apuntado por P:

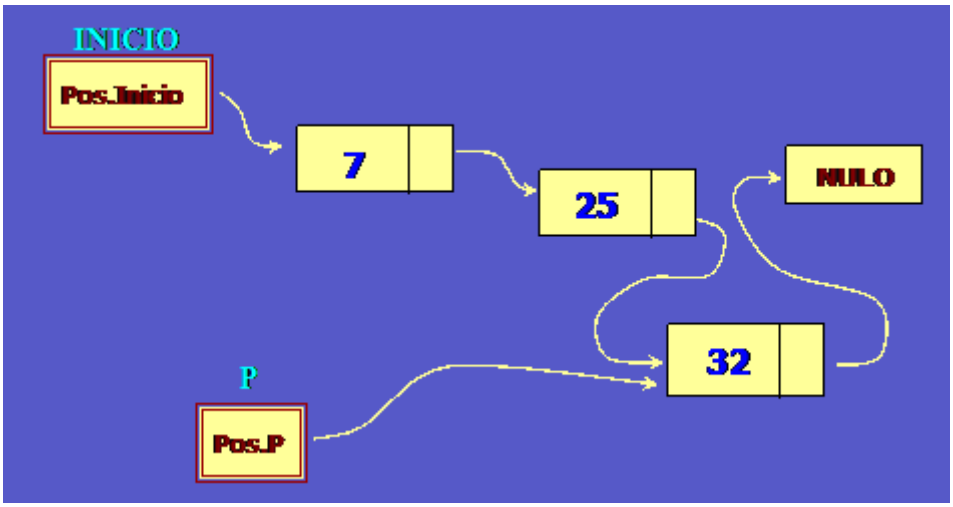



Tras la eliminación, la lista ha quedado vacía:



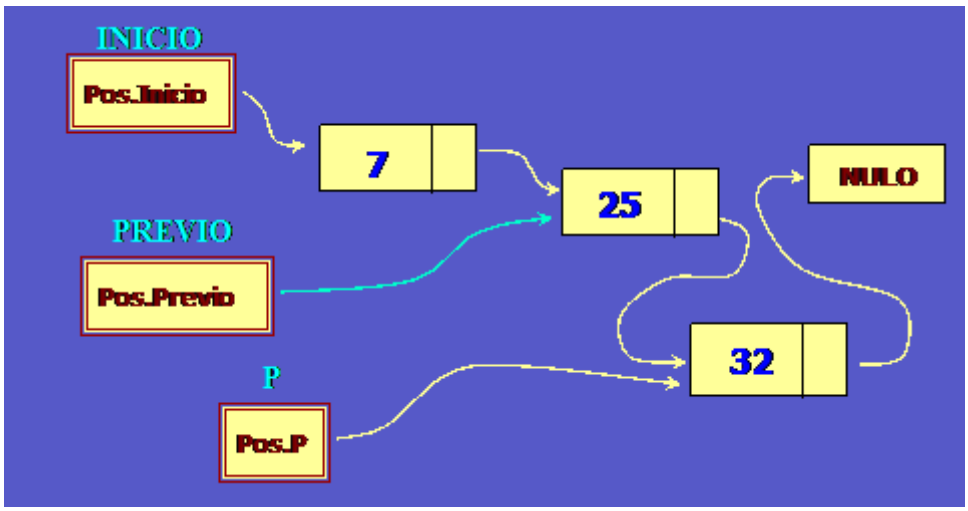

**CASO: ELIMINAR EL ÚLTIMO NODO DE UNA LISTA QUE NO QUEDARÁ VACÍA**

Supongamos que se desea eliminar el último nodo de la siguiente lista:

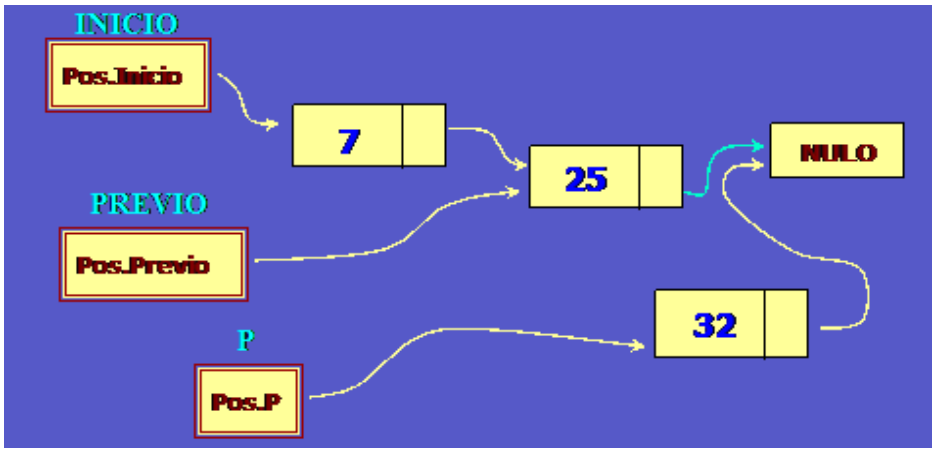



**Eliminar(INICIO, P)**

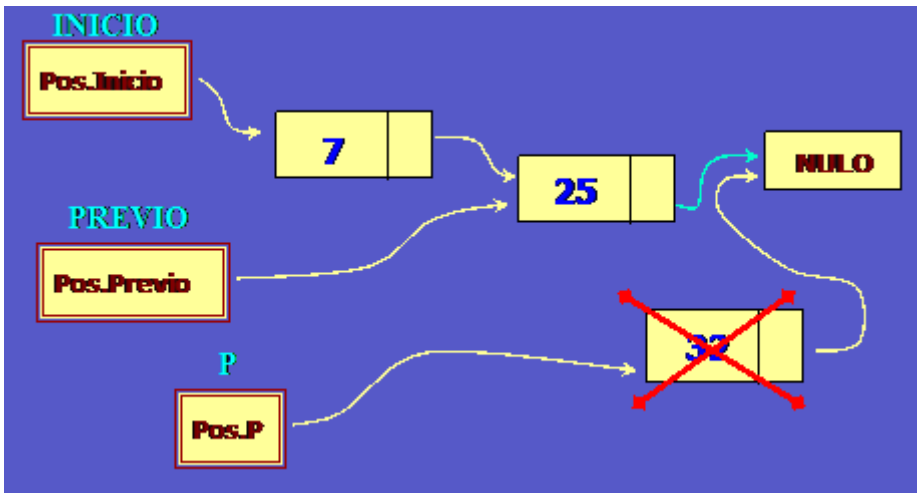
En primer lugar, se recorrerá la lista hasta encontrar el nodo que apunta al que se desea eliminar. El puntero **PREVIO** identificará dicho nodo:



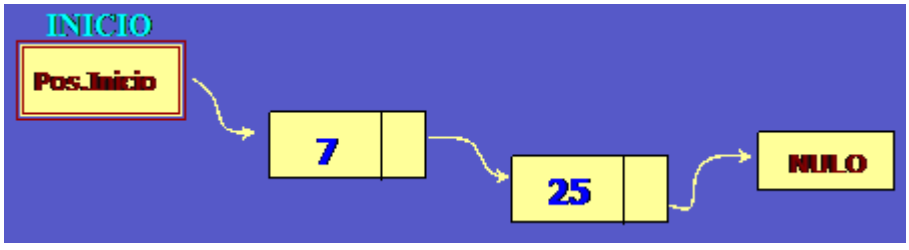

Se hace que el nodo previo al que se va a eliminar apunte al mismo sitio que este último, con lo que su enlace se hará nulo:




Se libera la memoria asociada al nodo que se va a eliminar:

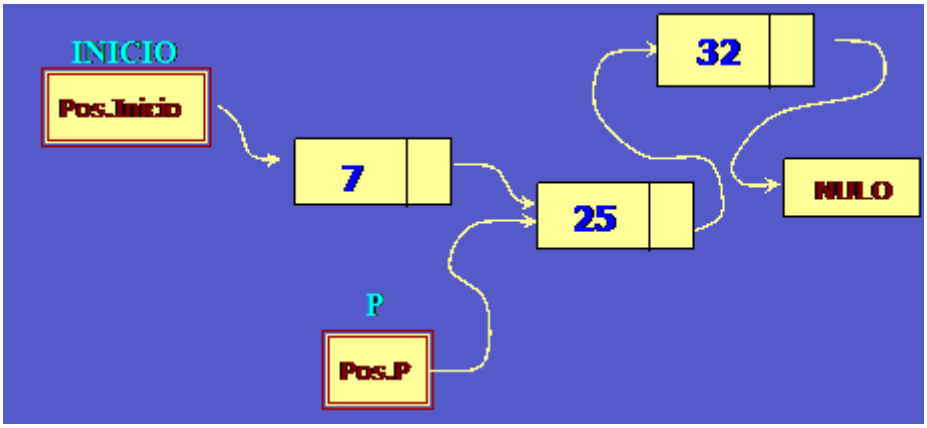



La lista quedará con dos nodos:



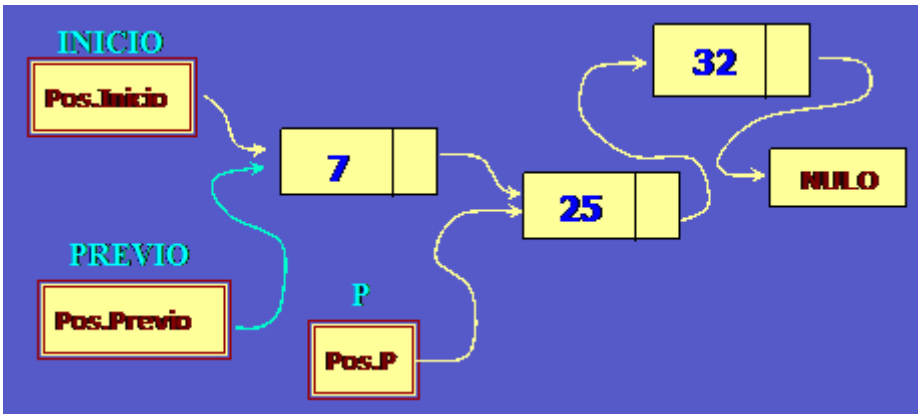
## CASO: ELIMINAR UN NODO INTERMEDIO DE UNA LISTA

En el siguiente ejemplo se intenta eliminar un nodo intermedio de una lista:

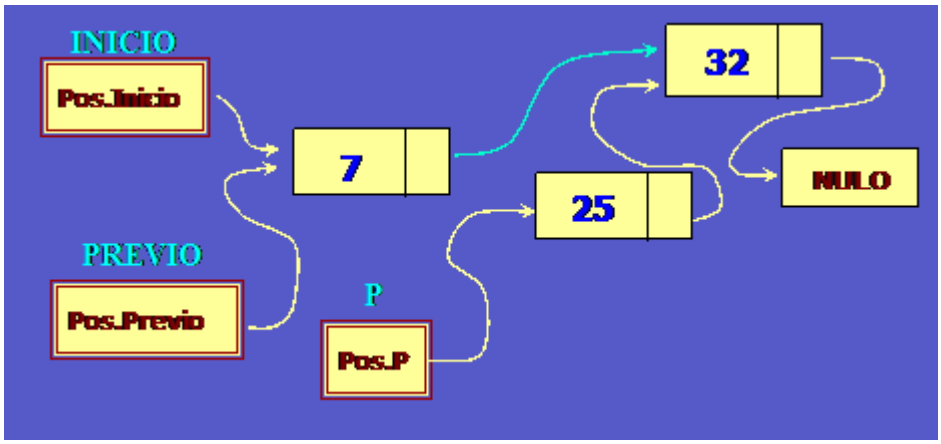



**Eliminar(INICIO, P)**

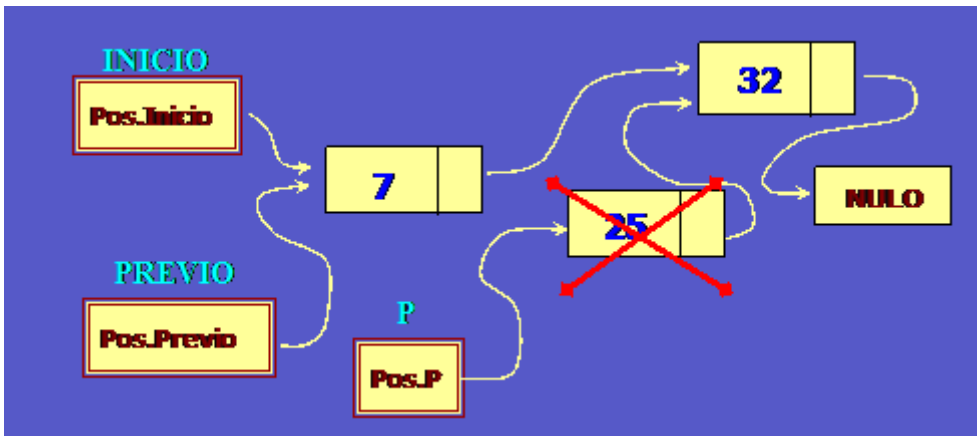
Como sólo hay un nodo antes del que se desea eliminar, la búsqueda concluye antes de entrar en el bucle Mientras:



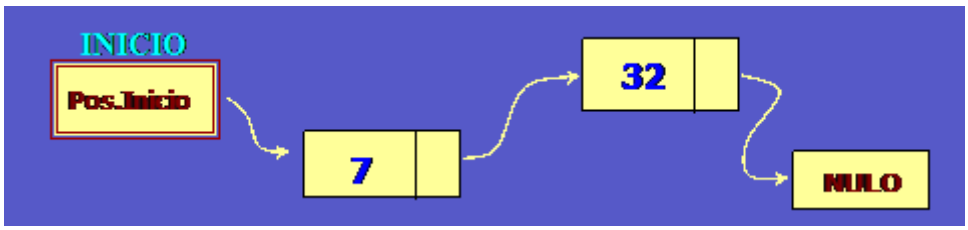

En enlace del nodo apuntado por **PREVIO** pasa a tomar el valor del enlace del nodo que se va a eliminar:




Se libera la memoria asociada al nodo que se va a eliminar:




La lista pasará a contener sólo 2 elementos:




**Eliminación de un elemento en una Lista Lineal Simplemente Enlazada (por su contenido)**

Si se desea eliminar de la lista un valor dado, se pueden combinar las operaciones de búsqueda y eliminación antes definidas. En primer lugar se comprobará si el valor buscado aparece en la lista, obteniendo en caso afirmativo un puntero al nodo que contiene tal valor. A continuación, se aplicará el módulo Eliminar para eliminar dicho nodo de la estructura.

Si el valor aparece varias veces en la lista, se eliminará el primero de todos ellos.

MÓDULO Eliminar_valor

DATOS

**PARÁMETROS**

**Transforma INICIO puntero a NODO_LISTA**

**Recibe VALOR de tipo T ** valor a eliminar**

**VARIABLES**

**P puntero a NODO_LISTA**

INICIO

**P <- Buscar(INICIO, VALOR)**

**Si (P = NULO) entonces**

```
            Escribir "El valor no forma parte de la lista"
        sino
            Eliminar (INICIO, P)
        fin-si
FIN
```

### Comprobación de Lista Lineal Simplemente Enlazada vacía

La lista estará vacía si el puntero a la misma es nulo:

```
MÓDULO Lista_vacia   devuelve lógico

DATOS
        PARÁMETROS
            Recibe INICIO puntero a NODO_LISTA
INICIO
        Si (INICIO = NULO) entonces
            DEVOLVER Verdadero          ** lista vacía
        sino
            DEVOLVER Falso              ** lista no vacía
        fin-si
FIN
```

### Recorrido de una Lista Lineal Simplemente Enlazada

Se puede aplicar un mismo tratamiento a todos los elementos de la lista (que representaremos mediante la operación **TRATAR( )**), utilizando un bucle que recorra la estructura, partiendo del nodo referenciado por el puntero a la misma. Si la lista está vacía, no se realizará ningún procesamiento.

```
MÓDULO Recorrer_lista

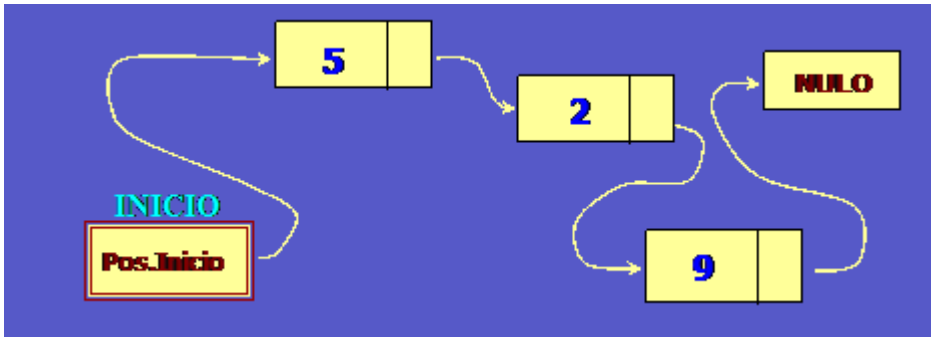
DATOS
        PARÁMETROS
            Recibe INICIO puntero a NODO_LISTA
        VARIABLES
            P puntero a NODO_LISTA
INICIO
```

```
P <- INICIO
Mientras (P ≠ NULO) hacer
        TRATAR (P↑.DATO)
        P <- P↑.SIGUIENTE
fin-mientras
```

FIN

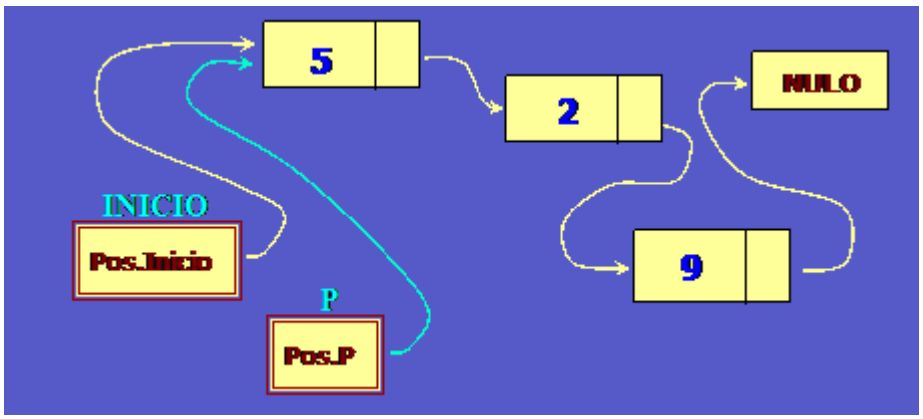
Ejemplo de operación de recorrido

Se procede a recorrer la siguiente lista:

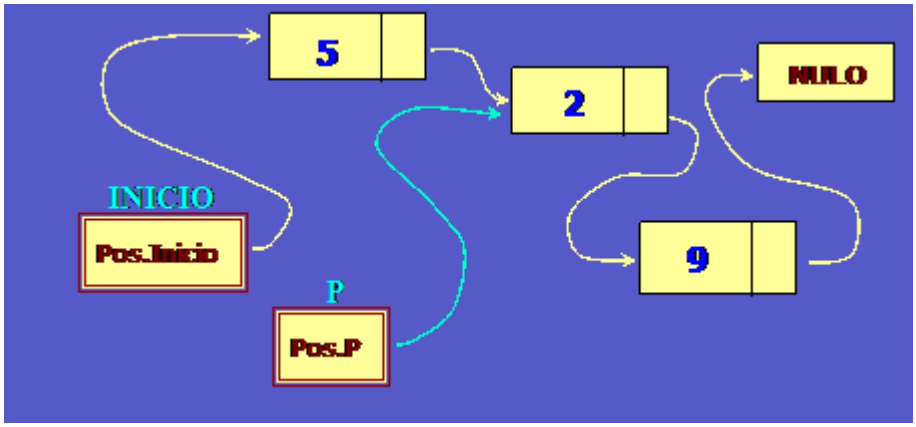



**Recorrer_lista(INICIO)**

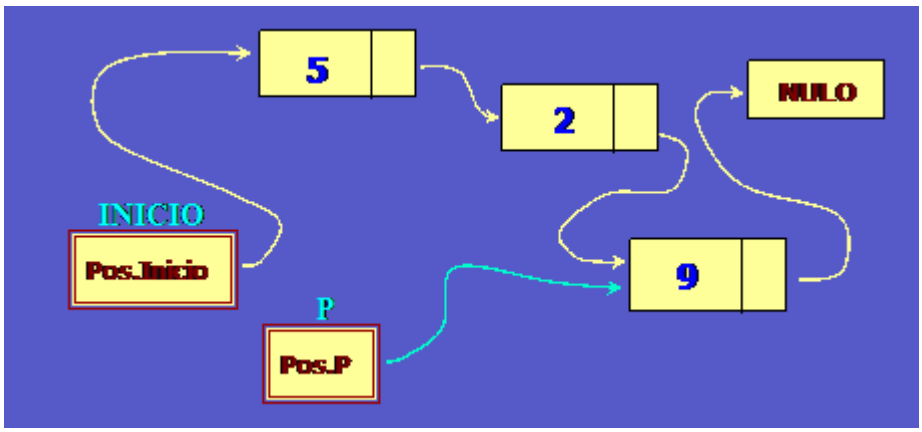
Inicialmente P apunta al nodo con valor 5:



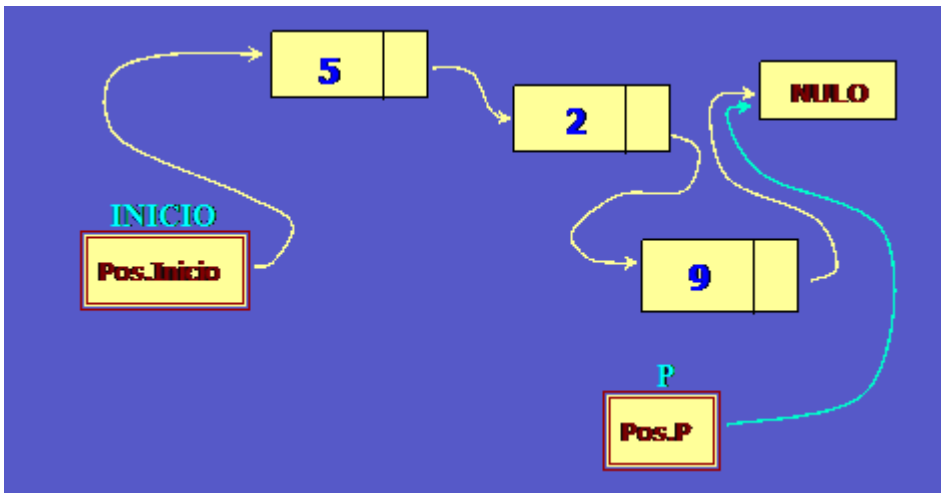

Se aplica el tratamiento a ese nodo y después el puntero avanza hasta el siguiente:




Se aplica el tratamiento al nodo con valor 2 y el puntero avanza:




Se trata el nodo con valor 9 y el puntero avanza, haciéndose nulo, con lo que concluyen las iteraciones:




**Acceso a un elemento de una Lista Lineal Simplemente Enlazada**

Para acceder al I-ésimo elemento de la lista, con I menor o igual que la longitud de la lista, debe recorrerse ésta partiendo del primer nodo, referenciado por el puntero a la estructura, y hasta alcanzar el I-ésimo.

```
MÓDULO Acceder_a_nodo

DATOS
        PARÁMETROS
                Recibe INICIO puntero a NODO_LISTA
                Recibe I entera
        VARIABLES
                J entera
                P puntero a NODO_LISTA
INICIO
        P <- INICIO

        J <- 1
        Mientras (J < I) hacer
                P <- P↑.SIGUIENTE
                J <- J+1
```

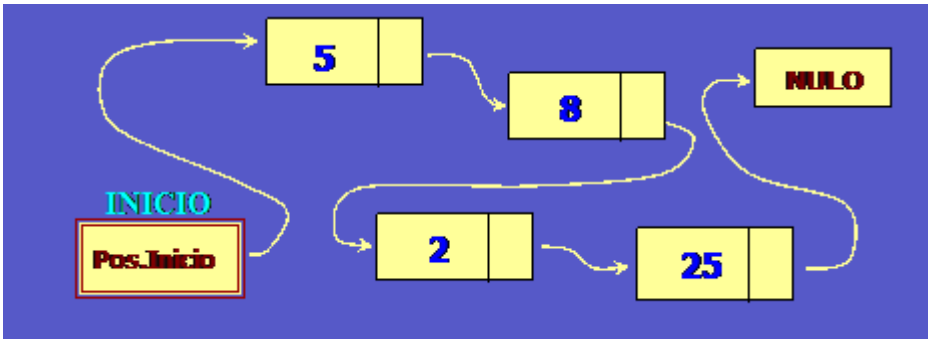
**fin-mientras**

**TRATAR (P)**

**FIN**

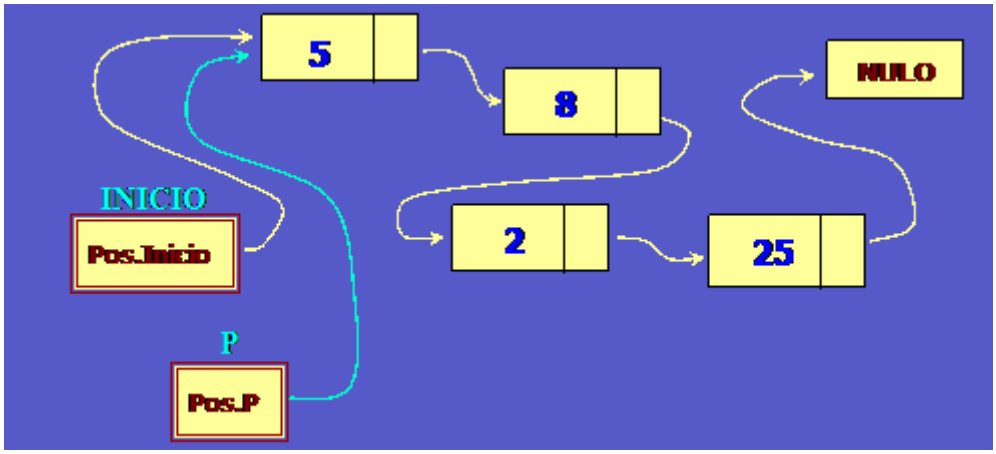
### Ejemplo de una operación de acceso a un elemento

Supongamos que se desea acceder al tercer elemento de la siguiente lista:

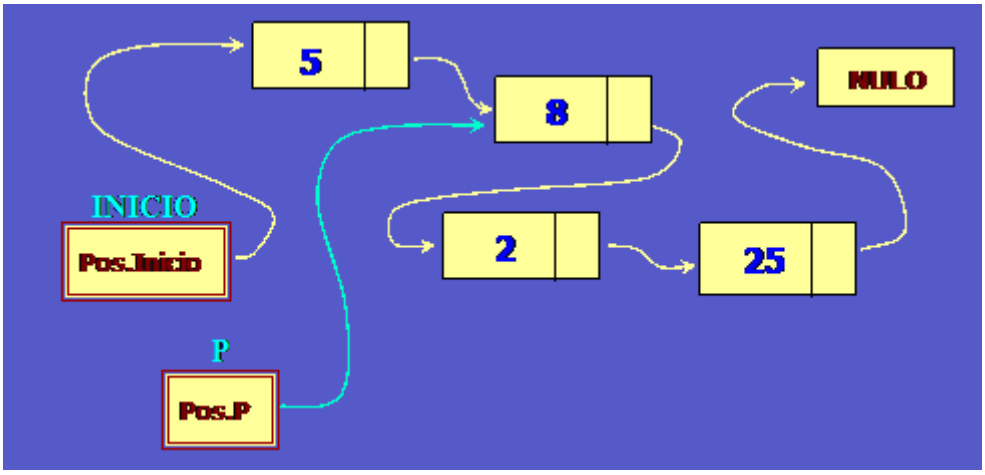



**Acceder_a_nodo(INICIO, 3)**

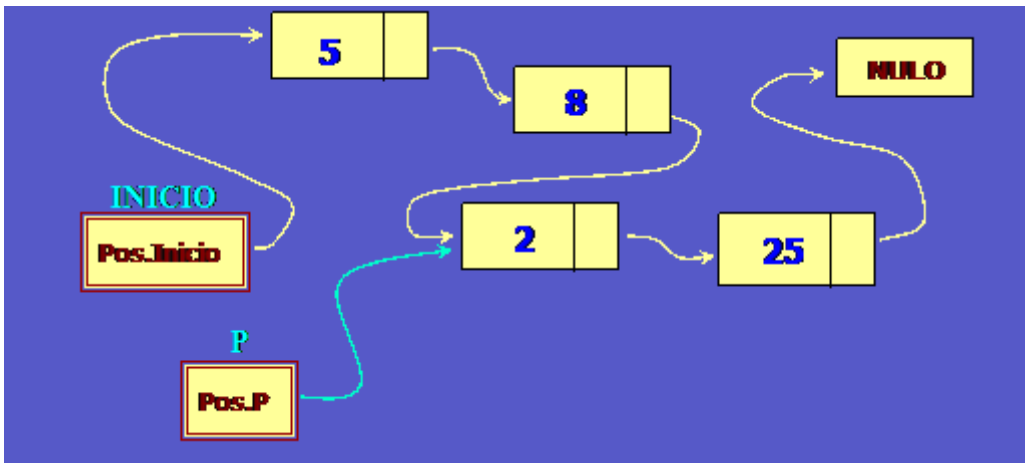
El puntero P apunta inicialmente al primer nodo (que sería el 1):




J se hace 1 y comienzan las iteraciones. En la primera iteración P pasa a apuntar al nodo con valor 8:




J se hace 2 y se pasa a la segunda iteración. P avanza un nodo:




J se hace 3, con lo que las iteraciones concluyen. En este momento P apunta al tercer nodo, por lo que se le puede aplicar el tratamiento elegido (lectura, escritura, modificación,...).

**Búsqueda de un elemento en una Lista Lineal Simplemente Enlazada**

Para localizar un elemento dentro de una lista enlazada no ordenada se realizará una búsqueda secuencial. La búsqueda se suele realizar en función del valor de alguno de los campos del nodo (por ejemplo, en una lista con información sobre personas, en función del nombre o edad de una persona). En nuestro caso supondremos que se realiza en función de todos los campos de datos.

Si la búsqueda tiene éxito, se retornará un puntero al nodo que contiene el valor buscado. En caso contrario, se retornará el valor NULO. Si el valor buscado aparece varias veces en la lista, se encontrará el más próximo al comienzo de la lista.

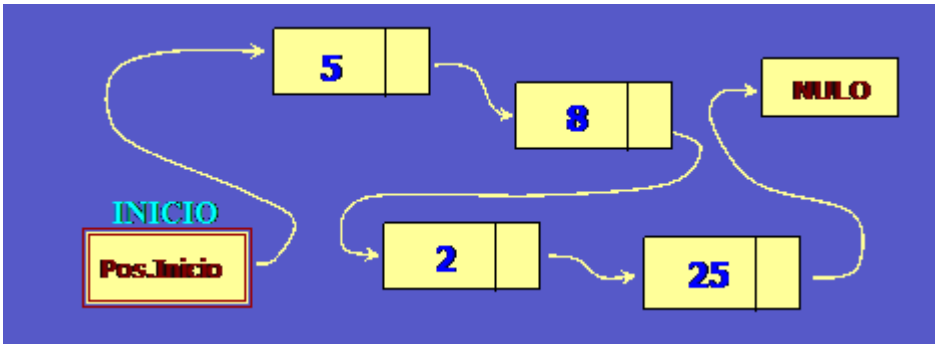
```
MÓDULO Buscar  devuelve puntero a NODO_LISTA
DATOS
        PARÁMETROS
                Recibe INICIO puntero a NODO_LISTA
                Recibe VALOR de tipo T          ** valor que se busca
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        P <- INICIO

        Mientras (P ≠ NULO) hacer
                Si (P↑.DATO = VALOR) entonces
                        DEVOLVER P
                fin-si
                P <- P↑.SIGUIENTE
        fin-mientras

        DEVOLVER NULO
FIN
```

**Ejemplo de operación de búsqueda con éxito**

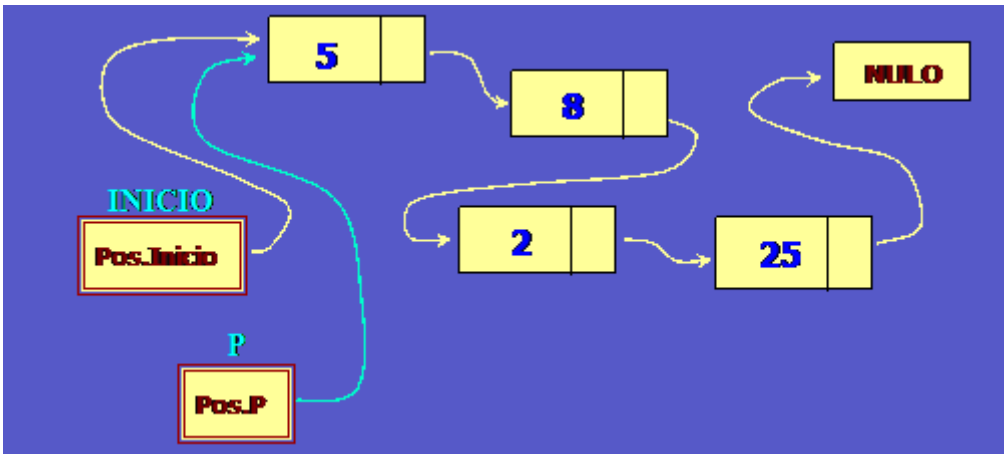
Consideremos la siguiente lista de enteros:



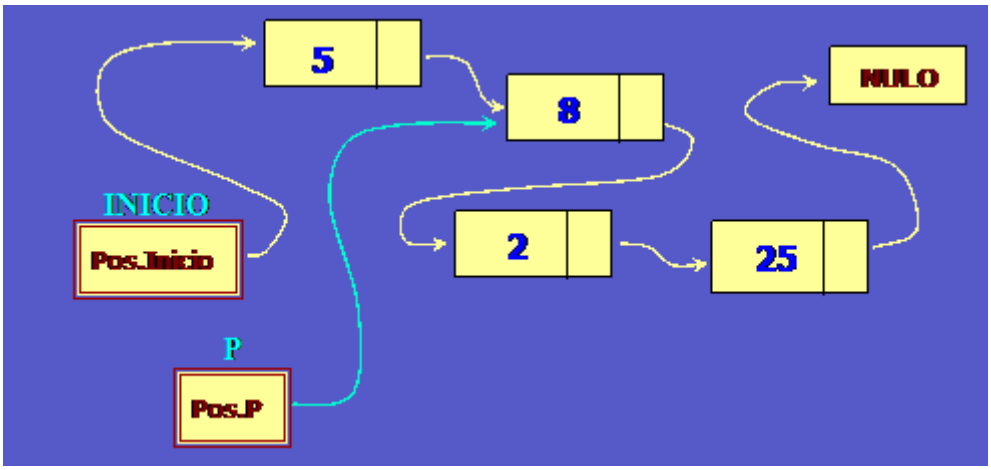

Supongamos que se desea buscar el valor 2:

**Buscar(INICIO, 2)**

El puntero P recorrerá los sucesivos elementos de la estructura, comenzando por el apuntado por **INICIO**:

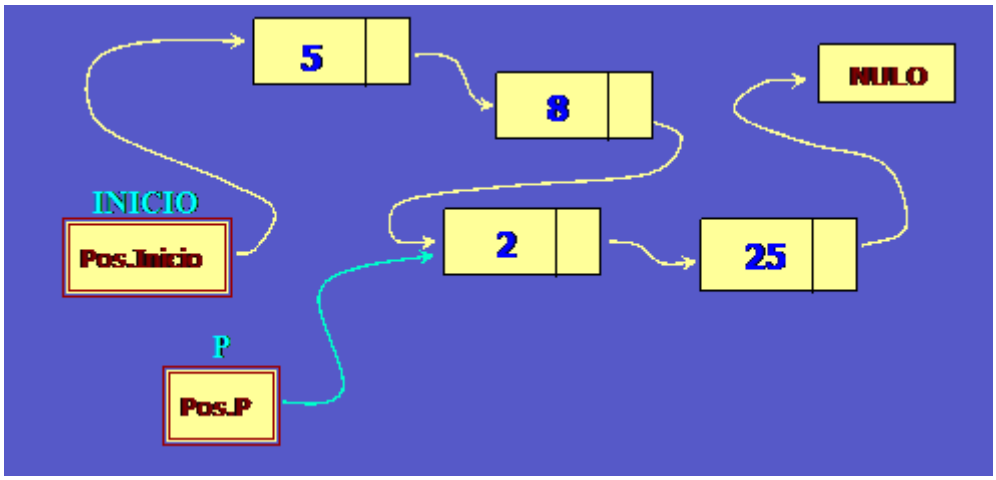



Como el primer nodo no contiene el valor buscado, se avanza el puntero P un nodo:




Como el segundo nodo tampoco contiene el valor buscado, se avanza otra posición en la

lista el puntero P:

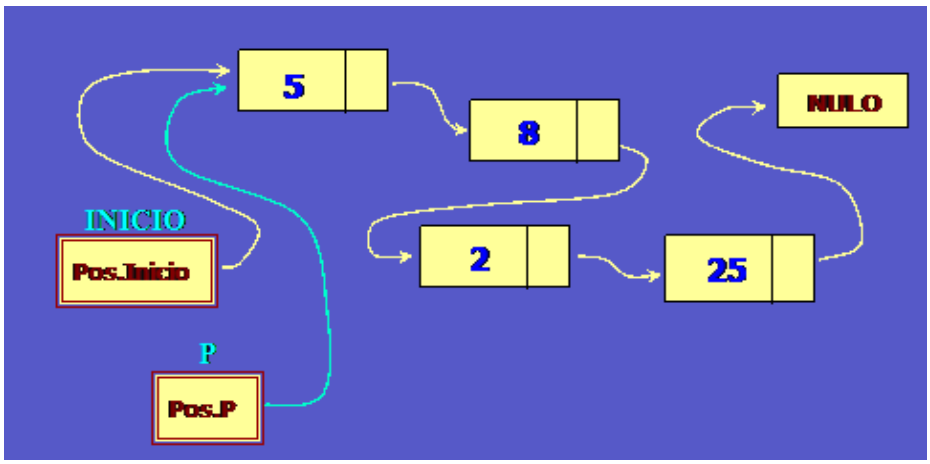



El bucle de búsqueda concluye cuando el puntero P apunta al nodo que contiene el valor buscado.

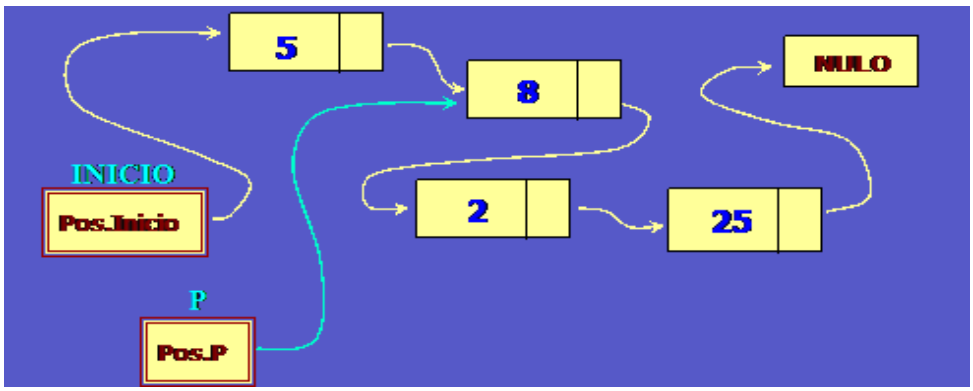
**Ejemplo de operación de búsqueda con fallo**

Si se intenta buscar un valor que no aparece en la lista, por ejemplo, el valor 4, el puntero P recorrerá todos los nodos de la misma, hasta hacerse NULO. Ello indicará que el valor buscado no aparece en la lista.

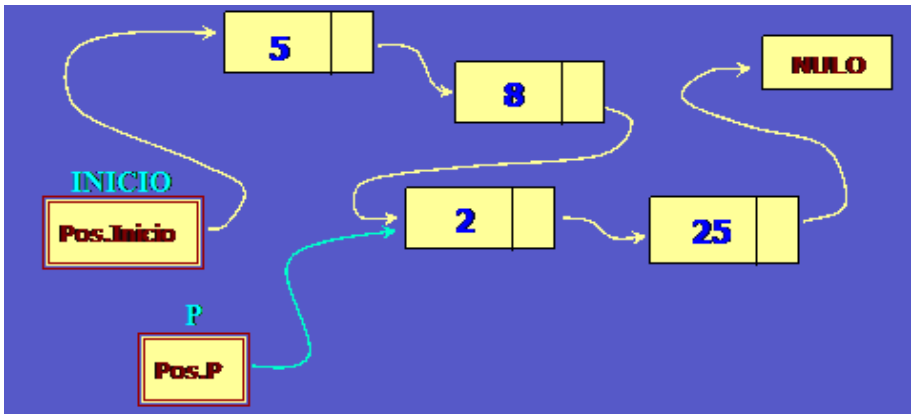
**Buscar(INICIO, 4)**



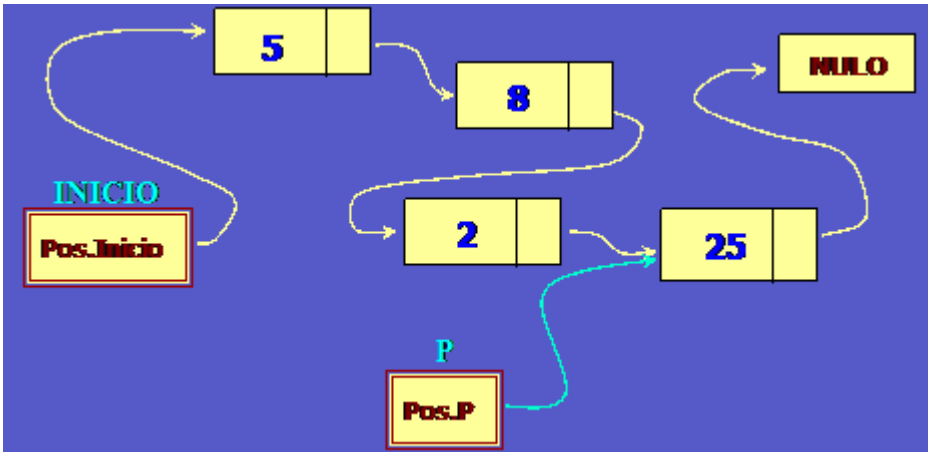

Siguiente paso



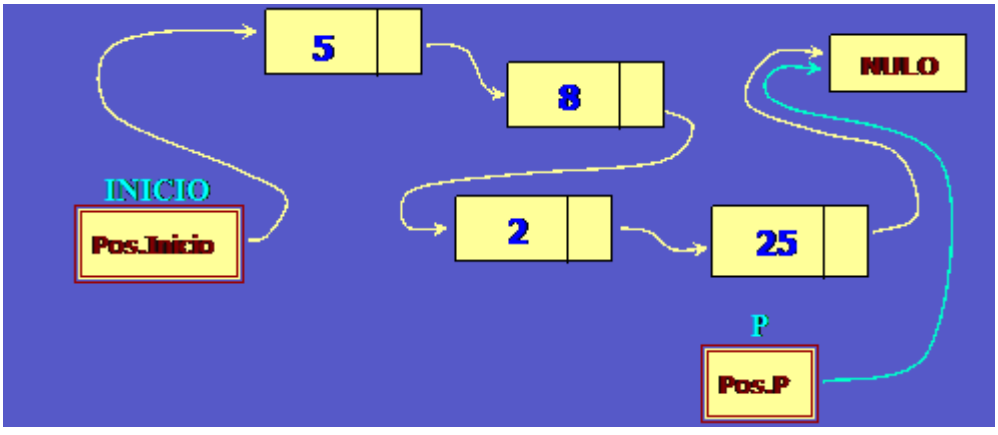

Siguiente paso




Siguiente paso




Siguiente paso



### Vaciado de una Lista Lineal Simplemente Enlazada

Para eliminar la lista se debe liberar la memoria asociada a los nodos de la misma y hacer nulo el puntero a la estructura.

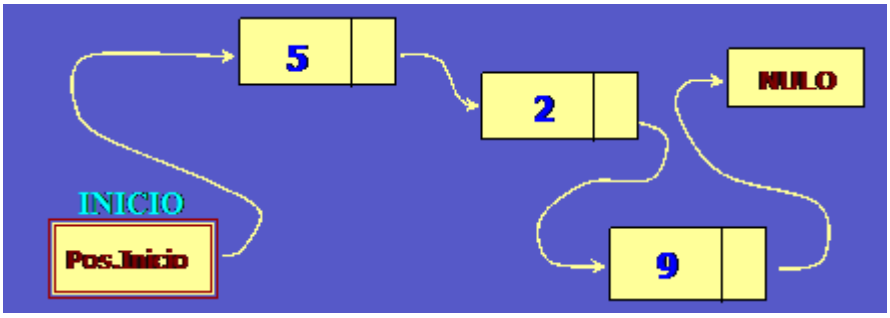
Se recorrerán los nodos de la lista desde el referenciado por el puntero a la estructura hasta el último, liberando la memoria que ocupa cada uno de ellos.

```
MÓDULO Vaciar_lista

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        Mientras (Lista_vacia(INICIO) = Falso) hacer
                P <- INICIO↑.SIGUIENTE
                LIBERAR(INICIO)
                INICIO <- P
        fin-mientras
FIN
```

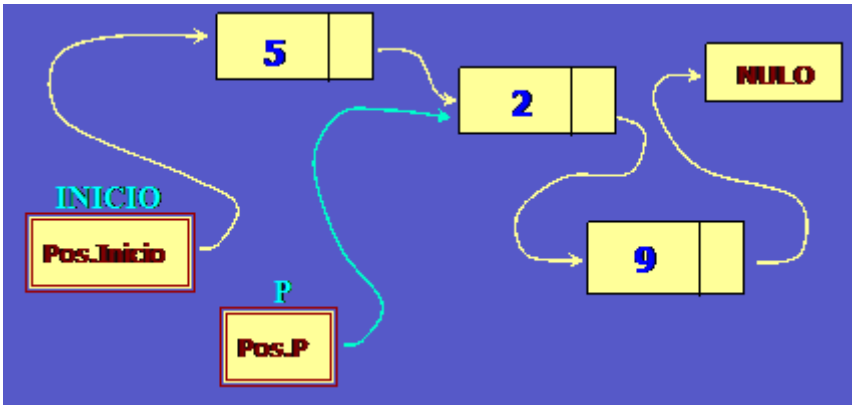
### Ejemplo de operación de vaciado de una Lista Lineal Simplemente Enlazada

Se procede a vaciar la siguiente lista:

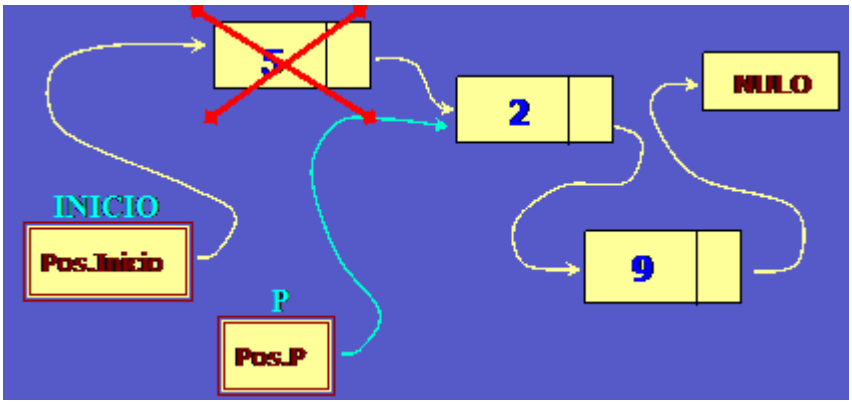



**Vaciar_lista(INICIO)**

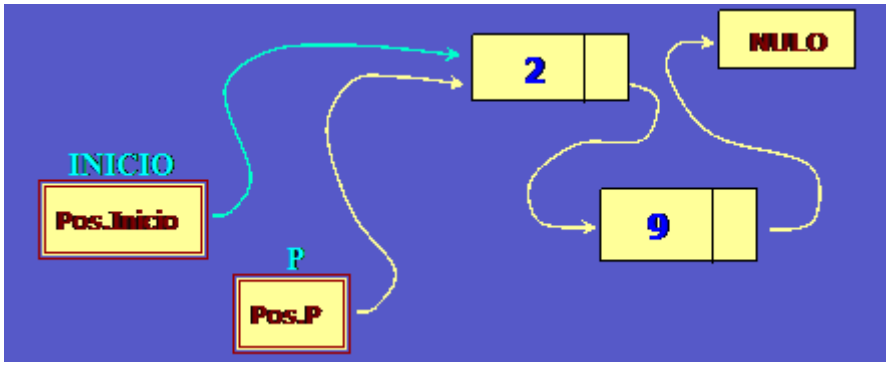
En la primera iteración, P apunta al nodo con valor 2:



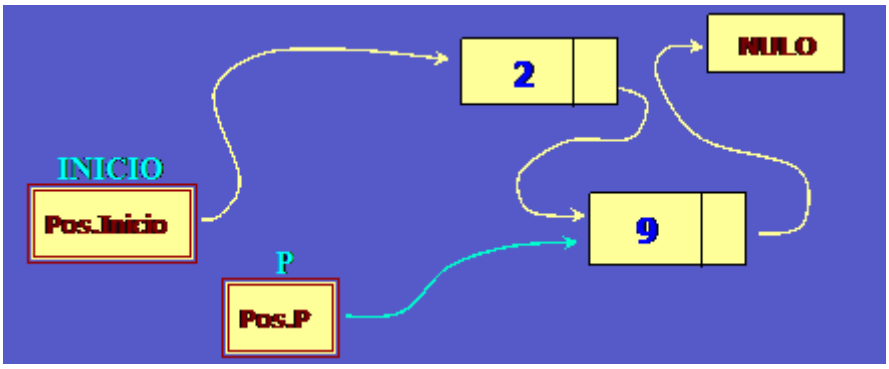
Se libera la memoria asociada al primer nodo:



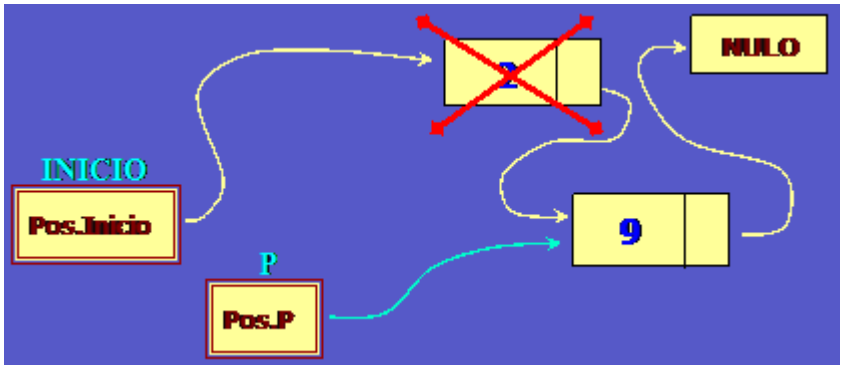

El puntero a la estructura toma el valor de P:



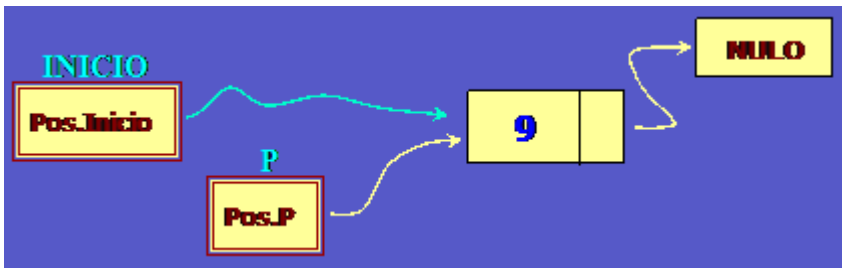

Comienza la segunda iteración. El puntero P avanza hasta el siguiente nodo:



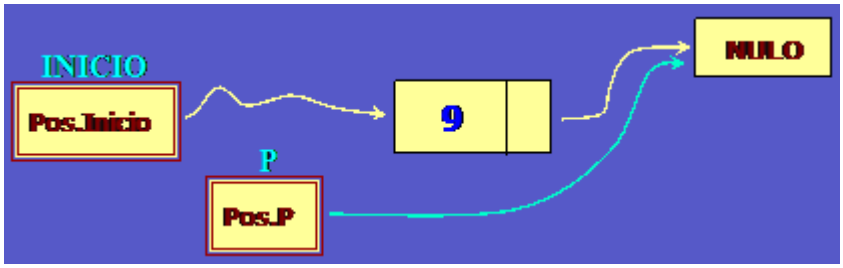

Se libera la memoria ocupada por el nodo con valor 2:



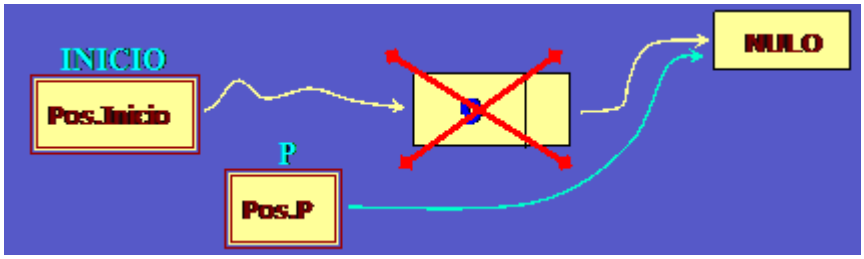

El puntero a la estructura toma el valor de P:



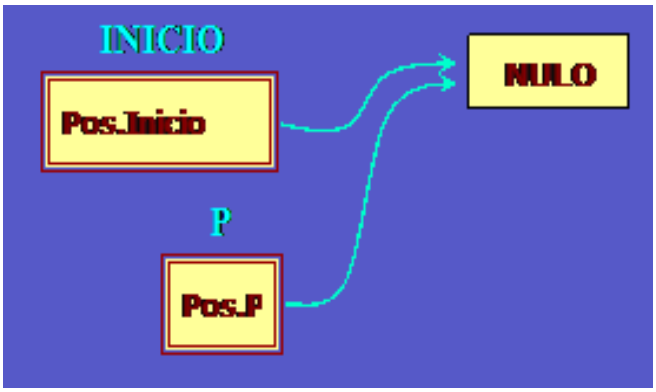

En la tercera iteración P toma valor nulo:




Se libera la memoria ocupada por el nodo con valor 9:

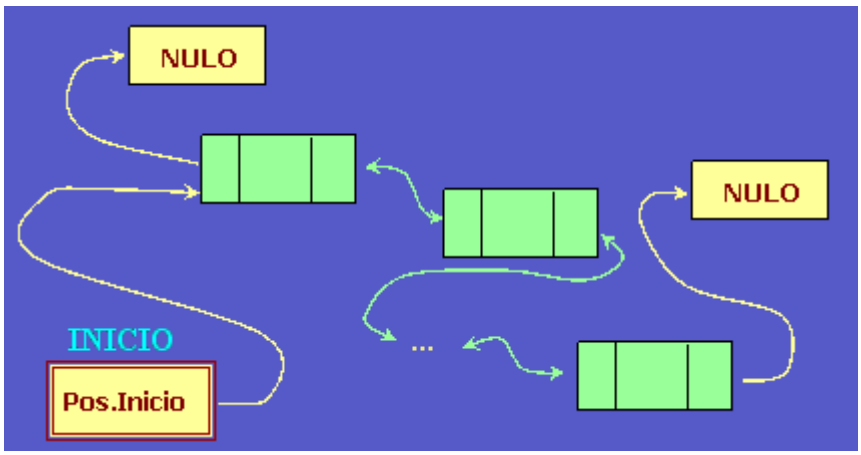



El puntero **INICIO** toma el valor de P, con lo que se establece la condición de lista vacía y las iteraciones concluyen:




## *Lista Lineal Doblemente Enlazada*

Las listas lineales doblemente enlazadas están formadas por nodos con 2 enlaces. Uno de los enlaces del nodo apuntará al nodo siguiente y el otro al anterior, salvo en los nodos de los extremos, que tienen uno de sus enlaces nulo (el primer nodo de la lista no tendrá elemento anterior y el último no tendrá elemento siguiente). Existe un puntero externo que apunta a uno de los extremos de la lista. Por similitud con el estudio de las listas simplemente enlazadas, consideraremos que este puntero apunta al elemento situado en la parte izquierda de la representación gráfica de la lista.




Algunas de las operaciones descritas para listas lineales con simple enlace son directamente aplicables a las listas lineales con doble enlace, por lo que no se volverán a comentar. Dichas operaciones son las siguientes:

- **Comprobación** de lista vacía
- **Recorrido** de la lista
- **Acceso** a un elemento
- **Búsqueda**
- **Vaciado** de la lista

### Creación de la Lista Lineal Doblemente Enlazada

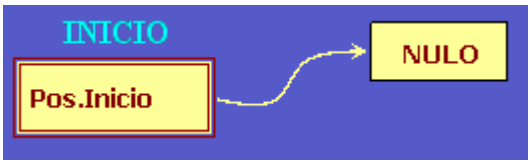
Para crear la lista, en primer lugar se define el tipo de sus nodos:

```
NODO_LISTA es registro compuesto por
        DATO de tipo T
        SIGUIENTE puntero a NODO_LISTA
        ANTERIOR puntero a NODO_LISTA
fin-registro
```

A continuación se define el puntero a la lista y se le da valor inicial nulo, para indicar que la lista está vacía:

```
INICIO puntero a NODO_LISTA

INICIO <- NULO
```




**Representación gráfica de lista lineal con enlace doble vacía**

### Inserción de un elemento en una Lista Lineal Doblemente Enlazada

Para añadir un nuevo elemento a la lista, se especificará el valor de dicho elemento (**ELEMENTO**) y un puntero a uno de los nodos de la lista (**P**), respecto al cuál se realizará la inserción. Como se indicó en el caso de las listas con enlace simple, el nuevo elemento se puede colocar antes o después del nodo identificado por dicho puntero, aunque nosotros analizaremos el segundo caso.

En primer lugar, se solicita memoria para el nuevo nodo de la lista. Si la solicitud no tiene éxito, se presenta un mensaje al usuario y la operación concluye; en caso contrario, se almacena el valor del nuevo elemento en el campo de datos del nodo.

Si la lista esta vacía, los dos campos de enlace del nuevo nodo se harán nulos y el puntero a la lista apuntará a dicho nodo. Cuando la lista está vacía, la inserción se realiza independientemente del valor pasado en el segundo parámetro, lo que permite realizar la primera inserción de datos.

Si la lista no está vacía, el nuevo nodo debe colocarse entre el apuntado por **P** y el siguiente (apuntado por **P↑.SIGUIENTE**). Si **P↑.SIGUIENTE** es NULO, el nuevo nodo queda colocado al final de la lista. Para ello hay que definir los dos campos de enlace del nuevo nodo. Además, se debe redefinir el enlace **SIGUIENTE** del nodo que quedará antes del nuevo y, si el nuevo nodo no va a quedar al final de la lista, el enlace **ANTERIOR** del nodo que va a quedar a continuación de él.

```
MÓDULO Insertar_despues

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T      ** valor a insertar
        VARIABLES

                NUEVO puntero a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO <- ELEMENTO

                Si (Lista_vacia(INICIO) = Verdadero) entonces
                        INICIO <- NUEVO
                        NUEVO↑.ANTERIOR <- NULO
                        NUEVO↑.SIGUIENTE <- NULO
                sino
                        NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
```

```
                        NUEVO↑.ANTERIOR <- P

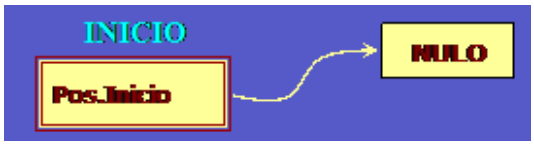
                        Si (P↑.SIGUIENTE ≠ NULO) entonces
                            P↑.SIGUIENTE↑.ANTERIOR <- NUEVO

                        fin-si

                        P↑.SIGUIENTE <- NUEVO
                    fin-si
                fin-si
            FIN
```

**CASO: INSERCIÓN EN LISTA VACÍA**

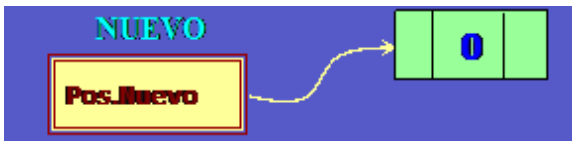
Supongamos una lista de enteros vacía:



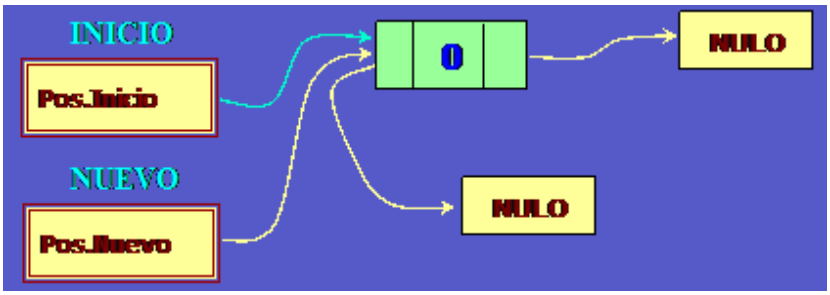

en la que deseamos insertar el valor 0:

**Insertar_despues(INICIO, P, 0)**

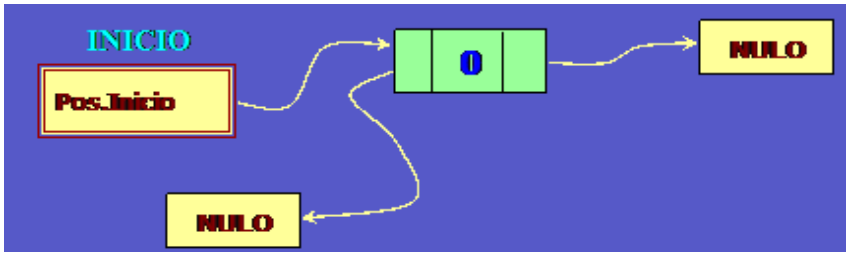
Tras reservar memoria para el nuevo nodo, se almacena el dato en el campo de datos:




A continuación se definen los enlaces, siendo ambos nulos dado que la lista estaba vacía. Además, se hace que el puntero a la lista pase a apuntar al nuevo nodo:

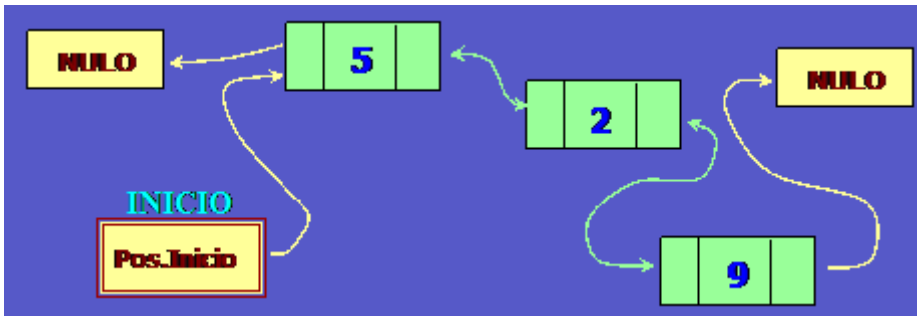



La lista quedará como sigue:

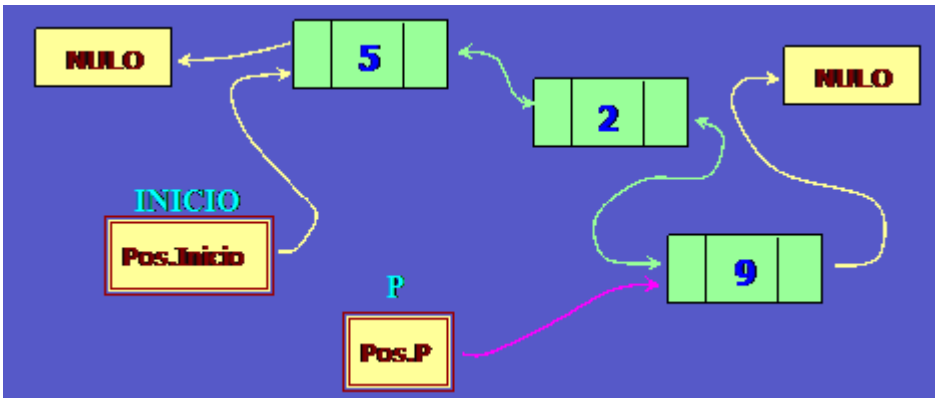



**CASO: INSERCIÓN DESPUÉS DEL ÚLTIMO NODO DE LA LISTA**

Consideremos la lista de enteros que se muestra en la siguiente figura:

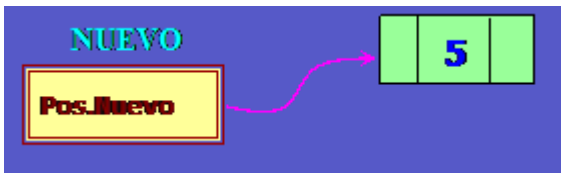



en la que deseamos insertar el dato 5 a continuación del nodo de la lista que almacena el 9. Por tanto, el puntero que identifica el punto de inserción debe apuntar al último nodo de la lista:

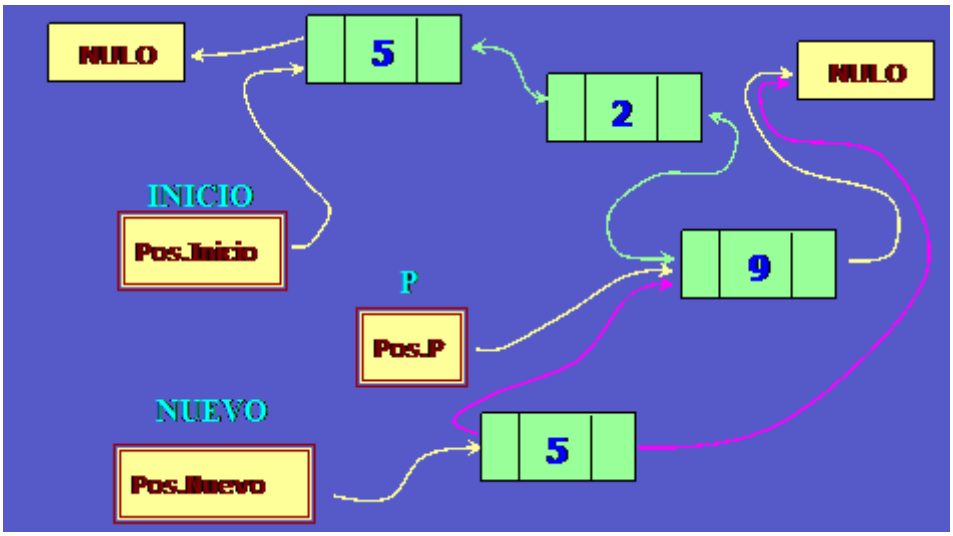



**Insertar_despues(INICIO, P, 5)**

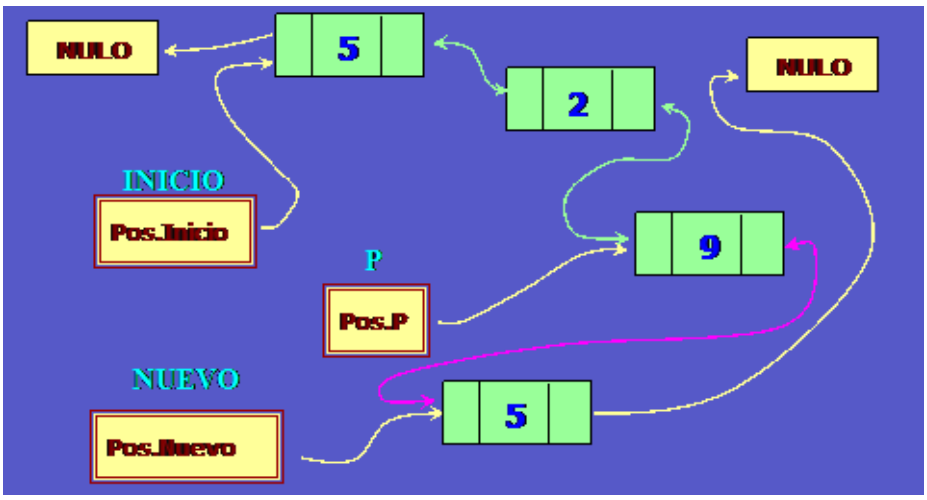
Tras obtener memoria para el nuevo nodo, se almacena el dato en el campo de datos:



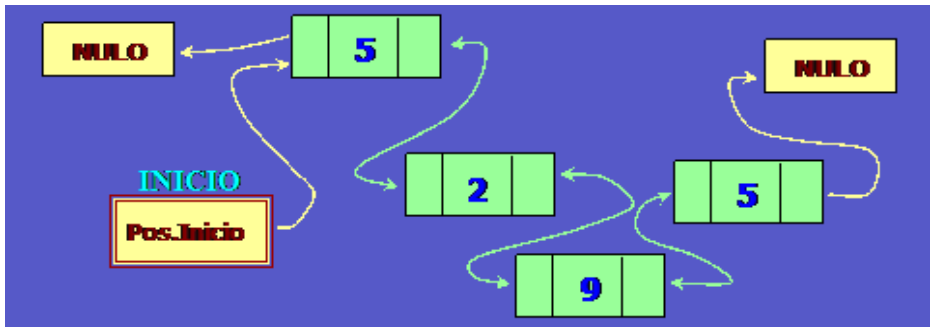

A continuación se definen los enlaces del nuevo nodo:




Se redefine el enlace **SIGUIENTE** del nodo apuntado por **P**, de forma que pase a apuntar al nuevo nodo:

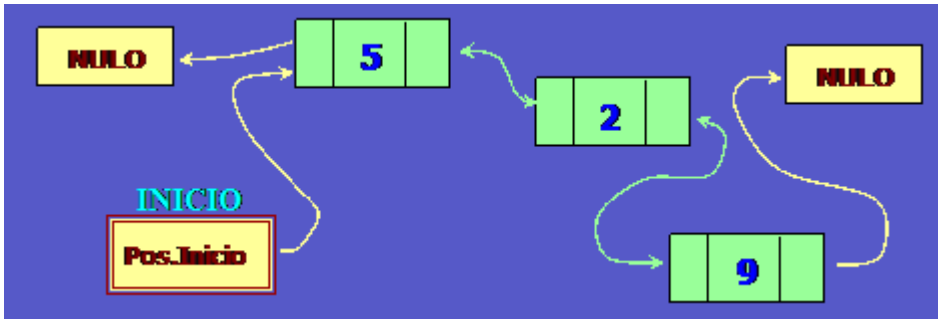



La lista quedaría como sigue:

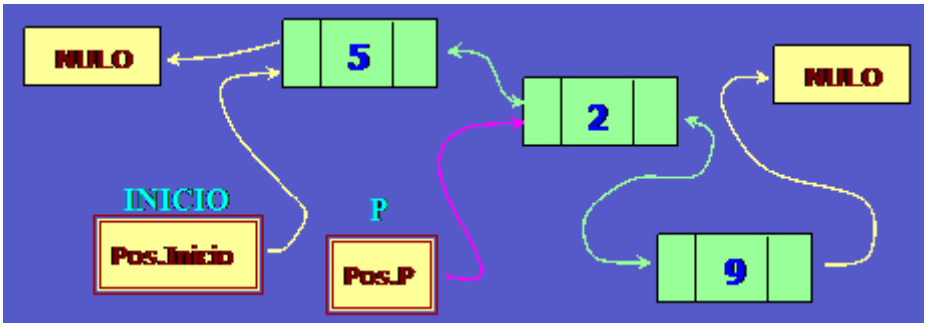



**CASO: INSERCIÓN EN UNA POSICIÓN INTERMEDIA DE LA LISTA**

Consideremos la lista de enteros que se muestra en la siguiente figura:



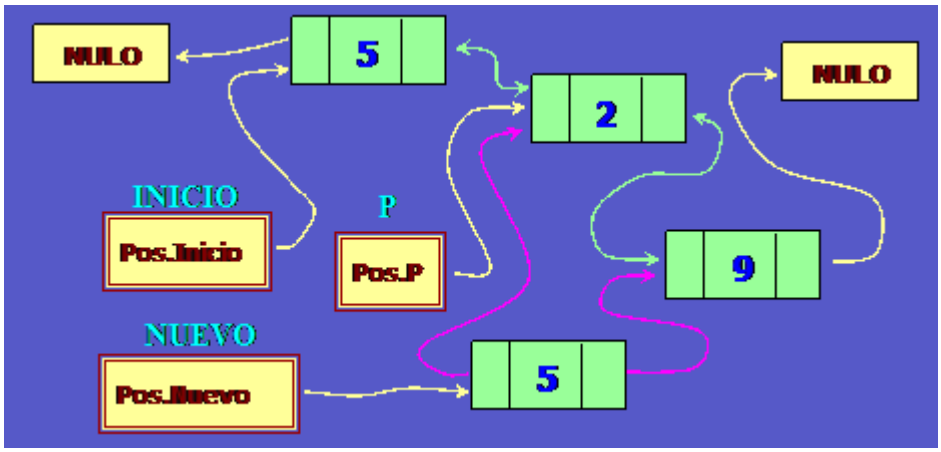

Insertamos el valor 5 a continuación del 2, estando éste identificado por el puntero que apunta al punto de inserción:



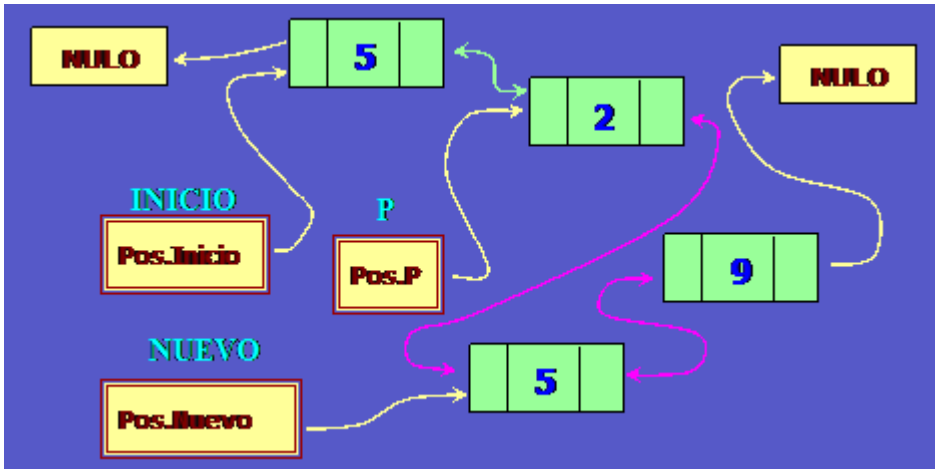

**Insertar_despues(INICIO, P, 5)**

Igual que en los casos anteriores, tras reservar memoria para el nuevo nodo, se almacena

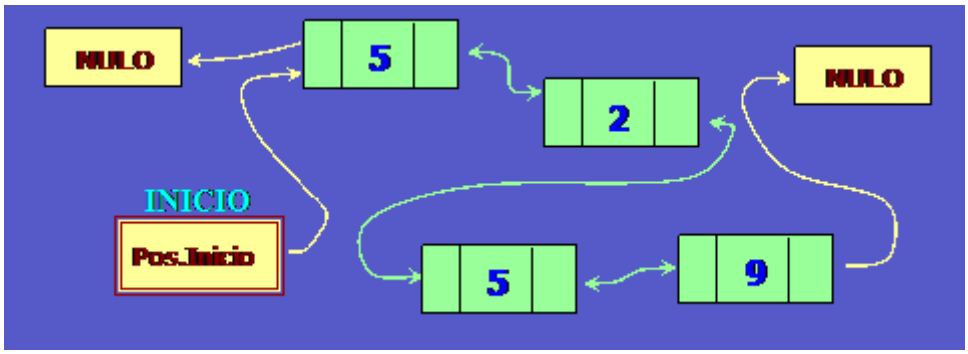
el dato en el campo de datos. A continuación se definen los enlaces del nuevo nodo:




Como el nuevo nodo no va a ser el último de la estructura, en primer lugar se redefine el enlace **ANTERIOR** del nodo que quedará a continuación del nuevo y, a continuación, el enlace **SIGUIENTE** del nodo apuntado por **P**:

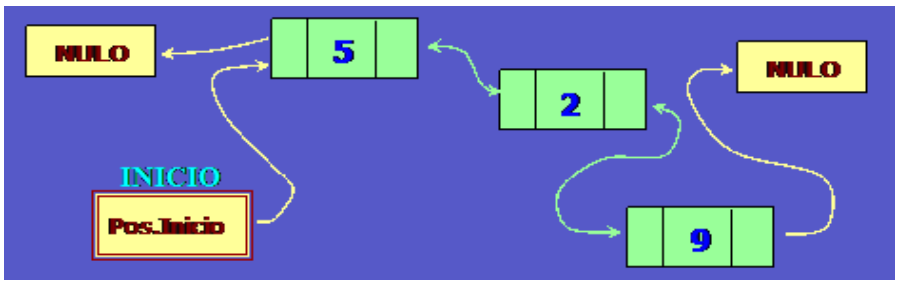



La lista resultante será:

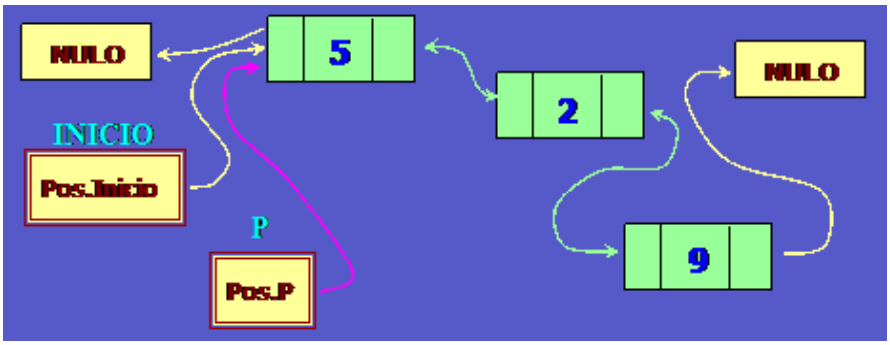



**CASO: INSERCIÓN DESPUÉS DEL PRIMERO DE LA LISTA**

Consideremos la lista de enteros que se muestra en la siguiente figura:

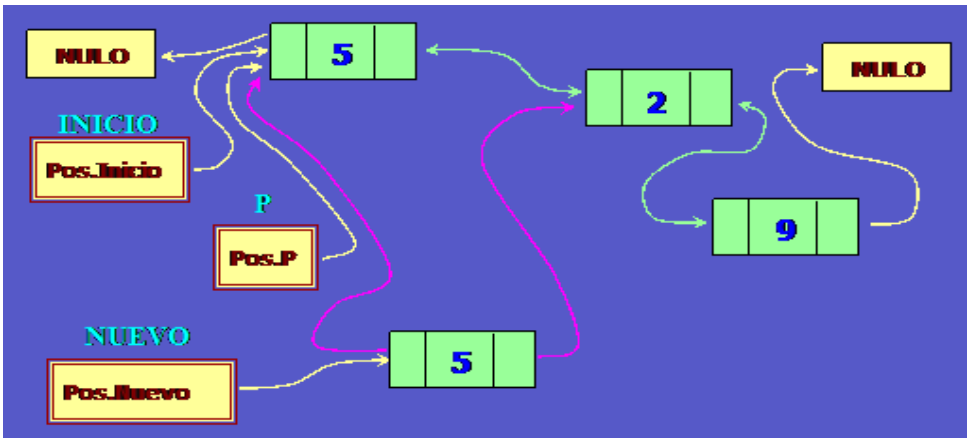



Se inserta ahora un valor respecto al primero de la estructura. Como en los casos anteriores el puntero **P** apunta al punto de inserción:

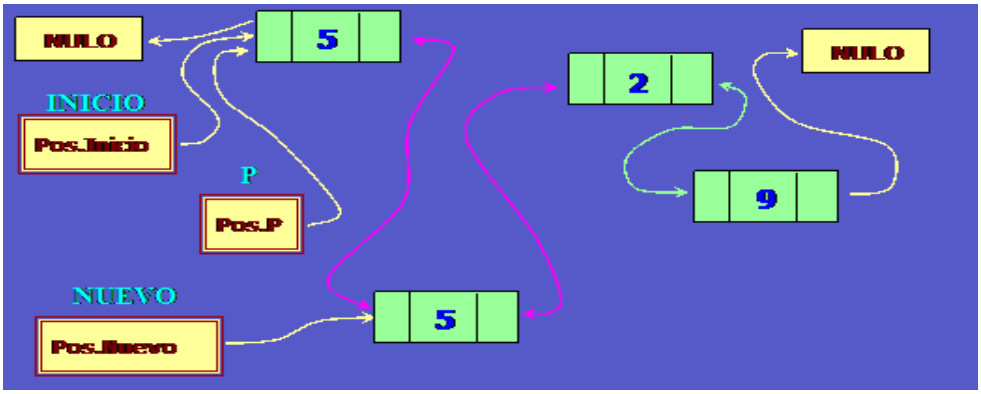



**Insertar_despues(INICIO, P, 5)**

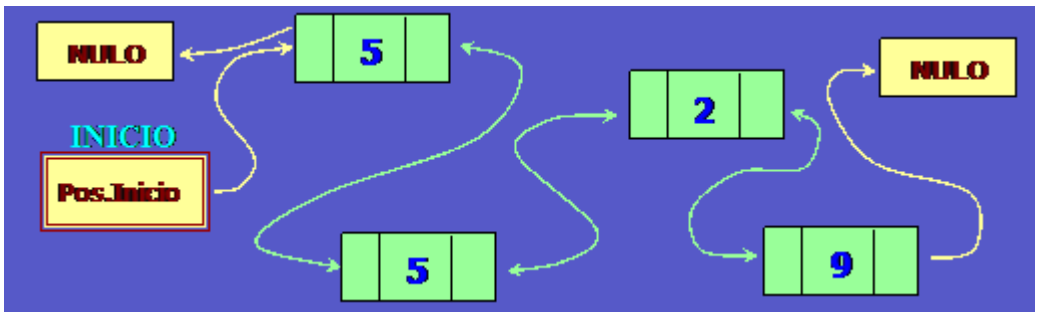
Tras obtener la memoria para el nuevo nodo, se da valor a todos sus campos:




Se conectan después con el nuevo nodo el nodo apuntado por **P** y el que le sigue (apuntado por **Pá.SIGUIENTE**):




La lista resultante es:




**Eliminación de un elemento en una Lista Lineal Doblemente Enlazada**

**Eliminación de un elemento en una Lista Lineal Doblemente Enlazada (por su posición)**
Para eliminar de la lista el nodo referenciado por el puntero **P**, en primer lugar se comprueba que la lista no esté vacía. En caso de estarlo, se anula la operación de eliminación. En caso de no estarlo, antes de liberar la memoria ocupada por el nodo a eliminar, se redefinen los enlaces de los nodos que vayan a permanecer en la estructura.

Si se va a eliminar el primer nodo de la lista, el puntero a la misma debe pasar a apuntar al que era el segundo nodo de la lista en este momento. Si la lista sólo contenía un nodo, el puntero a la lista debe hacerse **NULO**.

Si el nodo que se elimina no es el primero de la lista, tiene predecesor; si no es el último de la lista, tiene sucesor. Si el nodo que se elimina tiene predecesor, se hace que dicho predecesor se enlace con el nodo que sigue al que se va a eliminar (se redefine el valor de su campo **SIGUIENTE**). Si el nodo que se elimina tiene sucesor, se hace que dicho sucesor se enlace con el nodo que precede al que se va a eliminar (se redefine el valor de su campo **ANTERIOR**).

Una vez redefinidos los enlaces necesarios, se libera la memoria asociada al nodo que se debe eliminar.

```
MÓDULO Eliminar

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
INICIO
        Si (Lista_Vacia(INICIO) = Verdadero) entonces
                ESCRIBIR "Error: Lista vacía"
        sino
                Si (P = INICIO) entonces
                        INICIO <- INICIO↑.SIGUIENTE
                fin-si

                Si (P↑.ANTERIOR ≠ NULO) entonces
                        P↑.ANTERIOR↑.SIGUIENTE <- P↑.SIGUIENTE
```

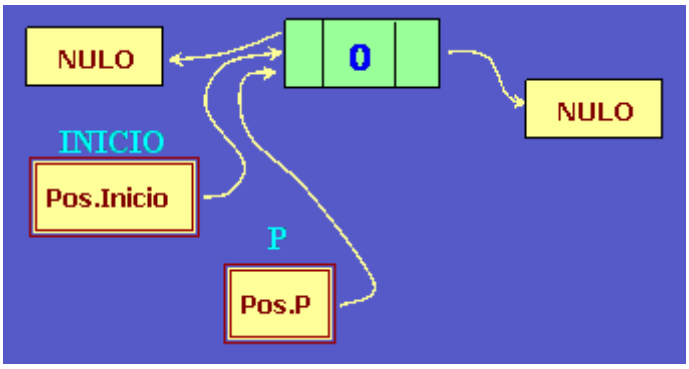
```
                    fin-si

                    Si (P↑.SIGUIENTE ≠ NULO) entonces
                        P↑.SIGUIENTE↑.ANTERIOR <- P↑.ANTERIOR
                    fin-si

                    LIBERAR(P)
                fin-si
        FIN
```

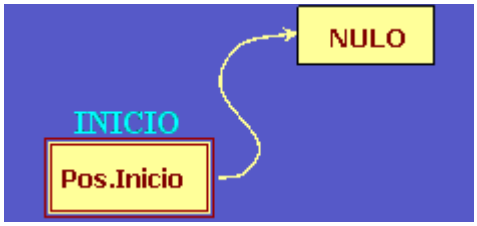
**CASO: SE ELIMINA EL ÚNICO NODO DE UNA LISTA**

Procedemos a eliminar el único nodo de la siguiente lista de enteros:

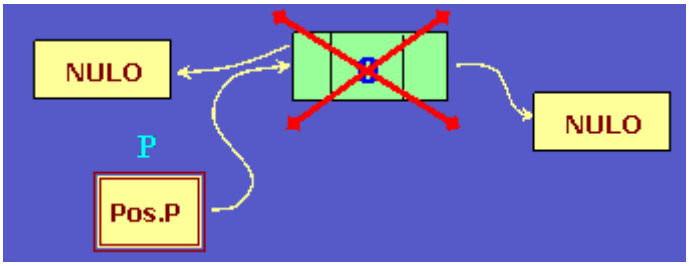



**Eliminar (INICIO, P)**

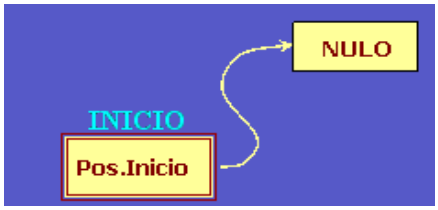
En primer lugar el puntero que da acceso a la estructura se hace NULO:




A continuación, se libera la memoria asociada al nodo:

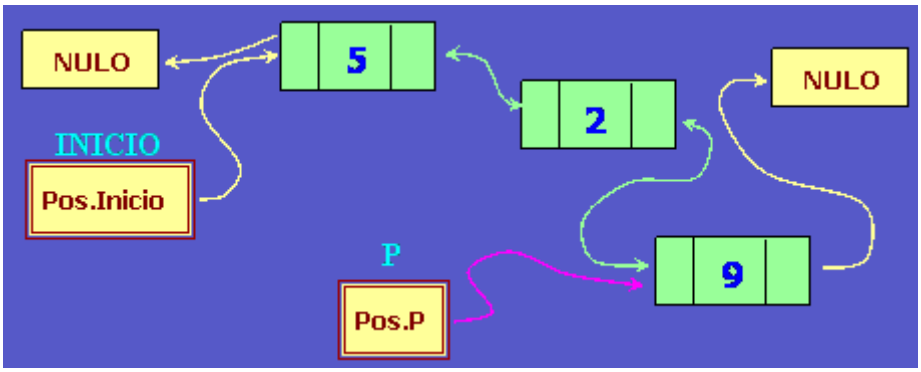



Tras esta eliminación la lista ha quedado vacía.



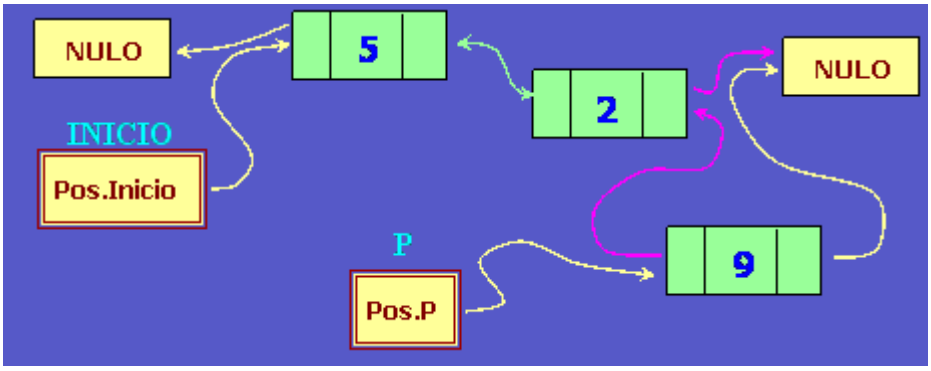

**CASO: SE ELIMINA EL ÚLTIMO NODO DE LA LISTA**

Se elimina el último nodo (el apuntado por **P**) de la siguiente lista:

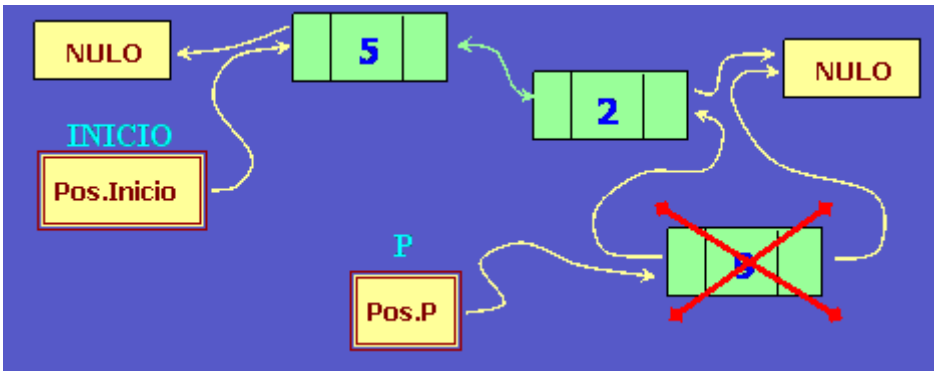



**Eliminar(INICIO, P)**

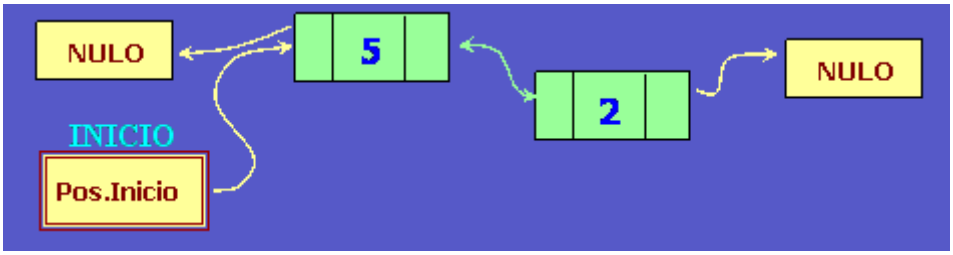
En primer lugar se redefine el enlace SIGUIENTE del nodo previo al que se va a eliminar:




A continuación, se libera la memoria asociada al nodo que se va a eliminar:

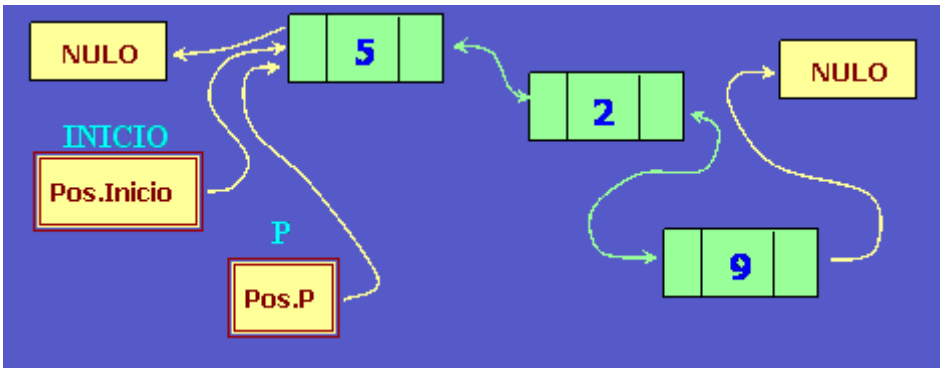



La lista habrá quedado con dos nodos:



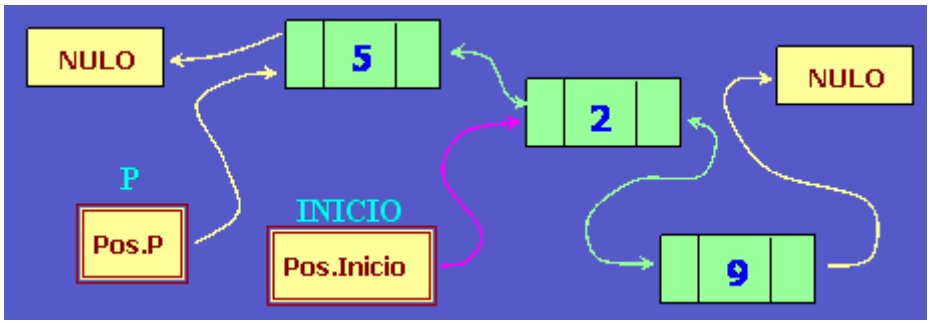

**CASO: SE ELIMINA EL PRIMER NODO DE LA LISTA**

Eliminamos ahora el nodo referenciado por el puntero que da acceso a la estructura:

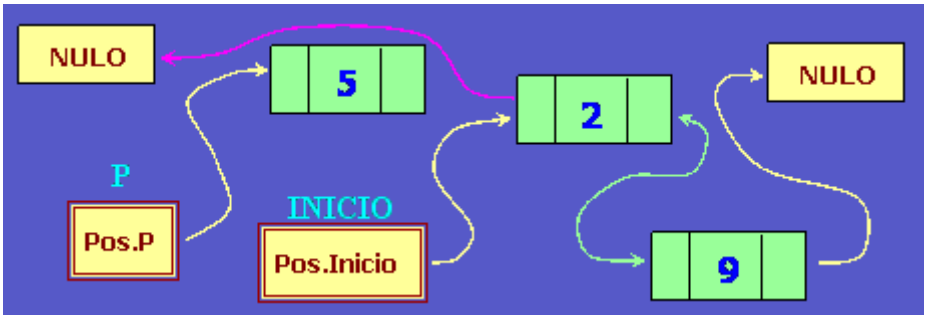



**Eliminar(INICIO, P)**

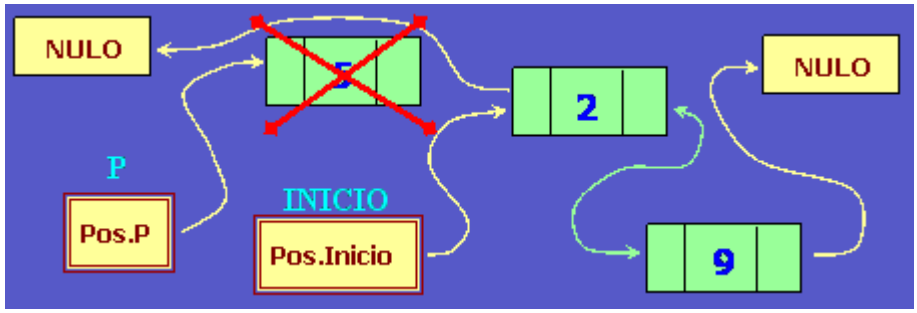
En primer lugar se hace que el puntero a la lista pase a apuntar al segundo elemento:



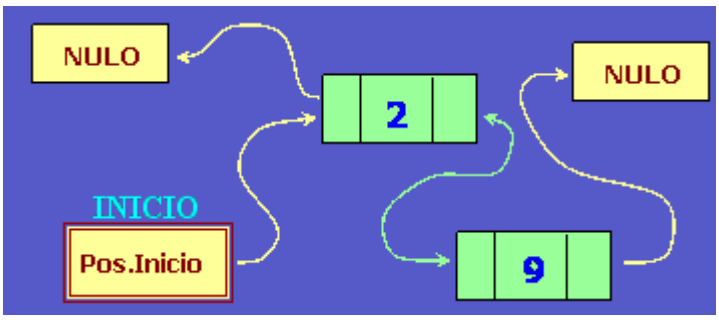

A continuación se redefine el enlace **ANTERIOR** del segundo nodo de la estructura:




Por último, se libera la memoria asociada al primer nodo de la lista:

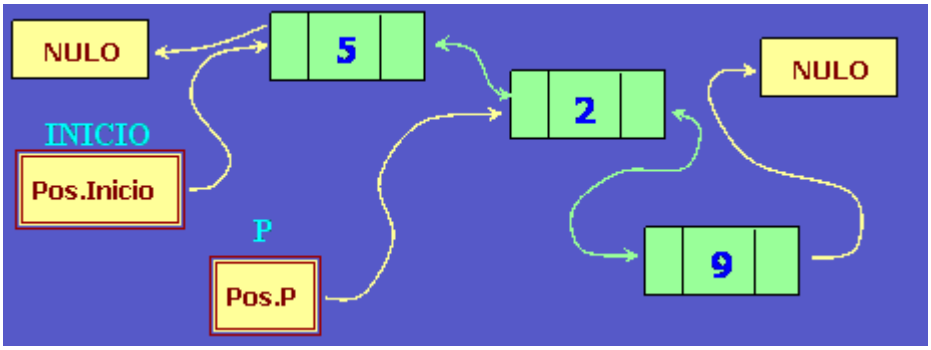



Se obtiene una lista con dos elementos:



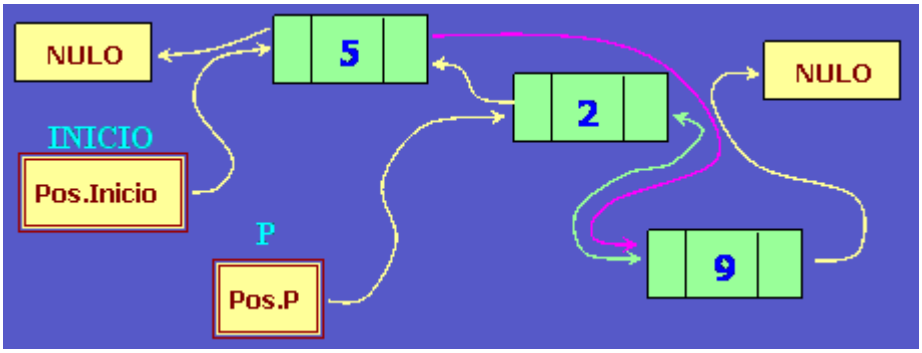

**CASO: SE ELIMINA UN NODO INTERMEDIO DE LA LISTA**

Por último, eliminamos un nodo intermedio de la estructura, estando éste referenciado por el puntero **P**:

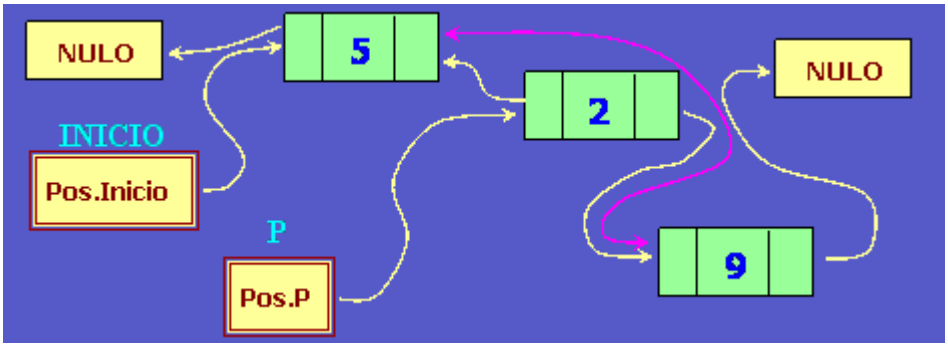



**Eliminar(INICIO, P)**

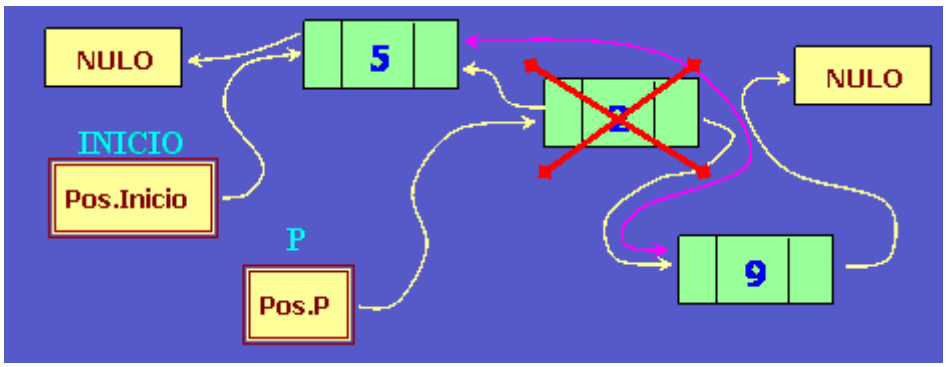
Como el elemento que se elimina no es el primero, se redefine el enlace **SIGUIENTE** del nodo previo al identificado por **P**:



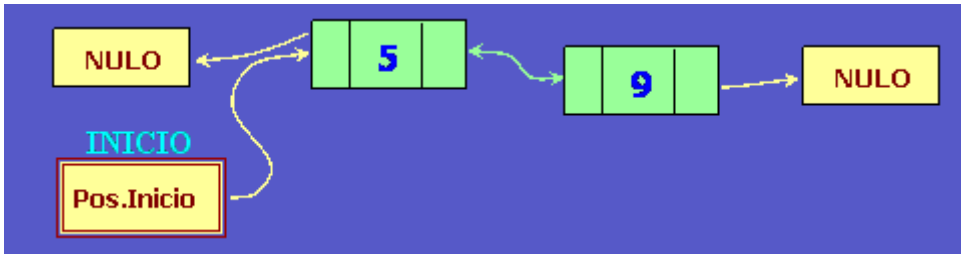

Como el elemento que se elimina no es el último, se redefine el enlace **ANTERIOR** del nodo siguiente al identificado por **P**:




A continuación se libera la memoria asociada al nodo que se desea eliminar de la lista:



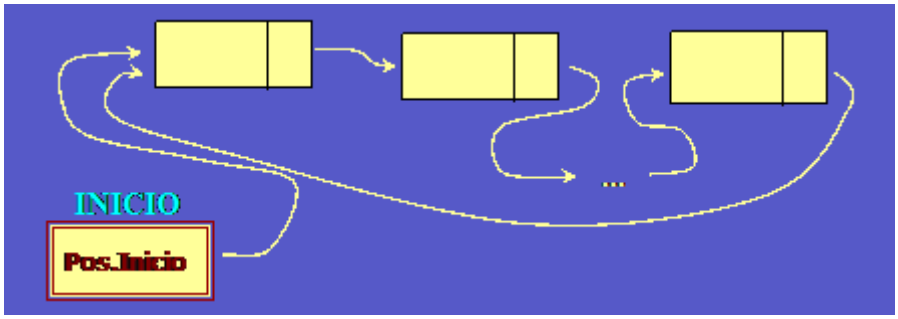

La lista quedará con dos nodos:




**Eliminación de un elemento en una Lista Lineal Doblemente Enlazada (por su contenido)**
Esta operación se puede realizar en función de los módulos de búsqueda y de eliminación por posición, de la misma forma que se comentó en el caso de la lista lineal con enlace simple.

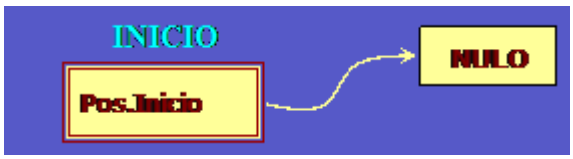
## *Lista Circular Simplemente Enlazada*

Una lista circular con enlace simple está formada por un conjunto de nodos que incluyen un campo de enlace que permite conectar a cada nodo de la lista con otro nodo de la misma. Existe un puntero a uno de los nodos de la lista que da acceso a la estructura.




Dado que este tipo de lista comparte la estructura de los nodos con la lista lineal con enlace simple, muchas de las operaciones son similares a las ya descritas para listas lineales. La diferencia fundamental reside en que las listas que ahora nos ocupan no incluyen ningún nodo con enlace nulo.

**Creación de la Lista Circular Simplemente Enlazada**
La operación de creación de una lista circular con enlace simple es idéntica a la comentada para una lista lineal con enlace simple:




**Representación gráfica de lista circular vacía**

### Inserción de un elemento en una Lista Circular Simplemente Enlazada

Para insertar un nuevo valor en la lista, además de dicho valor, se proporcionará un puntero **P** que determina la posición de la lista en la que se realizará la inserción. Si la lista está vacía, no se tiene en cuenta el valor de **P**. En caso contrario, **P** apuntará a un nodo de la lista y la inserción del nuevo elemento se realizará a continuación del nodo referenciado por **P**.

Una vez que se dispone de la memoria para el nuevo nodo de la lista, se almacena en dicho nodo el dato que debe contener. Para enlazar el nuevo nodo con el resto de elementos de la lista, se tiene en cuenta si la lista estaba vacía o no.

Si la lista estaba vacía, el nuevo nodo apuntará a sí mismo, al igual que el puntero que da acceso a la lista.

Si la lista no estaba vacía, el puntero a la misma no se verá afectado por la operación de inserción. El nuevo nodo debe colocarse entre el referencia por **P** y el siguiente, referenciado por **P↑.SIGUIENTE**. Si la lista contiene un único nodo, las referencias **P** y **P↑.SIGUIENTE** coinciden, por lo que el único nodo de la lista será tanto el predecesor como el sucesor del nuevo.

```
MÓDULO Insertar_despues

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T          *valor a insertar
        VARIABLES
                NUEVO puntero a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

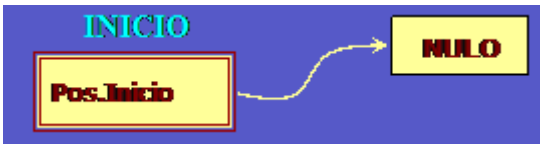
        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO <- ELEMENTO

                Si (Lista_vacia(INICIO) = Verdadero) entonces
                        NUEVO↑.SIGUIENTE <- NUEVO
                        INICIO <- NUEVO
                sino
```

```
                        NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                        P↑.SIGUIENTE <- NUEVO
                fin-si
        fin-si
FIN
```

**CASO: INSERCIÓN EN UNA LISTA VACÍA**

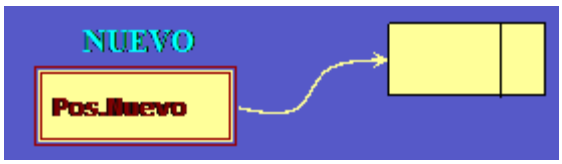
Consideremos una lista de enteros vacía, a la que se hará referencia mediante el puntero **INICIO**:



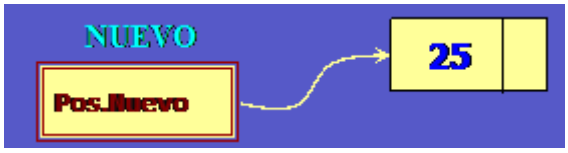

en la que se desea insertar el valor 25:

**Insertar_despues(INICIO, INICIO, 25)**

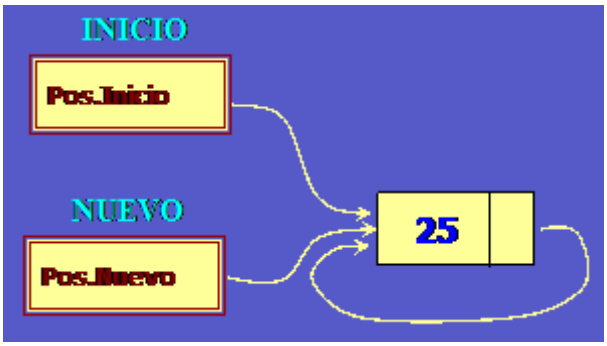
En primer lugar se solicita memoria para el nuevo nodo:



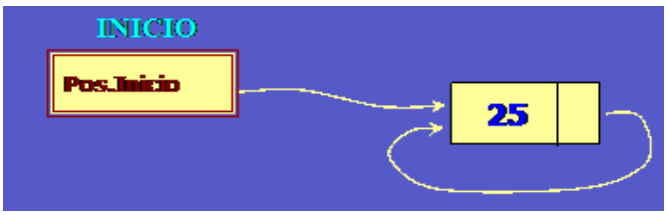

Se copia el valor 25 en el campo de datos:




Como la lista está vacía, tanto el campo de enlace del nuevo nodo como el puntero que da acceso a la estructura apuntarán al nuevo nodo.

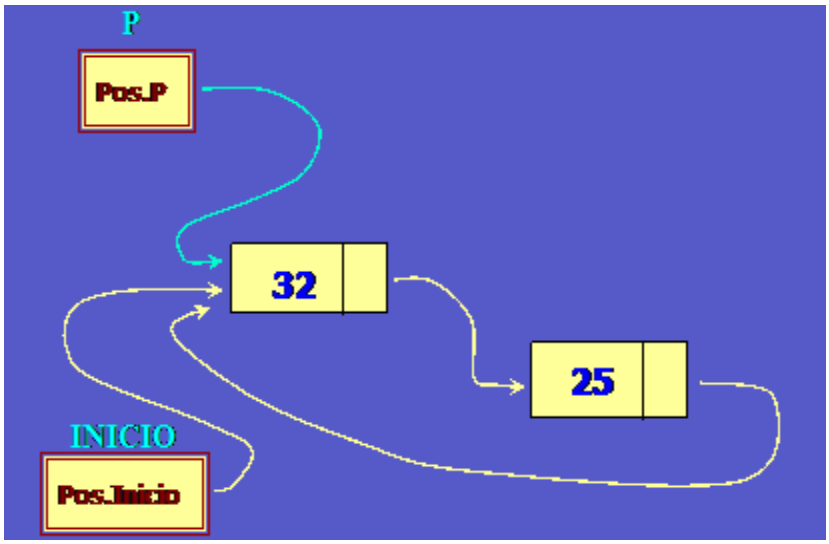



La lista resultante es:



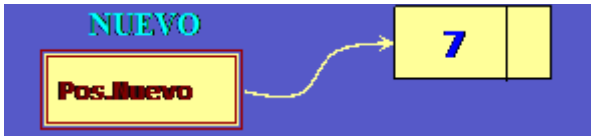
**CASO: INSERCIÓN DESPUÉS DEL NODO APUNTADO POR INICIO**

En la siguiente lista añadimos el valor 7 a continuación del 32:

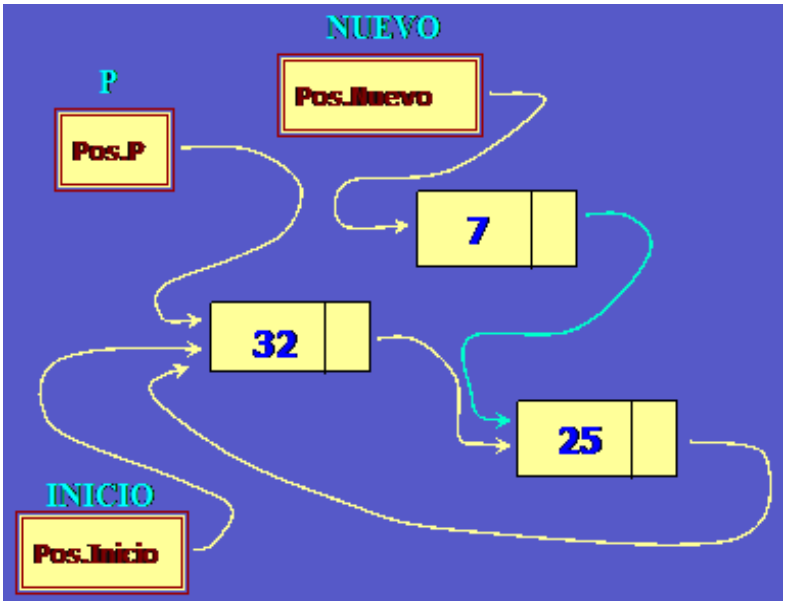



**Insertar_despues(INICIO, P, 7)**

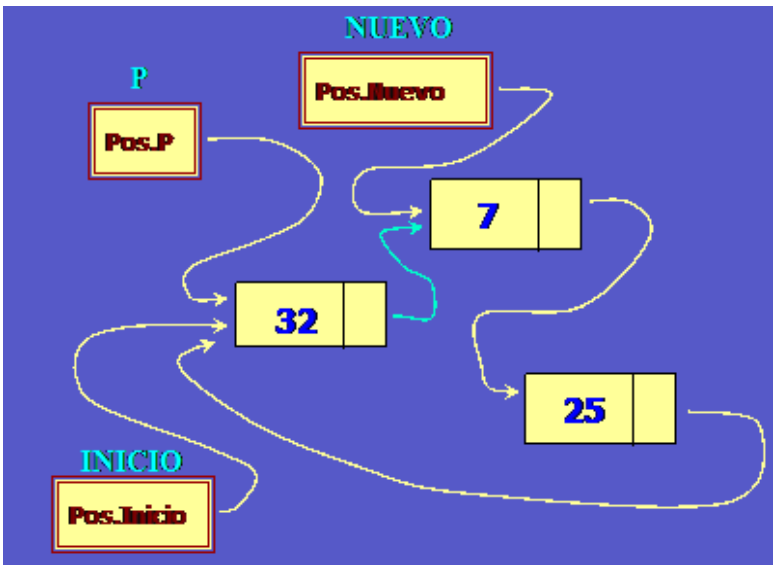
Una vez obtenida la memoria para el nuevo nodo, se copia en éste el valor 7:



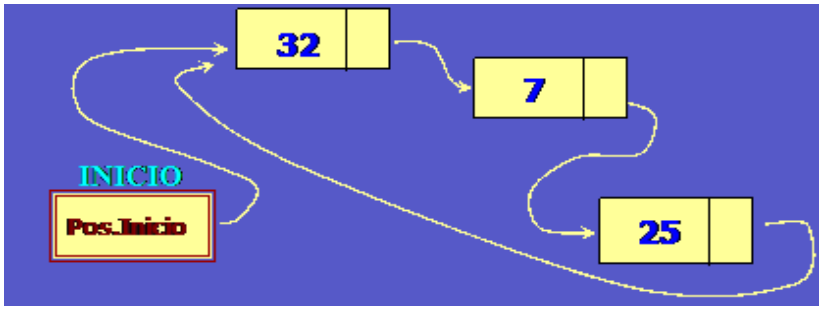

Como la lista no está vacía, se define en primer lugar el campo de enlace del nuevo nodo, de forma que apunte al nodo con valor 25:




Por último, se hace que el nodo apuntado por **P** pase a apuntar al nuevo:

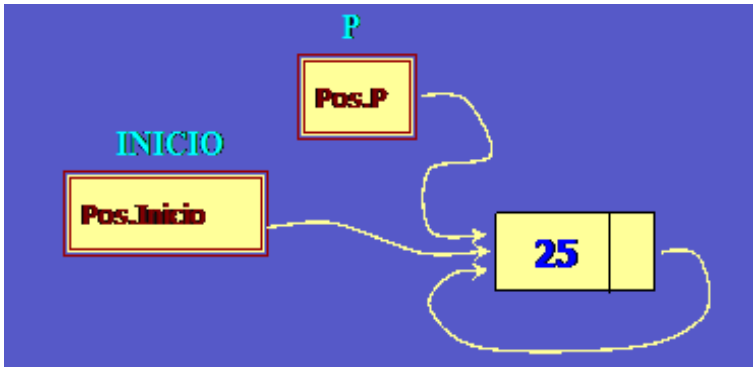



La lista queda como sigue:




**CASO: INSERCIÓN EN UNA LISTA QUE CONTIENE UN ÚNICO NODO**

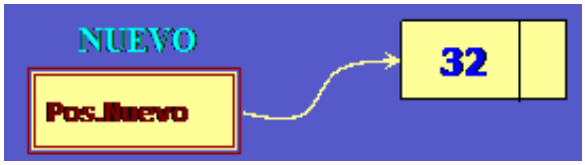
Añadimos ahora el valor 32 a continuación del 25, en una lista que contiene un único nodo al que apuntan el puntero a la estructura y **P**:



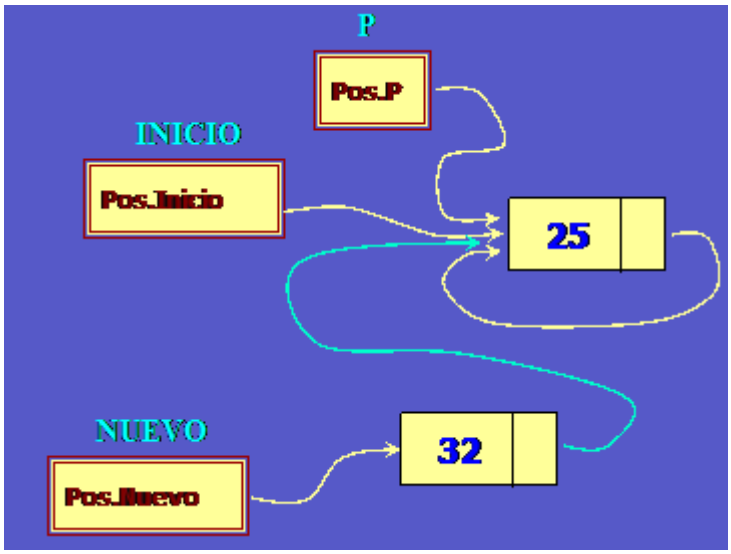

**Insertar_despues(INICIO, P, 32)**

Observar que el valor de **P** será igual al de **INICIO**.

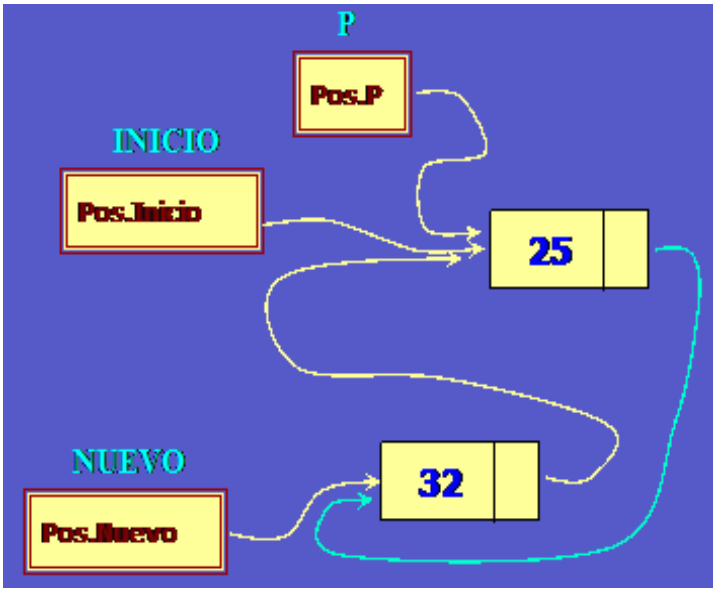
Una vez obtenida la memoria para el nuevo nodo, se almacena el valor 32 en el campo de datos:



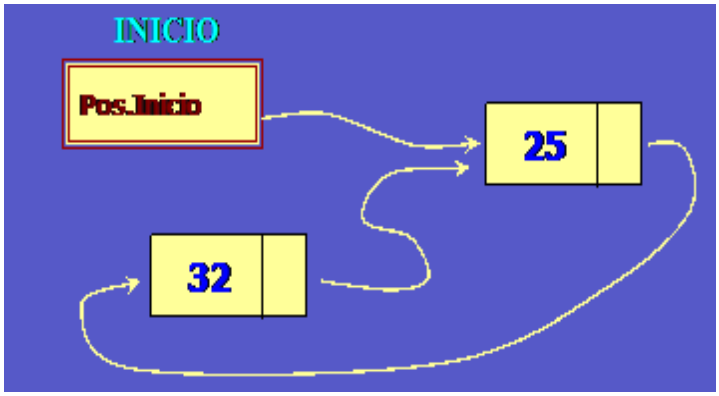
Como la lista no está vacía, se hace que el campo de enlace del nuevo nodo apunte al mismo nodo que el campo de enlace del nodo identificado por el puntero **P**:




Por último, se redefine el campo de enlace del nodo apuntado por **P**, de forma que pase a apuntar al nuevo nodo:




La lista queda como sigue:



### Eliminación de un elemento en una Lista Circular Simplemente Enlazada

### Eliminación de un elemento en una Lista Circular Simplemente Enlazada (por su posición)

Para eliminar un nodo de una lista no vacía, identificado mediante un puntero **P**, se deben tener en cuenta dos posibilidades: que la lista contenga un solo elemento o que contenga varios.

En el primer caso, tras liberar la memoria asociada al único nodo de la lista, ésta quedará vacía.

En el segundo caso, se debe conectar el nodo previo al que se va a eliminar con el posterior. Para ello se recorre la lista hasta encontrar el nodo previo al referenciado por **P**. Una vez encontrado, se redefine su enlace y, por último, se libera la memoria que ocupa el nodo elegido. Además, si se va a eliminar el nodo referenciado por el puntero a la estructura, antes de eliminarlo se hace que dicho puntero apunte al siguiente nodo.

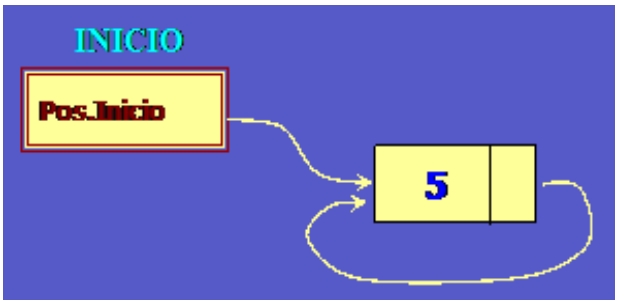
```
MÓDULO Eliminar
DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
        VARIABLES
                PREVIO puntero a NODO_LISTA
INICIO
        Si (Lista_vacia(INICIO) = Verdadero) entonces
                ESCRIBIR "Lista vacía"
        sino
                Si (P = P↑.SIGUIENTE) entonces
                        LIBERAR(P)
                        INICIO <- NULO
                sino
                        Si (INICIO = P) entonces
                                INICIO <- INICIO↑SIGUIENTE
                        fin-si

                        PREVIO <- INICIO
                        Mientras (PREVIO↑.SIGUIENTE ≠ P) hacer
                                PREVIO <- PREVIO↑.SIGUIENTE
                        fin-mientras

                        PREVIO↑.SIGUIENTE <- P↑.SIGUIENTE
                        LIBERAR(P)
                fin-si
        fin-si
FIN
```

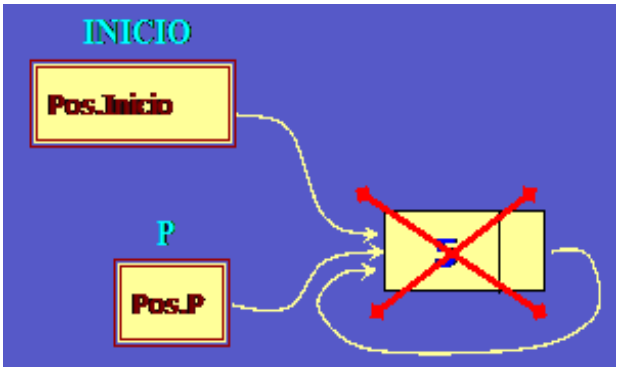
**CASO: LISTA CON UN ÚNICO NODO**

Supongamos que se desea eliminar el único elemento de la siguiente lista:

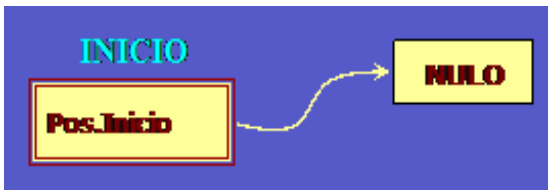



**Eliminar(INICIO, INICIO)**

Como sólo hay un elemento en la lista, se libera la memoria asociada a éste:

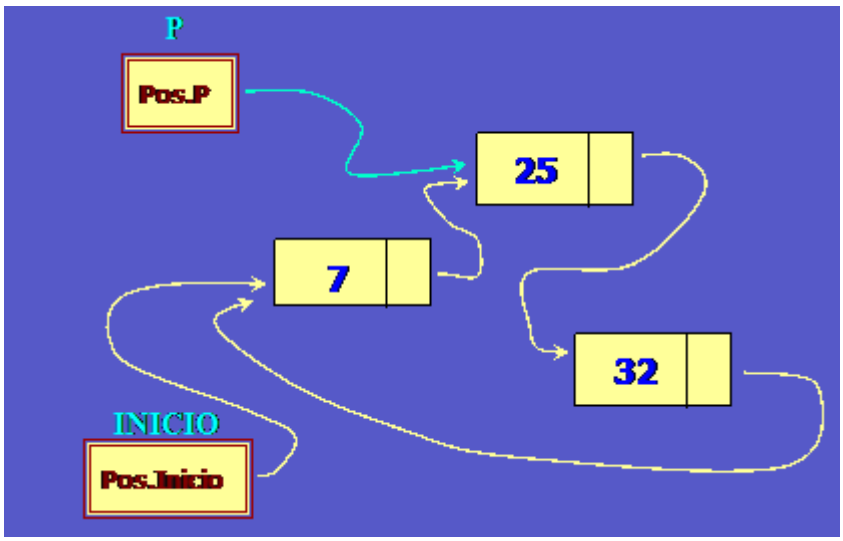



y a continuación se hace nulo el puntero a la lista:



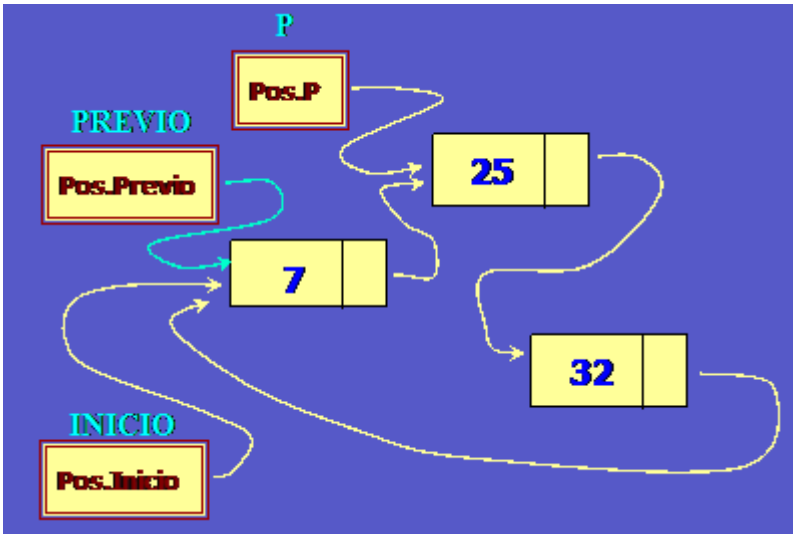

**CASO: ELIMINACIÓN DE UN NODO INTERMEDIO**

Se desea eliminar el nodo con valor 25 de la siguiente lista:

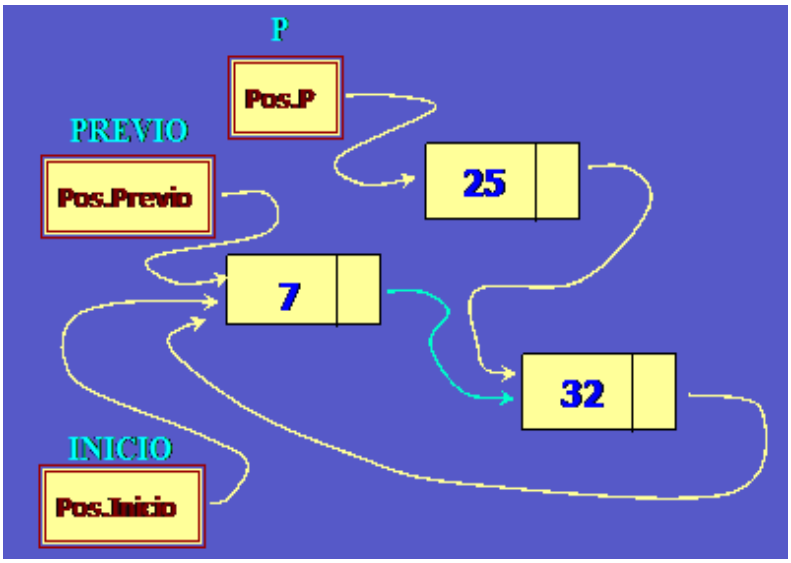



**Eliminar(INICIO, P)**

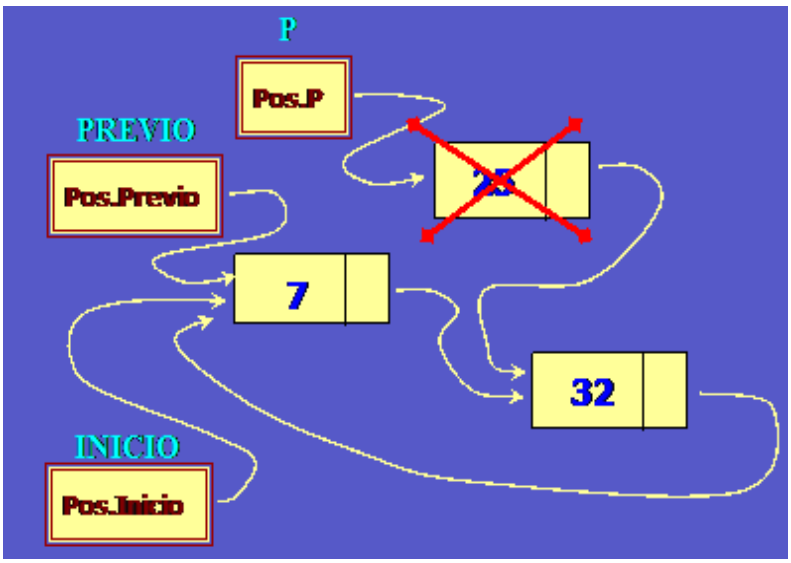
Se realiza una búsqueda para determinar el nodo que precede al que se desea eliminar. El puntero **PREVIO** apuntará a dicho nodo:



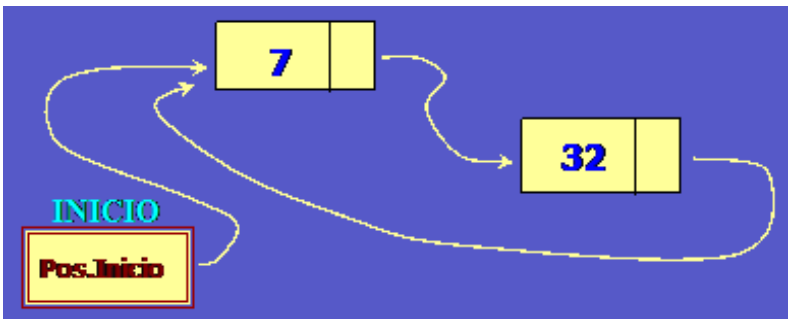

Se conecta el nodo previo con el que está a continuación del que se va a eliminar:




Se libera la memoria asociada al nodo que se debe eliminar de la lista:

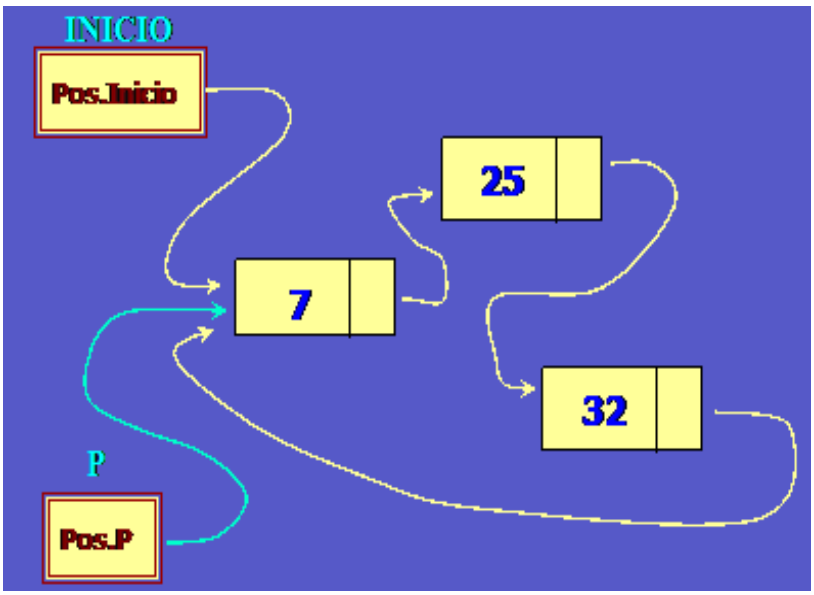



La lista resultante contendrá 2 elementos:



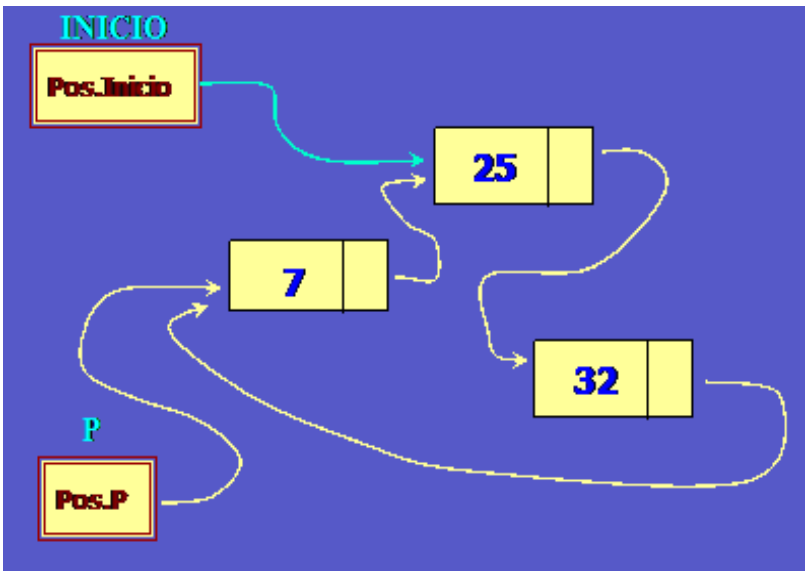

**CASO: ELIMINAR EL NODO REFERENCIADO POR EL PUNTERO A LA ESTRUCTURA**

En la siguiente lista se desea eliminar el nodo apuntado por el puntero que da acceso a la estructura:

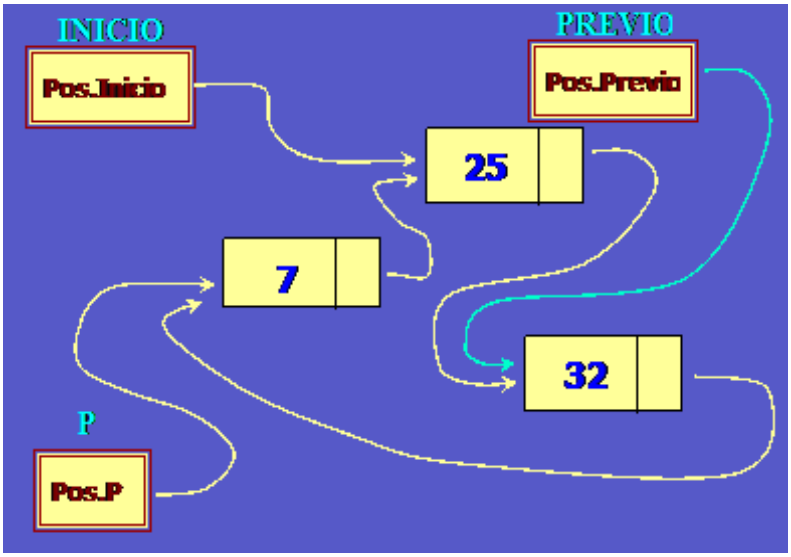



**Eliminar(INICIO, P)**

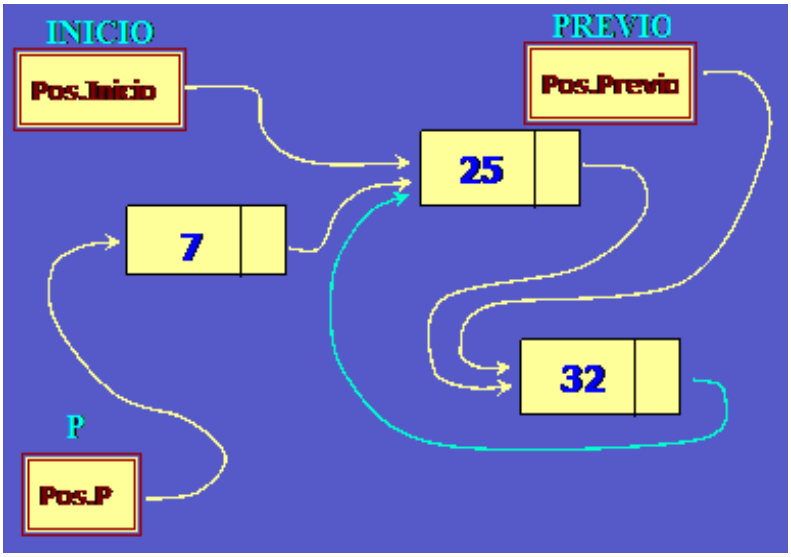
El puntero a la estructura pasa a apuntar al nodo siguiente al que se va a eliminar:



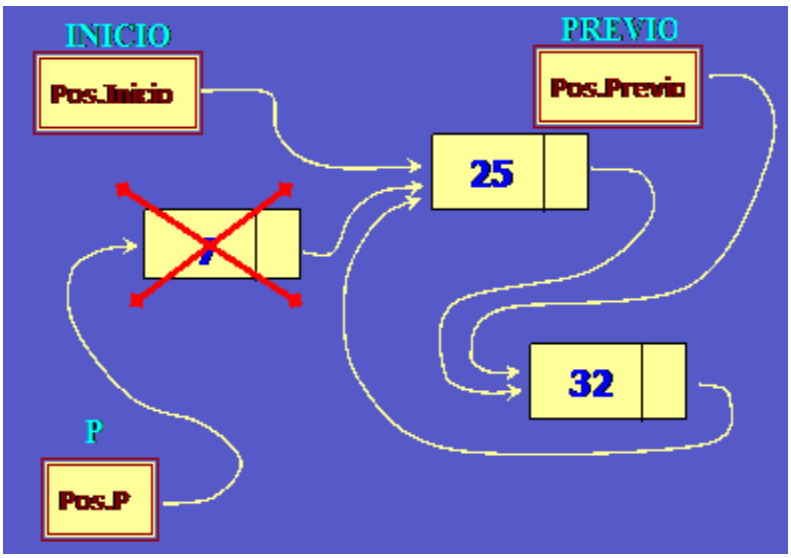

Se busca el nodo previo al que se va a eliminar, que queda apuntado por el puntero **PREVIO**:



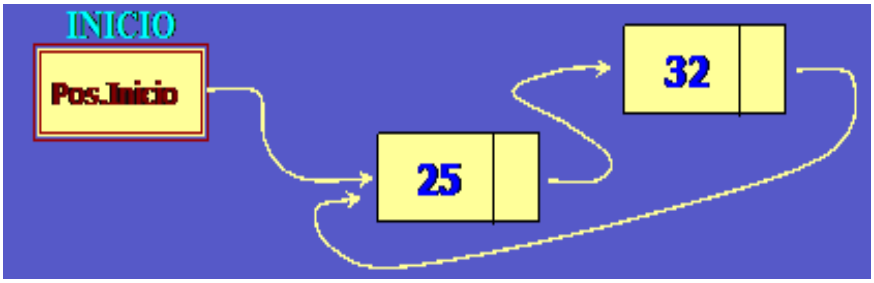

Se redefine el enlace del nodo apuntado por **PREVIO**:



Por último, se libera la memoria asociada al nodo apuntado por **P**:




La lista quedará con 2 nodos:




**Eliminación de un elemento en una Lista Circular Simplemente Enlazada (por su contenido)**

La eliminación por valor se puede realizar usando el mismo esquema que se propuso para la lista lineal con enlace simple, con llamadas a los módulos Buscar y Eliminar, o se puede escribir un nuevo módulo que combine las operaciones de búsqueda y eliminación.

**Comprobación de Lista Circular Simplemente Enlazada vacía**

La lista estará vacía si el puntero que da acceso a la estructura tiene como valor NULO (operación idéntica a la comentada para una lista lineal con enlace simple).

**Recorrido de una Lista Circular Simplemente Enlazada**

La operación de recorrido consiste en visitar todos y cada uno de los nodos de la lista, para poder realizar algún procesamiento sobre los mismos. Si la lista está vacía, no será necesario recorrerla. En caso contrario, se utiliza un puntero auxiliar, **P**, que inicialmente apunta al mismo nodo que **INICIO**; se aplica el tratamiento a ese primer nodo y se pasa al siguiente. El bucle de tratamiento concluye cuando el puntero **P** vuelva a ser igual a **INICIO**, lo que significa que ya se han tratado todos los nodos.

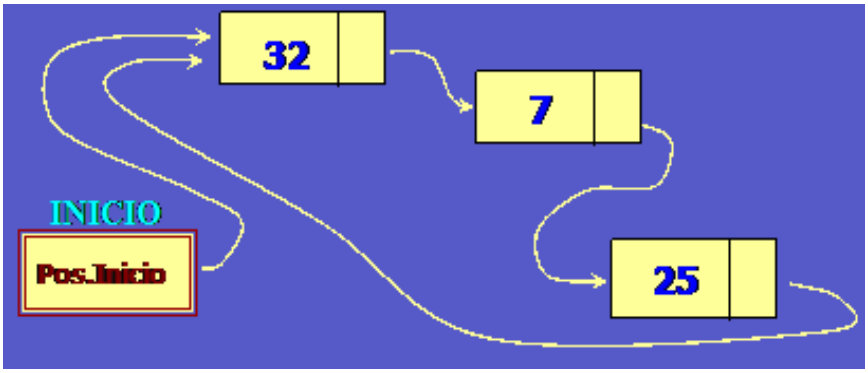
**MÓDULO Recorrer_lista**

**DATOS**

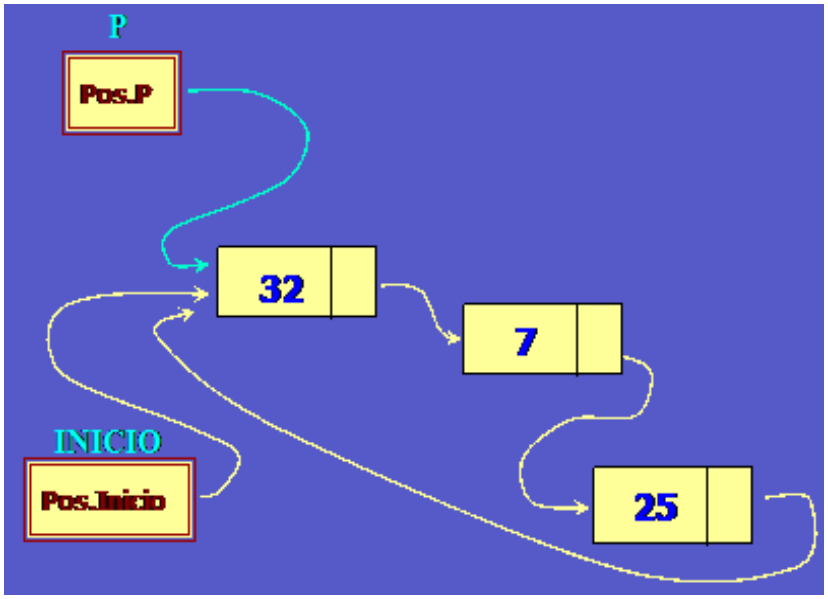
```
        PARÁMETROS
                Recibe INICIO puntero a NODO_LISTA
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        Si (Lista_vacia(INICIO) = Falso) entonces
                P <- INICIO
                Repetir
                        Tratar (P↑.DATO)
                        P <- P↑.SIGUIENTE
                Mientras (P ≠ INICIO)
        fin-si
FIN
```

**Ejemplo de operación de recorrido**

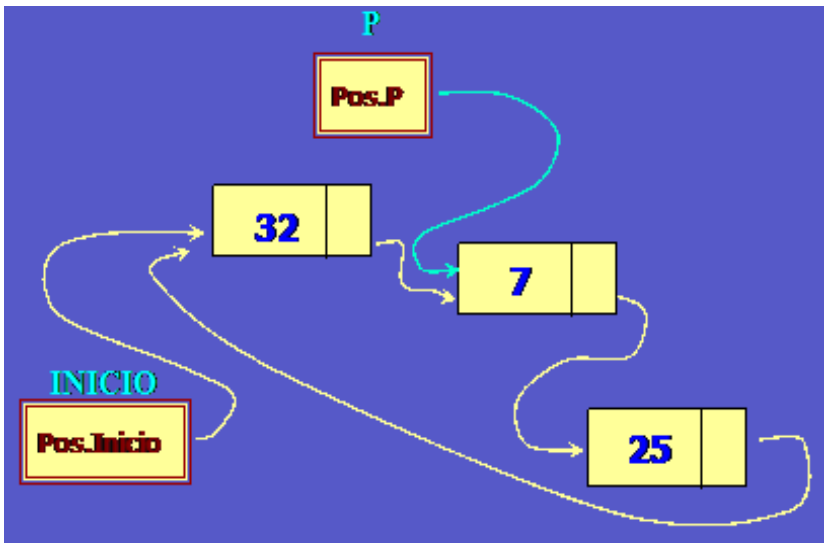
Supongamos la siguiente lista:



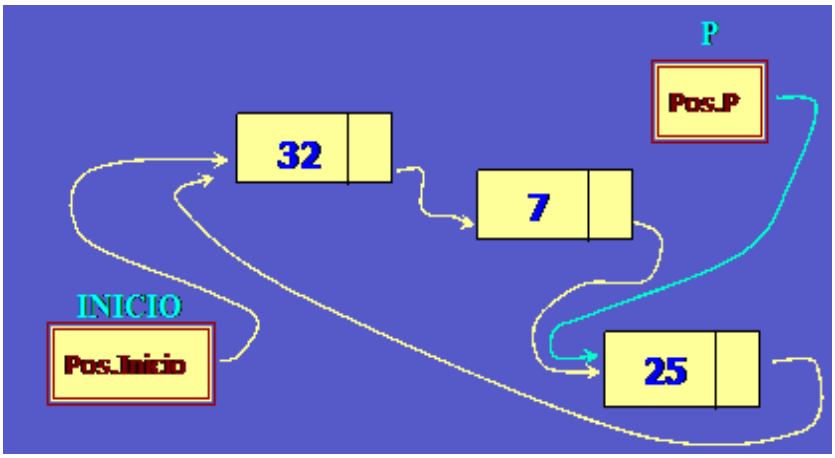

El recorrido comienza por el nodo con valor 32:



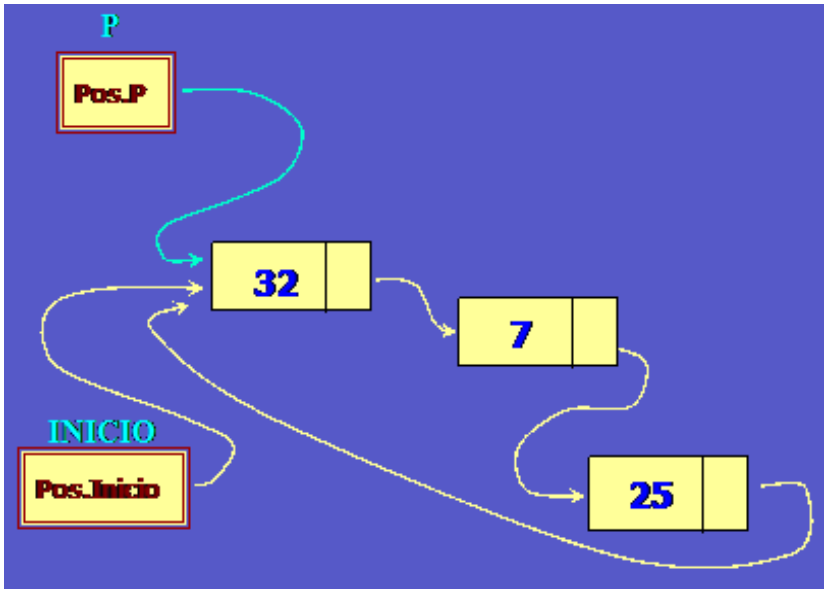

Se aplica el tratamiento elegido a dicho nodo y se avanza el puntero **P**:




Se aplica el tratamiento al nodo con valor 7 y se avanza el puntero:




Se aplica el tratamiento al nodo con valor 25 y se avanza el puntero:




En este momento concluyen las iteraciones, pues el puntero **P** vuelve a apuntar al primer nodo procesado.

### Acceso a un elemento de una Lista Circular Simplemente Enlazada

La operación de acceso a un elemento es similar a la descrita en el caso de listas lineales con enlace simple.

### Búsqueda de un elemento en una Lista Circular Simplemente Enlazada

Para buscar un elemento en la lista circular, se recorre ésta hasta encontrar el valor buscado o hasta que se hayan tratado todos los elementos de la misma, lo que primero ocurra. Comenzaremos el análisis de la lista por el nodo apuntado por el puntero que da acceso a la estructura y avanzando por los enlaces **SIGUIENTE** de los sucesivos nodos. Sabremos que se han procesado todos los nodos de la lista cuando el puntero auxiliar que se utiliza para hacer el recorrido sea nuevamente igual al puntero **INICIO**.

Como resultado de la búsqueda se obtiene el puntero **NULO** si la lista está vacía o si no se ha encontrado el valor. Si se ha encontrado, se obtiene un puntero al nodo que lo contiene. Si el valor buscado aparece varias veces en la lista, se encontrará el situado en el nodo más cercano al que da acceso a la estructura, en el sentido de recorrido fijado por el campo de enlace **SIGUIENTE** de los nodos.

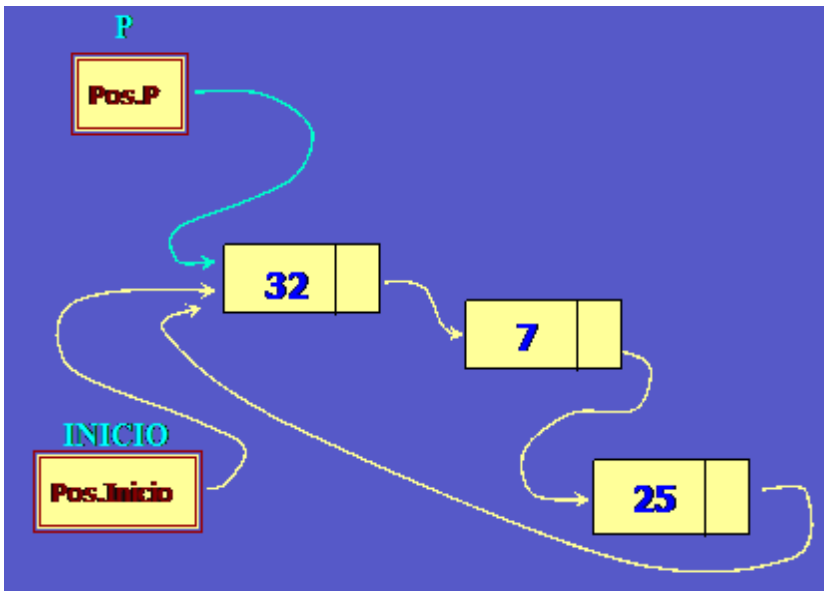
```
MÓDULO Buscar   devuelve puntero a NODO_LISTA

DATOS
        PARÁMETROS
                Recibe INICIO puntero a NODO_LISTA
                Recibe VALOR de tipo T          ** valor a buscar
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        Si (Lista_vacia(INICIO) = Falso) entonces
                P <- INICIO
                Repetir
                        Si (P↑.DATO = VALOR) entonces
                                DEVOLVER P    ** dato encontrado
                        fin-si
                        P <- P↑.SIGUIENTE
                Mientras (P ≠ INICIO)
        fin-si

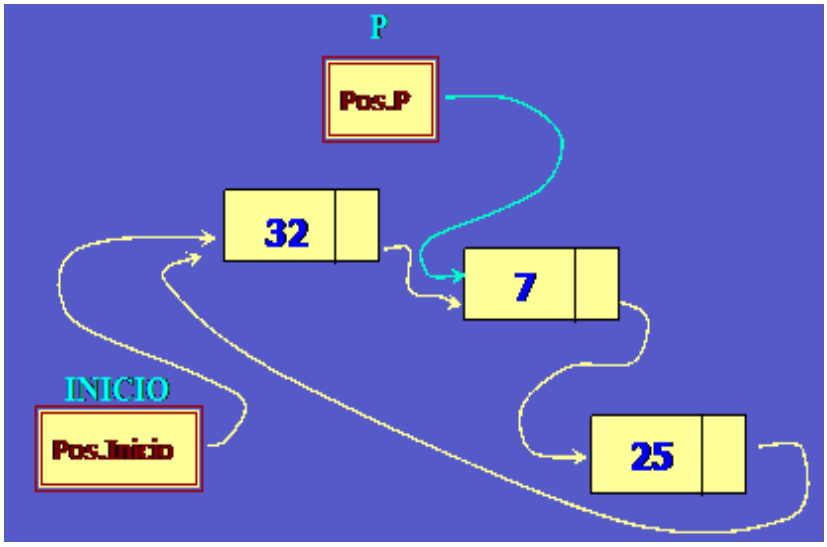
        DEVOLVER NULO                       ** dato no encontrado
FIN
```

### Ejemplo de operación de búsqueda con fallo

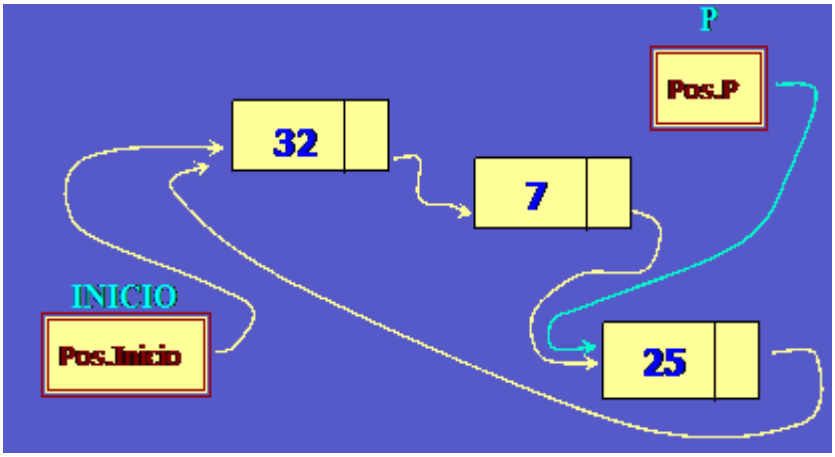
En la siguiente lista se busca el valor 33. La búsqueda comienza por el nodo con valor 32:



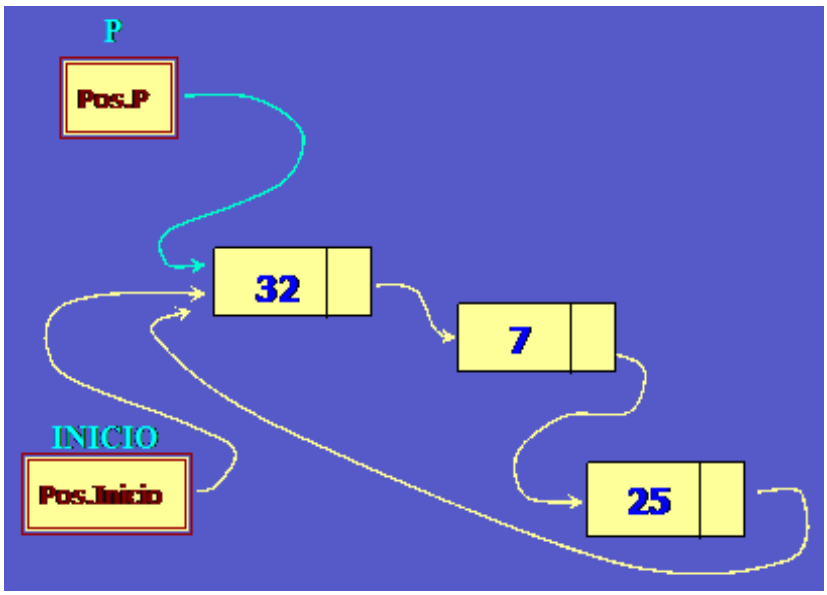

Como dicho nodo no contiene el valor buscado, se avanza el puntero **P**:




Como dicho nodo no contiene el valor buscado, se avanza el puntero **P**:




Como dicho nodo no contiene el valor buscado, se avanza el puntero **P**:



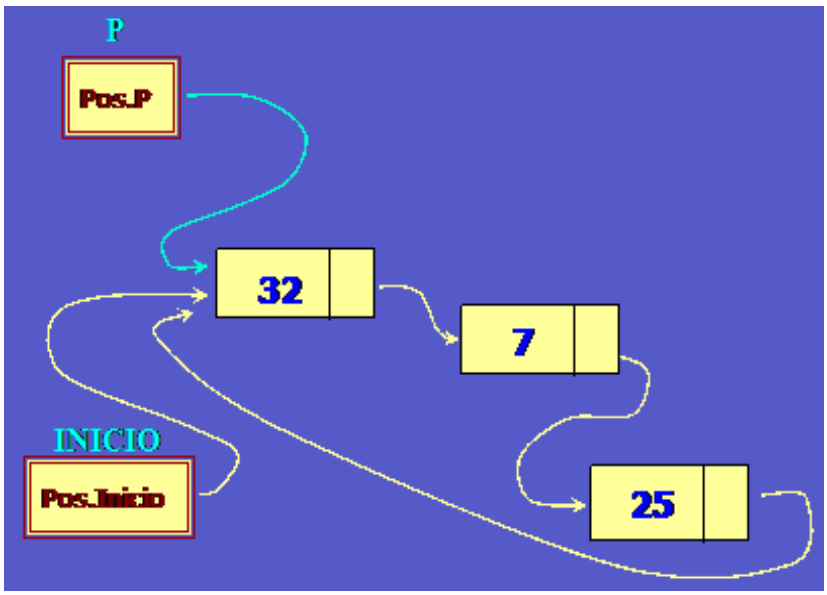

En este momento concluyen las iteraciones, pues el puntero **P** vuelve a apuntar al primer nodo procesado.

El módulo devolverá el valor NULO (fracaso en la búsqueda).

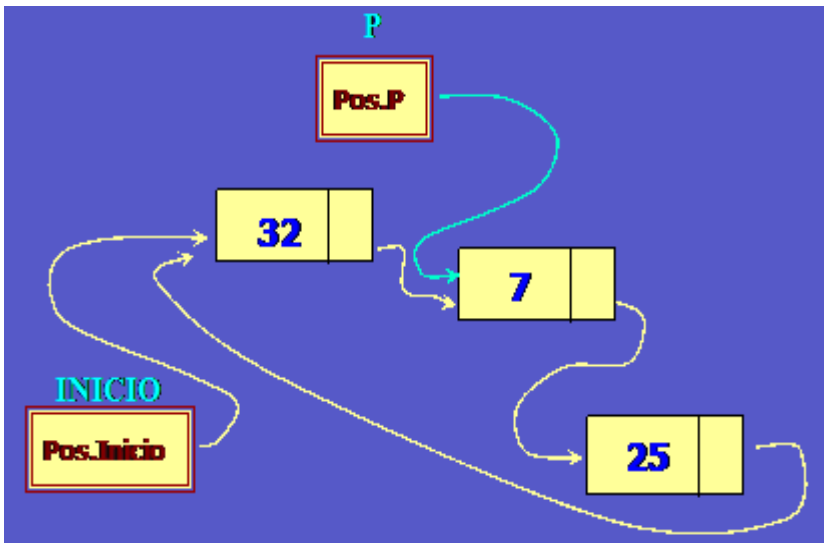
**Ejemplo de operación de búsqueda con éxito**

Se busca ahora el valor 7.

El primer nodo procesado es el de valor 32:




Como dicho nodo no contiene el valor buscado, se avanza el puntero **P**:




Como el nodo referenciado contiene el valor buscado, el módulo concluye, retornando el valor actual del puntero **P**.

### Vaciado de una Lista Circular Simplemente Enlazada

Para eliminar todos los nodos de la lista circular se utiliza un bucle que se repetirá mientras la lista no esté vacía. Dentro del bucle se distinguen dos situaciones: que en la lista quede un único nodo o que quede más de uno.

Si sólo queda un nodo, se libera la memoria que ocupa y el puntero a la estructura se hace nulo.

Si queda más de un nodo, se elimina el que está apuntado por el nodo referenciado por el puntero **INICIO**. Antes de liberar la memoria asociada a ese nodo, se conecta el nodo que le precede con el nodo que está a continuación de él. Cuando sólo queden dos nodos, el nodo predecesor y el sucesor del que se va a eliminar serán un mismo nodo.

```
MÓDULO Vaciar_lista

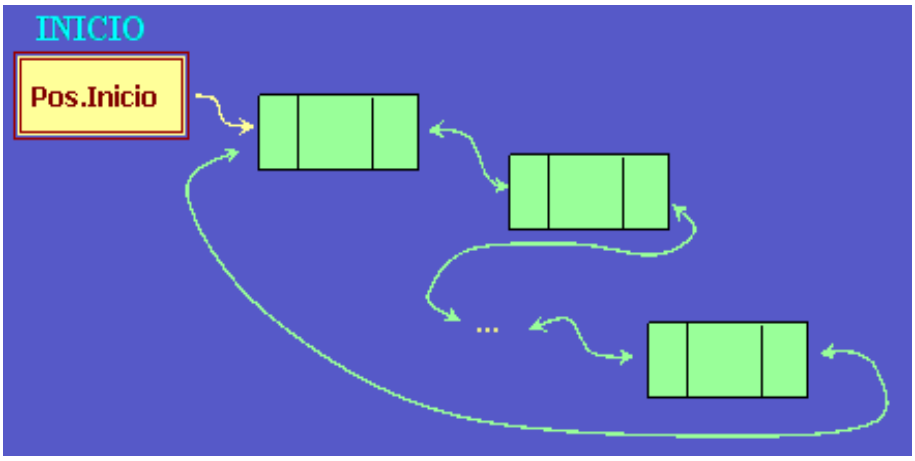
DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        Mientras (Lista_vacia(INICIO) ≠ Verdadero) hacer
                Si (INICIO = INICIO↑.SIGUIENTE) entonces
                        LIBERAR(INICIO)
                        INICIO <- NULO
                sino
                        P <- INICIO↑.SIGUIENTE
                        INICIO↑.SIGUIENTE <- P↑.SIGUIENTE
                        LIBERAR(P)
                fin-si
```

**fin-mientras**

**FIN**

## *Lista Circular Doblemente Enlazada*

Una lista circular con enlace doble incluye un conjunto de nodos cada uno de los cuales está conectado a otros dos nodos de la estructura. Existe un puntero externo que da acceso a la estructura.

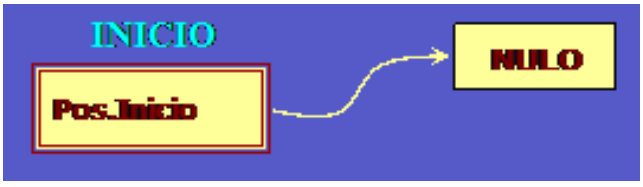



Esta lista comparte características con la lista lineal con enlace doble (debido a la estructura de los nodos) y con la lista circular con enlace simple (debido a la estructura de la lista), por lo que algunas operaciones son similares a las comentadas para dichos tipos de lista. Las operaciones descritas para listas circulares con enlace simple que son aplicables a las listas circulares con doble enlace no se volverán a comentar:

- **Comprobación** de lista vacía
- **Recorrido** de la lista
- **Acceso** a un elemento
- **Búsqueda**
- **Vaciado** de la lista

### Creación de la Lista Circular Doblemente Enlazada

La creación de este tipo de lista es idéntica a la comentada para una lista lineal con enlace doble, dado que ambos tipos de lista comparten la misma estructura de nodo.




**Representación gráfica de lista circular con enlace doble vacía**

### Inserción de un elemento en una Lista Circular Doblemente Enlazada

Para insertar un nuevo elemento en la lista, indicaremos el valor de dicho elemento, además de un puntero al nodo después del cual debe colocarse el nuevo elemento (**P**). Si la lista está vacía, el valor de **P** no se tendrá en cuenta al realizar la inserción.

Una vez obtenida la memoria para el nuevo nodo, se almacena el nuevo valor de la lista en el campo de datos de dicho nodo.

Si la lista está vacía, los campos de enlace del nuevo nodo apuntarán al propio nodo, al igual que el puntero a la estructura.

Si la lista no está vacía, el nuevo nodo debe colocarse entre el apuntado por **P** y el que va a continuación. Por tanto, los campos **ANTERIOR** y **SIGUIENTE** del nuevo nodo apuntarán a esos dos nodos respectivamente. Además, dichos nodos deben pasar a apuntar al nuevo.

```
MÓDULO Insertar_despues

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T       ** valor a insertar
        VARIABLES
                NUEVO puntero a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"  ** o devolver cierto valor
        sino
                NUEVO↑.DATO <- ELEMENTO

                Si (Lista_vacia(INICIO) = Verdadero) entonces
                        NUEVO↑.SIGUIENTE <- NUEVO
                        NUEVO↑ANTERIOR <- NUEVO
                        INICIO <- NUEVO
                sino
                        NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                        NUEVO↑.ANTERIOR <- P

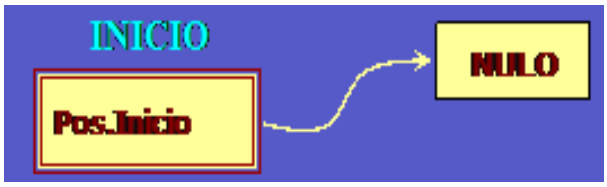
                        P↑.SIGUIENTE↑.ANTERIOR <- NUEVO
                        P↑.SIGUIENTE <- NUEVO
                fin-si
```

**fin-si**

**FIN**

### CASO: INSERCIÓN DEL PRIMER NODO

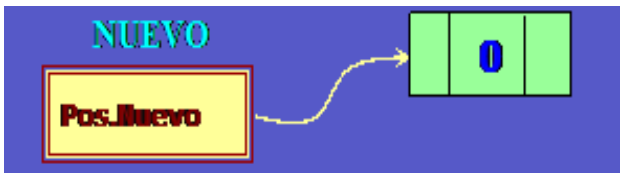
Supongamos una lista de enteros vacía:



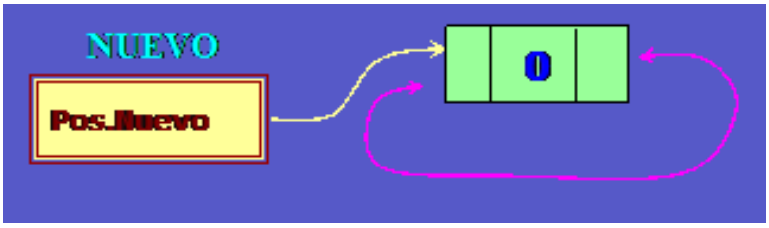

en la que deseamos insertar el valor 0:

**Insertar_despues(INICIO, NULO, 0)**

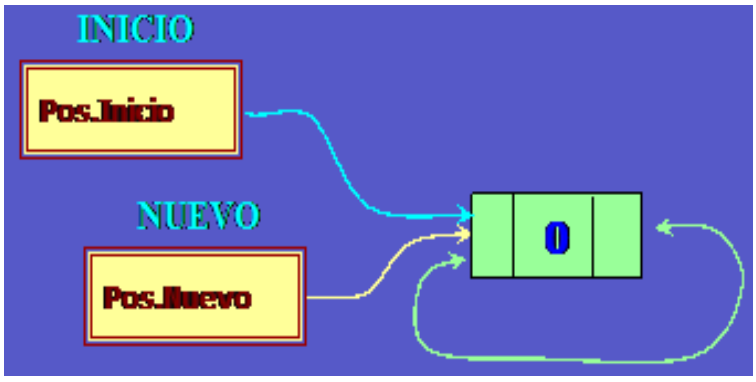
Una vez obtenida la memoria para el nuevo nodo, se almacena el valor 0 en el campo de datos:




Como la lista está vacía, los enlaces **SIGUIENTE** y **ANTERIOR** del nuevo nodo apuntan a dicho nodo:

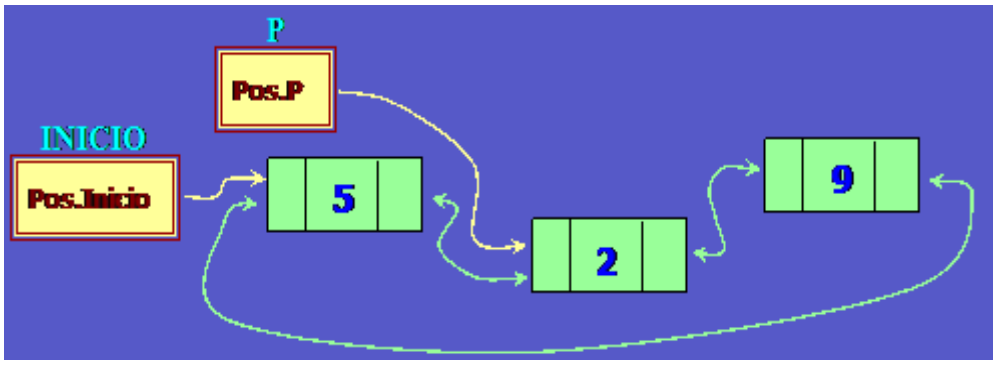



Por último, el puntero a la lista pasa a apuntar al nuevo nodo:



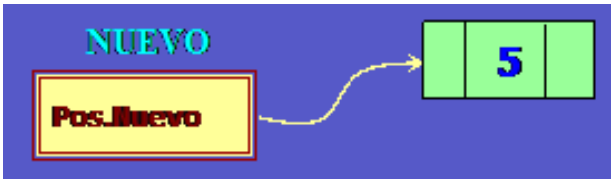

### CASO: INSERCIÓN EN UN PUNTO INTERMEDIO

En la siguiente lista de enteros se intenta insertar el valor 5 a continuación del 2:

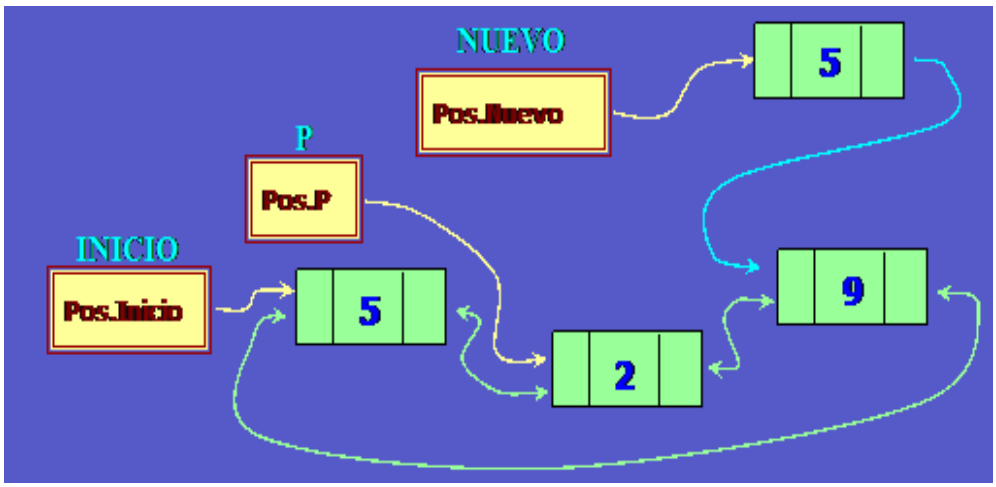



**Insertar_despues(INICIO, P, 5)**

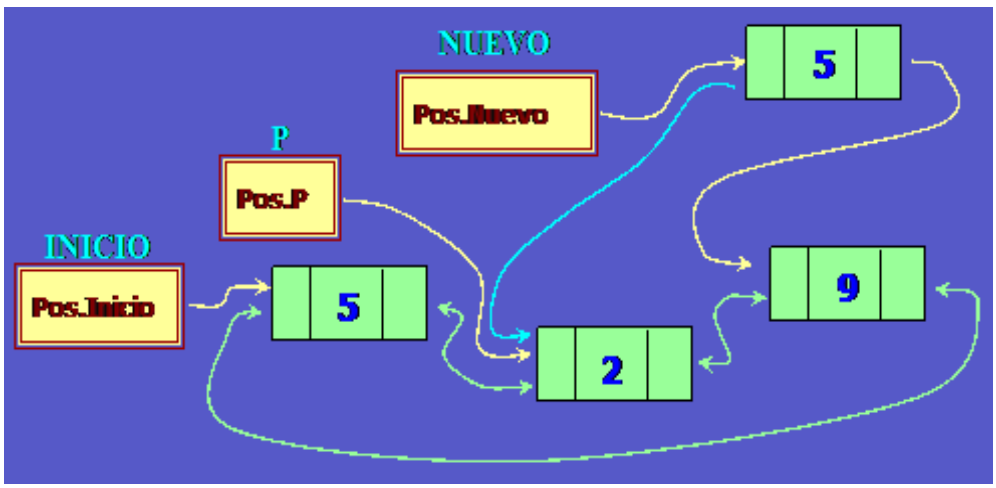
Tras reservar memoria para el nuevo nodo, se almacena el dato en el campo de datos:



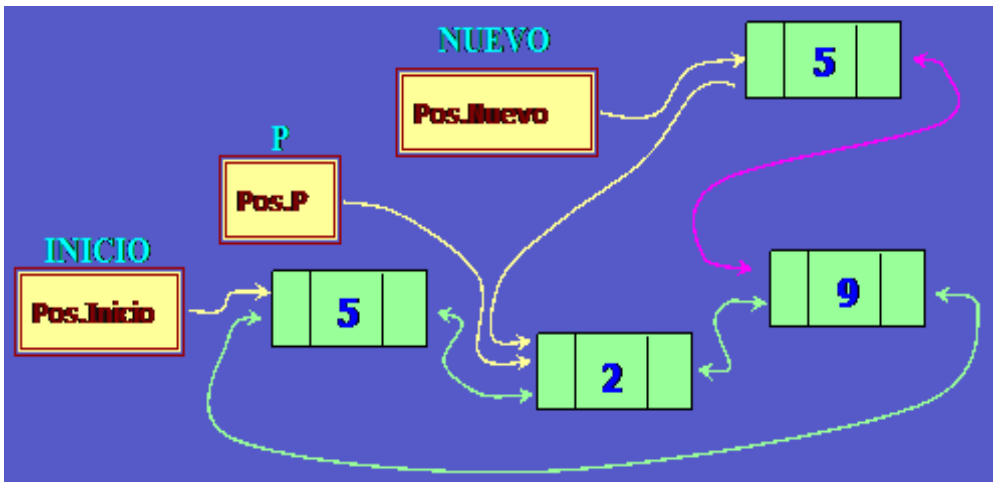

A continuación se definen los enlaces del nuevo nodo: en primer lugar se enlaza con el que debe quedar después de él:



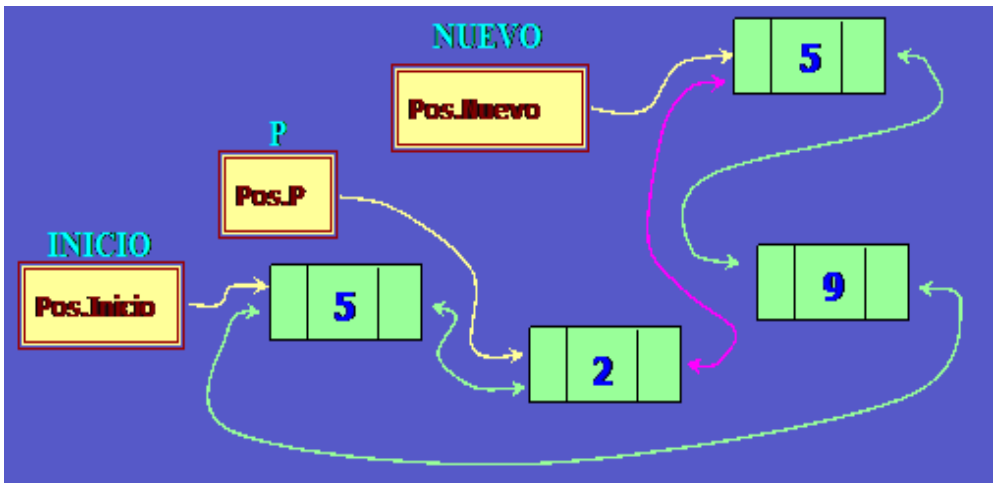

y a continuación se enlaza con el que debe quedar antes que él:



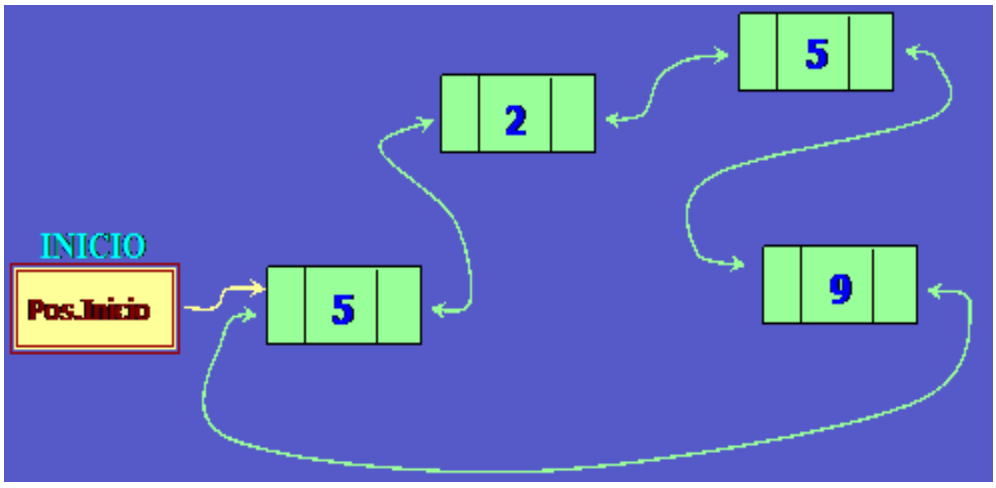
Se enlaza el nodo que debe quedar detrás del nuevo con éste:




Se enlaza con el nuevo nodo el que debe quedar antes que él:




La lista queda con 4 nodos:



### Eliminación de un elemento en una Lista Circular Doblemente Enlazada

**Eliminación de un elemento en una Lista Circular Doblemente Enlazada (por su posición)**
Para eliminar de una lista no vacía el nodo identificado por un puntero **P**, se deben tener en cuenta dos posibles casos: que la lista contenga un único nodo o que contenga dos o más nodos.

En el primer caso, tras liberar la memoria que ocupa el nodo la lista quedará vacía.

En el segundo caso se deben tratar por separado dos situaciones: que se vaya a eliminar el nodo referenciado por el puntero a la estructura o que no se vaya a eliminar dicho nodo. En la primera situación se hará que el puntero que da acceso a la estructura pase a apuntar al siguiente elemento de la misma; en la segunda situación no será necesario modificar el puntero **INICIO**. A continuación, en ambas situaciones, se enlaza el nodo previo al que se debe eliminar con el siguiente al que se desea eliminar y, por último, se libera la memoria asociada al nodo que debe desaparecer de la lista.

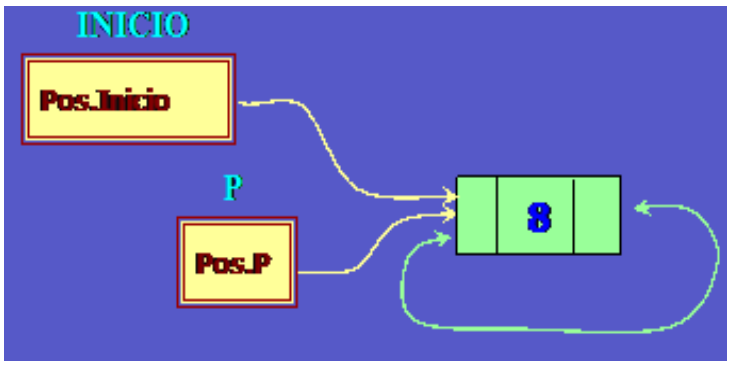
```
MÓDULO Eliminar

DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe P puntero a NODO_LISTA
INICIO
        Si (Lista_vacia(INICIO) = Verdadero) entonces
                ESCRIBIR "Lista vacía"
        sino
                Si (P = P↑.SIGUIENTE) entonces
                        LIBERAR(P)
                        INICIO <- NULO
                sino
                        Si (INICIO = P) entonces
                                INICIO <- INICIO↑.SIGUIENTE
                        fin-si

                        P↑.SIGUIENTE↑.ANTERIOR <- P↑.ANTERIOR
                        P↑.ANTERIOR↑.SIGUIENTE <- P↑.SIGUIENTE
                        LIBERAR(P)
                fin-si
        fin-si
FIN
```

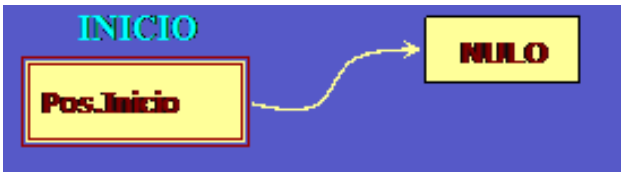
### CASO: ELIMINAR EL ÚNICO NODO DE UNA LISTA

Eliminamos el único nodo de la siguiente lista:



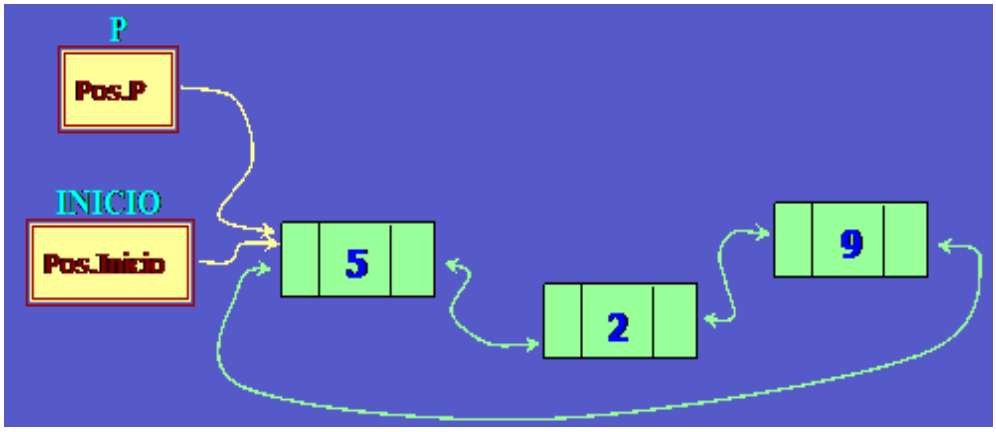

**Eliminar(INICIO, P)**

Como **P = P↑.SIGUIENTE**, se libera directamente la memoria del nodo y se hace nulo el puntero a la estructura, quedando la lista vacía:

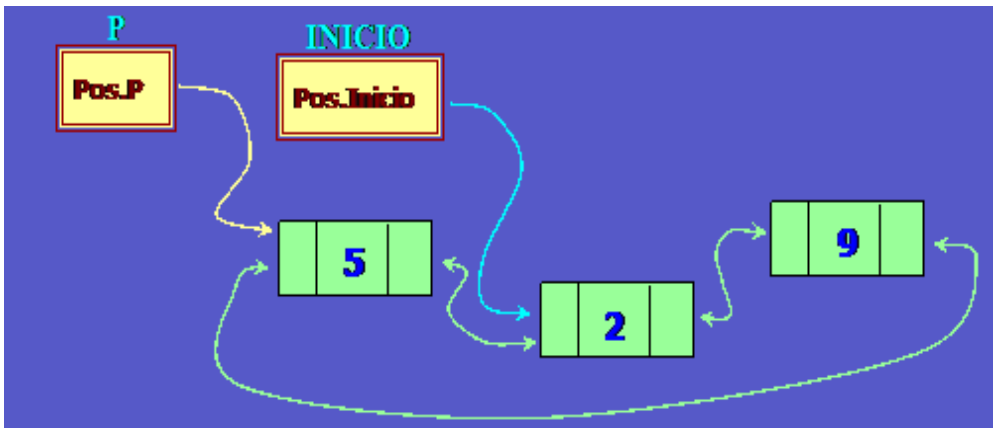



## CASO: ELIMINAR EL NODO REFERENCIADO POR EL PUNTERO A LA ESTRUCTURA

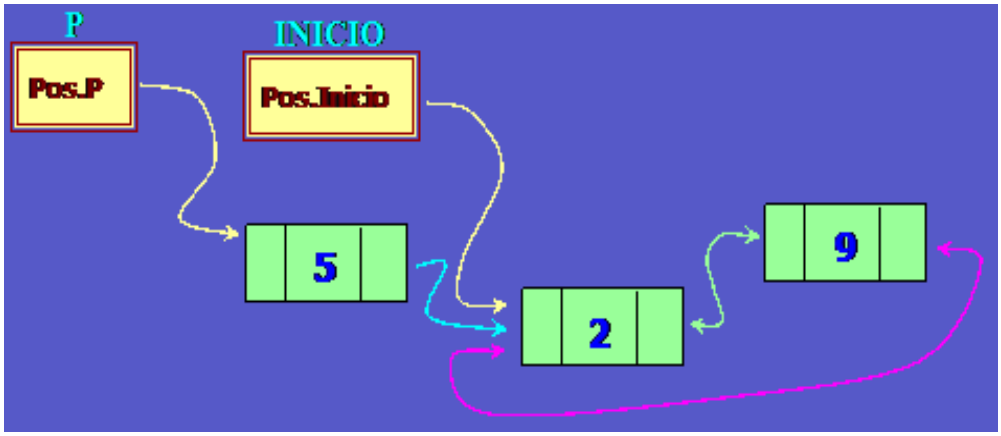
Eliminamos el nodo apuntado por el puntero que da acceso a la siguiente lista:



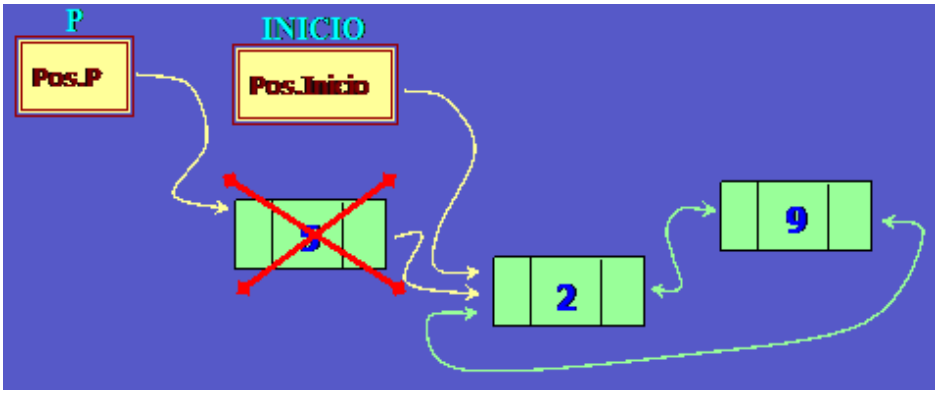

En primer lugar se hace que el puntero **INICIO** pase a apuntar al nodo con valor 2:



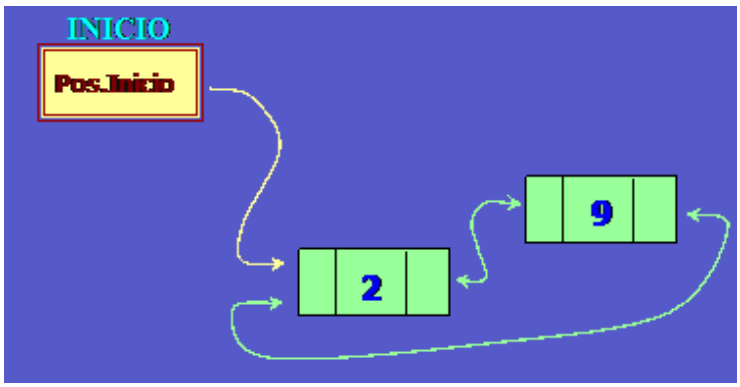

Se enlazan los nodos previo y posterior al que se debe eliminar:




Por último, se libera la memoria ocupada por el nodo con valor 5:

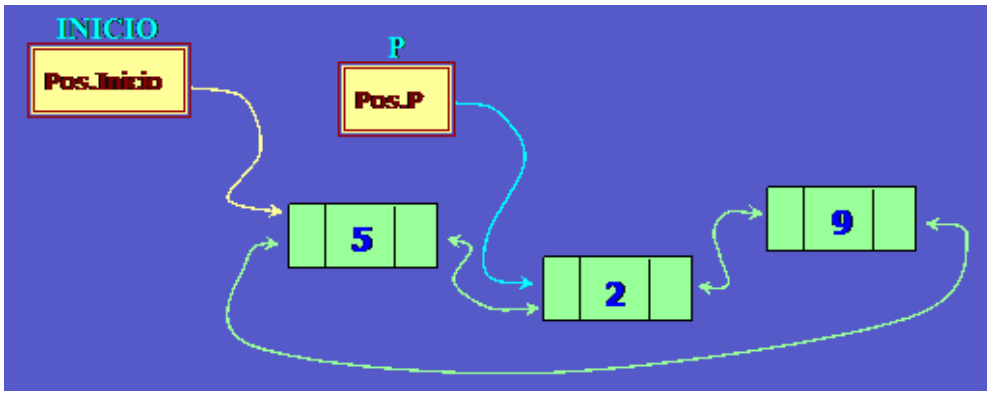



La lista queda con 2 nodos:

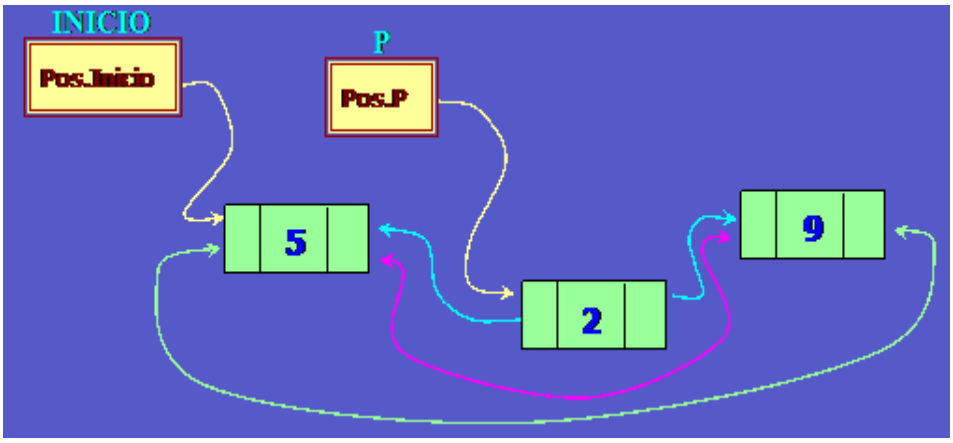


**CASO: ELIMINACIÓN DE UN NODO INTERMEDIO**

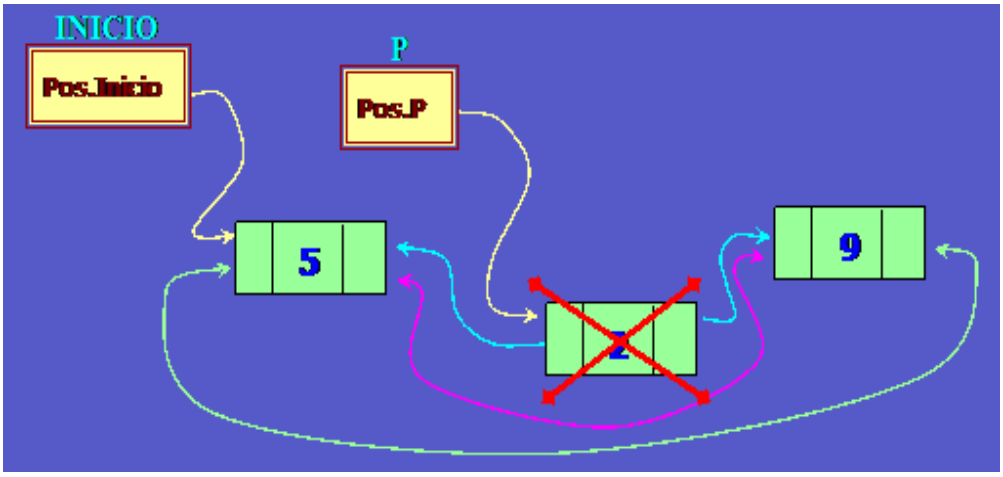
Se eliminará de la siguiente lista el nodo que contiene el valor 2:



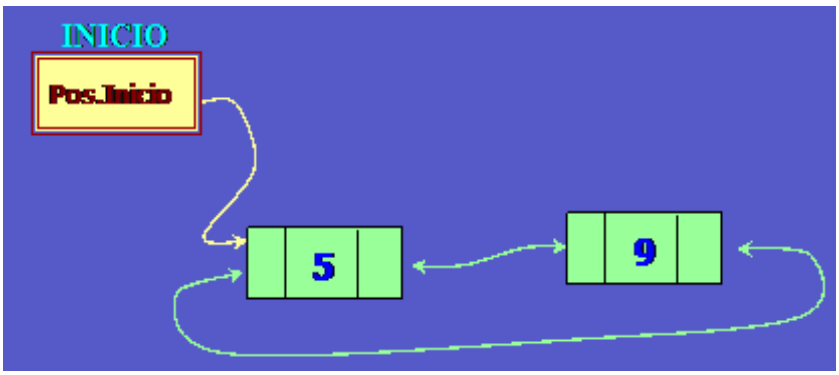

En primer lugar se enlazan los nodos previo y posterior al que se va a eliminar:




A continuación se libera la memoria asociada al nodo con valor 2:




La lista queda con 2 nodos:




**Eliminación de un elemento en una Lista Circular Doblemente Enlazada (por su contenido)**

Como se describió en el caso de la lista circular con enlace simple, se puede eliminar un valor de la lista utilizando las operaciones de búsqueda y eliminación. Se llama al módulo de búsqueda y, si se ha localizado el valor, se llama posteriormente al módulo de eliminación por posición descrito en el apartado anterior.

## *Listas Ordenadas*

Se puede trabajar con listas ordenadas, que bien se crean de forma que se mantenga el orden en todo momento, o bien se ordenan cuando ya contienen cierta información.

Una lista se podrá ordenar de forma **creciente** o **decreciente** por un campo o una combinación de campos de datos.

Para utilizar la misma estructura de nodos descrita en los apartados anteriores, supondremos un campo de datos de tipo T, que incluye como mínimo un subcampo, denominado **CLAVE**, por cuyo valor se ordenan los elementos de la lista.

**T es registro compuesto por**

**CLAVE de tipo T1**

***** otros campos***

**fin-registro**

En el caso más sencillo la estructura anterior se reducirá a un único campo.

### INSERCIÓN ORDENADA

La inserción ordenada implica 2 operaciones:

1- determinar la posición que debe ocupar en la lista el nuevo elemento, de modo que se mantenga el orden en los datos. Para ello se realizará una búsqueda que tendrá en cuenta el hecho de que la lista está ordenada.

2- colocar el elemento en dicha posición.

En los siguientes apartados se describirá la inserción ordenada para las diferentes representaciones de listas tratadas en este tema.

### Representación vectorial

```
MÓDULO Insertar_en_orden
DATOS
        PARÁMETROS
                Recibe ELEMENTO de tipo T
                Recibe N entera
                Transforma NUM_ELEM entera
                Transforma LISTA() de tipo T
        VARIABLES
                I  entera
INICIO
        Si ( Lista_llena ( NUM_ELEM, N ) = Verdadero ) entonces
                ESCRIBIR "Lista llena."
        sino
                I<- NUM_ELEM+1

                Mientras ( (LISTA(I-1).DATO.CLAVE > ELEMENTO.CLAVE)
                        Y  (I > 1) ) hacer
                        LISTA(I) <- LISTA(I-1)
                        I <- I-1
                fin-mientras

                LISTA(I) <- ELEMENTO
                NUM_ELEM <- NUM_ELEM +1
        fin-si
FIN
```

### Lista lineal con enlace simple

```
MÓDULO Insertar_en_orden
DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T
        VARIABLES
                NUEVO  puntero a NODO_LISTA
                P_ANTERIOR, P_SIGUIENTE punteros a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"  ** o devolver cierto valor
        sino
                NUEVO↑.DATO <- ELEMENTO

                P_ANTERIOR <- NULO
                P_SIGUIENTE <- INICIO

                Mientras (P_SIGUIENTE ≠ NULO)   Y
                 (P_SIGUIENTE↑.DATO.CLAVE < ELEMENTO.CLAVE) hacer
                        P_ANTERIOR <- P_SIGUIENTE
                        P_SIGUIENTE <- P_SIGUIENTE↑.SIGUIENTE
                fin-mientras

                NUEVO↑.SIGUIENTE <- P_SIGUIENTE

                Si (P_SIGUIENTE = INICIO) entonces
                        INICIO <- NUEVO
                sino
                        P_ANTERIOR↑.SIGUIENTE <- NUEVO
                fin-si
```

```
        fin-si
FIN
```

**Lista lineal con enlace doble**

```
MÓDULO Insertar_en_orden
DATOS
        PARÁMETROS
                Transforma INICIO puntero a NODO_LISTA
                Recibe ELEMENTO de tipo T
VARIABLES
                NUEVO puntero a NODO_LISTA
                P_ANTERIOR, P_SIGUIENTE punteros a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO <- ELEMENTO

                P_ANTERIOR <- NULO
                P_SIGUIENTE <- INICIO

                Mientras ( (P_SIGUIENTE ≠ NULO)   Y
                 (P_SIGUIENTE↑.DATO.CLAVE < ELEMENTO.CLAVE) ) hacer
                        P_ANTERIOR <- P_SIGUIENTE
                        P_SIGUIENTE <- P_SIGUIENTE↑.SIGUIENTE
                fin-mientras

                NUEVO↑.SIGUIENTE <- P_SIGUIENTE
                NUEVO↑.ANTERIOR <- P_ANTERIOR
```

```
            Si (P_SIGUIENTE ≠ NULO) entonces
                P_SIGUIENTE↑.ANTERIOR <- NUEVO
            fin-si

            Si (P_ANTERIOR ≠ NULO) entonces
                P_ANTERIOR↑SIGUIENTE <- NUEVO
            fin-si

            Si (P_SIGUIENTE = INICIO) entonces
                INICIO <- NUEVO
            fin-si
        fin-si
FIN
```

**Lista circular con enlace simple**

```
MÓDULO Insertar_en_orden
DATOS
        PARÁMETROS
            Transforma INICIO puntero a NODO_LISTA
            Recibe ELEMENTO de tipo T
        VARIABLES
            NUEVO, P punteros a NODO_LISTA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
            ESCRIBIR "No hay memoria"
        sino
            Si (Lista_vacia(INICIO) = Verdadero) entonces
                NUEVO↑.SIGUIENTE <- NUEVO
                NUEVO↑.DATO <- ELEMENTO
                INICIO <- NUEVO
```

```
            sino
                P <- INICIO

                Mientras (P↑.SIGUIENTE ≠ INICIO)
                 Y (P↑.DATO.CLAVE < ELEMENTO.CLAVE) hacer
                    P <- P↑.SIGUIENTE
                fin-mientras

                Si (P↑.DATO.CLAVE < ELEMENTO.CLAVE) entonces
                    ** insertar después
                    NUEVO↑.DATO <- ELEMENTO
                    NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                    P↑.SIGUIENTE <- NUEVO
                sino
                    ** insertar antes
                    NUEVO↑.DATO <- P↑.DATO
                    NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                    P↑.DATO <- ELEMENTO
                    P↑.SIGUIENTE <- NUEVO
                fin-si
            fin-si
        fin-si
FIN
```

**Lista circular con enlace doble**

```
MÓDULO Insertar_en_orden
DATOS
    PARÁMETROS
        Transforma INICIO puntero a NODO_LISTA
        Recibe ELEMENTO de tipo T
    VARIABLES
        NUEVO, P punteros a NODO_LISTA
```

```
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_LISTA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                Si (Lista_vacia(INICIO) = Verdadero) entonces
                        NUEVO↑.DATO <- ELEMENTO
                        NUEVO↑.SIGUIENTE <- NUEVO
                        NUEVO↑.ANTERIOR <- NUEVO
                        INICIO <- NUEVO
                sino
                        P <- INICIO

                        Mientras ( (P↑.SIGUIENTE ≠ INICIO)
                         Y (P↑.DATO.CLAVE < ELEMENTO.CLAVE) ) hacer
                                P <- P↑.SIGUIENTE
                        fin-mientras

                        Si (P↑.DATO.CLAVE < ELEMENTO.CLAVE) entonces
                                NUEVO↑.DATO <- ELEMENTO
                        sino
                                NUEVO↑.DATO <- P↑.DATO
                                P↑.DATO <- ELEMENTO
                        fin-si

                        NUEVO↑.SIGUIENTE <- P↑.SIGUIENTE
                        NUEVO↑.ANTERIOR <- P
                        NUEVO↑.ANTERIOR↑.SIGUIENTE <- NUEVO
                        NUEVO↑.SIGUIENTE↑.ANTERIOR <- NUEVO
                fin-si
        fin-si
```

**FIN**

## BÚSQUEDA EN UNA LISTA ORDENADA

Cuando se trabaja con listas ordenadas, el proceso de búsqueda se puede detener sin necesidad de evaluar todos los elementos de la estructura, siempre que se realice la búsqueda por el campo por el que está ordenada la lista.

Para buscar en una lista ordenada de forma creciente el elemento cuyo campo clave contiene cierto valor, se recorrerá la lista desde el elemento inicial hasta el que contiene el valor buscado (si existe en la lista) o hasta el primer elemento que supera el valor buscado (lo que significa que el valor buscado no existe).

En los siguientes apartados presentaremos algoritmos de búsqueda para los diferentes tipos de listas que hemos estudiado.

### Representación vectorial

Se pueden aplicar las técnicas de búsqueda en vectores descritas en el tema 2.

### Lista dinámica lineal

```
MÓDULO Buscar_elemento   devuelve puntero a NODO_LISTA
DATOS
        PARÁMETROS
                Recibe INICIO puntero a NODO_LISTA
                Recibe VALOR de tipo T
        VARIABLES
                P puntero a NODO_LISTA
INICIO
        P <- INICIO

        Mientras ( P ≠ NULO ) hacer
                Si (P↑.DATO.CLAVE = VALOR.CLAVE) entonces
                        DEVOLVER P
                sino
                        Si (P↑.DATO.CLAVE > VALOR.CLAVE) entonces
```

```
                    DEVOLVER NULO
                sino
                    P <- P↑.SIGUIENTE
                fin-si
            fin-si
        fin-mientras

        DEVOLVER NULO
FIN

                        Lista dinámica circular
MÓDULO Buscar_elemento  devuelve puntero a NODO_LISTA
DATOS
        PARÁMETROS
            Recibe INICIO puntero a NODO_LISTA
        Recibe VALOR de tipo T
VARIABLES
            P puntero a NODO_LISTA
INICIO
        Si (Lista_vacia(INICIO) = Falso) entonces
            P <- INICIO

            Repetir
                Si (P↑.DATO.CLAVE = VALOR.CLAVE) entonces
                    DEVOLVER P
                fin-si

                P <- P↑.SIGUIENTE
            Mientras ((P ≠ INICIO) Y (P↑.DATO.CLAVE ≤ VALOR.CLAVE))
        fin-si

        DEVOLVER NULO
```

**FIN**

## ORDENACIÓN DE LISTAS

Es posible ordenar en cualquier momento una lista que se haya creado sin orden, u ordenar por otro campo una lista ya ordenada. Para ello se pueden aplicar las técnicas de ordenación interna descritas en el primer tema. Tales técnicas resultan directamente aplicables a las listas representadas de forma estática, pudiéndose adaptar para su aplicación a listas representadas de forma dinámica.

### Representación vectorial

Se pueden aplicar las técnicas de ordenación de vectores descritas en el tema 2.

### Representación enlazada

Se podrán aplicar las técnicas de ordenación vistas para vectores, pero teniendo en cuenta que en este caso se accede a los elementos de la lista mediante punteros y no mediante índices.

A continuación se muestra el algoritmo de ordenación por selección directa aplicable a una lista lineal con enlace simple. Recordemos que este método toma el menor de los elementos que quedan por ordenar y lo coloca a continuación de los elementos ya ordenados.

Para realizar la ordenación, consideraremos dos listas auxiliares: una formada por los elementos ya ordenados y la otra formada por los elementos que quedan por ordenar. Inicialmente la primera lista estará vacía, mientras que la segunda será igual a la que se pretende ordenar. Para ordenar la lista original se tomará el menor elemento de la lista no ordenada y se colocará al final de la lista ordenada. Una vez procesados todos los elementos de la lista no ordenada la operación habrá concluido. Dado que en la lista no ordenada se realizarán búsquedas que operan sobre todos los nodos, resulta necesario un puntero al comienzo de dicha lista. Sin embargo, para la lista ordenada resulta útil mantener un segundo puntero al final de la lista, ya que las inserciones de nuevos nodos se realizan siempre al final de la estructura. Aunque se podría mantener la referencia a los nodos no ordenados con el puntero a la lista que se pretende ordenar (**INICIO**), usaremos un puntero auxiliar (**no_orden_i**) para hacer más sencilla la explicación.

**MÓDULO Metodo_seleccion_lista_lineal_simple**

**DATOS**

**PARÁMETROS**

**Transforma INICIO puntero a NODO_LISTA**

**VARIABLES**

**** punteros a los extremos inicial y final de la lista ordenada**

**orden_i, orden_f punteros a NODO_LISTA**

**no_orden_i punteros a NODO_LISTA**

```
        p_min, p_min_prev, q, q_prev punteros a NODO_LISTA
INICIO
    Si (Lista_vacia(INICIO) = Verdadero) entonces
        ACABAR
    fin-si

    orden_i <- NULO
    orden_f <- NULO
    no_orden_i <- INICIO

    Mientras (no_orden_i↑.SIGUIENTE ≠ NULO) hacer
        ** -A- Se busca el menor elemento de la lista no ordenada
        p_min_prev <- NULO
        p_min <- no_orden_i

        q_prev <- no_orden_i
        q <- no_orden_i↑.SIGUIENTE

        Mientras (q ≠ NULO) hacer
            Si( q↑.DATO < p_min↑.DATO ) entonces
                p_min_prev <- q_prev
                p_min <- q
            fin-si

            ** se avanza un nodo en la lista no ordenada
            q_prev <- q
            q <- q↑.SIGUIENTE
        fin-mientras

        ** -B- Se desconecta de la lista no ordenada
        Si (p_min_prev ≠ NULO) entonces
            p_min_prev↑.SIGUIENTE <- p_min↑.SIGUIENTE
```

```
            fin-si

            Si (p_min = no_orden_i) entonces
                  no_orden_i <- no_orden_i↑.SIGUIENTE
            fin-si

            ** -C- Se conecta a la lista ordenada
            Si (orden_i = NULO) entonces
                  orden_i <- p_min
            sino
                  orden_f↑.SIGUIENTE <- p_min
            fin-si

            orden_f <- p_min

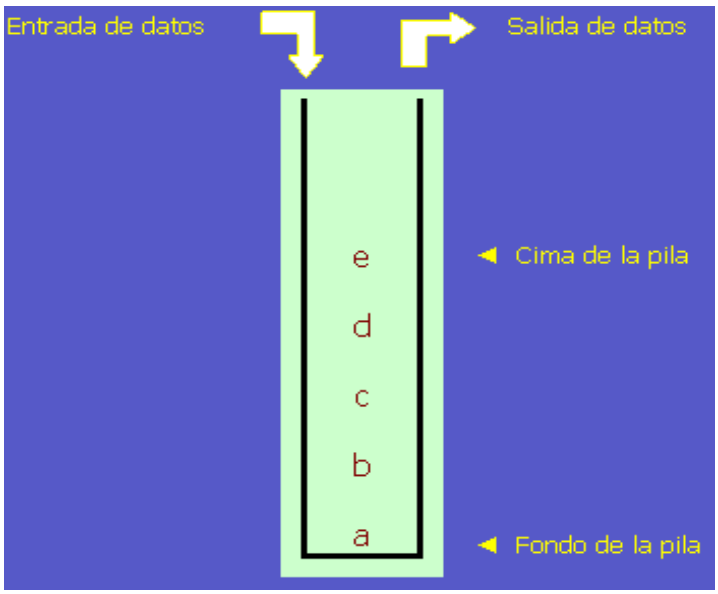
            p_min↑.SIGUIENTE <- NULO
      fin-mientras

      ** se enlaza a la lista ordenada el último nodo
      orden_f↑.SIGUIENTE <- no_orden_i
      INICIO <- orden_i
FIN
```

## 3.2 Pilas

### *3.2.1 Definición*

Una pila es un conjunto ordenado de datos, caracterizado porque las operaciones de inserción y eliminación de elementos se realizan solamente en un extremo de la estructura, que suele denominarse **cima** o **tope** de la pila.




**Esquema de pila**

Por tanto, una pila es una estructura de tipo **LIFO (Last In First Out)**, en la que el último elemento introducido es el primero que sale de la estructura.

Dada una pila como la representada en la figura anterior, que contiene un conjunto de elementos (a, b, c, d, e), se dice que el elemento más inaccesible de la pila está en el **fondo** de la pila y que el elemento más accesible está en la **cima**.

### *3.2.2 Operaciones Básicas con Pilas*

Las operaciones relacionadas con el trabajo sobre la estructura de datos pila son:

- **Creación de la pila:** debe ser la primera operación que se realiza sobre la pila. La pila se crea vacía.
- **Apilado:** permite añadir un nuevo elemento a la pila, que se colocará en la cima.
- **Desapilado:** permite eliminar el elemento situado en la cima de la pila.
- **Acceso al elemento situado en la cima:** permite obtener el elemento situado en la cima de la pila. A diferencia de la operación de desapilado, en este caso el valor consultado permanece en la pila.
- **Comprobación de pila vacía:** permite comprobar si la pila contiene algún elemento. Es necesario realizar esta operación antes de un desapilado o un acceso al valor de la cima, para evitar un error al intentar acceder a un elemento inexistente.

Para describir cada una de las operaciones, tendremos en cuenta la forma de representación considerada para la pila.

Si se utiliza una representación estática de la estructura, será necesaria una operación de **comprobación de pila llena** antes de realizar un apilado.

## *3.2.3 Representación de Pilas*

### *Representación Vectorial de Pilas*

Se puede utilizar un vector para almacenar una pila, que en este caso estará limitada en su capacidad por el tamaño del vector.

Para definir el vector de datos de algún tipo, T, indicaremos el nombre del vector, el tamaño, N, y el tipo. En pseudocódigo:

**PILA(N) de tipo T**

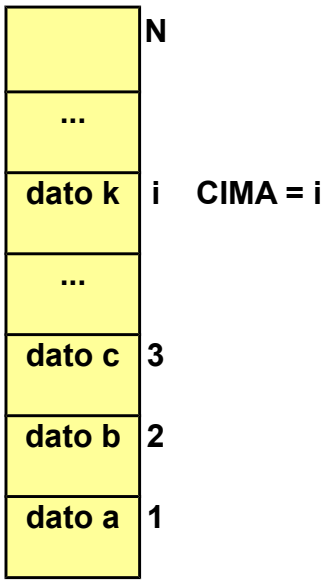
El tamaño del vector condicionará la capacidad de almacenamiento de la pila.

El primer elemento introducido en la pila se almacenará en PILA(1). Los sucesivos elementos se irán colocando en las siguientes posiciones del vector.

Para determinar cuántos elementos contiene la pila en cada momento, se utiliza una variable auxiliar entera, que indicará la posición del último elemento de la pila en cada momento:

**CIMA entera**

Cuando la pila no contenga ningún elemento, tendrá asociado el valor 0. Al añadir un nuevo elemento, se incrementará su valor en 1, llegando como máximo al valor N. Al eliminar un elemento de la pila, se reducirá el valor de la variable en 1.




**Esquema de pila implementada mediante un vector**

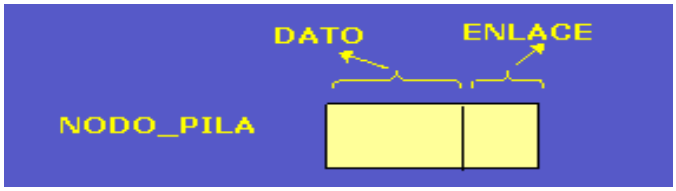
### *Representación de Pilas mediante el uso de asignación de Memoria Dinámica*

Una segunda posibilidad para representar pilas consiste en utilizar una lista enlazada. En este caso cada uno de los elementos de la pila estará almacenado en un nodo de la lista.

Para trabajar con esta representación en primer lugar hay que definir el tipo de los nodos de la lista enlazada. Cada nodo contendrá un campo de tipo puntero, que permite enlazar

unos nodos con otros, y uno o más campos que almacenen la información de la pila propiamente dicha (en este caso consideraremos un solo campo de algún tipo T)

**NODO_PILA es registro compuesto por**
**DATO de tipo T**
**ENLACE puntero a NODO_PILA**
**fin-registro**



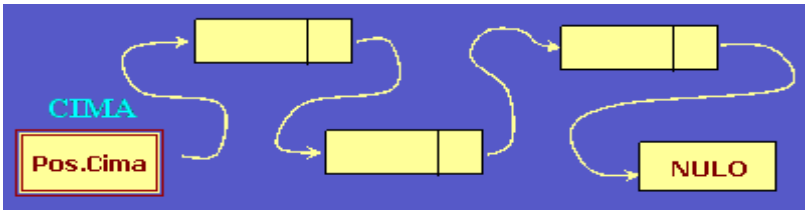

**Representación gráfica de un nodo**

Una vez definido el tipo de cada nodo, se declara un puntero a elementos de dicho tipo, que dará acceso a la pila, apuntando permanentemente a la cima. Inicialmente se le asocia el valor **NULO**, indicando que la pila se crea vacía:

**CIMA puntero a NODO_PILA**

**CIMA <- NULO**

Dado que en las pilas todas las operaciones se realizan sobre uno de sus extremos, sólo es necesario un puntero para gestionar la estructura de datos.




**Esquema de pila**
**implementada en Mem. Dinámica**

Una ventaja de esta representación frente a la anterior reside en que el tamaño máximo de la pila no está limitado de antemano.

### *3.2.4 Operaciones con Pilas*

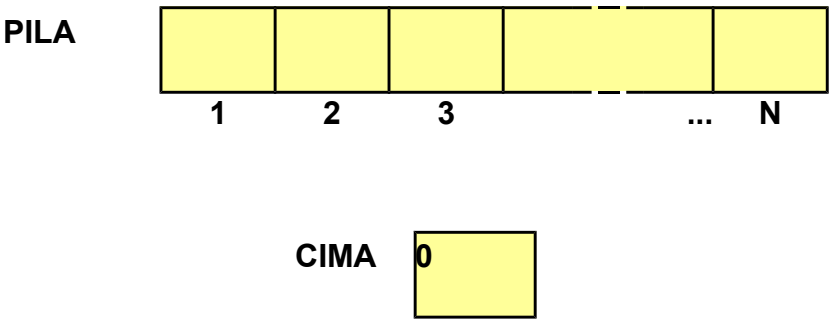
## *Operaciones mediante representación vectorial*

**Creación de la pila mediante representación vectorial**

La creación de la pila implica declarar el vector que almacenará la información, declarar la variable que indicará la posición de la cima e inicializar dicha variable al valor 0, indicando que la creación implica que la pila está vacía.

**PILA(N) de tipo T**
**CIMA entera**
**CIMA <- 0**

**Creación de una pila - implementación vectorial**




**Representación estática de una pila y de la variable cima asociada**

### Comprobación de pila vacía

Para comprobar si la pila está vacía, hay que determinar si la variable CIMA es 0.

```
MÓDULO Pila_vacia   devuelve lógico
DATOS
        PARÁMETROS
                Recibe CIMA entera
INICIO
        Si (CIMA = 0) entonces
                DEVOLVER Verdadero        ** pila vacía
        sino
                DEVOLVER Falso             ** pila no vacía
        fin-si
FIN
```

### Comprobación de pila Llena

Al trabajar con una pila implementada mediante un vector de tamaño fijo, N, se debe comprobar que no se exceda la capacidad del vector antes de intentar añadir nuevos elementos.

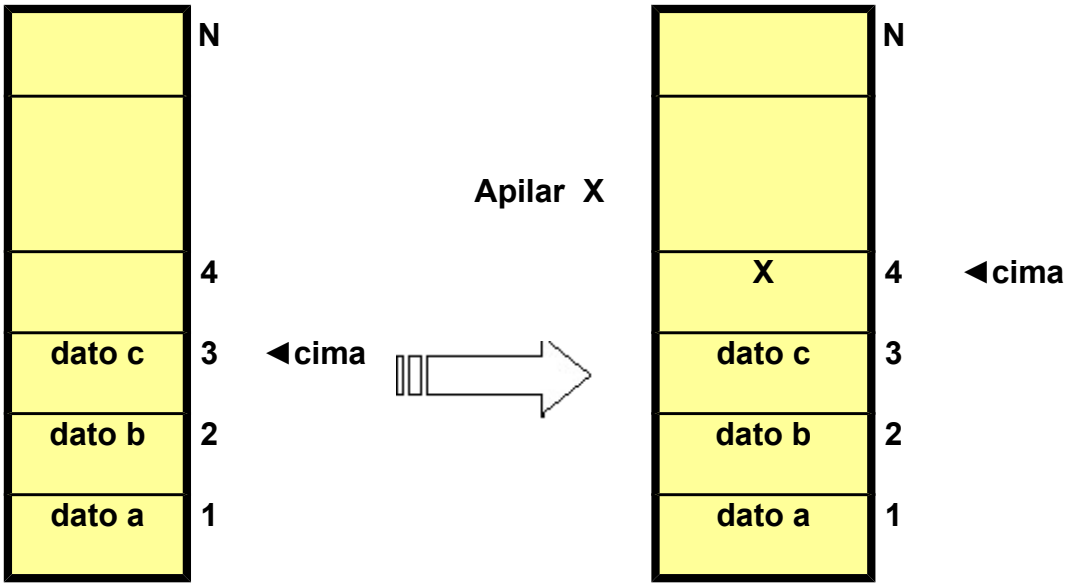
La pila estará llena si la CIMA es igual al tamaño del vector que la representa.

```
MÓDULO Pila_llena   devuelve lógico
DATOS
        PARÁMETROS
                Recibe CIMA entera      ** posición de la cima
                Recibe N entera          ** tamaño del vector
INICIO
        Si (CIMA = N) entonces
                DEVOLVER Verdadero         ** pila llena
        sino
                DEVOLVER Falso              ** pila no llena
        fin-si
FIN
```

### Operación de Apilado

Al implementar la pila mediante un vector, sólo se podrá realizar un apilado si la pila no está llena.

La operación de apilado consiste en añadir un nuevo elemento a la pila, que pasará a colocarse en la posición que sigue a la identificada por CIMA. El valor de la variable CIMA se incrementará en 1, para hacer referencia a la posición del nuevo valor introducido.

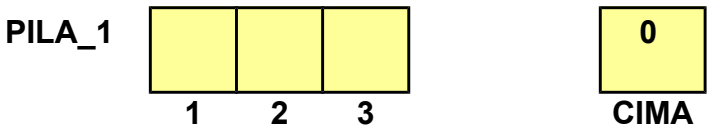



**Apilado del dato X**

```
MÓDULO Apilar
DATOS
        PARÁMETROS
                Transforma CIMA entera        ** posición de la cima
                Transforma PILA() de tipo T   ** vector pila
                Recibe ELEMENTO de tipo T     ** dato a apilar
                Recibe N entera               ** tamaño del vector
INICIO
        Si (Pila_llena(CIMA, N) = Verdadero) entonces
                ESCRIBIR "Pila  llena. No se puede apilar"
        sino
                CIMA <- CIMA + 1
                PILA(CIMA) <- ELEMENTO
        fin-si
FIN
```

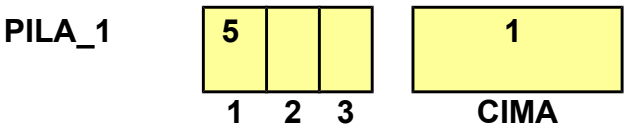
**Apilado - implementación vectorial**

Consideremos una pila representada mediante un vector con capacidad para 3 enteros, que actualmente está vacía:

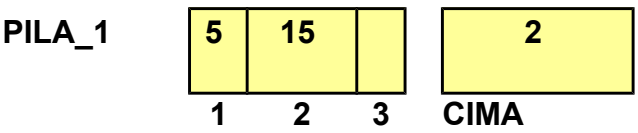



Comprobemos el resultado obtenido al apilar sucesivos valores:

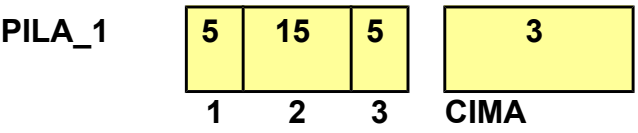
**Apilar(CIMA, PILA_1, 5, 3)**




**Apilar(CIMA, PILA_1, 15, 3)**




**Apilar(CIMA, PILA_1, 5, 3)**




**Apilar(CIMA, PILA_1, 10, 3)**

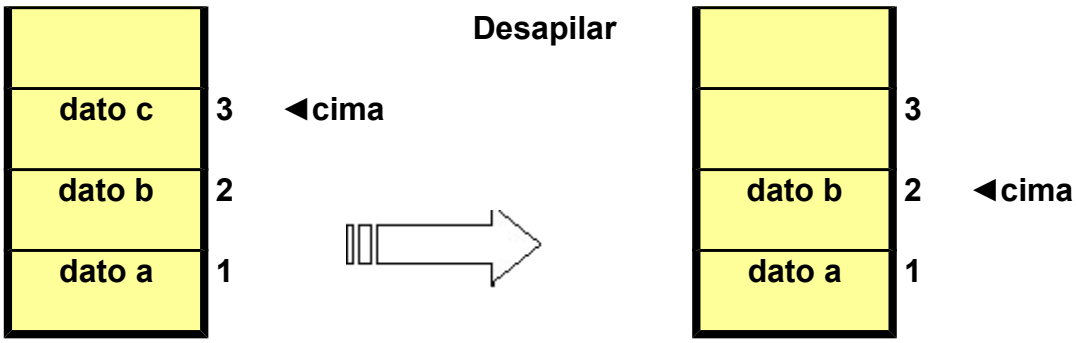
La última operación de apilado no se puede completar, dado que la pila está llena. Por tanto, en este caso se muestra un mensaje en la pantalla, para informa al usuario.

**Pila llena. No se puede apilar**

**Operación de desapilado**

La operación de desapilado elimina de la estructura el elemento situado en la posición indicada por la variable CIMA, siempre que exista algún elemento en la pila. Por ello, antes de proceder a eliminar dicho valor, se debe comprobar que la pila no está vacía. La operación se completa reduciendo en 1 el valor de la variable CIMA.





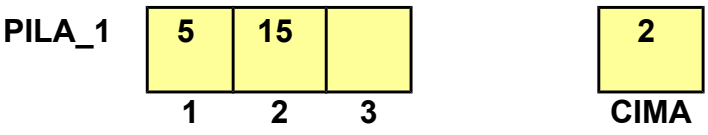

**Desapilado de un dato**

```
MÓDULO Desapilar   devuelve lógico
DATOS
      PARÁMETROS
            Transforma ELEMENTO de tipo T   ** dato desapilado
            Transforma CIMA entera          ** posición de la cima
            Transforma PILA() de tipo T     ** vector pila
INICIO
      Si ( Pila_vacia(CIMA) = Falso ) entonces
            ELEMENTO <- PILA(CIMA)
            CIMA <- CIMA - 1
            DEVOLVER Verdadero      ** operación realizada
      sino
            Escribir "Pila vacía. No se puede desapilar"
            DEVOLVER Falso          ** operación no realizada
      fin-si
FIN
```

Como normalmente el módulo llamante realizará algún tipo de procesamiento sobre el dato desapilado, se devolverá un valor que indique el resultado de la operación. En este caso si la operación de desapilado concluye con éxito, se devuelve un valor Verdadero, devolviendo Falso en caso contrario. Si el módulo llamante obtiene Falso como retorno de la llamada, no deberá intentar acceder al primer argumento (ELEMENTO), pues no contendrá un valor vólido.

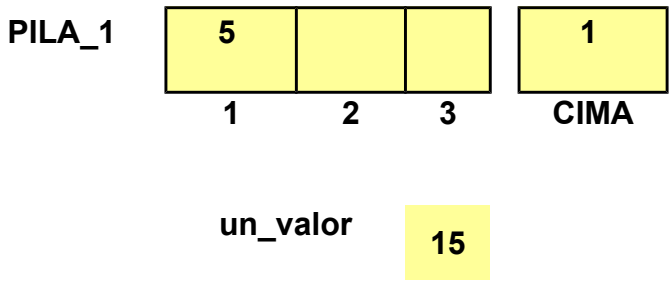
**Desapilado - implementación vectorial**

Consideremos una pila representada mediante un vector con capacidad para 3 enteros, que actualmente contiene dos valores:

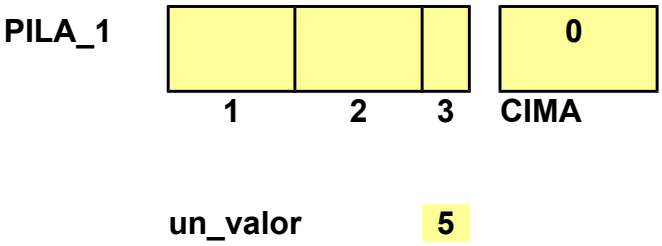



Comprobemos el resultado obtenido al desapilar sucesivos valores:

**Desapilar(un_valor, CIMA, PILA_1)**




**Desapilar(un_valor, CIMA, PILA_1)**




**Desapilar(un_valor, CIMA, PILA_1)**

La última operación de desapilado no se puede completar, dado que la pila está vacía. Por tanto, en este caso se muestra un mensaje en la pantalla, para informa al usuario.

**Pila vacía. No se puede desapilar**

Aunque en la representación gráfica anterior se eliminan del vector que representa la pila los valores que se van desapilando, recordemos que la operación de eliminación de un valor de un vector realmente no suprime dicho valor. La modificación de la variable CIMA permite descartar el valor de la posición que se acaba de desapilar, aunque el valor siga físicamente almacenado hasta que se haga un nuevo apilado, que reemplazará dicho valor por otro.

### Comprobación del valor de la cima de la pila

Esta operación es parecida a la de desapilado, salvo porque el valor no se saca de la pila, sólo se consulta. Por tanto, el módulo propuesto es muy parecido al anterior, aunque los parámetros CIMA y PILA no se modifican:

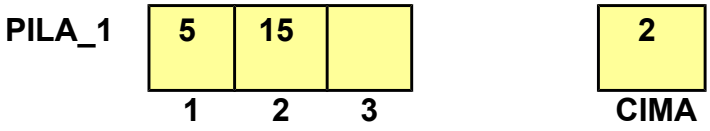
```
MÓDULO Valor_cima   devuelve lógico
DATOS
    PARÁMETROS
        Transforma ELEMENTO de tipo T   ** valor consultado
        Recibe CIMA entera              ** posición de la cima
        Recibe PILA() de tipo T         ** vector pila
INICIO
    Si ( Pila_vacia(CIMA) = Falso ) entonces
        ELEMENTO <- PILA(CIMA)
        DEVOLVER Verdadero        ** operación realizada
    sino
```

```
            Escribir "Pila vacía"
            DEVOLVER Falso              ** operación no realizada
        fin-si
    FIN
```

**Comprobación del valor de la cima - implementación vectorial**

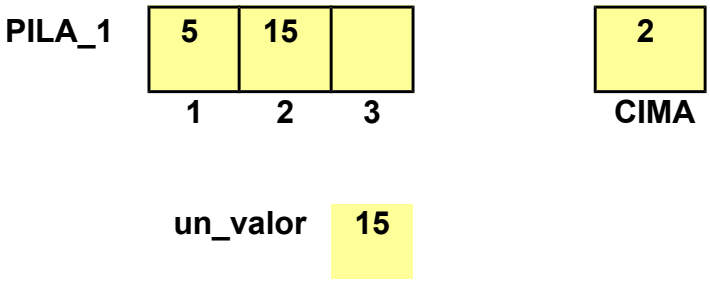
Por los mismos motivos que en la operación de desapilado, se informará al módulo llamante del resultado de las operaciones.

Consideremos la misma pila del ejemplo anterior:




Si deseamos comprobar el valor de la cima, observamos que la pila no se modifica:

**Valor_cima(un_valor, CIMA, PILA_1)**



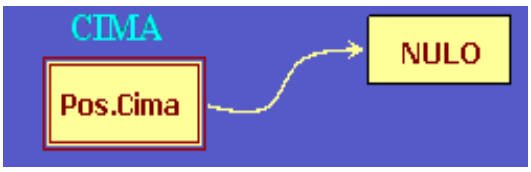

## *Operaciones mediante representación de Memoria Dinámica*

**Creación de la pila mediante representación con Memoria Dinámica**

La creación de la pila implica definir el tipo de los nodos que formarán la pila, definir la variable puntero que dará acceso a la pila e inicializar dicha variable al valor **NULO**, indicando que la creación implica que la pila está vacía.

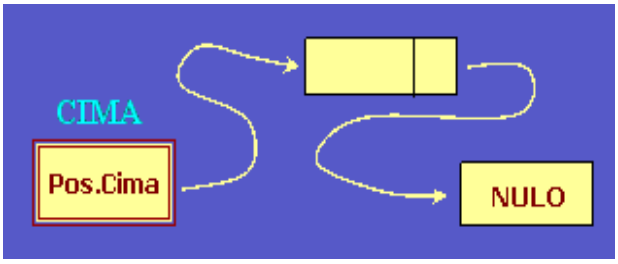
```
NODO_PILA es registro compuesto por
        DATO de tipo T
        ENLACE puntero a NODO_PILA
fin_registro
CIMA puntero a NODO_PILA
CIMA <- NULO
```

Con estas operaciones habremos reservado en memoria el espacio necesario para el puntero a la pila. Inmediatamente después de crear la pila ésta permanece vacía:




**Representación gráfica de pila vacía**

Cuando se añada el primer elemento a la pila, el puntero CIMA apuntará a dicho elemento:




**Pila con un nodo**

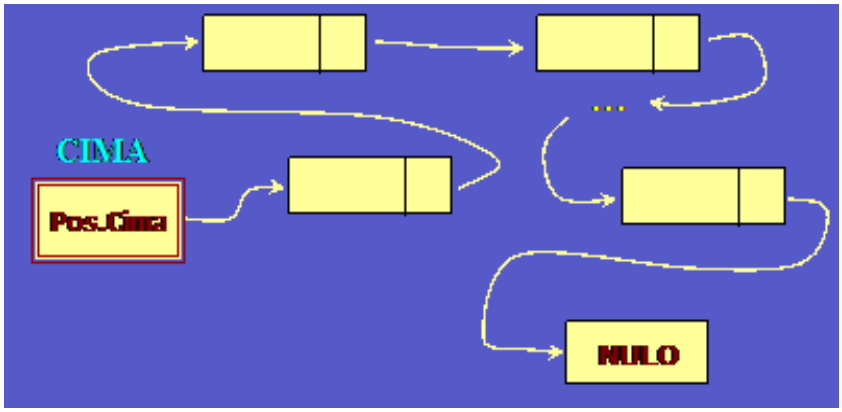
### Comprobación de pila vacía

Para comprobar si la pila está vacía, comprobaremos si el puntero CIMA es nulo.

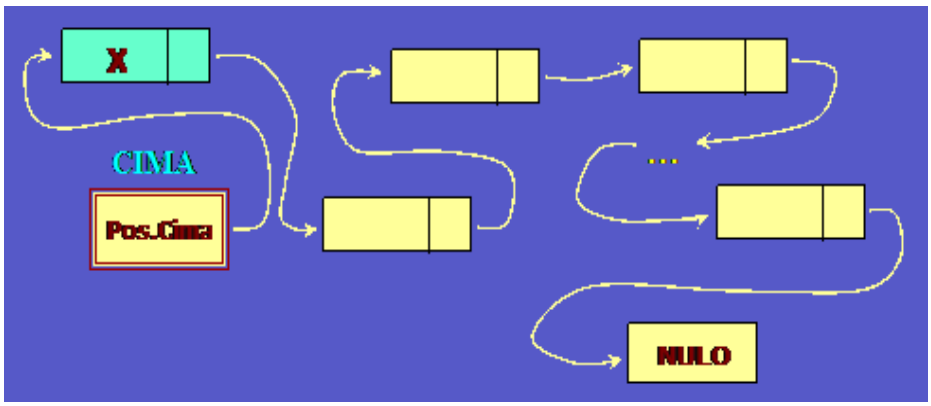
```
MÓDULO Pila_vacia   devuelve lógico
DATOS
        PARÁMETROS
                Recibe CIMA puntero a NODO_PILA
INICIO
        Si (CIMA = NULO) entonces
                DEVOLVER Verdadero              ** pila vacía
        sino
                DEVOLVER Falso                  ** pila no vacía
        fin-si
FIN
```

### Operación de Apilado

El apilado de un nuevo elemento supone añadir un nuevo nodo a la estructura, que se colocará en la cima.




**Apilar(X)**




El módulo que realiza el apilado recibe como argumentos el elemento que se desea incluir en la pila (ELEMENTO) y el puntero que da acceso a la misma:

```
MÓDULO Apilar
DATOS
        PARÁMETROS
                Recibe ELEMENTO de tipo T
                Transforma CIMA puntero a NODO_PILA
        VARIABLES
                NUEVO puntero a NODO_PILA

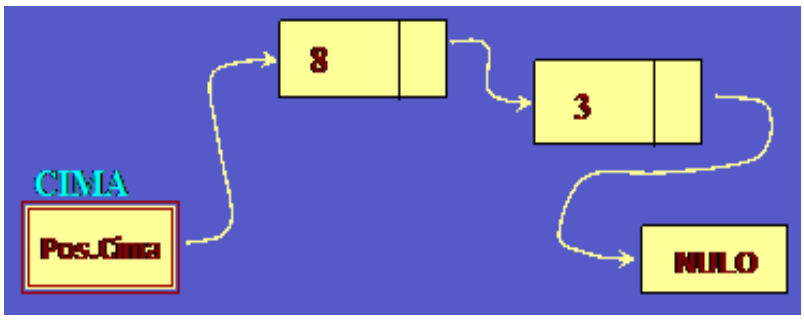
INICIO
        NUEVO ← RESERVAR_MEMORIA(tamaño(NODO_PILA))

        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO ← ELEMENTO
                NUEVO↑.ENLACE ← CIMA
                CIMA ← NUEVO
        fin-si
FIN
```

En primer lugar se solicita memoria para un nuevo nodo. Si la solicitud de memoria no tuvo éxito, se presenta un mensaje al usuario y el módulo concluye. Si la solicitud tuvo éxito, se almacena en el campo de datos del nuevo nodo el valor que se desea introducir en la pila. A continuación se hace que el nuevo nodo apunte al que hasta ahora era el último, es decir, al que estaba apuntado por **CIMA**. Por último, se actualiza el puntero **CIMA**, haciendo que apunte al nuevo nodo, es decir, dándole el valor del puntero **NUEVO**.

Consideremos una pila que almacena enteros a la que se accede mediante el puntero CIMA.

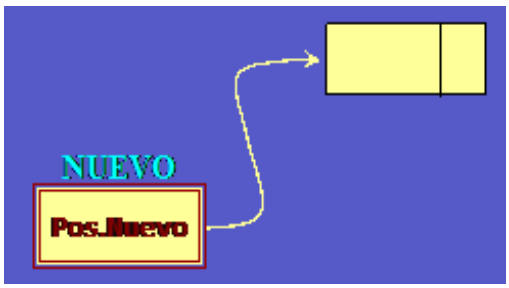
Supongamos que en este momento la pila contiene 2 enteros: el primero que se apiló fue el 3 y el segundo el 8:



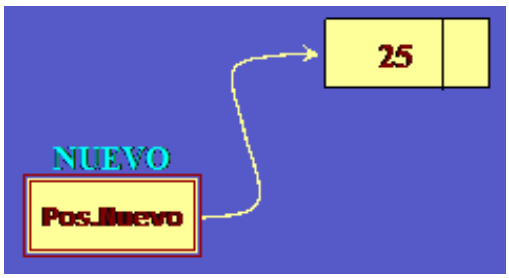

y se desea apilar el valor 25:

**Apilar(25,CIMA)**

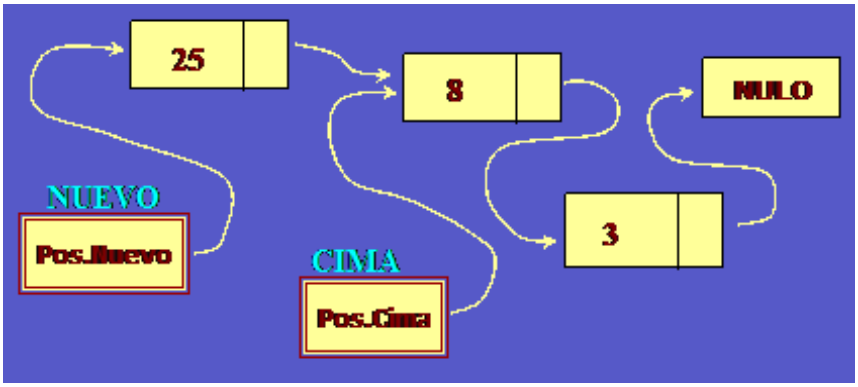
Para completar el apilado, en primer lugar se solicita un bloque de memoria dinámicamente. Suponiendo que no ha habido problemas con la solicitud, el puntero **NUEVO** apuntará a dicho bloque de memoria



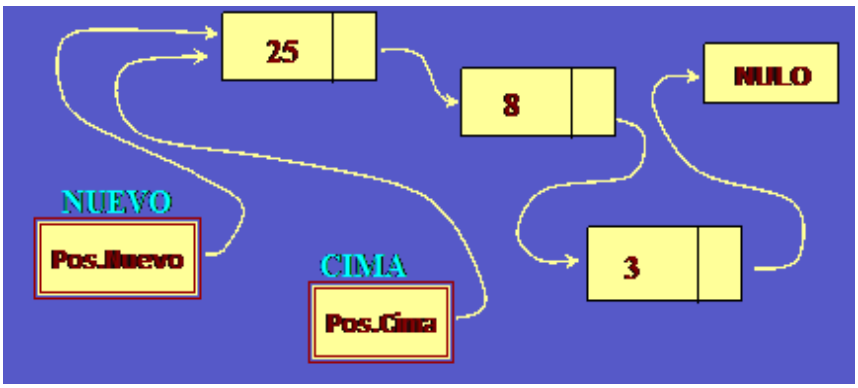

A continuación, se almacena el valor 25 en el campo de datos del nodo que se acaba de crear:



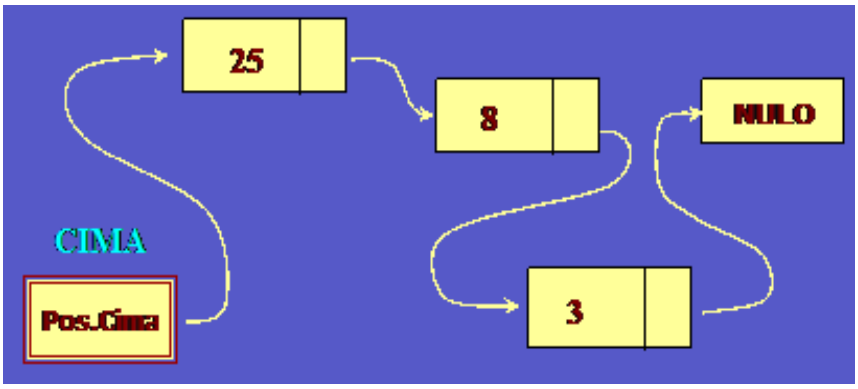

Se hace que el campo de enlace apunte al último elemento que se había introducido en la pila:




Por último, se hace que el puntero **CIMA** apunte a la nueva cima:

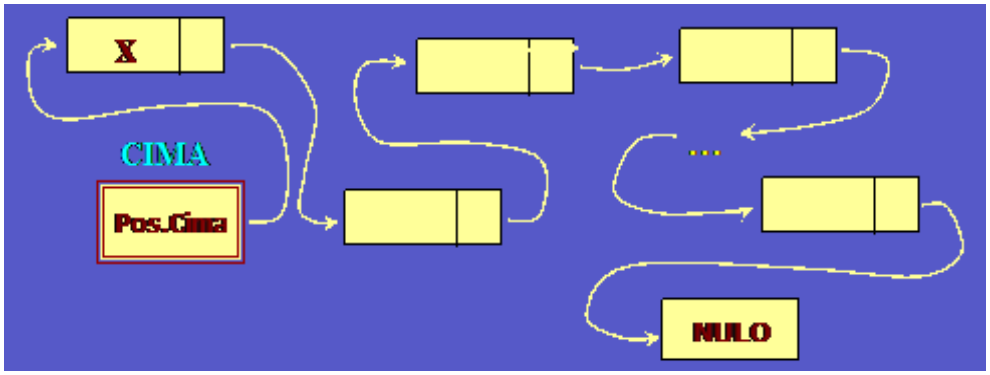



La lista quedará como sigue:

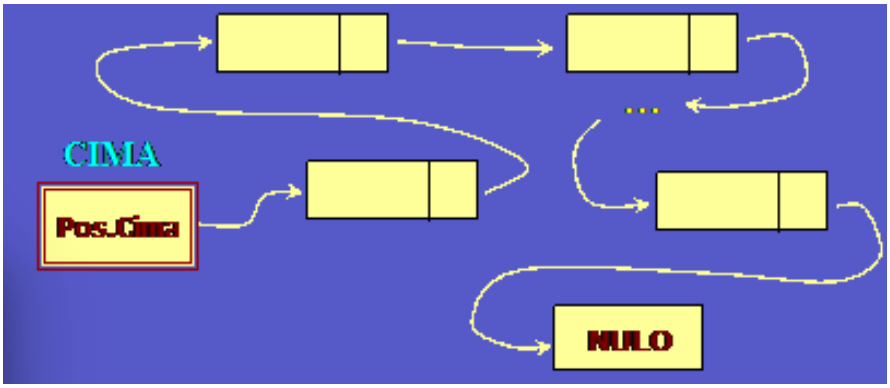



**Operación de Desapilado**

La operación de desapilado implica eliminar de la estructura el nodo situado en la cima:




**Desapilar(...)**

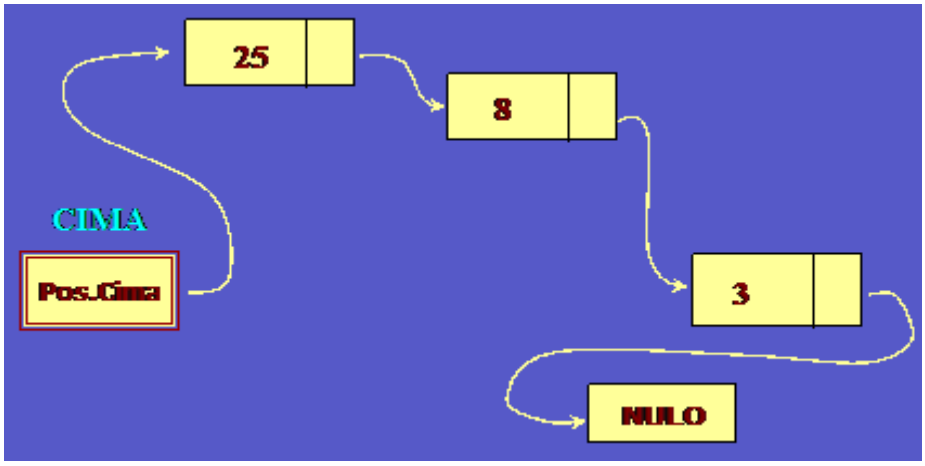


```
MÓDULO Desapilar  devuelve lógico
DATOS
    PARÁMETROS
        Transforma ELEMENTO de tipo T   ** dato desapilado
        Transforma CIMA puntero a NODO_PILA
    VARIABLES
        NODO puntero a NODO_PILA
INICIO
    Si ( Pila_vacia(CIMA) = Verdadero ) entonces
        ESCRIBIR "Pila vacía. No se puede desapilar"
        DEVOLVER Falso          ** operación no completada
    sino
        ELEMENTO <- CIMA↑.DATO
        NODO <- CIMA↑.ENLACE
        LIBERAR(CIMA)
        CIMA <- NODO
        DEVOLVER Verdadero      ** operación completada
    fin-si
FIN
```

Antes de proceder a desapilar un elemento, se debe comprobar que la pila no está vacía. En caso de estarlo, será imposible realizar un desapilado.

Si la pila no está vacía, se recuperará el campo de datos del nodo situado en la cima de la misma y, a continuación, se eliminará el nodo que contenía dicho dato. Por último, se hace que el puntero CIMA apunte a la nueva cima. Si al realizar una operación de desapilado la pila queda vacía, el puntero CIMA se hará **NULO**; en caso contrario, apuntará al elemento que hasta ahora estaba apuntado por el nodo que se ha eliminado de la pila.

Consideremos la pila obtenida en el ejemplo de apilado:



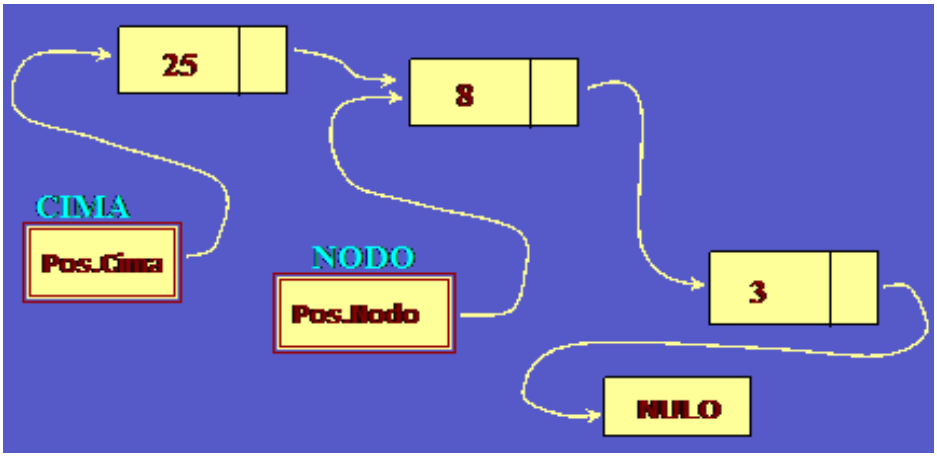

Observemos los pasos realizados al solicitar un desapilado:

**Desapilar(un_valor, CIMA)**

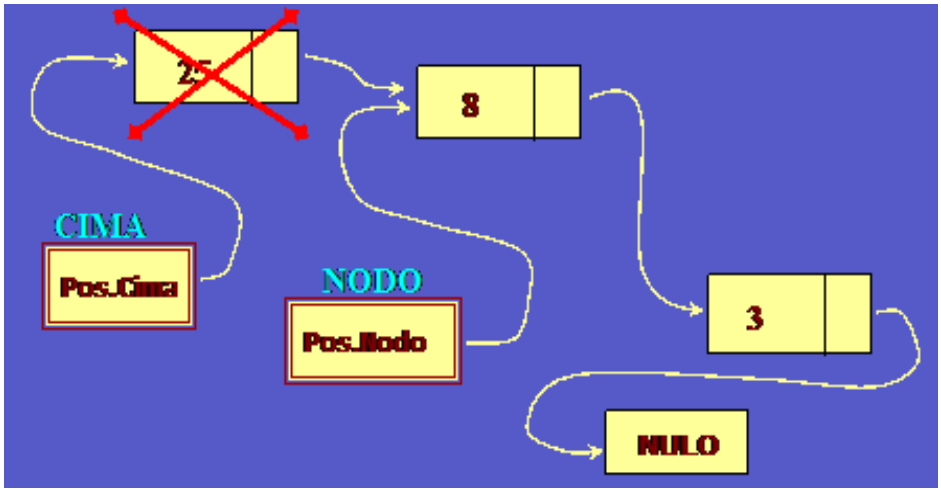
En primer lugar se copia en la variable **un_valor** el dato almacenado en el nodo que se va a eliminar:



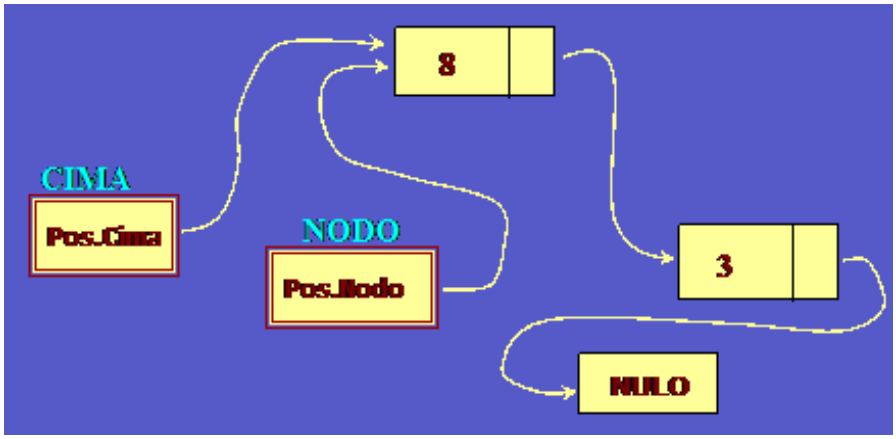

A continuación, se hace que el puntero **NODO** apunte al nodo siguiente al que va a ser eliminado:



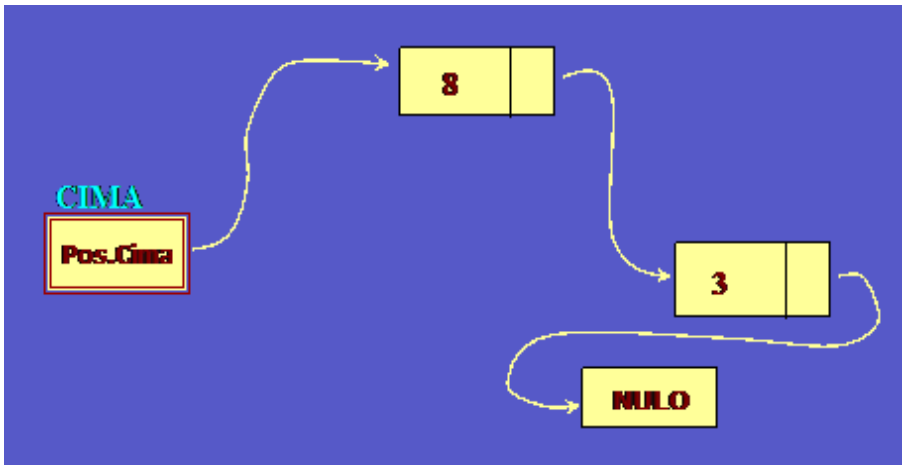

Se libera la memoria asociada al nodo apuntado por el puntero **CIMA**:




Por último, el puntero **CIMA** pasa a apuntar al mismo nodo que el puntero **NODO**:



La lista quedará con dos nodos:




### Comprobación del valor de la cima de la pila

La operación de comprobación del valor de la cima permite obtener el contenido del campo de datos del nodo situado en la cima de la pila, si ésta contiene algún elemento:

```
MÓDULO Valor_cima   devuelve lógico
DATOS
        PARÁMETROS
                Transforma ELEMENTO de tipo T
                Recibe CIMA puntero a NODO_PILA
INICIO
        Si ( Pila_vacia(CIMA) = Verdadero ) entonces
                ESCRIBIR "La pila está vacía"
                DEVOLVER Falso              ** operación no realizada
        sino
                ELEMENTO <- CIMA↑.DATO
                DEVOLVER Verdadero        ** operación realizada
        fin-si
FIN
```

### Operación de eliminación de la pila

Cuando se implementa una pila utilizando memoria dinámica, es conveniente eliminarla en el momento en que deje de ser necesaria.

El módulo que realiza la eliminación debe liberar cada uno de los nodos que queden en la pila. Para ello, puede utilizar un bucle que, mientras quede algún elemento en la pila, libere el que está en la cima. Con ello, el módulo se puede implementar realizando llamadas a los módulos **Pila_vacia** y **Desapilar**.

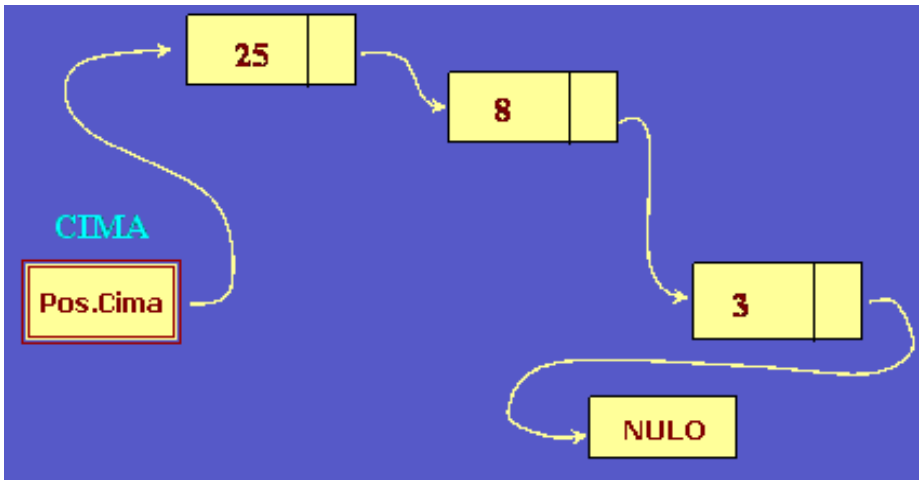
```
MÓDULO Eliminar
DATOS
        PARÁMETROS
                Transforma CIMA puntero a NODO_PILA
        VARIABLES
                ELEMENTO de tipo T
INICIO
        Mientras (Pila_vacia(CIMA) = Falso ) hacer
                Desapilar (ELEMENTO,CIMA)
        fin-mientras
FIN
```

También se puede construir un módulo que realice la eliminación sin realizar llamadas al módulo **Desapilar**:

```
MÓDULO Eliminar_2
DATOS
        PARÁMETROS
                Transforma CIMA puntero a NODO_PILA
        VARIABLES
                NODO puntero a NODO_PILA
INICIO
        Mientras (Pila_vacia(CIMA) = Falso ) hacer
                NODO <- CIMA.ENLACE
                LIBERAR(CIMA)
                CIMA <- NODO
        fin-mientras
FIN
```

En ambos casos, al concluir la llamada al módulo de eliminación, el puntero **CIMA** será **NULO**.

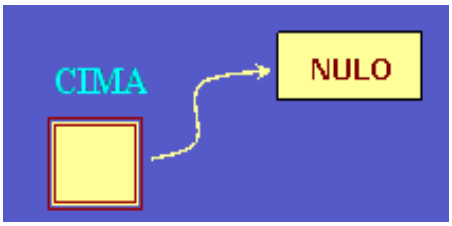
Supongamos que queremos vaciar la siguiente pila:




Al hacer una llamada al módulo **Eliminar**:

**Eliminar(CIMA)**

El resultado final será:




Es decir, se libera la memoria asociada a todos los nodos de la pila y el puntero **CIMA** se hace **NULO**, para indicar que la pila ha quedado vacía.

## 3.3 Colas

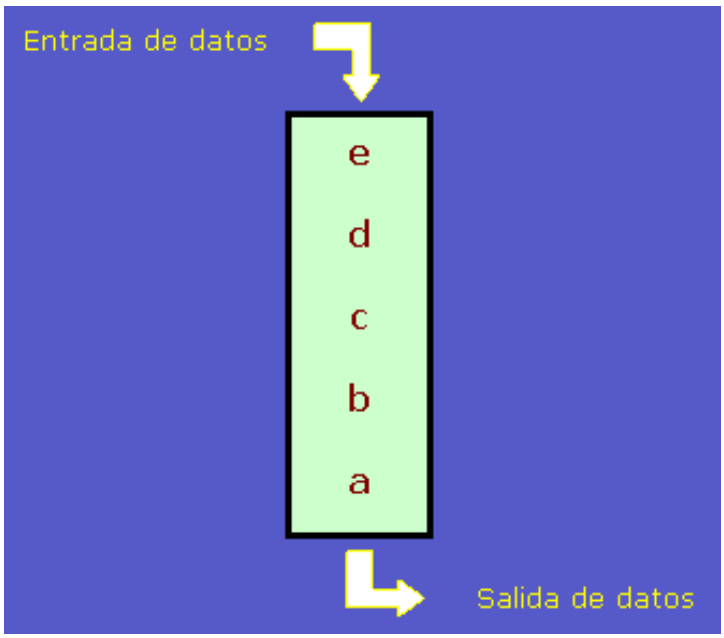
### *3.3.1 Definición*

Una cola es un conjunto ordenado de datos, caracterizado porque las operaciones de inserción y eliminación de elementos se realizan sobre extremos opuestos de la estructura.

Las colas son estructuras de tipo **FIFO (First In First Out)**, es decir, el primer elemento introducido en la estructura es el primero que sale de ella.

En la cola representada en la figura, se han introducido varias letras en orden alfabético (a, b, c, d, e). El primer elemento que abandonará la cola es la letra 'a', pues es la que lleva más tiempo en la estructura. El último elemento que ha entrado en la cola es la letra 'e'.

Al igual que ocurre con las pilas, una cola puede almacenar infinitos valores. Sin embargo, al representarla en un ordenador, su tamaño estará condicionado bien por la memoria de la máquina o bien por el tipo de representación que elijamos.




**Esquema de una cola**

### *3.3.2 Operaciones Básicas con Colas*

Las operaciones básicas sobre una cola son las siguientes:

- **Crear la cola:** debe ser la primera operación que se realiza sobre la cola. La cola se crea vacía.
- **Añadir un elemento a la cola:** permite colocar un elemento en el extremo de entrada a la estructura. Esta operación se suele denominar ***encolado***.
- **Eliminar un elemento de la cola:** permite eliminar de la estructura el elemento situado en el extremo de salida. Esta operación se suele denominar ***desencolado***. Requiere realizar una comprobación previa de cola no vacía, para evitar errores de operación.
- **Acceder al elemento situado en la salida de la cola:** consiste en comprobar el valor del elemento de la cola situado en la posición de salida, sin eliminar éste de la estructura. Esta operación también requiere una comprobación previa de cola no vacía.
- **Comprobar si la cola está vacía:** permite determinar si la cola no contiene ningún elemento.

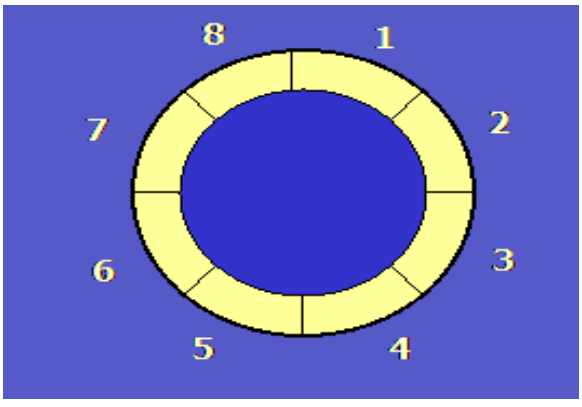
Para describir las operaciones anteriores, tendremos en cuenta la forma de representación considerada para la cola.

Si se utiliza una representación estática de la estructura, será necesaria una operación de **comprobación de cola llena** antes de realizar un encolado.

### *3.3.3 Representación de Colas*

#### *Representación Vectorial de Colas*

Existen varias formas de utilizar un vector para representar una cola, pero una de las más eficientes es representar el vector cola como si fuese circular. Es decir, dado un vector de tamaño N, consideramos que la posición 1 es la que sigue a la N. En la siguiente figura se representa un vector de 8 elementos como si fuera circular:

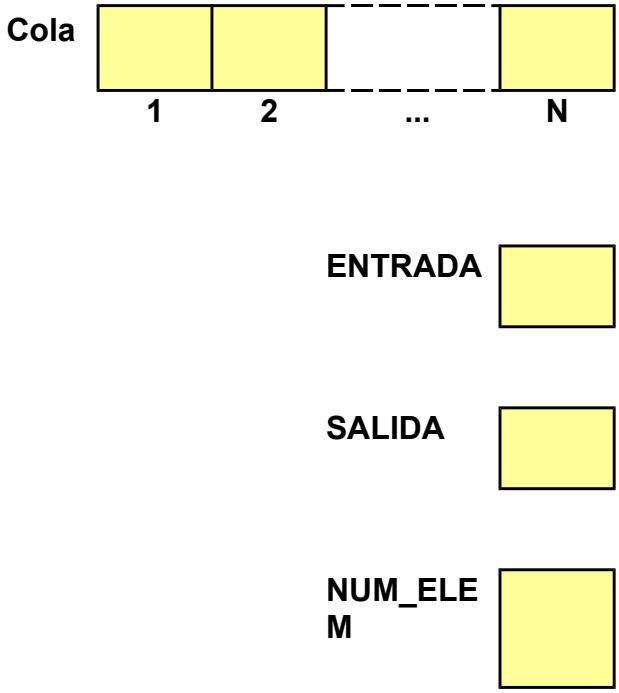



Para representar la cola mediante un vector circular se considerará un vector de N elementos y dos variables enteras, que llamaremos **ENTRADA** y **SALIDA**, que harán

referencia a posiciones del vector: ENTRADA hará referencia a la posición del vector en la que se realizó la última entrada de datos; SALIDA hará referencia a la posición del vector que contiene el dato que saldrá de la cola en la siguiente operación de desencolado. Se utiliza una variable entera, que denominaremos **NUM_ELEM**, para determinar el número de elementos que contiene la cola en cada momento.

**Cola(N) de tipo T**

**ENTRADA, SALIDA, NUM_ELEM enteras**




Cuando la cola está vacía **NUM_ELEM** será 0; cuando está llena será igual a N. Cuando la cola está vacía, tanto **ENTRADA** como **SALIDA** serán 0 (índice no válido dentro del vector).

Al encolar un valor, se avanza el índice **ENTRADA** una posición y se almacena el dato en dicha posición. Si la cola estaba vacía, el índice **SALIDA** toma el valor de **ENTRADA**. Al desencolar un valor, se toma el elemento de la posición referenciada por **SALIDA** y, a continuación, se avanza el índice **SALIDA** una posición. Antes de desencolar, se debe comprobar que la cola no está vacía. Dada la limitación de almacenamiento impuesta por la representación vectorial, antes de encolar un nuevo elemento se debe comprobar que la cola no está llena. Cuando al realizar un desencolado la cola quede vacía, se volverán a poner a 0 las variables **ENTRADA** y **SALIDA** (además del contador de elementos de la cola).

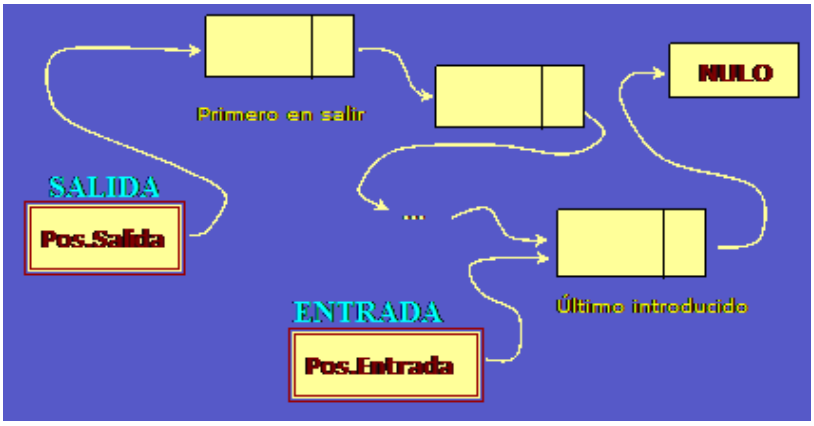
Al trabajar con un vector considerado como circular, se utilizará la operación módulo para determinar la siguiente posición en el vector. Dado un vector de N elementos, la posición siguiente a la I-ésima se determina aplicando la expresión: (**I MOD N**) + 1, donde MOD representa la operación módulo.

### *Representación mediante el uso de asignación de Memoria Dinámica*

Se puede representar una cola utilizando una lista enlazada. Para trabajar con esta representación en primer lugar hay que definir el tipo de los nodos de la lista enlazada. Cada nodo contendrá un campo de tipo puntero, que permite enlazar unos nodos con otros, y uno o más campos que almacenen la información de la cola propiamente dicha (en este caso consideraremos un solo campo de algún tipo T):

**NODO_COLA es registro compuesto por**
**DATO de tipo T**
**ENLACE puntero a NODO_COLA**
**fin-registro**

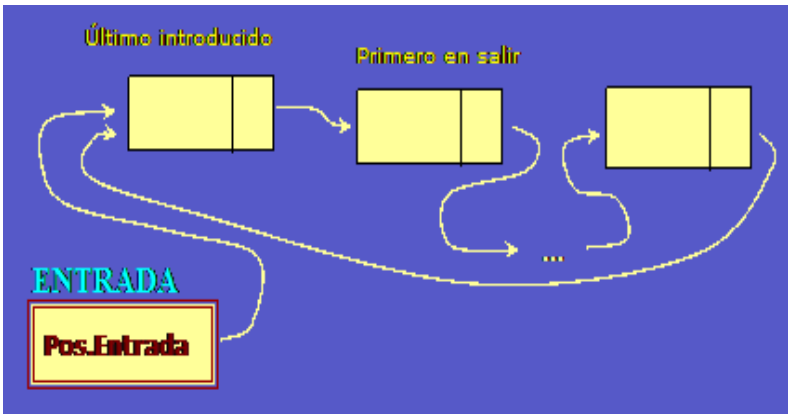
Se puede representar una cola mediante una lista lineal enlazada cuyos extremos estén apuntados por dos punteros, que identificarán el punto de entrada y el de salida de la cola.




El puntero **ENTRADA** apuntará al último nodo introducido en la cola, mientras que **SALIDA** apuntará al primer nodo que abandonará la cola. Si la cola está vacía, ambos punteros serán nulos; si no lo está, apuntarán a algún nodo de la cola.

Aunque podríamos utilizar un único puntero para gestionar la cola (el puntero **SALIDA** según el esquema propuesto), de forma similar a como estudiamos en el caso de las pilas, las operaciones en tal caso serían más costosas.

También se puede implementar una cola circular utilizando memoria dinámica. La cola circular estará formada por nodos similares a los de la cola dinámica lineal, y tendrá un puntero que permite acceder a la misma:



El puntero a la cola será inicialmente nulo. A medida que se añadan elementos a la cola, dicho puntero apuntará siempre al último elemento que haya entrado en la cola (por ello lo hemos denominado **ENTRADA**). El último nodo introducido apuntará siempre al primer nodo candidato a abandonar la cola (el que estaba apuntado por el puntero **SALIDA** en la representación lineal).

Igual que en la representación lineal, se podría trabajar con dos punteros a la estructura, aunque no resulta imprescindible.

Cuando representamos una cola utilizando memoria dinámica, debemos liberar la memoria que ocupa una vez que deje de ser necesaria.

### *3.3.4 Operaciones con Colas*

## *Operaciones mediante representación vectorial*

**Creación de la cola mediante representación vectorial**

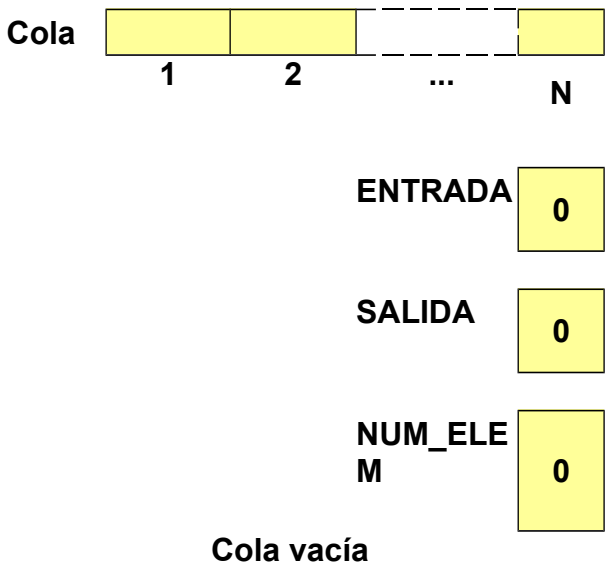
La creación de la cola consiste en definir la variable de tipo vector que almacenará los valores de la cola, así como las variables que identificarán los extremos de la estructura y el contador de elementos de la cola:

**COLA(N) de tipo T**
**ENTRADA, SALIDA, NUM_ELEM enteras**

A continuación, se indicará que la cola está vacía, haciendo cero el contador de elementos de la cola. Además, se hacen cero las variables que indican la posición de entrada y de salida a la cola.

**NUM_ELEM <- 0**

**ENTRADA <- 0**
**SALIDA <- 0**




**Cola vacía**

### Siguiente elemento de un vector circular

Antes de comenzar a describir las operaciones sobre colas, propondremos un módulo que permite determinar el elemento siguiente a uno dado en un vector de N elementos considerado como circular.

```
MÓDULO Siguiente   devuelve entero
DATOS
        PARÁMETROS
                Recibe INDICE entera    ** una posición del vector
                Recibe N entera          ** tamaño del vector
INICIO
        DEVOLVER (INDICE MOD N ) +1
FIN
```

### Comprobación de Cola vacía

La cola está vacía si el contador de elementos ésta a 0.

```
MÓDULO Cola_vacia   devuelve lógico
DATOS
        PARÁMETROS
                Recibe NUM_ELEMENTOS entera
INICIO
        Si (NUM_ELEMENTOS = 0) entonces
                DEVOLVER Verdadero          ** cola vacía
        sino
                DEVOLVER Falso              ** cola no vacía
        fin-si
FIN
```

### Comprobación de Cola llena

Al utilizar una representación estática para la cola, ésta puede llenarse en algún momento, lo que impedirá añadir nuevos elementos a la estructura. Por ello, antes de intentar encolar un nuevo elemento se debe comprobar si la cola aún no está llena.

La cola estará llena si contiene tantos elementos como permite la capacidad del vector que la almacena.

```
MÓDULO Cola_llena   devuelve lógico
DATOS
        PARÁMETROS
                Recibe NUM_ELEM entera     ** elementos de la cola
                Recibe N entera            ** tamaño del vector
INICIO
        Si (NUM_ELEM = N) entonces
```

```
            DEVOLVER Verdadero          ** cola llena
        sino
            DEVOLVER Falso              ** cola no llena
        fin-si
FIN
```

### Operación de Encolado

La operación de encolado permite añadir un elemento a la estructura.

Al utilizar una implementación estática de la estructura cola, antes de intentar encolar se debe determinar si existe espacio en la cola. Una vez comprobado que la cola no está llena, se determina la posición que debe ocupar el nuevo elemento de la misma (la siguiente a la ocupada en el último encolado). Si antes de realizar este encolado la cola estaba vacía, el indicador de posición de salida de la cola debe pasar a hacer referencia a la posición que ocupará el dato que se va a encolar, que se convierte en el único de la cola. Por último, se almacena el nuevo dato en la posición identificada y se incrementa en uno el contador de elementos de la cola.

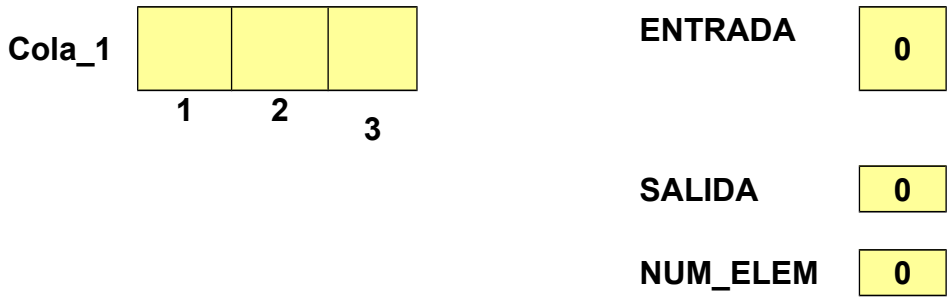
```
MÓDULO Encolar
DATOS
    PARÁMETROS
        Transforma COLA() de tipo T     ** vector cola
        Transforma ENTRADA entera       ** posición de entrada
        Transforma SALIDA entera        ** posición de salida
        Transforma NUM_ELEM entera      ** num. de elementos
        Recibe N entera                 ** tamaño del vector COLA
        Recibe ELEMENTO de tipo T       ** dato a encolar
INICIO
    Si (Cola_llena(NUM_ELEM, N) = Verdadero) entonces
        ESCRIBIR "Cola llena. No se puede encolar"
    sino
        ENTRADA <- Siguiente(ENTRADA, N)

        Si (Cola_vacia(NUM_ELEM) = Verdadero) entonces
            SALIDA <- ENTRADA
        fin-si

        COLA(ENTRADA) <- ELEMENTO
        NUM_ELEM <- NUM_ELEM + 1
    fin-si
FIN
```

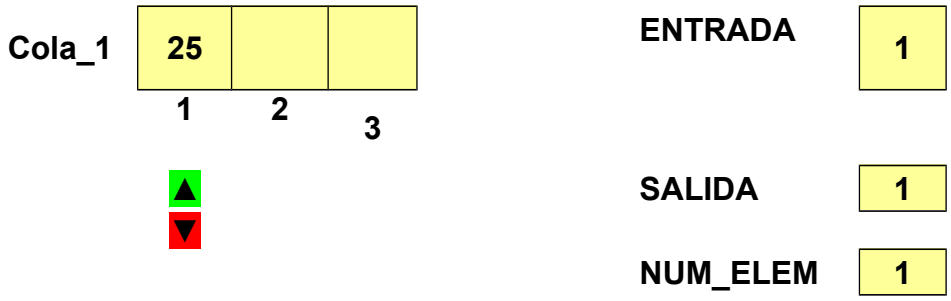
### Ejemplo de operación de encolado

Supongamos una cola de enteros representada mediante un vector de 3 elementos, que inicialmente está vacía:

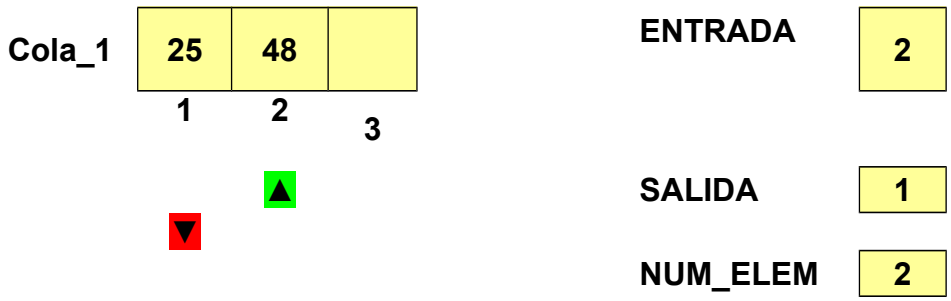



Mostraremos las modificaciones producidas en la cola al intentar encolar varios elementos:

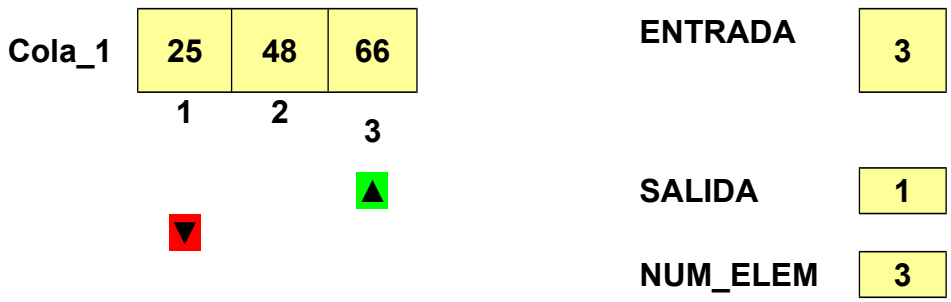
**Encolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, 25)**




**Encolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, 48)**




**Encolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, 66)**



**Encolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, 100)**

En este último caso no se puede completar la operación de encolado, dado que el vector que representa la cola está lleno. Por tanto, se mostrará un mensaje informativo:

**"Cola llena. No se puede encolar"**

### Operación de Desencolado

La operación de desencolado recupera el dato almacenado en la posición de salida de la cola y actualiza el valor del indicador de salida, de forma que pase a hacer referencia al siguiente candidato a abandonar la cola.

```
MÓDULO Desencolar  devuelve lógico
DATOS
   PARÁMETROS
      Recibe COLA() de tipo T             ** vector cola
      Transforma ENTRADA entera           ** posición de entrada
      Transforma SALIDA entera            ** posición de salida
      Transforma NUM_ELEM entera          ** num. de elementos
      Recibe N entera                     ** tamaño del vector COLA
      Transforma ELEMENTO de tipo T       ** dato a encolar
INICIO
   Si (Cola_vacia(NUM_ELEM) = Verdadero) entonces
      ESCRIBIR "Cola vacía. No se puede desencolar"
      DEVOLVER Falso                      ** operación cancelada
   sino
      ELEMENTO <- COLA(SALIDA)
      SALIDA <- Siguiente(SALIDA, N)
      NUM_ELEM <- NUM_ELEM -1

      Si (Cola_vacia(NUM_ELEM) = Verdadero) entonces
         ENTRADA <- 0
         SALIDA <- 0
      fin-si

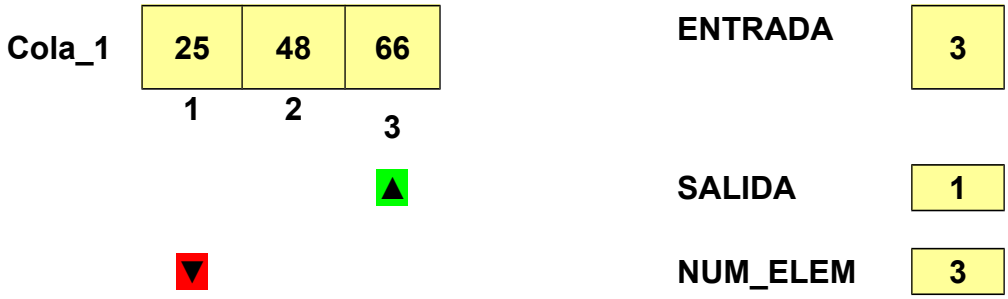
      DEVOLVER Verdadero                  ** operación realizada
   fin-si
FIN
```

En primer lugar se comprueba que la cola no está vacía, pues en caso de estarlo no se podrá realizar ningún desencolado. Si la cola no está vacía, se extrae el elemento situado en la posición de salida. A continuación se avanza el indicador de posición de salida hasta el elemento que abandonará la cola en la siguiente operación de desencolado, y se reduce en 1 el nmero de elementos que contiene la cola.

Tras esta operación de desencolado la estructura ha podido quedar vacía. En ese caso se vuelven a poner a 0 los indicadores de posición de entrada y salida.

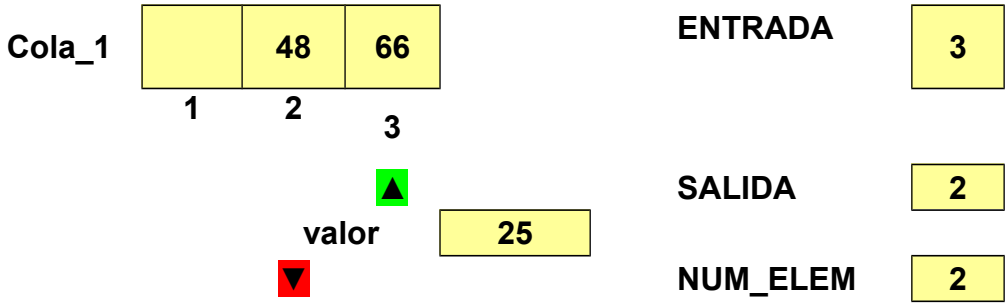
**Ejemplo de operación de desencolado**

Consideremos la cola creada en el apartado de encolado:

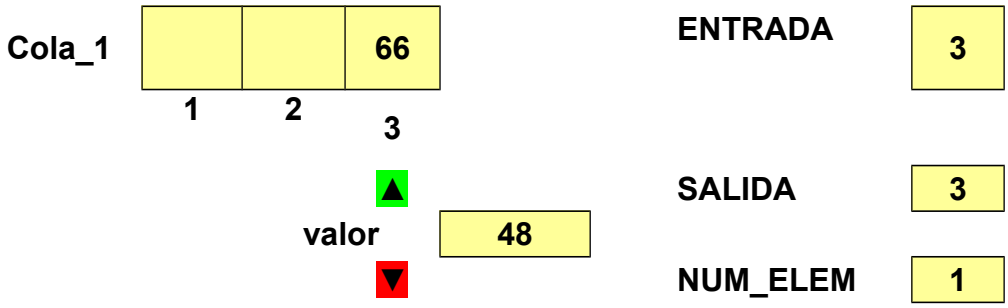



Mostraremos la evolución de la cola al realizar sucesivos desencolados:

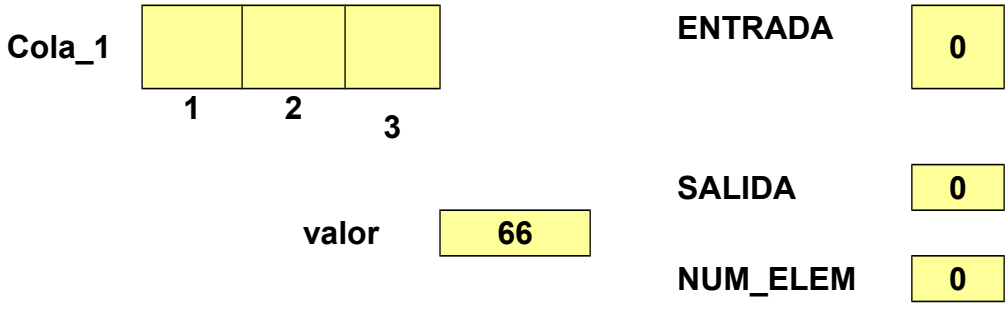
**Desencolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, valor)**




**Desencolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, valor)**




**Desencolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, valor)**




**Desencolar(Cola_1, ENTRADA, SALIDA, NUM_ELEM, 3, valor)**

En este último caso no se puede completar la operación de desencolado, dado que la cola

está vacía. Por tanto, se mostrará un mensaje informativo:

**"Cola vacía. No se puede desencolar"**

### Acceso al elemento situado en la salida de la cola

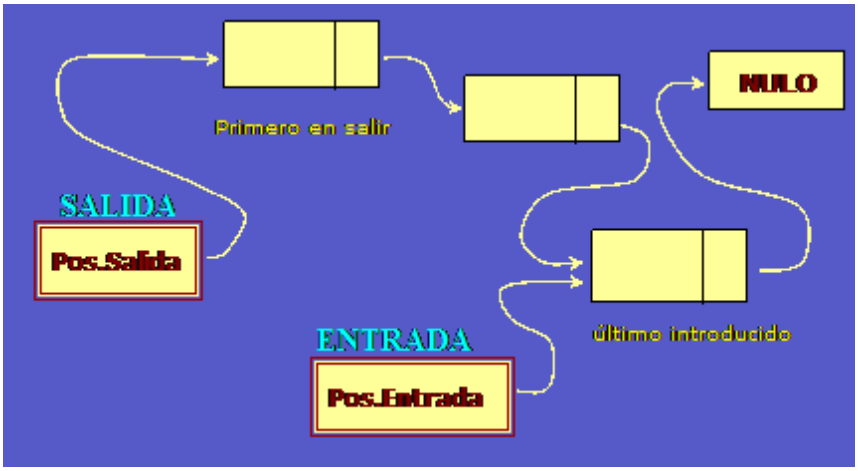
Esta operación permite obtener una copia del valor del elemento situado en la posición de salida de la cola. Antes de intentar acceder al elemento se debe comprobar que la cola no está vacía.

```
MÓDULO Acceder   devuelve lógico
DATOS
    PARÁMETROS
        Recibe COLA() de tipo T              ** vector cola
        Recibe SALIDA entera                 ** posición de salida
        Recibe NUM_ELEM entera               ** elementos de la cola
        Transforma ELEMENTO de tipo T        ** dato consultado
INICIO
    Si (Cola_vacia(NUM_ELEM) = Verdadero) entonces
        ESCRIBIR "La cola está vacía"
        DEVOLVER Falso                       ** operación no realizada
    sino
        ELEMENTO <- COLA( SALIDA )
        DEVOLVER Verdadero                   ** operación realizada
    fin-si
FIN
```

## *Operaciones mediante representación de Memoria Dinámica*

### Implementación con Cola Lineal

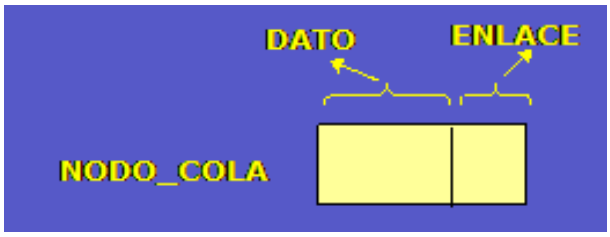
Esta cola se representa mediante una lista enlazada lineal con enlace simple, apuntada por dos punteros que identifican el punto de entrada y de salida de la estructura.



### Creación de la Cola Lineal mediante representación con Memoria Dinámica

Para crear la cola en primer lugar definiremos el tipo de los nodos que formarán la cola:

**NODO_COLA es registro compuesto por**
**DATO de tipo T**
**ENLACE puntero a NODO_COLA**
**fin-registro**




**Esquema de un nodo de la cola**

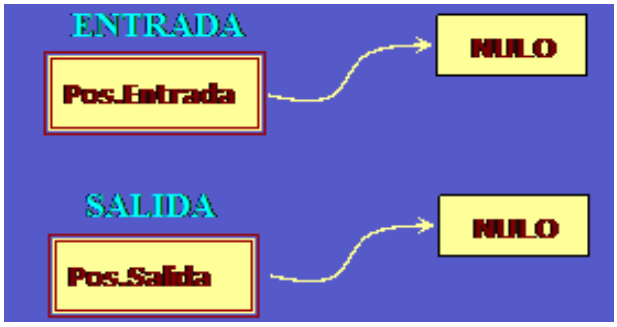
A continuación se definirán los dos punteros que dan acceso a la cola y, por último, se indicará que la cola inicialmente está vacía, haciendo nulos ambos punteros.

**ENTRADA, SALIDA punteros a NODO_COLA**

**ENTRADA <- NULO**
**SALIDA <- NULO**

El puntero **ENTRADA** identificará el extremo de la cola por el que entran nuevos elementos, mientras que el puntero **SALIDA** identificará el extremo opuesto, por el que salen los elementos de la cola.

Con estas operaciones habremos reservado en memoria el espacio necesario para los dos punteros. Inmediatamente después de crear la cola ésta permanece vacía:




**Representación gráfica de cola vacía**

### Comprobación de Cola Lineal vacía

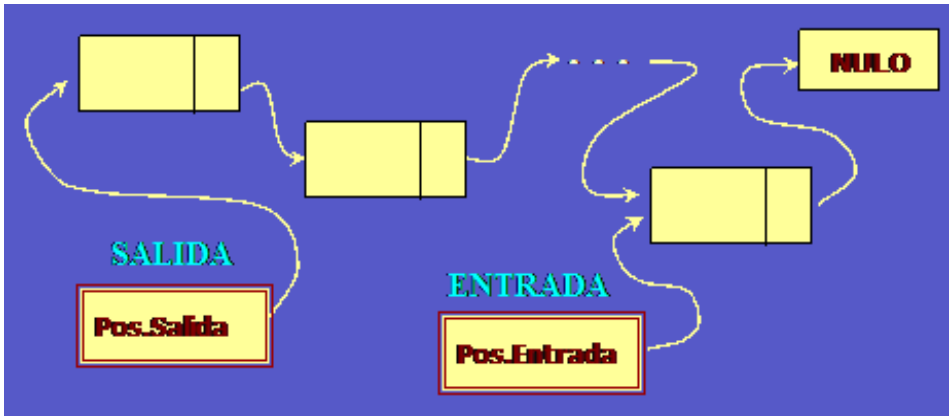
La cola estará vacía si los dos punteros a la misma son nulos. La comprobación se realizará sobre cualquiera de los dos punteros

**MÓDULO Cola_vacia devuelve lógico**

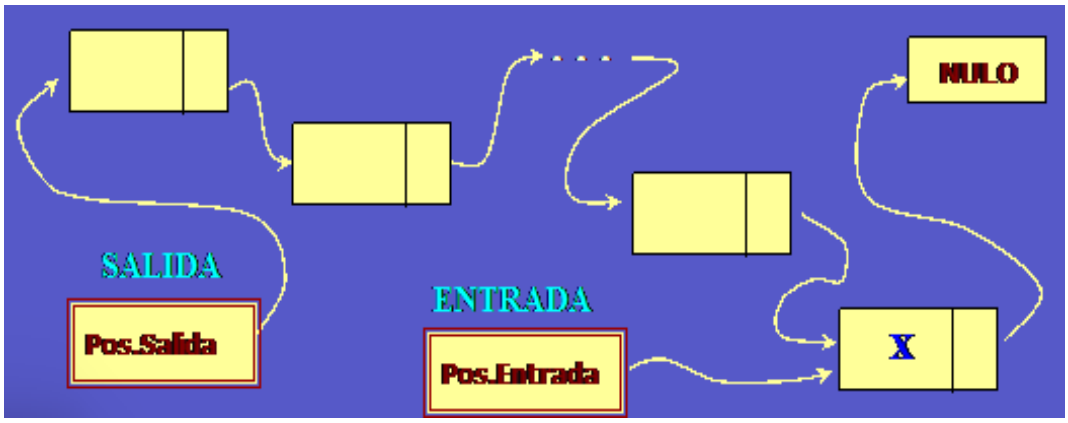
```
DATOS
        PARÁMETROS
                Recibe PTRO_COLA puntero a NODO_COLA
INICIO
        Si (PTRO_COLA = NULO) entonces
                DEVOLVER Verdadero          ** cola vacía
        sino
                DEVOLVER Falso              ** cola no vacía
        fin-si
FIN
```

### Operación de Encolado sobre Cola Lineal

La operación de encolado permite añadir un nuevo nodo a la estructura, que se colocará en el extremo de entrada:




**Encolar X**




Para añadir un nuevo elemento a la cola, la primera operación a realizar consiste en solicitar memoria para el nuevo nodo. Si la reserva tiene éxito, se copia en el campo de datos del nodo el valor del nuevo elemento. El campo **ENLACE** del nuevo nodo se hará nulo siempre, pues el nuevo nodo pasa a colocarse al final de la lista enlazada que representa la cola (al extremo derecho en la representación gráfica de la cola).

Si la cola estaba vacía, el nuevo elemento pasa a ser el único de la cola. Por ello, tanto el

puntero **ENTRADA** como el puntero **SALIDA** pasarán a apuntar a este nuevo elemento.

Si la cola ya contenía algún valor, el nodo que contiene el último de los valores encolados pasará a apuntar al nuevo. Además, el puntero **ENTRADA** también pasará a apuntar al nuevo elemento. En este caso el puntero **SALIDA** no se modifica.

```
MÓDULO Encolar

DATOS

        PARÁMETROS

                Recibe ELEMENTO de tipo T     ** dato a encolar

                Transforma ENTRADA puntero a NODO_COLA

                Transforma SALIDA puntero a NODO_COLA

        VARIABLES

                NUEVO puntero a NODO_COLA

INICIO

        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_COLA))

        Si (NUEVO = NULO) entonces

                ESCRIBIR "No hay memoria"

        sino

                NUEVO↑.DATO <- ELEMENTO

                NUEVO↑.ENLACE <- NULO

                Si (Cola_vacia(SALIDA) = Verdadero) entonces

                        SALIDA <- NUEVO

                sino

                        ENTRADA↑.ENLACE <- NUEVO

                fin-si

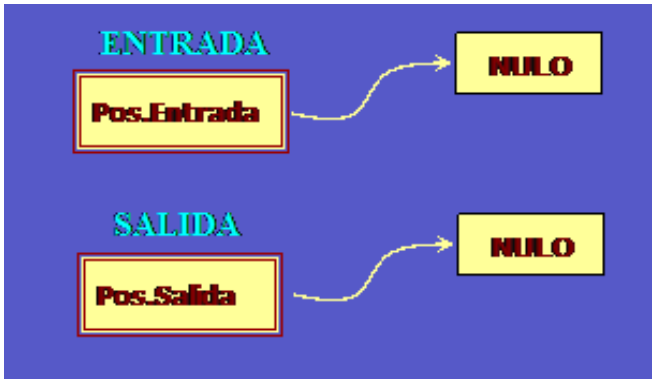
                ENTRADA <- NUEVO

        fin-si
```

**FIN**

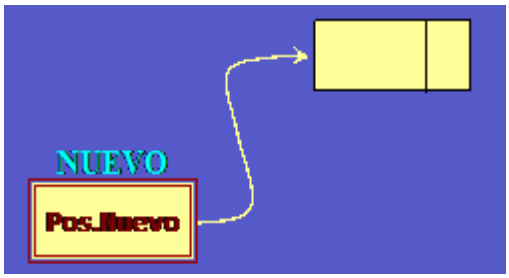
### Ejemplo de operación de encolado

Consideremos una cola que almacena enteros a la que se accede mediante los punteros **ENTRADA** y **SALIDA**. Supondremos que la cola está vacía y se intentan encolar dos valores: el 25 y el 30.

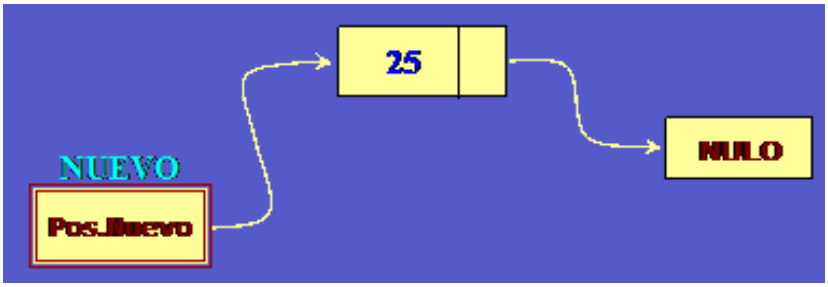



**Encolar(25, ENTRADA, SALIDA)**

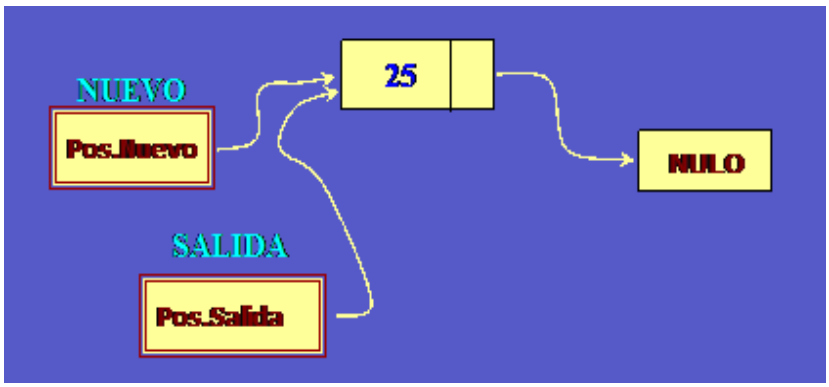
En primer lugar se solicita memoria para el nuevo nodo:



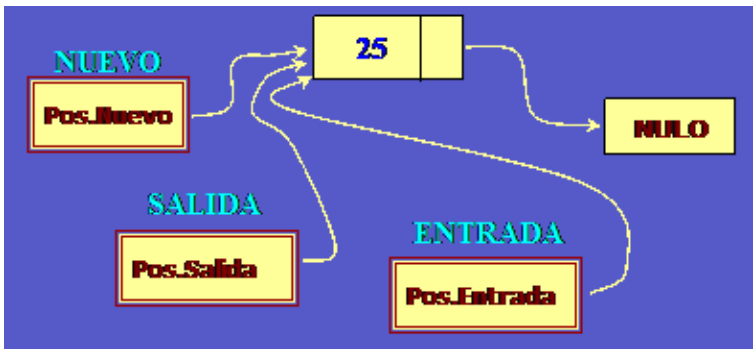

Si la solicitud tuvo éxito, se almacena el valor 25 en el campo de datos del nuevo nodo y se hace nulo el campo de enlace:



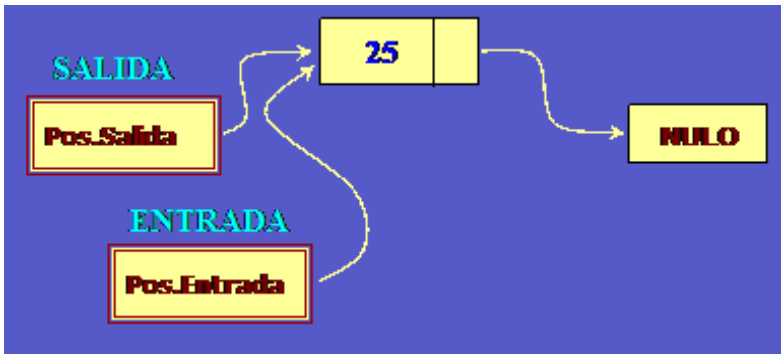

Como la cola estaba vacía, el puntero SALIDA pasa a apuntar al nuevo nodo:




Por último, el puntero ENTRADA pasa a apuntar al nuevo nodo:



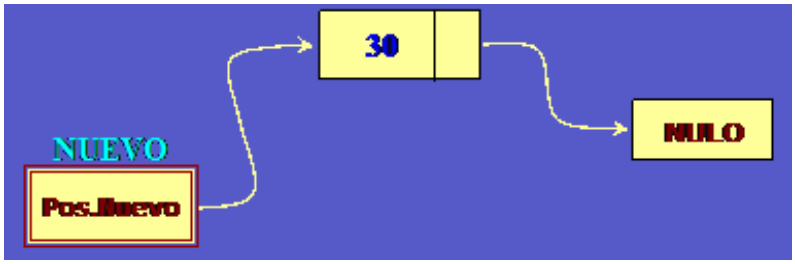

La cola quedará con un nodo:



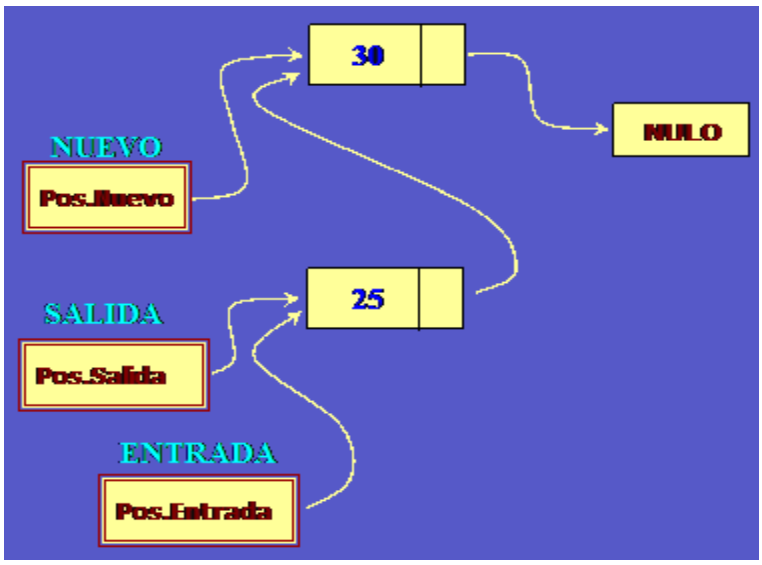

Se encola un nuevo elemento:

**Encolar(30, ENTRADA, SALIDA)**

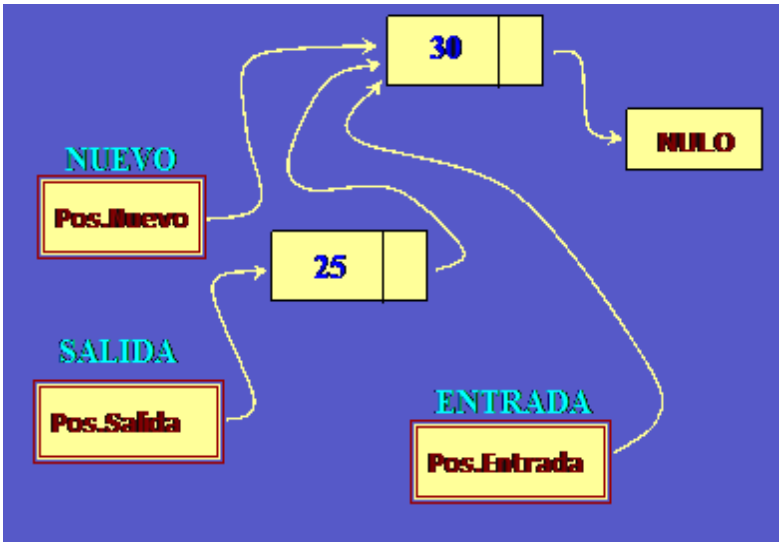
Una vez obtenida la memoria para el nuevo nodo, se almacena el valor 30 en su campo de datos y el valor NULO en su campo de enlace:



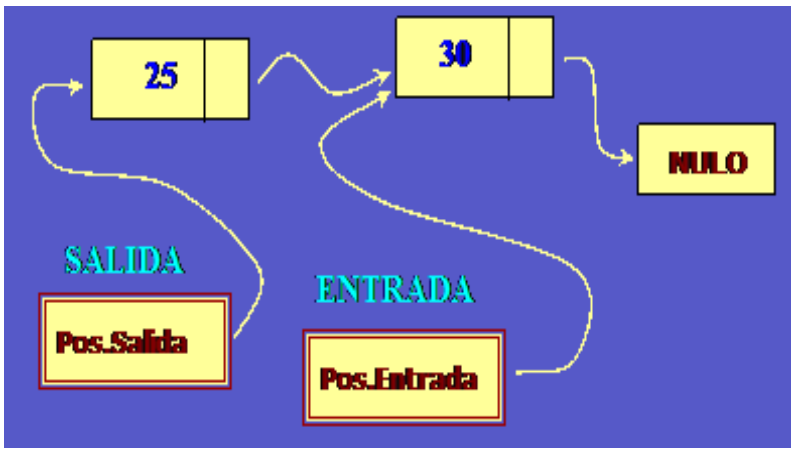

A continuación se hace que el último nodo que entró en la estructura pase a apuntar al nuevo:




Por último, el puntero ENTRADA pasará a apuntar al nuevo nodo, que es el último que ha entrado en la estructura:

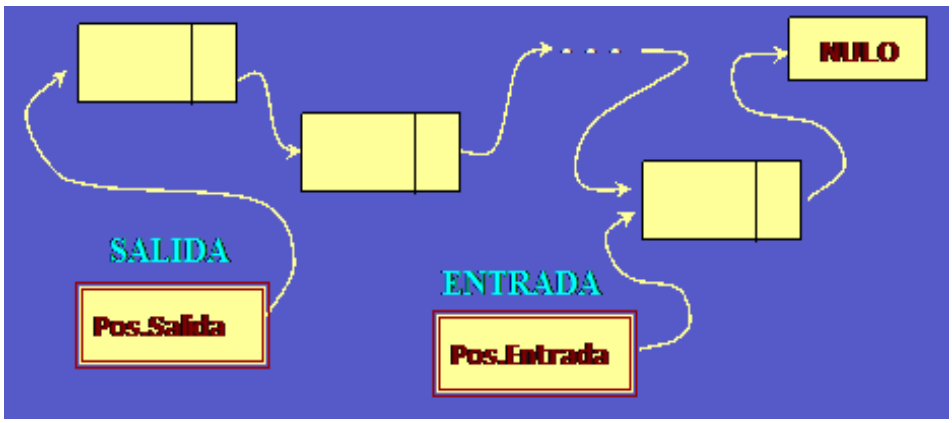



La cola almacena ahora 2 elementos:

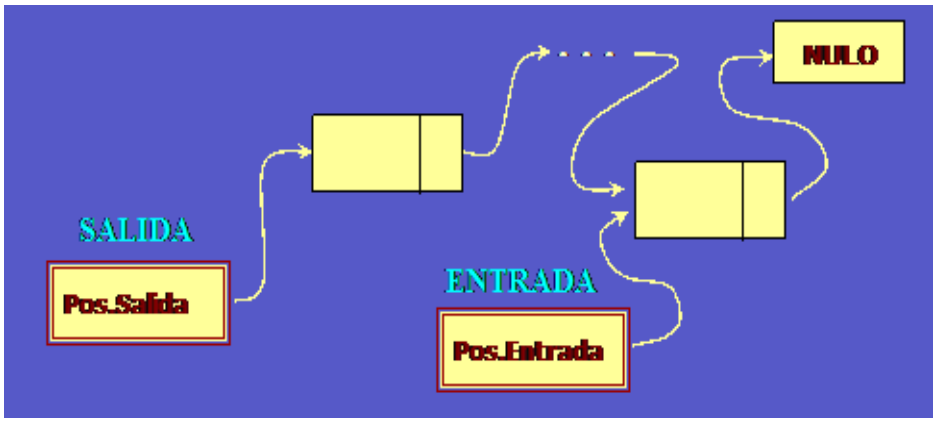



**Operación de Desencolado sobre Cola Lineal**

La operación de desencolado elimina de la estructura el nodo identificado por el puntero **SALIDA**:




**Desencolar**




Para eliminar un elemento de la cola, en primer lugar se comprueba que ésta contenga elementos. Si no está vacía, se recupera la parte de datos del nodo candidato a abandonar la cola (identificado por el puntero **SALIDA**) y se elimina el nodo correspondiente.

Si la cola sólo contenía un elemento, quedará vacía tras el desencolado, con lo que los punteros **ENTRADA** y **SALIDA** serán nulos.

Si contenía más de un elemento, el que era segundo candidato a abandonar la cola pasa a ser el primero, con lo que el puntero **SALIDA** debe apuntar a dicho elemento. En este caso el puntero **ENTRADA** no se verá afectado.

```
MÓDULO Desencolar  devuelve lógico
DATOS
    PARÁMETROS
        Transforma ELEMENTO de tipo T   ** dato desencolado
        Transforma ENTRADA puntero a NODO_COLA
        Transforma SALIDA puntero a NODO_COLA
    VARIABLES
        P puntero a NODO_COLA
INICIO
    Si (Cola_vacia(ENTRADA) = Verdadero) entonces
```

```
            ESCRIBIR "La cola está vacía"
            DEVOLVER Falso          ** operación no realizada
        sino
            ELEMENTO <- SALIDA↑.DATO
            P <- SALIDA
            SALIDA <- SALIDA↑.ENLACE

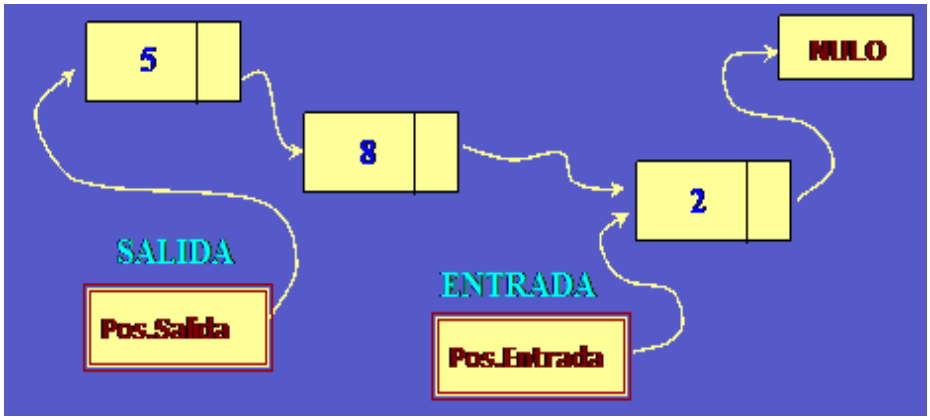
            Si (SALIDA = NULO) entonces
                ENTRADA <- NULO
            fin-si

            LIBERAR(P)

            DEVOLVER Verdadero   ** operación realizada
        fin-si
    FIN
```

**Ejemplo de operación de desencolado**

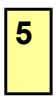
Consideremos una cola de enteros que contiene 3 elementos:



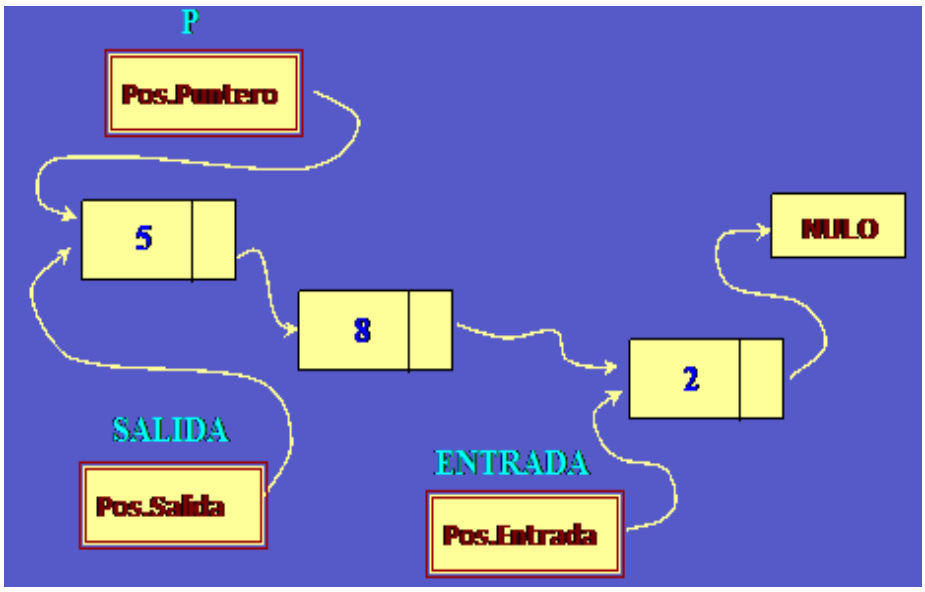

Sacamos el primer elemento:

**Desencolar(un_valor, ENTRADA, SALIDA)**

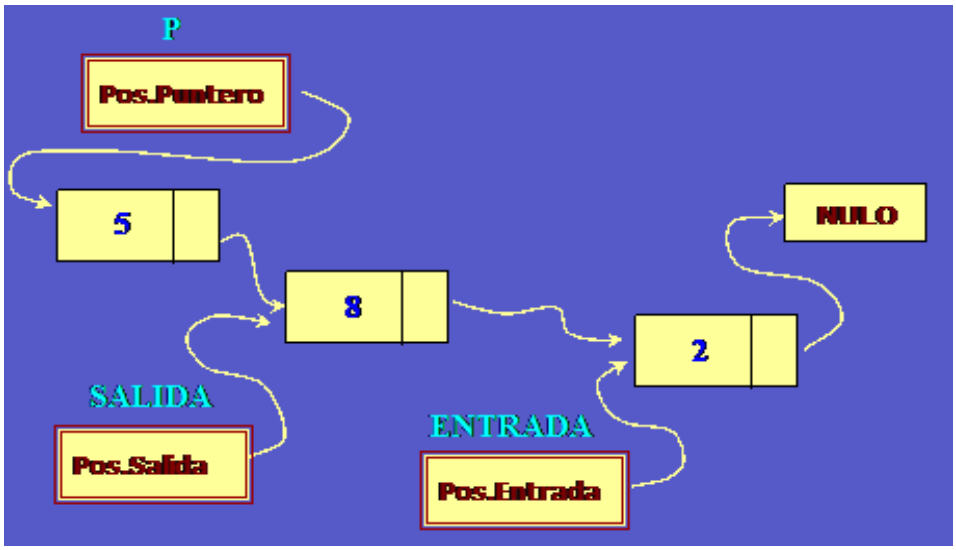
Dado que la cola no está vacía, se copia en el primer parámetro el dato almacenado en el nodo que saldrá de la cola:



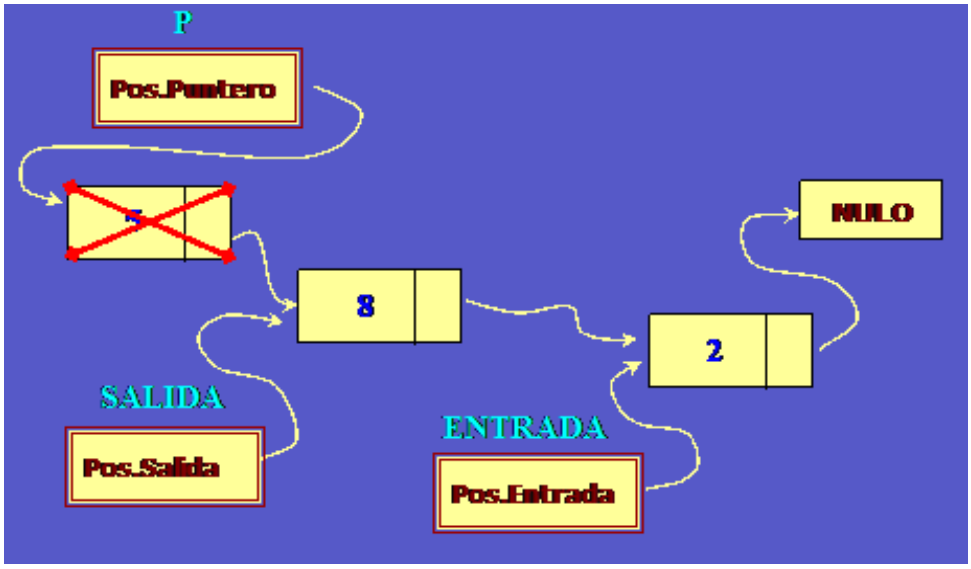

Se hace que el puntero P apunte al nodo que se va a desencolar:



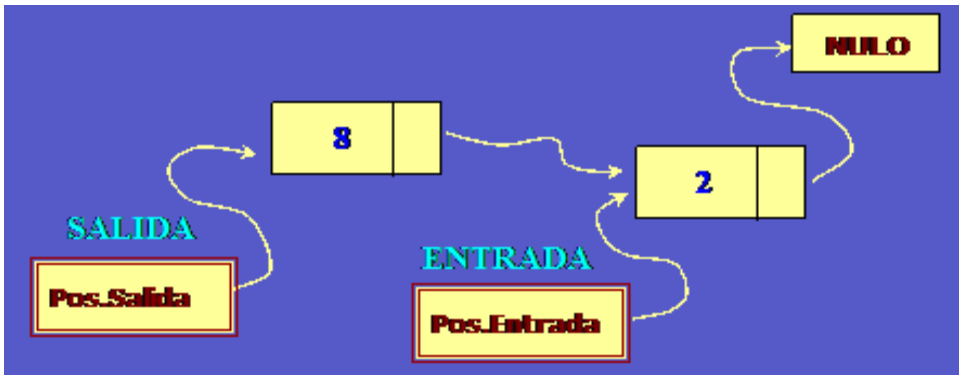

Se hace que el puntero SALIDA avance hasta el siguiente nodo:



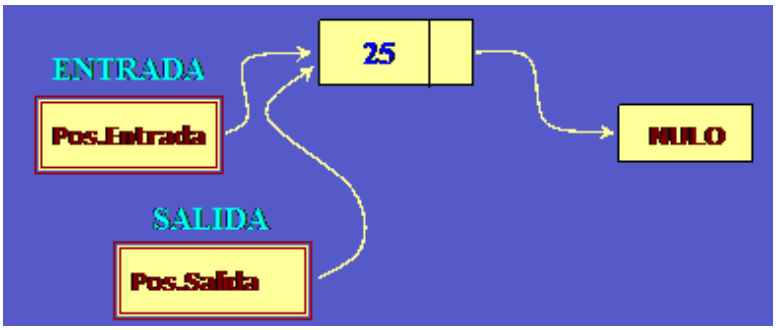

Se libera la memoria que ocupa el nodo que se va a desencolar:



La cola queda con 2 elementos:




Supongamos ahora una cola con un único elemento, en la que se realiza un desencolado:

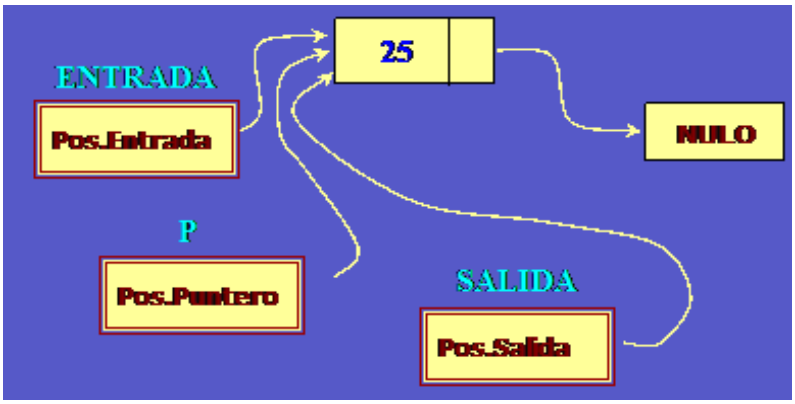



**Desencolar(un_valor, ENTRADA, SALIDA)**

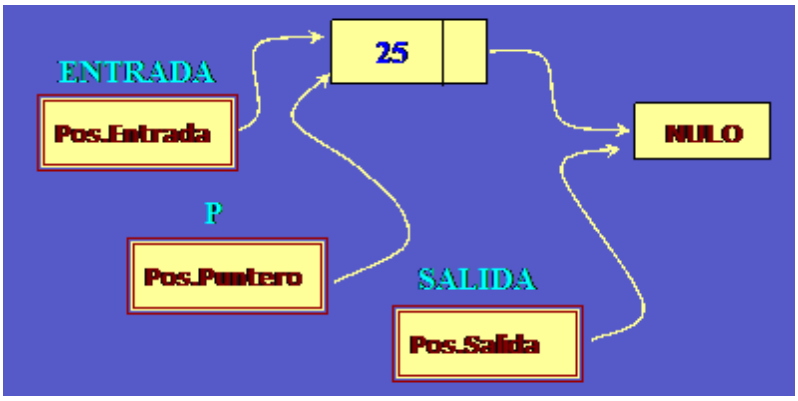
Se copia el valor del nodo en el parámetro:

**un_valor** **25**

A continuación el puntero auxiliar pasa a apuntar al nodo:



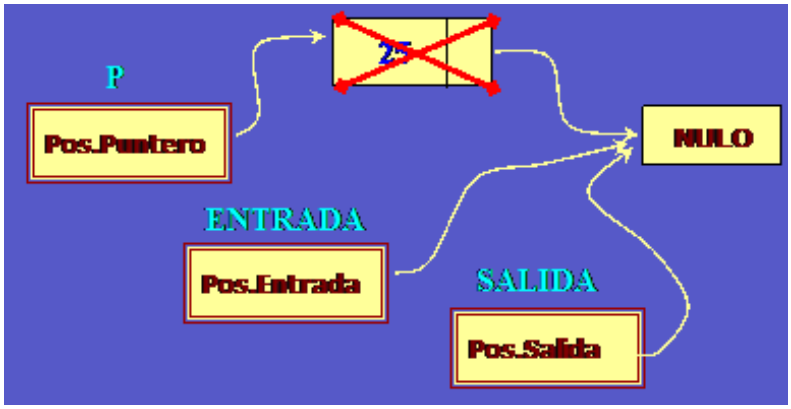

Se avanza el puntero a la salida de la cola, con lo que tomará el valor nulo:



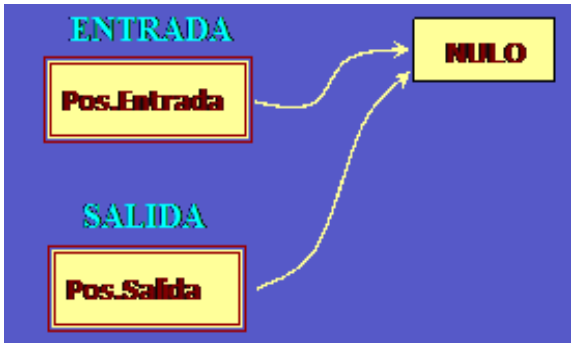

Dado que la cola va a quedar vacía, se actualiza el puntero a la entrada de la misma:




y, a continuación, se libera la memoria ocupada por el nodo:




La cola ha quedado vacía:




**Acceso al elemento situado en la salida de la Cola Lineal**

Para obtener el valor situado en la posición de salida de la cola, se accede al dato almacenado en el nodo apuntado por el puntero **SALIDA**, siempre que la cola no esté

vacía.

```
MÓDULO Acceder   devuelve logico
DATOS
        PARÁMETROS
                Recibe SALIDA puntero a NODO_COLA
                Transforma ELEMENTO de tipo T    ** dato consultado
INICIO
        Si (Cola_vacia(SALIDA) = Verdadero) entonces
                ESCRIBIR "La cola está vacía"
                DEVOLVER Falso              ** operación no realizada
        sino
                ELEMENTO <- SALIDA↑.DATO
                DEVOLVER Verdadero        ** operación realizada
        fin-si
FIN
```

### Operación de vaciado de la Cola Lineal

Cuando representamos una cola utilizando memoria dinámica, debemos liberar la memoria que ocupa una vez que deje de ser necesaria.

Se puede vaciar la cola realizando llamadas al módulo de desencolado:

```
MÓDULO Vaciar
DATOS
        PARÁMETROS
                Transforma ENTRADA puntero a NODO_COLA
                Transforma SALIDA puntero a NODO_COLA
        VARIABLES
                ELEMENTO del tipo T
INICIO
        Mientras (Cola_vacia(ENTRADA) = Falso) hacer
                Desencolar (ELEMENTO, ENTRADA, SALIDA)
        fin-mientras
FIN
```

Observar que en este módulo, dado que utilizamos una llamada al módulo Desencolar, que opera sobre 3 argumentos, tenemos que pasar el argumento **ELEMENTO**, aunque realmente el valor que toma no se utilice.

También podemos escribir un módulo que vacíe la cola incluyendo directamente las operaciones realizadas por el módulo de desencolado:

```
MÓDULO Vaciar
DATOS
        PARÁMETROS
                Transforma ENTRADA puntero a NODO_COLA
                Transforma SALIDA puntero a NODO_COLA
        VARIABLES
                P_NODO es puntero a NODO_COLA
INICIO
        Mientras (Cola_vacia(SALIDA) = Falso) hacer
                P_NODO <- SALIDA
                SALIDA <- SALIDA↑.ENLACE
                LIBERAR(P_NODO)
        fin-mientras

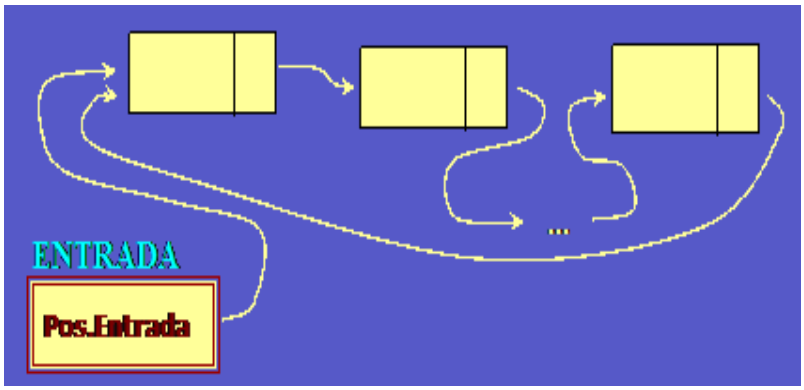
        ENTRADA <- NULO
FIN
```

Las operaciones se repiten hasta que se hayan sacado todos los nodos de la cola. En cada iteración se saca el nodo situado en la posición de salida de la cola.

Al concluir el proceso de vaciado de la cola, los punteros **ENTRADA** y **SALIDA** serán nulos, habiéndose restablecido la condición de cola vacía.

### Implementación con Cola Circular

La cola circular estará formada por un conjunto de nodos cada uno de los cuales apunta a otro nodo de la estructura. Existirá un puntero externo, que denominamos **ENTRADA**, que da acceso a la estructura:



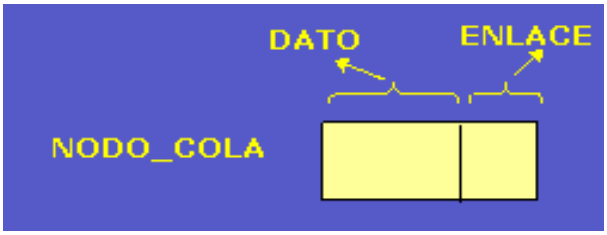

El último elemento encolado estará apuntado por **ENTRADA**. El primer elemento que se desencolará está identificado por **ENTRADA↑.ENLACE.**

### Creación de la Cola Circular mediante representación con Memoria Dinámica

Para crear la cola circular se define el tipo de nodos que almacenará la cola:

```
NODO_COLA es registro compuesto por
        DATO de tipo T
```

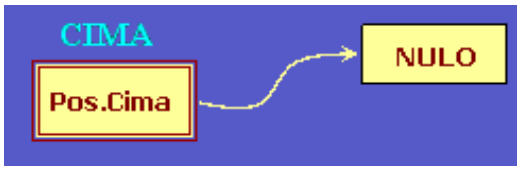
```
        ENLACE puntero a  NODO_COLA
fin-registro
```




**Esquema de un nodo de la cola**

A continuación se declara un puntero a elementos de la cola, que se iniciará al valor **NULO** para indicar que la cola se crea vacía:

```
ENTRADA puntero a NODO_COLA
ENTRADA <- NULO
```




**Cola circular vacía**

### Comprobación de Cola Circular vacía

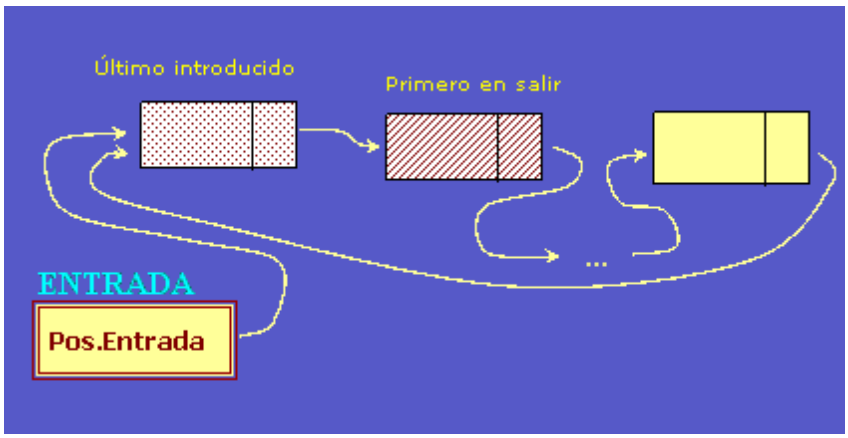
La cola está vacía si el puntero a la estructura es nulo. La comprobación es idéntica a la realizada para la cola dinámica lineal.

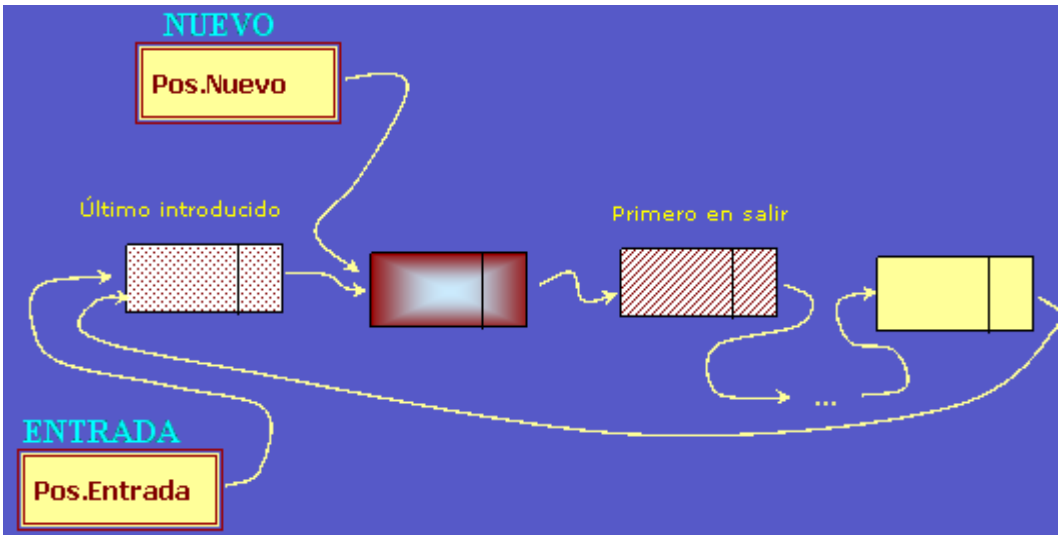
```
MÓDULO Cola_vacia  devuelve lógico
DATOS
        PARÁMETROS
                Recibe PTRO_COLA puntero a NODO_COLA
INICIO
        Si (PTRO_COLA = NULO) entonces
                DEVOLVER Verdadero          ** cola vacía
        sino
                DEVOLVER Falso              ** cola no vacía
        fin-si
FIN
```

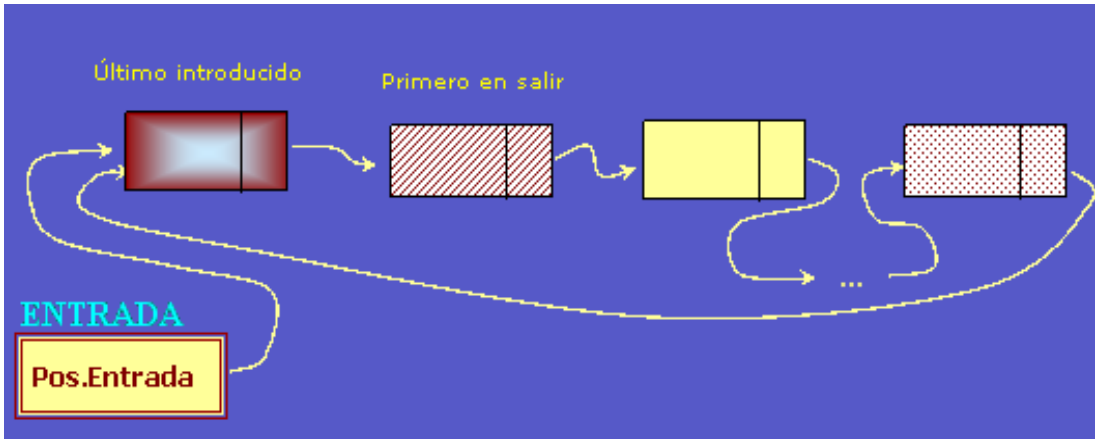
### Operación de Encolado sobre Cola Circular

Encolar un elemento supone colocarlo entre el último nodo que se encoló y el primero que se desencolará.

En la siguiente figura se muestra la modificación producida en la cola al añadir un nuevo nodo (el que aparece sombreado con un degradado). Este se coloca entre el último nodo candidato a salir de la cola (con relleno punteado) y el primer candidato a abandonarla (con rayado oblicuo).




**Encolado en una cola dinámica circular**

```
MÓDULO Encolar
DATOS
        PARÁMETROS
                Recibe ELEMENTO de tipo T    ** dato a encolar
                Transforma ENTRADA puntero a NODO_COLA
        VARIABLES
                NUEVO puntero a NODO_COLA
INICIO
        NUEVO <- RESERVAR_MEMORIA(tamaño(NODO_COLA))
```

```
        Si (NUEVO = NULO) entonces
                ESCRIBIR "No hay memoria"
        sino
                NUEVO↑.DATO <- ELEMENTO

                Si (Cola_vacia(ENTRADA) = Verdadero ) entonces
                        NUEVO↑.ENLACE <- NUEVO
                sino
                        NUEVO↑.ENLACE <- ENTRADA↑.ENLACE
                        ENTRADA↑.ENLACE <- NUEVO
                fin-si

                ENTRADA <- NUEVO
        fin-si
FIN
```

En primer lugar se solicita memoria para un nuevo nodo. Si la solicitud no tiene éxito, se muestra un mensaje y la operación concluye. En caso contrario, se procede a copiar en el campo de datos del nodo el valor que se desea encolar.

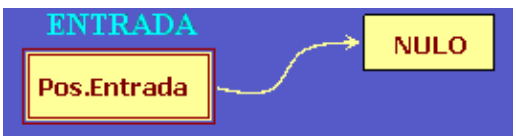
Para conectar el nuevo nodo a la estructura hay que distinguir dos situaciones:

- Si la cola estaba vacía, el nuevo nodo pasará a ser el único de la cola. Por ello, estará apuntado por el puntero a la cola (**ENTRADA**) y además su campo **ENLACE** apuntará al propio nodo.
- Si la cola no estaba vacía, el nuevo nodo se coloca entre el que era último candidato a abandonar la cola (identificado por **ENTRADA**) y el primero (identificado por **ENTRADA↑.ENLACE** ).

En ambos casos el puntero a la estructura pasará a apuntar al nuevo nodo, identificando así la última entrada producida en la cola.

### Ejemplo de operación de encolado

Consideremos una cola circular que almacena números enteros y que inicialmente está vacía:



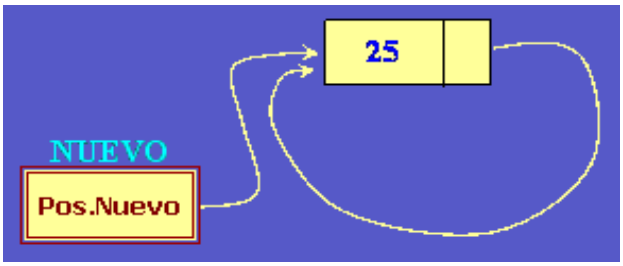

Se intenta encolar el valor 25:

**Encolar(25, ENTRADA)**

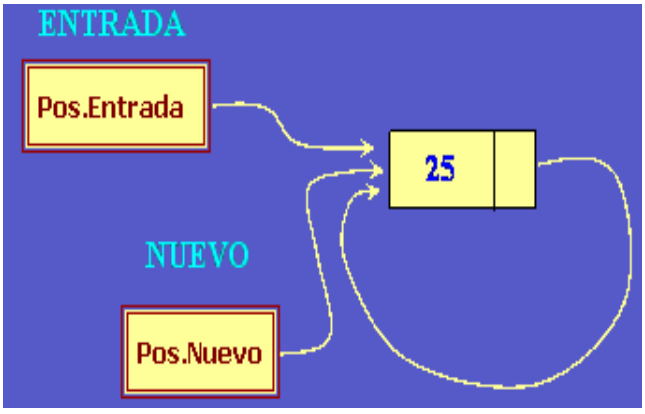
Una vez obtenida la memoria solicitada para el nuevo nodo, se almacena el valor 25 en el campo de datos:



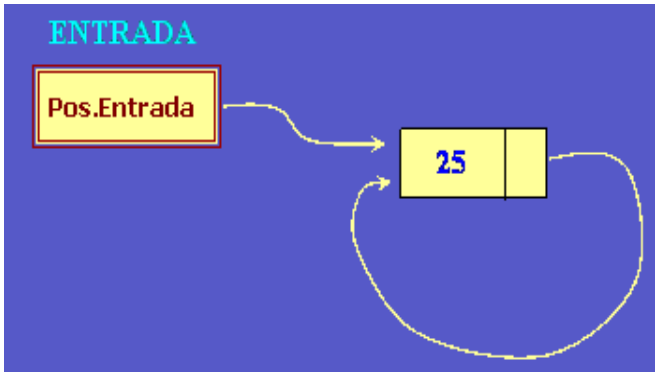

Como la cola estaba vacía, el campo de enlace del nuevo nodo apunta a dicho nodo, definiendo así una estructura circular:




Por último, se debe hacer que el puntero a la estructura pase a apuntar al nuevo nodo:




La cola ha quedado formada por un único nodo:



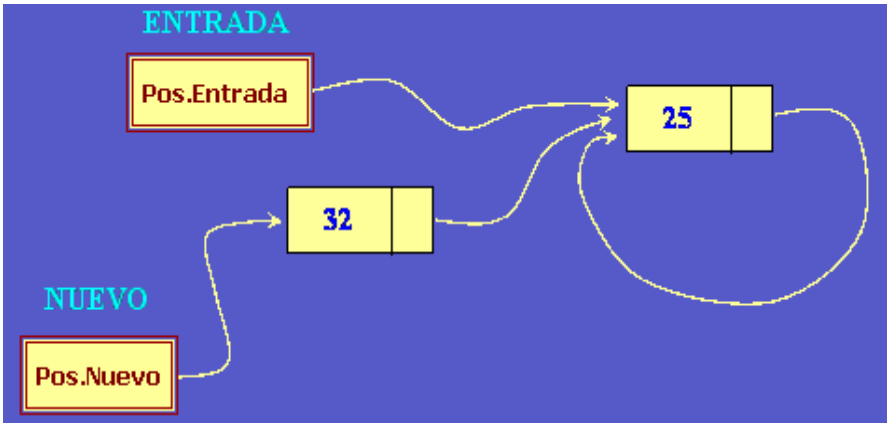

Se intenta encolar el valor 32:

**Encolar(32, ENTRADA)**

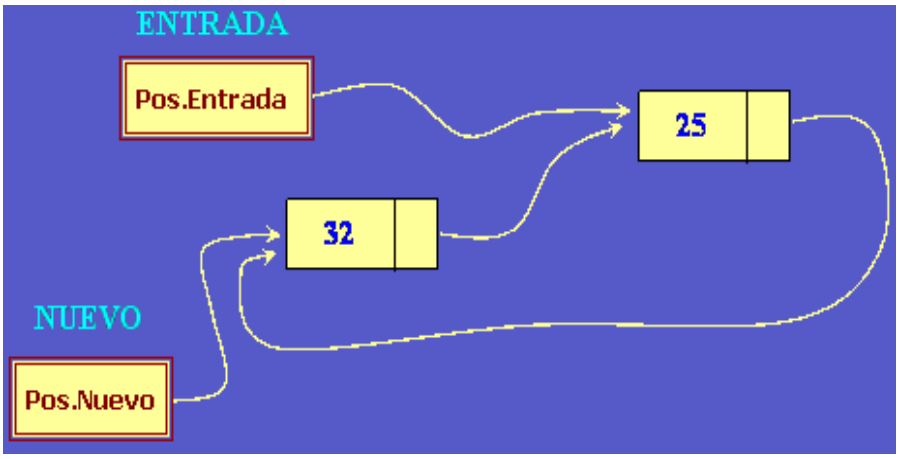
Una vez obtenida la memoria para el nuevo nodo, se almacena el valor 32 en el campo de datos:



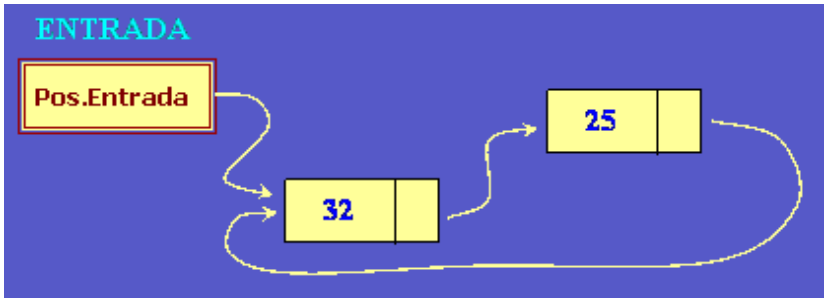

Como la cola no estaba vacía, el campo de enlace del nuevo nodo apunta al nodo que lleva más tiempo en la estructura (referenciado por **ENTRADA↑.ENLACE** ):




A continuación se hace que **ENTRADA↑.ENLACE** apunte al nuevo nodo:




Por último, se hace que el puntero a la cola pase a apuntar al nuevo nodo, ya que representa la última entrada en la estructura. La cola ha quedado formada por dos nodos:



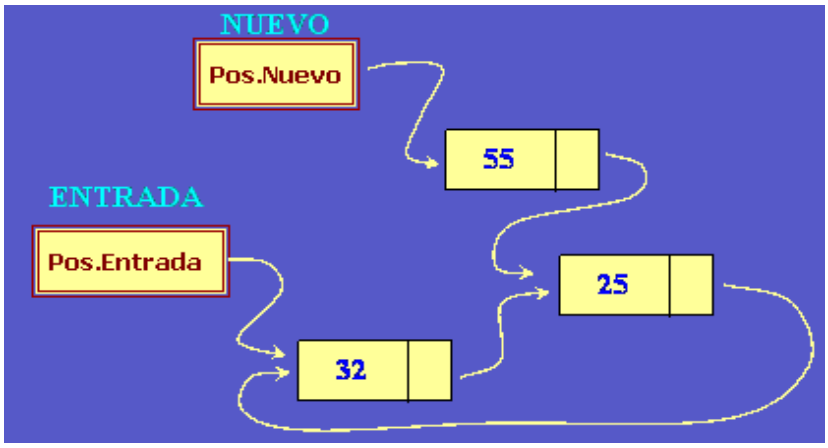

Se intenta encolar el valor 55:

**Encolar(55, ENTRADA)**

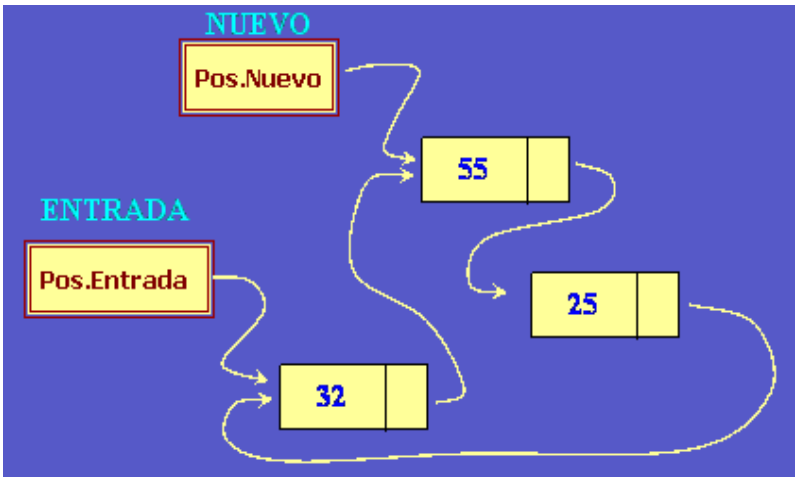
El valor 55 se copia en el campo de datos del nuevo nodo:



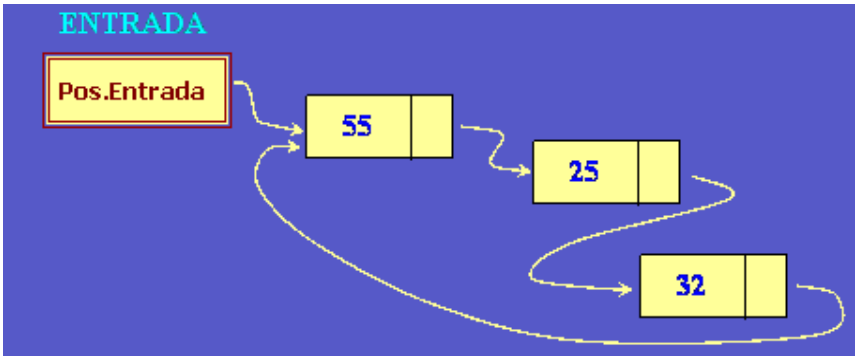

El campo de enlace del nuevo nodo apunta al primer nodo candidato a salir de la estructura, es decir, el que contiene el valor 25 (referenciado por **ENTRADA↑.ENLACE**):




A continuación el último nodo que entró en la estructura pasará a apuntar al nuevo:



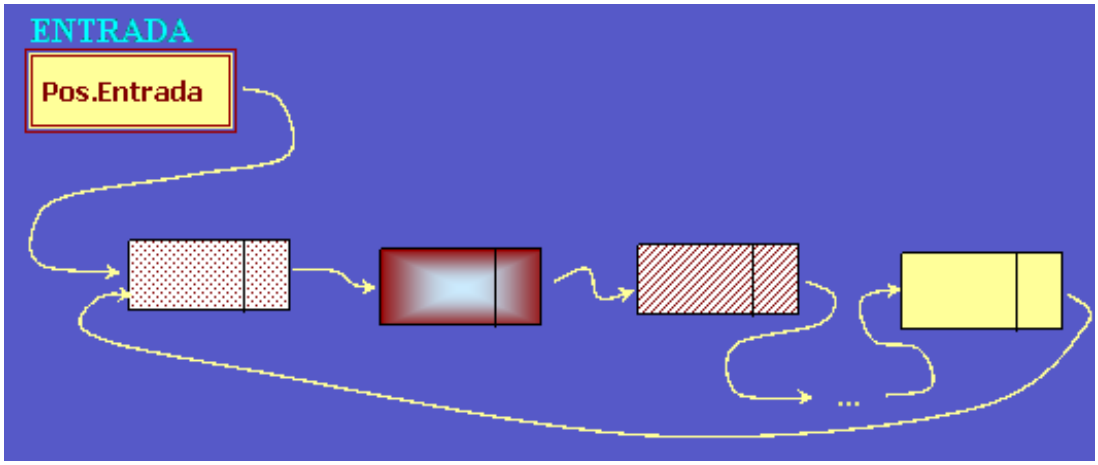

El puntero a la cola pasará a apuntar al nuevo nodo:

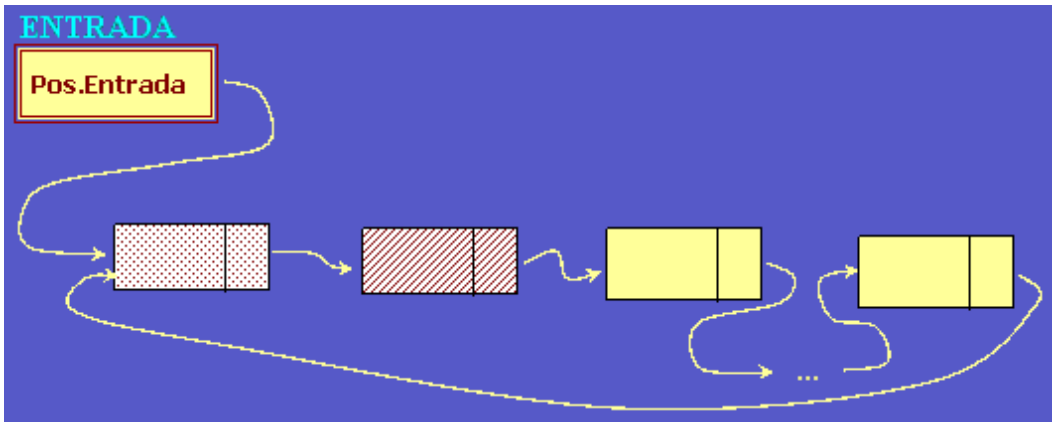


### Operación de Desencolado sobre Cola Circular

Al realizar un desencolado saldrá de la estructura el nodo apuntado por el que está referenciado por el puntero a la estructura.

En la figura se muestra la modificación producida en la cola al realizar un desencolado. El nodo que abandona la estructura es el referenciado por **ENTRADA↑.ENLACE** (con un degradado de color):




**Desencolado en una cola dinámica circular**

```
MÓDULO Desencolar  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ELEMENTO de tipo T    ** dato desencolado
                Transforma ENTRADA puntero a NODO_COLA
        VARIABLES
                P puntero a NODO_COLA
INICIO
        Si (Cola_vacia(ENTRADA) = Verdadero) entonces
                ESCRIBIR "Cola vacía. No se puede desencolar"
                DEVOLVER Falso     ** operación no realizada
        sino
                ELEMENTO <- ENTRADA↑.ENLACE↑.DATO
```

```
            Si (ENTRADA = ENTRADA↑.ENLACE) entonces
                LIBERAR(ENTRADA)
                ENTRADA <- NULO
            sino
                P <- ENTRADA↑.ENLACE
                ENTRADA↑.ENLACE <- P↑.ENLACE
                LIBERAR(P)
            fin-si

            DEVOLVER Verdadero   ** operación realizada
        fin-si
    FIN
```

Antes de proceder a desencolar un elemento, se comprueba que la cola no esté vacía. En caso de estarlo la operación se anula.

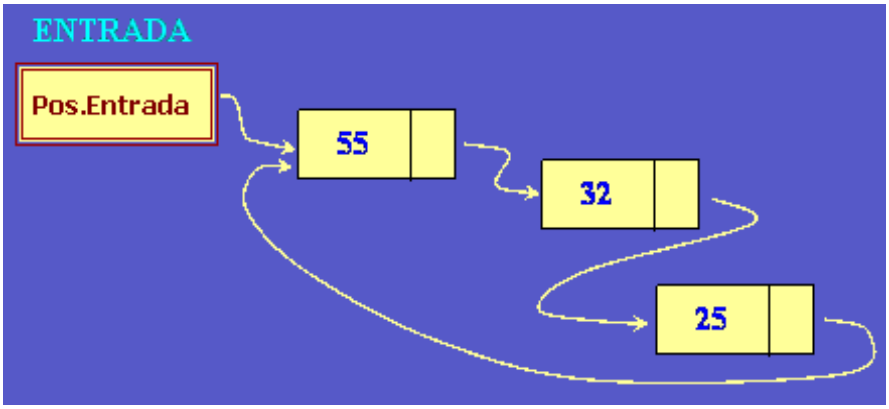
Si la cola no está vacía, se hace una copia del dato almacenado en el nodo que va a abandonar la cola y, a continuación, se libera la memoria que ocupa.

Si la cola contiene más de un elemento, antes de liberar la memoria asociada al nodo situado en la salida se debe hacer que el nodo que lo precede apunte al nodo que lo sucede.

Si la cola contiene un solo elemento, quedará vacía tras la eliminación. En este caso se libera la memoria asociada al único nodo de la cola y se da después la condición de cola vacía.

**Ejemplo de operación de desencolado**

Se realiza un desencolado en la siguiente estructura:



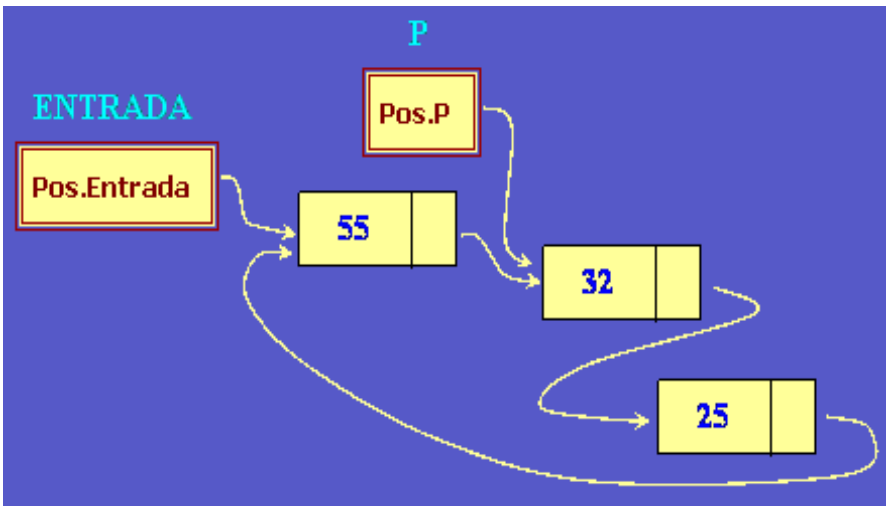

**Desencolar (un_valor, ENTRADA)**

Como la cola no está vacía, en primer lugar se copia en el parámetro un_valor el campo de datos del nodo que abandonará la cola

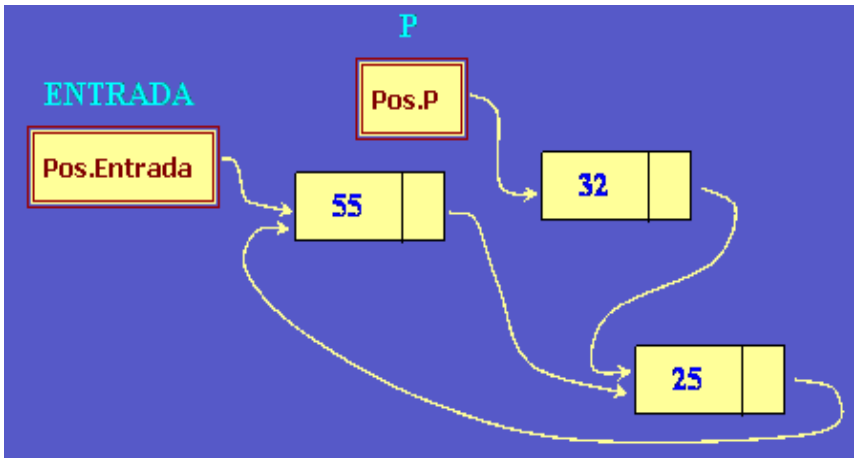
**un_valor** | **32**

Dado que la cola incluye más de un nodo, se hace que el puntero P apunte al nodo que

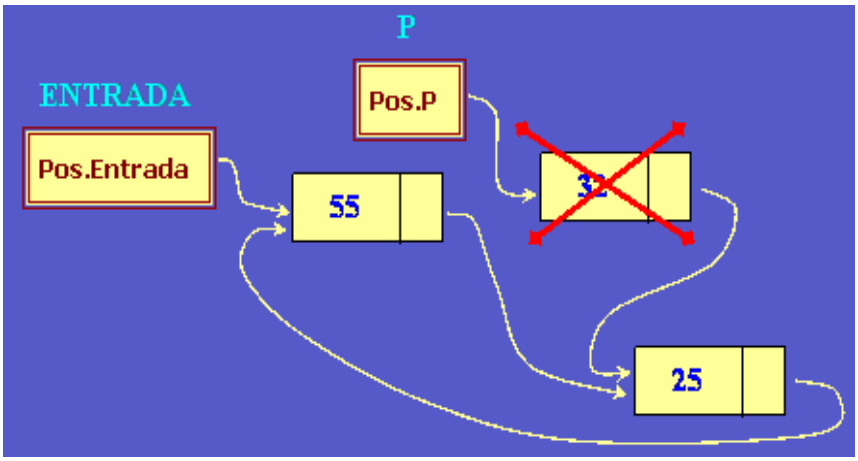
se debe eliminar:



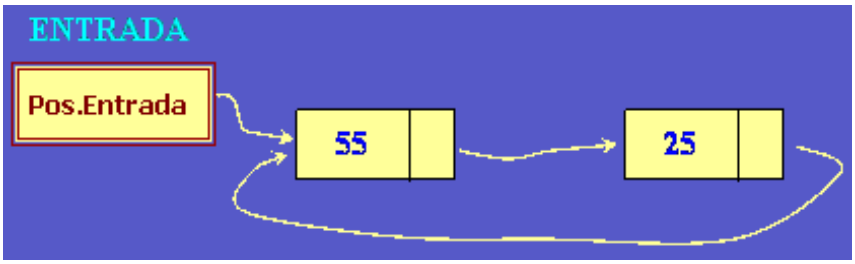

Se hace que el último nodo que entró en la cola pase a apuntar al primer nodo que la abandonará cuando se produzca un nuevo desencolado:



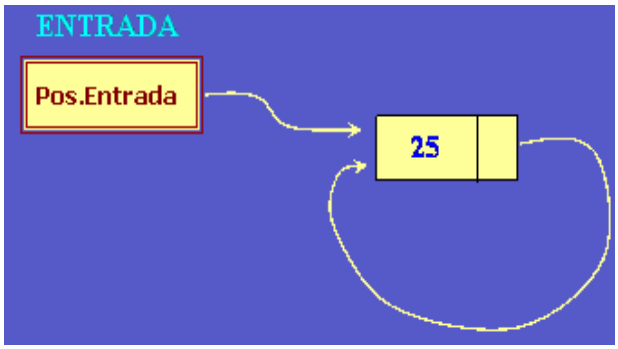

Se libera la memoria ocupada por el nodo que va a abandonar la estructura:




La cola queda con 2 nodos:



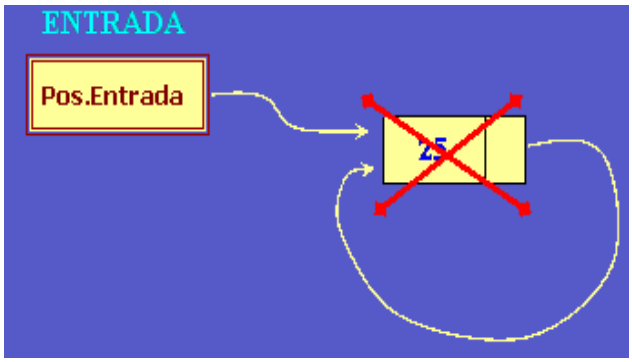

Supongamos ahora que se solicita un desencolado sobre una cola que contiene un solo nodo:




En primer lugar se copia sobre el parámetro el valor almacenado en la zona de datos del nodo:




A continuación se libera la memoria que ocupa el nodo:




Por último, se hace nulo el puntero a la estructura, indicando que la cola ha quedado vacía:




### Acceso al elemento situado en la salida de la Cola Circular

Para obtener el valor situado en la posición de salida de la cola, se accede al dato almacenado en el nodo apuntado por el puntero **ENTRADA↑.ENLACE**, siempre que la cola no está vacía.

**MÓDULO Acceder  devuelve lógico**

```
DATOS
        PARÁMETROS
                Recibe ENTRADA puntero a NODO_COLA
                Transforma ELEMENTO de tipo T   ** dato consultado
INICIO
        Si (Cola_vacia(ENTRADA) = Verdadero) entonces
                ESCRIBIR "La cola está vacía"
                DEVOLVER Falso          ** operación no realizada
        sino
                ELEMENTO <- ENTRADA↑.ENLACE↑.DATO
                DEVOLVER Verdadero  ** operación realizada
        fin-si
FIN
```

### Operación de vaciado de la Cola Circular

Cuando representamos una cola utilizando memoria dinámica, debemos liberar la memoria que ocupa una vez que deje de ser necesaria.

Podemos definir la operación de vaciado de la cola en términos de la operación de desencolado, realizando desencolados hasta que la cola quede vacía:

```
MÓDULO Vaciar
DATOS
        PARÁMETROS
                Transforma ENTRADA puntero a NODO_COLA
        VARIABLES
                ELEMENTO de tipo T
INICIO
        Mientras (Cola_vacia(ENTRADA) = Falso) hacer
                Desencolar (ELEMENTO, ENTRADA)
        fin-mientras
FIN
```

También podemos escribir un módulo de vaciado:

```
MÓDULO Vaciar
DATOS
        PARÁMETROS
                Transforma ENTRADA puntero a NODO_COLA
        VARIABLES
                P puntero a NODO_COLA
```

```
INICIO
        Mientras (Cola_vacia(ENTRADA) = Falso) hacer
                Si (ENTRADA = ENTRADA↑.ENLACE) entonces
                        LIBERAR(ENTRADA)
                        ENTRADA <- NULO
                sino
                        P <- ENTRADA↑.ENLACE
                        ENTRADA↑.ENLACE <- P↑.ENLACE
                        LIBERAR(P)
                fin-si
        fin-mientras
FIN
```

# 4 ESTRUCTURAS DE DATOS NO LINEALES

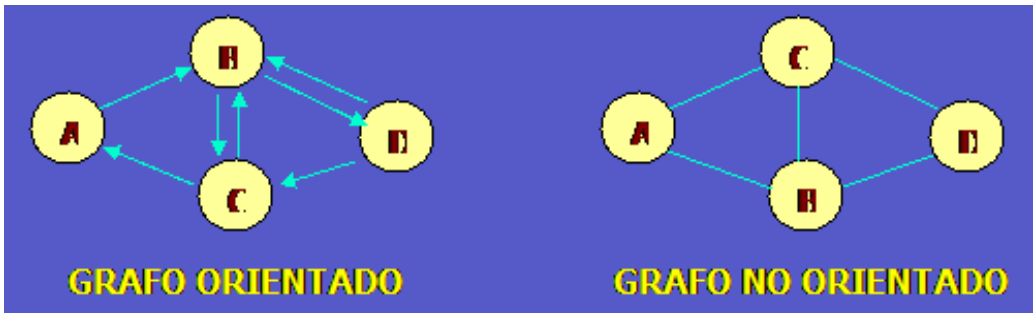
Este módulo presenta las estructuras de datos internas no lineales más comunes: grafos y árboles.

## 4.1 Grafos

### *4.1.1 Definición*

Un **grafo** viene dado por un par de conjuntos, G = {V, A}, siendo V un conjunto no vacío denominado conjunto de **vértices** o **nodos** del grafo y A un subconjunto de V, denominado conjunto de **conexiones**, formado por pares (i, j), donde i y j son elementos de V.

Los nodos de un grafo pueden tener asociados nombres y diferentes propiedades. Las conexiones entre vértices pueden incluir o no incluir orientación, dando lugar a dos tipos de grafos: **grafos orientados** y **grafos no orientados**, respectivamente. Las conexiones no orientadas se denominan **aristas** y las orientadas se denominan **arcos**.




Dada una arista que une los nodos i y j, i y j se denominan **extremos** de la arista. Si dichos nodos están unidos por un arco que va desde i hasta j, i se denomina **origen** del arco y j se denomina **extremo** del arco.

Si las conexiones tienen asociado algún valor, denominado **coste** o **peso**, se dice que el grafo es **ponderado**.

Se denomina **orden** del grafo al número de vértices del mismo y **tama****o** del grafo al número de aristas o arcos.

Se dice que un **grafo** es **simple** cuando cada par de vértices están unidos a lo sumo por una arista, en el caso de grafos no orientados, o por dos arcos, uno en cada sentido, en el caso de grafos orientados; en caso contrario se dice que el **grafo** es **múltiple**.

En este tema nos centraremos en el trabajo con grafos simples.

Dado un *grafo no orientado*, una **cadena** es una secuencia de aristas consecutivas del grafo. La **longitud de una cadena** es el número de aristas que la componen, si el grafo no es ponderado, y la suma de los costes de las mismas, si el grafo es ponderado. Si la cadena empieza y termina en el mismo vértice se denomina **ciclo**. Un ciclo que incluye una sola arista se denomina **bucle**. Si el *grafo* es *orientado*, el concepto de cadena se sustituye por el de **camino** y el de ciclo por el de **circuito**.

Un grafo no dirigido es **conexo** si existe una cadena entre cualquier par de nodos, siendo **no conexo** o inconexo en caso contrario. Un grafo dirigido es conexo si existe una cadena entre cualquier par de nodos y es **fuertemente conexo** si existe un camino entre cualquier par de nodos. Cuando un grafo es conexo consta de una sola **componente**; cuando es no conexo consta de dos o más componentes conexas. Si un grafo dirigido no es fuertemente conexo, incluirá dos o más componentes fuertemente conexas.

### *4.1.2 Representación de Grafos*

#### *Representación mediante Matriz de Adyacencia*

Dado un grafo con N nodos, la **matriz de adyacencia** que representa el grafo es una matriz cuadrada de orden N. El elemento (i, j) almacenará un 0 si no existe conexión desde el nodo i al j y un 1 en caso contrario (grafo simple).

**ADY(N, N) de enteros**

Si existe un arco desde el nodo i al j, se almacenará un uno en la posición (i, j) de la matriz. Si ambos nodos están conectados por una arista, se almacenará un uno en las posiciones (i, j) y (j, i) de la matriz.

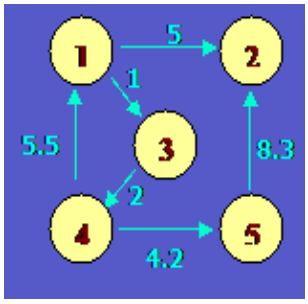
Si el grafo es ponderado, además de la información sobre las conexiones existentes, es necesario almacenar los pesos asociados a éstas. La **matriz de costes** tendrá los mismos elementos que la matriz de adyacencia, pudiendo almacenar reales o enteros.

**COSTES(N, N) de reales**

El elemento (i, j) de la matriz de costes almacenará el peso asociado a la conexión existente entre los vértices i y j, siendo infinito si no existe conexión. Si el grafo es no dirigido, los elementos (i, j) y (j, i) de la matriz almacenarán el mismo valor.

**EJEMPLO**: grafo con 5 nodos

$$ADY = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \qquad COSTES = \begin{bmatrix} \infty & 5 & 1 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 2 & \infty \\ 5.5 & \infty & \infty & \infty & 4.2 \\ \infty & 8.3 & \infty & \infty & \infty \end{bmatrix}$$



## *Representación mediante Listas de Adyacencia*

La representar de un grafo mediante **listas de adyacencia** se basa en la utilización de listas enlazadas. Para cada vértice del grafo se almacena en una lista la información sobre los vértices a los que está conectado. Esta representación sólo almacena información sobre las conexiones existentes.

Los nodos del grafo se identifican mediante enteros diferentes. En primer lugar se define una lista enlazada que almacenará información sobre los vértices del grafo. El nodo de la lista relacionado con el vértice I del grafo dará acceso a la lista de adyacencia de dicho vértice. Los nodos de la lista de vértices incluyen como mínimo tres campos: uno para identificar el vértice del grafo (**ID_VERTICE**), otro para dar acceso a la lista de adyacencia del vértice (**P_LISTA**) y otro para enlazar entre sí los nodos de la lista de vértices:

```
NODO_V es registro compuesto por
        ID_VERTICE entero
        P_LISTA puntero a NODO_L
        ENLACE puntero a NODO_V
fin-registro
```

Si los vértices tienen asociado un nombre u otro parámetro, se pueden añadir campos para almacenar esta información.

Los nodos de una lista de adyacencia incluyen como mínimo dos campos: uno para almacenar el identificador del vértice conectado (mediante un arco o una arista) con el vértice asociado a la lista (**ID_VERTICE**), y otro para enlazar entre sí los nodos de la lista. Si el grafo es ponderado, se incluye un tercer campo para almacenar el peso de una conexión cuyo extremo es el vértice identificado en el nodo (**COSTE**):

```
NODO_L es registro compuesto por
        ID_VERTICE entera
        COSTE real
        ENLACE puntero a NODO_L
fin-registro
```

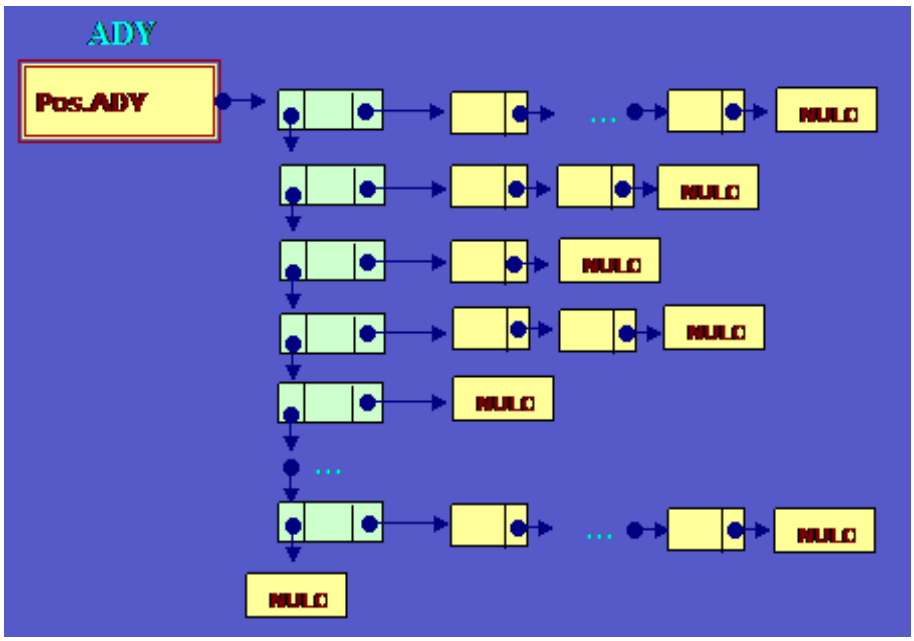
Llamaremos **ADY** al puntero que da acceso a la información que define el grafo. Dicho puntero apuntará a la lista de vértices:

```
ADY puntero a NODO_V
```

Inicialmente el puntero se hace nulo, para indicar que el grafo no incluye ni vértices ni conexiones.

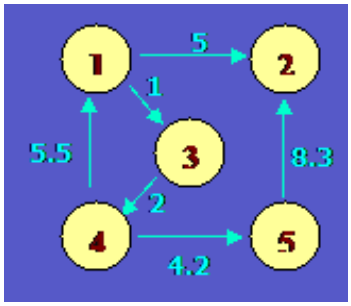
```
ADY <- NULO
```

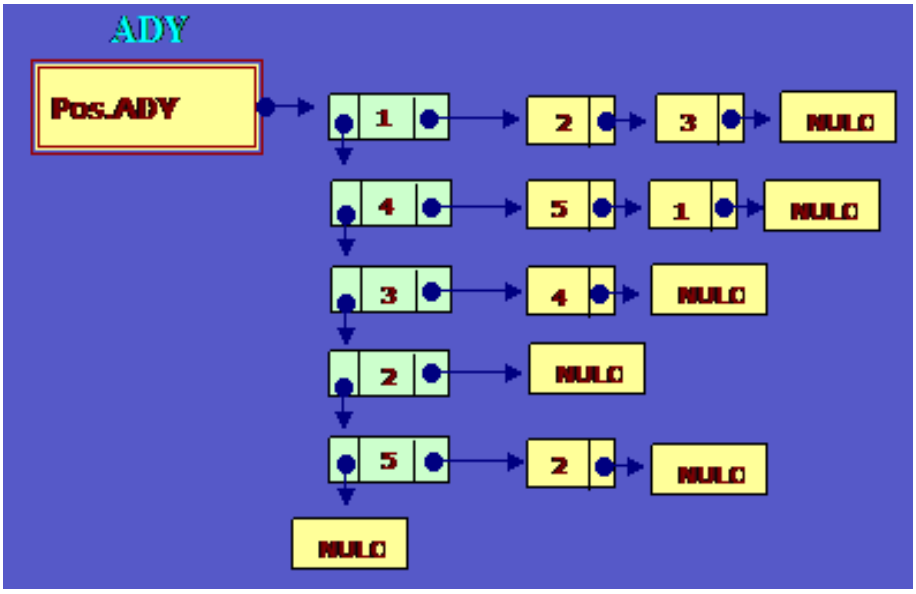
La siguiente figura muestra una representación gráfica de las listas de adyacencia que definen un grafo.




**Esquema de listas de adyacencia**

Representación mediante listas de adyacencia del grafo de la siguiente figura (en los nodos de las listas de adyacencia se omite el peso asociado a la conexión, para simplificar el esquema):



### *4.1.3 Operaciones en la representación mediante Matriz de Adyacencia*

En la representación mediante matriz de adyacencia consideramos un grafo con N nodos, por lo que se reserva espacio para una matriz cuadrada de orden N. Por tanto, en esta representación no se consideran las operaciones de adición y eliminación de nodos del grafo. Los nodos se numeran de 1 a N, de forma que coincidan con las filas y columnas de la matriz.

- Creación
- Comprobación de la existencia de un vértice
- Comprobación de la existencia de un arco
- Comprobación de la existencia de una arista
- Añadir un arco
- Añadir una arista
- Eliminar un arco
- Eliminar una arista

#### *Creación*

Para crear el grafo, se reserva espacio para la matriz de adyacencia del mismo, con tantas filas y columnas como nodos incluya el grafo, N:

**ADY(N, N) de enteros**

Además, se deben poner a cero todas las posiciones de la matriz, para indicar que inicialmente no se ha definido ninguna conexión entre los vértices del grafo.

Si el grafo es ponderado, también se debe definir la matriz de costes, que supondremos de tipo real para que sea más general:

**COSTES(N, N) de reales**

y darle valor inicial a sus componentes.

#### *Añadir un Arco*

Para añadir un arco entre los vértices V1 y V2, se debe poner a 1 la componente (V1, V2) de la matriz de adyacencia. Si el grafo es ponderado, será necesario añadir el coste del nuevo arco a la matriz de costes.

```
MÓDULO Añadir_arco
DATOS
        PARÁMETROS
                Transforma ADY(N, N) de enteros
                Transforma COSTES(N, N) de reales
                Recibe V1, V2 enteros   ** origen y extremo del arco
                Recibe C real           ** coste del arco
```

```
INICIO
        ADY(V1, V2) <- 1
        COSTES(V1, V2) <- C
FIN
```

## *Añadir una Arista*

Para añadir una arista entre los nodos V1 y V2, se pondrán a uno las componentes (V1, V2) y (V2, V1) de la matriz de adyacencia. Si el grafo es ponderado, se almacenará el coste de la arista en las componentes (V1, V2) y (V2, V1) de la matriz de costes.

```
MÓDULO Añadir_arista
DATOS
        PARÁMETROS
                Transforma ADY(N, N) de enteros
                Transforma COSTES(N, N) de reales
                Recibe V1, V2 enteros   ** extremos de la arista
                Recibe C real           ** coste de la arista
INICIO
        ADY(V1, V2) <- 1
        ADY(V2, V1) <- 1

        COSTES(V1, V2) <- C
        COSTES(V2, V1) <- C
FIN
```

## *Comprobación de la existencia de un Vértice*

Para un grafo de N vértices representado mediante su matriz de adyacencia, los vértices se numerarán de 1 a N, de forma que correspondan con filas/columnas de la matriz. Por tanto, para comprobar si existe un vértice VI, se comprobará si el valor VI está entre 1 y N.

```
MÓDULO Existe_vertice  devuelve lógico
        PARÁMETROS
                Recibe VI entera
                Recibe N entera
INICIO
        Si ( (VI ≥ 1)  Y  (VI ≤ N) ) entonces
                DEVOLVER Verdadero
        sino
                DEVOLVER Falso
        fin-si
FIN
```

## *Comprobación de la existencia de un Arco*

Para comprobar si existe un arco desde el vértice V1 al V2, se comprueba si no hay un cero en la posición (V1, V2) de la matriz de adyacencia.

```
MÓDULO Existe_arco  devuelve lógico
DATOS
        PARÁMETROS
                Recibe ADY(N, N) de enteros
                Recibe V1, V2 enteros
INICIO
        Si (ADY(V1, V2) ≠ 0) entonces
                DEVOLVER Verdadero          ** existe arco
        sino
                DEVOLVER Falso               ** no existe arco
        fin-si
FIN
```

## *Comprobación de la existencia de una Arista*

Para comprobar si existe una arista entre los vértices V1 y V2, se comprueba si no hay un cero en las posiciones (V1, V2) y (V2, V1) de la matriz de adyacencia.

```
MÓDULO Existe_arista  devuelve lógico
DATOS
        PARÁMETROS
                Recibe ADY(N, N) de enteros
                Recibe V1, V2 enteros
INICIO
        Si (  (ADY(V1, V2) ≠ 0)   Y   (ADY(V2, V1) ≠ 0) ) entonces
                DEVOLVER Verdadero            ** existe arista
        sino
                DEVOLVER Falso                 ** no existe arista
        fin-si
FIN
```

## *Eliminar un Arco*

Para eliminar un arco existente entre los vértices V1 y V2, se pone a 0 la componente de la matriz de adyacencia que representa dicha conexión. Si el grafo es ponderado se debe eliminar el coste asociado al arco.

```
MÓDULO Eliminar_arco
DATOS
        PARÁMETROS
                Transforma ADY(N, N) de enteros
                Transforma COSTES(N, N) de enteros
                Recibe V1, V2 enteros
```

```
INICIO
        ADY(V1, V2) <- 0
        COSTES(V1, V2) <- INFINITO
FIN
```

### *Eliminar una Arista*

Para eliminar una arista existente entre los vértices V1 y V2, se pondrán a cero las posiciones (V1, V2) y (V2, V1) de la matriz de adyacencia. Si el grafo es ponderado se debe eliminar también el coste asociado a la arista.

```
MÓDULO Eliminar_arista
DATOS
        PARÁMETROS
                Transforma ADY(N, N) de enteros
                Transforma COSTES(N, N) de enteros
                Recibe V1, V2 enteros
INICIO
        ADY(V1, V2) <- 0
        ADY(V2, V1) <- 0
        COSTES(V1, V2) <- INFINITO
        COSTES(V2, V1) <- INFINITO
FIN
```

## *4.1.4 Operaciones en la representación mediante Listas de Adyacencia*

En la representación mediante listas de adyacencia, la memoria que ocupa la estructura que almacena el grafo se debe liberar cuando deje de ser necesaria, por lo que se añade una operación de eliminación de las listas.

- Creación
- Comprobación de la existencia de un vértice
- Comprobación de la existencia de un arco
- Comprobación de la existencia de una arista
- Añadir un vértice
- Añadir un arco
- Añadir una arista
- Eliminar un vértice
- Eliminar un arco
- Eliminar una arista
- Eliminar las listas

## *Creación*

Para crear el grafo, en primer lugar se define la estructura de los nodos de las listas de adyacencia:

```
NODO_L es registro compuesto por
        ID_VERTICE entera
        COSTE real
        ENLACE puntero a NODO_L
fin-registro
```

Si el grafo no es ponderado, se elimina el segundo campo.

En segundo lugar, se define la estructura de los nodos de la lista de vértices:

```
NODO_V es registro compuesto por
        ID_VERTICE entera
        P_LISTA puntero a NODO_L
        ENLACE puntero a NODO_V
fin-registro
```

A continuación, se define el puntero que dará acceso a la lista de vértices:

```
ADY puntero a NODO_V
```

Inicialmente no existirá ningún vértice en el grafo y todas las listas estarán vacías, por lo que el puntero ADY se hace nulo:

```
ADY <- NULO
```

## *Comprobación de la existencia de un Vértice*

Para comprobar si existe un vértice VI se busca éste en la lista de vértices.

```
MÓDULO Existe_vertice  devuelve lógico
        PARÁMETROS
                Recibe ADY puntero a NODO_V
                Recibe VI entero
INICIO
        Si (Buscar_vertice(ADY, VI) != NULO) entonces
                DEVOLVER Verdadero      ** vértice encontrado
        sino
                DEVOLVER Falso          ** vértice no encontrado
        fin-si
FIN
```

Para realizar esta comprobación se utiliza un módulo auxiliar (**Buscar_vertice**), que también se utilizará en operaciones posteriores. Busca un vértice V en la lista de vértices asociada al grafo. Retorna NULO si el vértice no aparece en la lista de vértices (lo que significa que no forma parte del grafo), y un puntero no NULO si ha encontrado el vértice. En este segundo caso el puntero apuntará al nodo de la lista de vértices que identifica al vértice buscado.

```
MÓDULO Buscar_vertice  devuelve puntero a NODO_V
DATOS
        PARÁMETRO
                Recibe P puntero a NODO_V
                Recibe V entero
        VARIABLES
                P_V puntero a NODO_V
INICIO
        P_V <- P

        Mientras (P_V != NULO) hacer
                Si (P_V↑.ID_VERTICE = V) entonces
                        DEVOLVER P_V                    ** vértice encontrado
                fin-si
                P_V <- P_V↑.ENLACE
        fin-mientras

        DEVOLVER NULO                                   ** vértice no
encontrado
FIN
```

## *Comprobación de la existencia de un Arco*

Para comprobar si existe un arco desde el vértice V1 al V2, se buscará un nodo con valor V2 en la lista de adyacencia de V1.

```
MÓDULO Existe_arco   devuelve lógico
        PARÁMETROS
            Recibe ADY puntero a NODO_V
            Recibe V1 entera
            Recibe V2 entera
        VARIABLES
            P_V1 puntero a NODO_V
            P_V2 puntero a NODO_L
INICIO
        P_V1 <- Buscar_vertice(ADY, V1)

        Si( P_V1 ≠ NULO) entonces
            P_V2 <- P_V1↑.P_LISTA

            Mientras ( (P_V2 ≠ NULO) Y (P_V2↑.ID_VERTICE ≠ V2) ) hacer
                    P_V2 <- P_V2↑.ENLACE
            fin-mientras

            Si (P_V2 ≠ NULO) entonces
```

```
                DEVOLVER Verdadero   ** encontrado
            fin-si
        fin-si

        DEVOLVER Falso                          ** no encontrado
FIN
```

## *Comprobación de la existencia de una Arista*

Para comprobar si existe una arista entre los vértices V1 y V2, se debe buscar el vértice V2 en la lista de adyacencia de V1 y el vértice V1 en la lista de adyacencia de V2.

```
MÓDULO Existe_arista  devuelve lógico
DATOS
        PARÁMETROS
                Recibe ADY puntero a NODO_V
                Recibe V1, V2 enteros
INICIO
        Si ( (Existe_arco(ADY, V1, V2) = Verdadero)  Y
             (Existe_arco(ADY, V2, V1) = Verdadero) ) entonces
                DEVOLVER Verdadero          ** encontrado
        sino
                DEVOLVER Falso              ** no encontrado
        fin-si
FIN
```

## *Añadir un Vértice*

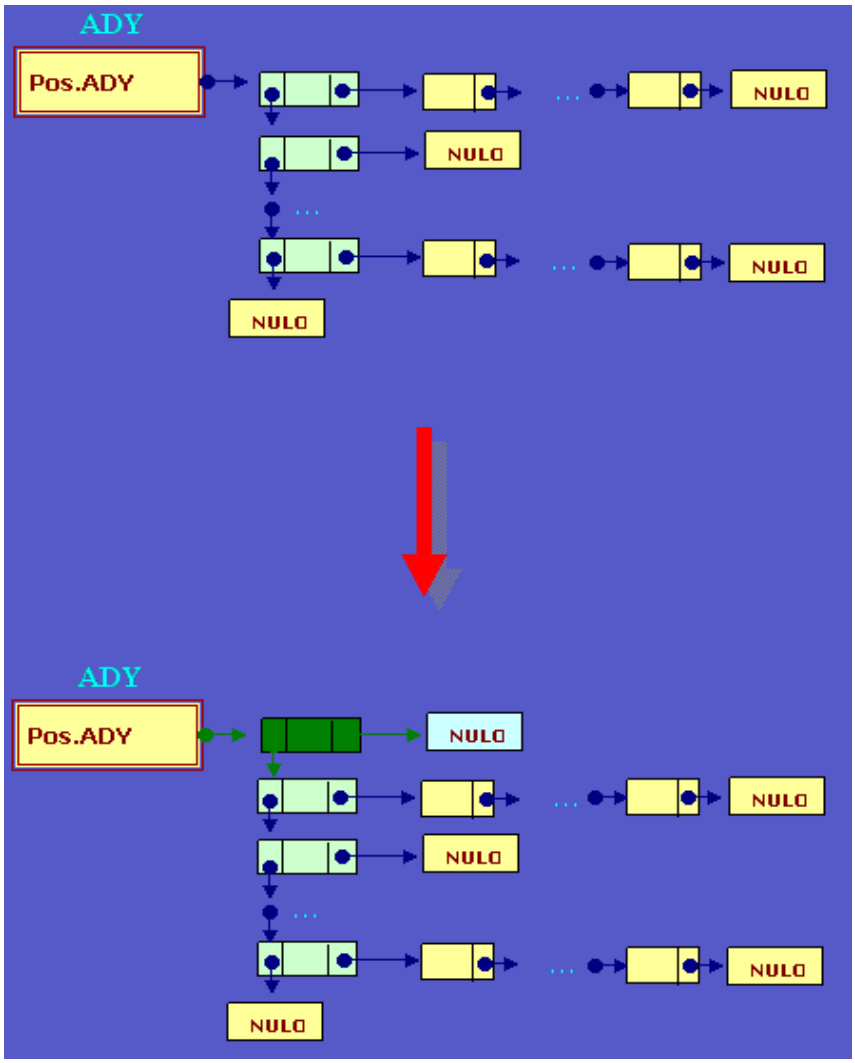
Para añadir un vértice VI al grafo, se debe insertar un nuevo nodo en la lista de vértices y hacer que referencie a VI. El nuevo nodo almacenará el identificador del nuevo vértice, tendrá a nulo el puntero a la lista de adyacencia de VI (pues inicialmente está vacía) y se enlazará con el primer nodo de la lista de vértices.

```
MÓDULO Añadir_vertice  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ADY puntero a NODO_V
                Recibe VI entera
        VARIABLES
                NUEVO puntero a NOVO_V
INICIO
        NUEVO ← RESERVAR_MEMORIA(tamaño(NODO_V))

        Si (NUEVO = NULO) entonces
                Escribir "Error al solicitar memoria"
                DEVOLVER Falso              ** inserción no realizada
```

```
        sino
                NUEVO↑.ID_VERTICE ← VI
                NUEVO↑.P_LISTA ← NULO
                NUEVO↑.ENLACE ← ADY

                ADY ← NUEVO
                DEVOLVER Verdadero          ** inserción realizada
        fin-si
FIN
```




## *Añadir un Arco*

Para añadir un arco desde el vértice V1 a V2, se debe añadir un nodo a la lista de adyacencia de V1 que identifique a V2 como extremo del arco. V1 ya debe formar parte del grafo, para que exista su lista de adyacencia. El nuevo nodo se inserta al principio de dicha lista.

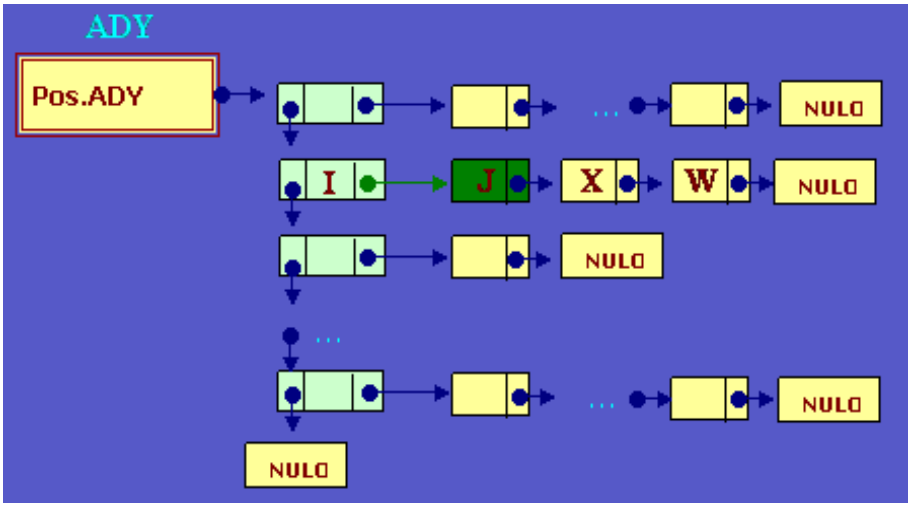
```
MÓDULO Añadir_arco  devuelve lógico
DATOS
        PARÁMETRO
                Transforma ADY puntero a NODO_V
                Recibe V1 entera
```

```
                Recibe V2 entera
                Recibe C real
        VARIABLES
                NUEVO puntero a NODO_L
                P_V1 puntero a NODO_V
INICIO
        P_V1 ← Buscar_vertice(ADY, V1)

        Si (P_V1 = NULO) entonces
                ESCRIBIR "El primer vértice no existe"
                DEVOLVER Falso                  ** inserción no realizada
        sino
                NUEVO ← RESERVAR_MEMORIA(tamaño(NODO_L))

                Si (NUEVO = NULO) entonces
                        Escribir "Error al solicitar memoria"
                        DEVOLVER Falso          ** inserción no realizada
                sino
                        NUEVO↑.ID_VERTICE ← V2
                        NUEVO↑.COSTE ← C
                        NUEVO↑.ENLACE ← P_V1↑.P_LISTA
                        P_V1↑.P_LISTA ← NUEVO

                        DEVOLVER Verdadero    ** inserción realizada
                fin-si
        fin-si
FIN
```



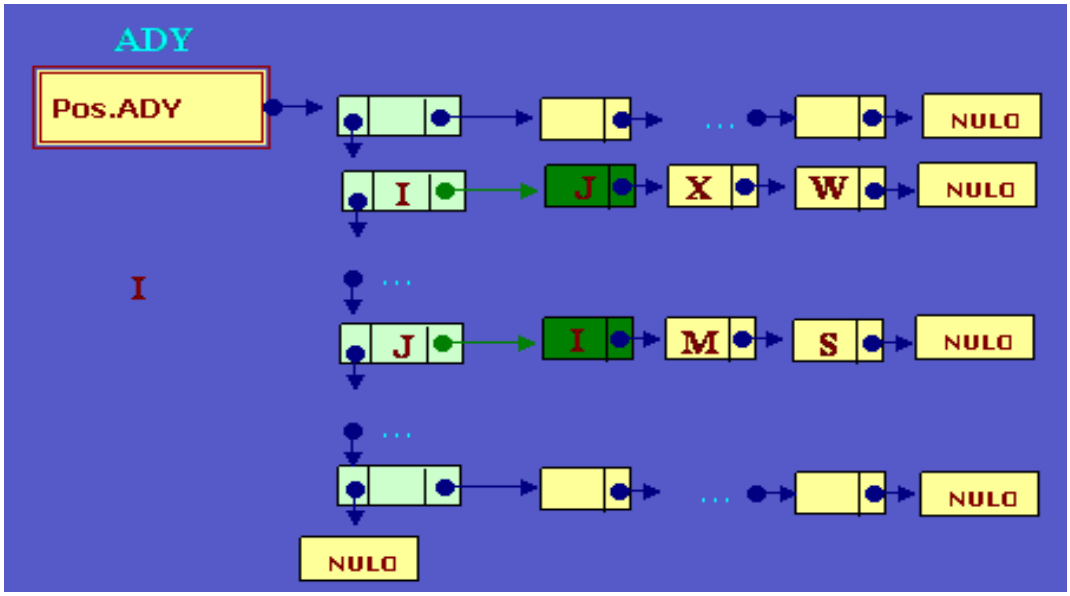

**Esquema de listas enlazadas tras añadir un arco de I a J**

## *Añadir una Arista*

Para añadir una arista entre dos vértices, V1 y V2, se pueden hacer dos llamadas al módulo que inserta un arco, solicitando en una de ellas la creación del arco (V1, V2) y en la otra la creación del arco (V2, V1). Los vértices V1 y V2 ya deben formar parte del grafo.

```
MÓDULO Añadir_arista  devuelve lógico
DATOS
        PARÁMETRO
                Transforma ADY puntero a NODO_V
                Recibe V1 entera
                Recibe V2 entera
                Recibe C real
        VARIABLES
                P_V1, P_V2 punteros a NODO_V
INICIO
        P_V1 ← Buscar_vertice(ADY, V1)
        P_V2 ← Buscar_vertice(ADY, V2)

        Si ( (P_V1 = NULO)  O  (P_V2 = NULO) ) entonces
                ESCRIBIR "No existe alguno de los extremos"
                DEVOLVER Falso              ** inserción no realizada
        sino
                Si (Añadir_arco(ADY, V1, V2, C) = Verdadero) entonces
                        DEVOLVER Añadir_arco(ADY, V2, V1, C)
                sino
                        DEVOLVER Falso     ** inserción realizada
                fin-si
        fin-si
FIN
```




**Esquema de listas enlazadas tras añadir una arista entre I y J**

## *Eliminar un Vértice*

Para eliminar un vértice, se debe eliminar el nodo de la lista de vértices que lo identifica, junto con la lista de adyacencia que tiene asociado este vértice. Además, se deben recorrer las listas de adyacencia de los restantes vértices del grafo para eliminar cualquier referencia al vértice que va a desaparecer del grafo.

```
MÓDULO Eliminar_vertice  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ADY puntero a NODO_V
                Recibe VI entera
        VARIABLES
                P1, PREVIO_P1 punteros a NODO_V
INICIO
        Si (Existe_vertice(ADY, VI) = Falso) entonces
                ESCRIBIR "El vértice no forma parte del grafo"
                DEVOLVER Falso
        fin-si

        PREVIO_P1 ← NULO
        P1 ← ADY

        Mientras (P1 ≠ NULO) hacer

                ** si el nodo actual referencia al vértice que se debe eliminar
                Si (P1↑.ID_VERTICE = VI) entonces

                        Si (P1↑.P_LISTA ≠ NULO) entonces
                                Vaciar_lista(P1↑.P_LISTA)
                        fin-si

                        Eliminar(ADY, P1)

                        Si (PREVIO_P1 = NULO) entonces
                                P1 ← ADY
                        sino
                                P1 ← PREVIO_P1↑.ENLACE
                        fin-si

                ** si no referencia al vértice que se debe eliminar
                sino
                        Eliminar_todos(P1, VI)
                         PREVIO_P1 ← P1
                        P1 ← P1↑.ENLACE
                fin-si
```

```
        fin-mientras

        DEVOLVER Verdadero
    FIN
```

El módulo **Vaciar_lista** elimina la lista de adyacencia de un vértice. El módulo **Eliminar** elimina de la lista de vértices el nodo apuntado por el puntero P1. Estas operaciones se describieron en el tema referente a listas. El módulo **Eliminar_todos** recorre una lista de adyacencia y elimina todos los nodos que hagan referencia al vértice pasado como argumento.

```
    MÓDULO Eliminar_todos
    DATOS
        PARÁMETROS
            Recibe P puntero a NODO_V
            Recibe VI entera
        VARIABLES
            P2, PREVIO_P2 punteros a NODO_L
    INICIO
        P2 ← P↑.P_LISTA
        PREVIO_P2 ← NULO

        Mientras(P2 ≠ NULO) hacer

            Si (P2↑.ID_VERTICE = VI) entonces
                Si (P2 = P↑.P_LISTA) entonces
                    P↑.P_LISTA ← P2↑.ENLACE
                sino
                    PREVIO_P2↑.ENLACE ← P2↑.ENLACE
                fin-si

                LIBERAR(P2)
            fin-si

            PREVIO_P2 ← P2
            P2 ← P2↑.ENLACE
        fin-mientras
    FIN
```

## *Eliminar un Arco*

Para eliminar un arco desde el vértice V1 hasta el V2, se debe eliminar el nodo que contenga el valor V2 en la lista de adyacencia asociada a V1, siempre y cuando exista el vértice V1 y esté conectado con V2.

```
MÓDULO Eliminar_arco  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ADY puntero a NODO_V
                Recibe V1 entera
                Recibe V2 entera
        VARIABLES
                P_V1 puntero a NODO_V
                RET lógica
INICIO
        P_V1 ← Buscar_vertice(ADY, V1)

        Si (P_V1 ≠ NULO) entonces
                RET ← Eliminar_valor(P_V1↑P_LISTA, V2)
                DEVOLVER RET
        sino
                ESCRIBIR "No existe el primer vértice"
                DEVOLVER Falso
        fin-si
FIN
```

Si existe un arco desde V1 hasta V2, el módulo **Eliminar_valor** eliminará de la lista de adyacencia del vértice V1 el nodo que contenga la referencia al vértice V2. El módulo recibe como argumentos el puntero que da acceso a la lista de adyacencia de un vértice del grafo y el identificador de otro vértice. Se describió en el tema referente a listas, aunque en este caso incluimos un valor de retorno que informa del éxito o el fracaso de la operación de eliminación.

## *Eliminar una Arista*

Para eliminar una arista entre dos vértices V1 y V2, se debe eliminar de la lista de adyacencia de V1 el nodo que haga referencia a V2, y de la lista de adyacencia de V2 el nodo que referencie a V1. Por tanto, la eliminación se puede realizar haciendo dos llamadas al módulo que elimina un arco, invirtiendo el orden de los vértices en ambas llamadas.

```
MÓDULO Eliminar_arista  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ADY puntero a NODO_V
                Recibe V1 entera
                Recibe V2 entera
INICIO
        Si (Existe_arista(ADY, V1, V2) = Verdadero) entonces
                Eliminar_arco (ADY, V1, V2)
                Eliminar_arco(ADY, V2, V1)
```

```
                DEVOLVER Verdadero
        sino
                ESCRIBIR "No existe la arista"
                DEVOLVER Falso
        fin-si
FIN
```

En primer lugar se comprueba que exista la arista que se desea eliminar. En caso afirmativo, se elimina la conexión en ambos sentidos.

### *Eliminar las Listas de Adyacencia*

Cuando la estructura grafo deje de ser necesaria, debe liberarse la memoria asociada a las listas de adyacencia. Esta eliminación supone borrar las N listas asociadas a los N vértices del grafo y, además, borrar la lista de vértices.

Se recorrerá la lista de vértices, eliminando en primer lugar la lista de adyacencia asociada al vértice y, a continuación, el nodo de la lista de vértices que daba acceso a dicha lista enlazada.

```
MÓDULO Borrar_listas_adyacencia
DATOS
        PARÁMETROS
                Transforma ADY puntero a NODO_V
        VARIABLES
                P puntero a NODO_V
INICIO
        Mientras ( ADY ≠ NULO) hacer
                Vaciar_lista (ADY↑.P_LISTA )
                P ← ADY
                ADY ← ADY↑.ENLACE
                LIBERAR(P)
        fin-mientras
FIN
```

El módulo **Vaciar_lista**, que vacía una lista dinámica lineal con enlace simple, se comentó en el tema referente a listas.

## *4.1.5 Recorrido de un Grafo*

Recorrer un grafo supone intentar alcanzar todos los vértices que estén relacionados con uno dado, que tomaremos como vértice inicial del recorrido. Existen dos técnicas básicas para recorrer un grafo: el **recorrido en anchura** y el **recorrido en profundidad**. Ambas técnicas se pueden aplicar tanto a grafos dirigidos como no dirigidos.

Dado que un vértice del grafo normalmente está conectado con varios, al realizar un recorrido podemos alcanzar un mismo vértice por varios caminos. Para evitar visitar varias veces un mismo vértice, se deben marcar de algún modo los vértices ya visitados. Para

ello utilizaremos un vector de N elementos, en el que la posición i-ésima indique si el vértice i-ésimo ya ha sido visitado (Verdadero) o no (Falso):

**VISITADO(N) de tipo lógico**

Antes de comenzar el recorrido del grafo se pondrán a Falso todos los elementos del vector (mediante un módulo que llamaremos **Iniciar_vector_visitados**) y, a medida que se vayan visitando vértices, se pondrá la posición correspondiente del vector a Verdadero.

Para hacer una descripción más general de los algoritmos de recorrido, la variable G representa el grafo de N vértices que se desea recorrer, que puede estar representado bien mediante su matriz de adyacencia o mediante listas de adyacencia. El vértice de partida del recorrido será el V0. Se supone que el objetivo del recorrido del grafo es aplicar algún tratamiento a todos los vértices, que representaremos por la operación, TRATAR() (por ejemplo, visualizar su contenido, acumular su valor en una operación,...).

## *Recorrido en Anchura*

El recorrido en anchura supone recorrer el grafo en niveles, partiendo de un vértice dado. El vértice procesado en primer lugar es el de partida del recorrido, a continuación se recorren los vértices que están a una distancia de un arco/arista del vértice de salida, después los que están a dos arcos/aristas de distancia, y así sucesivamente hasta alcanzar todos los vértices a los que se pueda llegar desde el vértice de salida. Por tanto, al visitar el nodo de partida se debe almacenar una referencia a todos sus adyacentes. Cuando se visite un nodo de este conjunto, se deben almacenar también las referencias a los vértices adyacentes al segundo que se ha tratado, teniendo en cuenta que tales vértices deben ser procesados sólo cuando se hayan procesado los del primer conjunto. El orden en que se debe ir aplicando el tratamiento a los nodos sugiere el uso de una cola para almacenar las referencias a los vértices adyacentes a los que se van visitando.

Para hacer el algoritmo más general, se utilizará una variable del tipo COLA, para representar cualquier implementación de cola. Las operaciones a realizar sobre la cola serán función de la representación elegida.

```
MÓDULO Primero_en_anchura
DATOS
    PARÁMETROS
        Recibe G de tipo GRAFO      ** grafo a recorrer
        Recibe V0 entera            ** vértice de partida
        Recibe N entera             ** vértices del grafo
    VARIABLES
        C de tipo COLA
        V1, V2 enteras
        VISITADO(N) de tipo lógico
INICIO
    Iniciar_vector_visitados(N, VISITADO)
    Crear_Cola(C)
```

```
        Encolar(C, V0)
        VISITADO(V0) ← Verdadero

        Mientras  (Cola_vacia(C) = Falso)  hacer
            V1 ← Desencolar(C)
            TRATAR(V1)

            Para todo vértice, V2, adyacente a V1 hacer
                Si (VISITADO(V2) = Falso) entonces
                    Encolar(C, V2)
                    VISITADO(V2) ← Verdadero
                fin-si
            fin-para
        fin-mientras
    FIN
```

Aunque la variable G no se usa explícitamente en ninguna instrucción, se utilizará para determinar los vértices adyacentes a uno dado, por lo que es necesario incluirla en la lista de parámetros del módulo. Esta operación depende de la forma de representación del grafo.

Si el grafo no es conexo, el módulo propuesto recorrerá la componente conexa a la que pertenece el vértice V0. Para recorrer todo el grafo se debe hacer una nueva llamada al módulo, tomando como vértice de partida alguno que no pertenezca a la componente conexa recorrida. Esta información se puede obtener del vector VISITADO, pasando el vector como parámetro al módulo que realiza el recorrido.

La diferencia al adaptar el algoritmo general para cada una de las implementaciones de la estructura de datos grafo, residirá en la manera de averiguar los diferentes vértices adyacentes a uno dado. Si se utiliza la matriz de adyacencia se tendrán que comprobar las posiciones de la fila de la matriz correspondiente al vértice V1. Si se utilizan listas de adyacencia, los vértices adyacentes a uno dado son los que aparecen en su lista de adyacencia.

## *Recorrido en Profundidad*

El recorrido en profundidad trata de buscar los caminos que parten desde el vértice de salida hasta que ya no es posible avanzar más (no se puede pasar a ningún otro vértice, o todos aquellos a los que se puede pasar ya forman parte del camino). Cuando ya no se puede avanzar más sobre el camino elegido, se vuelve atrás en busca de caminos alternativos que no se analizaron previamente.

En primer lugar se visita el vértice de partida del recorrido, a continuación se visita uno de sus vértices adyacentes, a continuación un vértice no visitado adyacente a este último, y así sucesivamente hasta que se llegue a un vértice para el que no existen vértices adyacentes aún no visitados. En ese momento se vuelve atrás un nivel y se busca un nuevo camino por el que seguir el recorrido. La estructura de datos adecuada para poder

avanzar todos los niveles posibles en el recorrido es una pila. Al visitar el primer vértice, se apilan los vértices adyacentes. A continuación, se desapila uno de dichos vértices y se apilan los vértices no visitados adyacentes a él, y así sucesivamente.

Para hacer el algoritmo más general, se utilizará una variable del tipo PILA, para representar cualquier implementación de pila. Las operaciones a realizar sobre la pila serán función de la representación elegida.

```
Módulo Primero_en_profundidad
DATOS
    PARÁMETROS
        Recibe G de tipo GRAFO     ** grafo a recorrer
        Recibe V0 entera           ** vértice de partida
        Recibe N entera            ** vértices del grafo
    VARIABLES
        P de tipo PILA
        V1, V2 enteras
        VISITADO(N) de tipo lógico
INICIO
    Iniciar_vector_visitados(N, VISITADO)
    Crear_pila(P)
    Apilar(P, V0)
    VISITADO(V0) ← Verdadero

    Mientras  (Pila_vacia(P) = Falso)  hacer
        V1 ← Desapilar(P)
        TRATAR(V1)

        Para todo vértice, V2, adyacente a V1 hacer
            Si (VISITADO(V2) = Falso) entonces
                Apilar(P, V2)
                VISITADO(V2) ← Verdadero
            fin-si
        fin-para
    fin-mientras
FIN
```

Observamos que la estructura del algoritmo es idéntica a la propuesta para el recorrido en anchura. La única diferencia reside en el uso de una pila en lugar de una cola.

El recorrido en profundidad también se puede implementar mediante una función recursiva, lo que hace innecesario el uso de una pila. Se elige un vértice de partida, se marca como visitado y se recorren los vértices no visitados adyacentes al de partida, usando recursivamente la búsqueda primero en profundidad. El primer elemento tratado es el vértice de partida. Una vez aplicado el tratamiento, se marca como visitado. A continuación, se entra en un bucle, que se repite para cada vértice adyacente al de partida. Si se encuentra un vértice adyacente al de partida que aún no haya sido visitado,

se realiza una llamada recursiva al módulo, pasando como argumentos el grafo, el vértice adyacente encontrado, el número de vértices del grafo y el vector de visitados. En esta nueva llamada al módulo se repite el proceso, pero partiendo ahora de un nuevo vértice.

```
Módulo Primero_en_profundidad_rec
DATOS
    PARÁMETROS
        Recibe G de tipo GRAFO     ** grafo a recorrer
        Recibe V0 entero           ** origen del recorrido
        Recibe N entera            ** vértices del grafo
        Transforma VISITADO(N) de tipo lógico
    VARIABLES
        V1 entera
INICIO
    VISITADO (V0) ← Verdadero
    TRATAR(V0)

    Para todo vértice, V1, adyacente a V0 hacer
        Si  (VISITADO(V1) = Falso )  entonces
            Primero_en_profundidad_rec (G, V1, N, VISITADO)
        fin-si
    fin-para
FIN
```

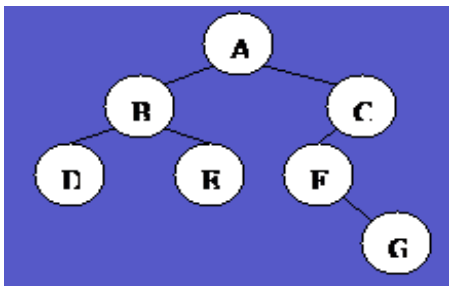
Por el mismo motivo que se comentó en el recorrido en anchura, para recorrer todos los vértices de un grafo que incluya varias componentes conexas o fuertemente conexas, llamaremos varias veces al módulo que realiza el recorrido.

## 4.2 Árboles

### *4.2.1 Definición - Árboles binarios*

Un **árbol binario** es un conjunto finito de elementos que o está vacío o dividido en 3 subconjuntos disjuntos. Un subconjunto contiene un elemento denominado raíz del árbol, que es único. Los otros dos subconjuntos, denominados **subárbol izquierdo** y **subárbol derecho**, son por si mismos árboles binarios. Las raíces de los subárboles se denominan **hijo izquierdo** e **hijo derecho**, respectivamente, de la raíz del árbol inicial.

Dado un grafo G = (V, A) no dirigido, llamaremos **árbol** de G a todo subgrafo de G conexo y sin ciclos. Utilizaremos la notación T = (V, E) para representar un árbol, formado por un conjunto de nodos V y un conjunto de conexiones E (o **ramas**). Los nodos de un árbol se suelen representar gráficamente mediante círculos y las conexiones mediante segmentos que conectan dos círculos.




Un árbol es una estructura jerárquica que permite representar relaciones entre los nodos, siendo la fundamental la relación padre-hijo. Si existe una conexión desde el nodo A al B, se dice que A es el **padre** de B y que B es **hijo** de A. Los hijos del mismo padre se denominan **hermanos**.

Los nodos del árbol se pueden clasificar en tres tipos:

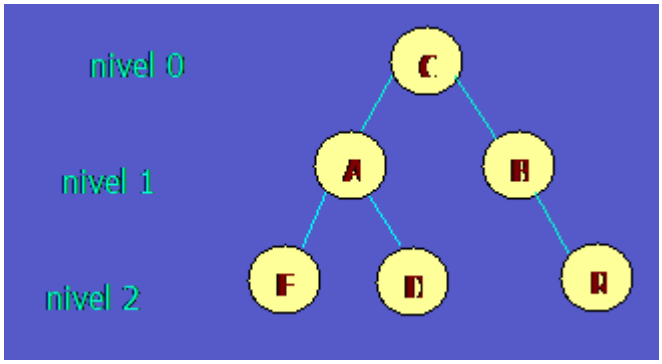
- **raíz** : nodo especial en el árbol, que no tiene padre y es único. Si un árbol no tiene raíz se denomina *árbol libre*.
- **nudos** o **nodos internos** : cualquier nodo del árbol que tiene padre y al menos un hijo.
- **hojas**, **vértices externos** o **colgantes** : nodos que no tiene hijos. Todo árbol debe contener como mínimo 2 hojas.

Una vez fijada la raíz de un árbol, se pueden clasificar los nodos en **niveles**. La raíz estará en el nivel 0, sus hijos en el nivel 1, los hijos de éstos en el nivel 2, y así sucesivamente.

Se puede definir el **nivel** o **profundidad de un nodo** como la longitud del camino que permite ir desde la raíz al nodo.

Se denomina **altura de un árbol** al número de niveles del árbol. La altura de un árbol vacío es 0.

**EJEMPLO**:




Nodos del árbol: {C, A, B, E, D, R}

Ramas del árbol: { (C, A), (C, B), (A, E), (A, D), (B, R) }

Raíz: C

Nodos hoja: E, D, R

Nudos: A, B

Altura del árbol: 3

Hijos de la raíz: A y B

Hijos del nodo B: R

Padre del nodo D: A

Padre del nodo E: A

Nodos hermanos: A y B; E y D

### *4.2.2 Operaciones básicas*

Las operaciones fundamentales que nos permiten definir un árbol binario son las siguientes:

- **Crear el árbol**: es la primera operación que se realizará sobre la estructura de datos. Permitirá describir la estructura de la información con la que se representará cada nodo del árbol, reservar memoria para almacenar la información que define el árbol o para un puntero a la estructura. El árbol se crea vacío.
- **Crear un nodo**: permite añadir un nuevo nodo al árbol. Supondremos que para cada nuevo nodo su padre ya forma parte del árbol, salvo para la raíz, lógicamente.
- **Eliminar un nodo**: permite eliminar un nodo del árbol.
- **Modificar el valor asociado a un nodo**: permite modificar el dato asociado a un nodo del árbol.
- **Comprobar si el árbol está vacío**: permite comprobar si el árbol no contiene ningún nodo. Se debe comprobar que el árbol no esté vacío antes de intentar eliminar algún elemento o acceder al mismo.

Si el árbol está representado mediante un vector, se considera además la operación de **comprobación de árbol lleno**, necesario para evitar intentar añadir nodos al árbol cuando el vector ya no puede alojar más.

### *4.2.3 Representación de Árboles binarios*

Los árboles binarios se pueden representar dentro de un programa de dos formas básicas:

- utilizando un vector
- utilizando memoria dinámica

En ambos casos se almacenará información sobre cada nodo:

- un posible *dato* asociado al nodo

- una referencia a los *hijos* de éste
- una referencia a su *padre*

En ocasiones se omite la referencia al padre, ya que se puede localizar el padre de cualquier nodo realizando un recorrido del árbol (como se comentará en apartados posteriores). Con esto se reduce el espacio de memoria necesario para almacenar el árbol.

## *Representación vectorial de árboles*

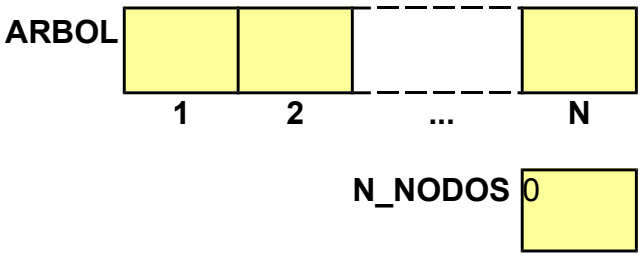
Se utilizará un vector con tantas posiciones como nodos pueda incluir el árbol. Las operaciones de creación de nodos deben comprobar siempre que no se exceda el espacio reservado para el vector.

**ARBOL(N) de tipo NODO**

Utilizaremos una variable entera, denominada **N_NODOS**, para determinar los nodos que contiene el árbol en cada momento. Dicha variable tomará valores entre 0 (árbol vacío) y N (árbol lleno).

**N_NODOS entero**

Cuando se crea el árbol se da valor 0 al contador de nodos, creando un árbol vacío:



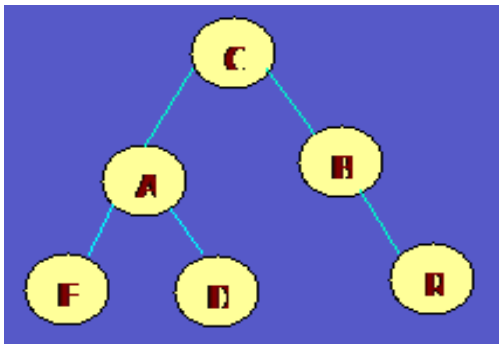

**Variables necesarias para trabajar con un árbol representado de forma estática**

Para cada nodo del árbol se almacenará cierta información, de algún tipo T, además de enteros que determinen la posición del padre, el hijo izquierdo y el derecho de dicho nodo dentro del vector que representa el árbol:

```
NODO es registro compuesto por
        DATO de tipo T                ** información asociada al nodo
        HIJO_IZQUIERDO entero   ** posición del hijo izquierdo
        HIJO_DERECHO entero     ** posición del hijo derecho
        PADRE entero                  ** posición del padre
fin-registro
```

En nuestro caso los campos **HIJO_DERECHO**, **HIJO_IZQUIERDO** y **PADRE** tomarán valores entre 1 y **N**, salvo cuando no exista el elemento al que hacen referencia, en cuyo caso almacenaremos el valor -1 para indicarlo.

EJEMPLO:




Mostramos una posible forma de almacenar los nodos del árbol de la figura dentro del vector **ARBOL**. El orden en que se pueden almacenar los nodos dentro del vector no es único. Para cada nodo se muestran en 4 filas sucesivas su etiqueta, su hijo izquierdo, su hijo derecho y su padre:

| | C | A | B | F | D | R |
|---|---|---|---|---|---|---|
| **ARBOL** | 2 | 4 | -1 | -1 | -1 | -1 |
| | 3 | 5 | 6 | -1 | -1 | -1 |
| | -1 | 1 | 1 | 2 | 2 | 3 |
| | **1** | **2** | **3** | **4** | **5** | **6** |

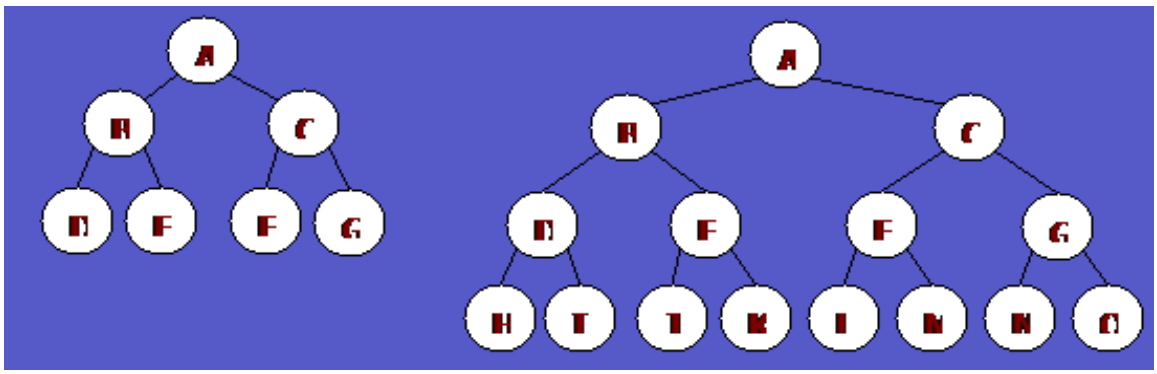
| | |
|---|---|
| **N_NODOS** | 6 |

**Representación vectorial de árboles - un caso especial**

Los árboles binarios llenos o completos se pueden representar de forma eficiente utilizando una forma vectorial simplificada:

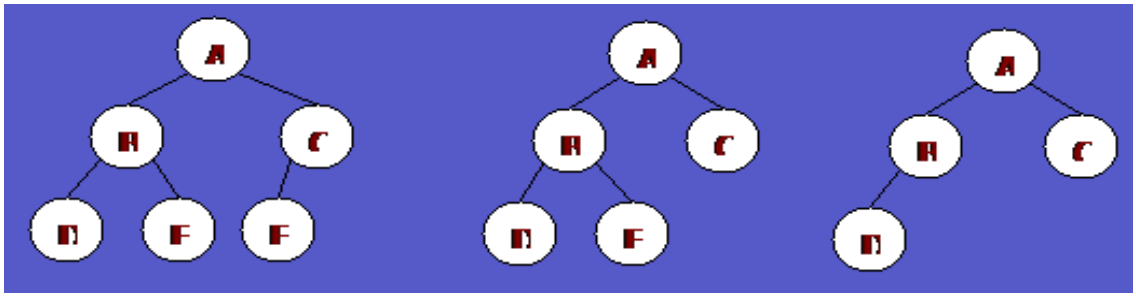
**Árboles Binarios llenos y completos**

Se dice que un **árbol binario** está **lleno** si es un árbol binario de profundidad k que tiene 2k-1 nodos. Es decir, un árbol binario lleno es aquel que contiene el número máximo de posibles nodos en función de su profundidad.



**Ejemplo de árboles binarios llenos**

Un árbol binario de profundidad k es un **árbol binario completo** si cualquier nodo i a un nivel menor que k-2 tiene dos hijos; además, todo nodo i con un descendiente derecho en el nivel k-1 debe tener un hijo izquierdo y cada descendiente izquierdo de i es una hoja en el nivel k-1 o tiene dos hijos.



**Ejemplo de árboles binarios completos**

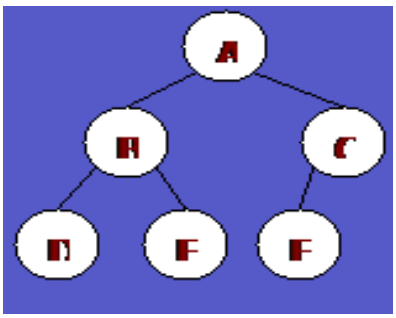
Estos dos tipos de árbol se pueden representar mediante un vector de forma sencilla. Para ello los **N** nodos del árbol se numeran de 1 a **N** comenzando por la raíz y avanzando por los sucesivos niveles de izquierda a derecha. Los nodos del árbol se pueden almacenar en un vector de **N** elementos, colocando cada uno en la posición que corresponde a su número. No es necesario almacenar referencias a las posiciones del padre o los hijos, dado que se cumplen las siguientes relaciones:

- Si el nodo i tiene padre, estará ubicado en la posición (i div 2) del vector.
- Si el nodo i tiene hijo izquierdo, estará ubicado en la posición (2i).
- Si el nodo i tiene hijo derecho, estará ubicado en la posición (2i+1).

El nodo i, con valores entre 1 y **N**, tendrá:

- padre si no es la raíz del árbol (i>1).
- hijo izquierdo si la posición que debe ocupar éste en el vector, de existir, es menor o igual que el número de nodos del árbol (2i ≤ **N** )
- hijo derecho si la posición que debe ocupar éste en el vector, de existir, es menor o igual que el número de nodos del árbol (2i+1 ≤ **N**).

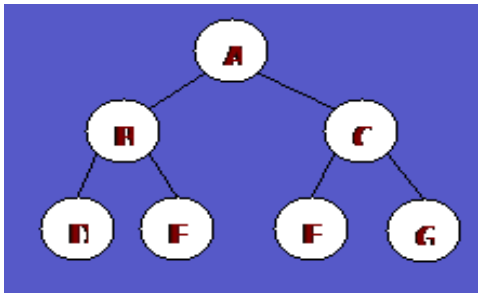
**EJEMPLOS**:



*1 2 3 4 5 6*

| A | B | C | D | E | F |
|---|---|---|---|---|---|

Árbol completo y su representación vectorial



*1 2 3 4 5 6 7*

| A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|

Árbol lleno y su representación vectorial

## *Representación basada en el uso de memoria dinámica*

También se puede representar un árbol trabajando con memoria dinámica. La información almacenada para cada nodo del árbol es similar a la descrita para la representación vectorial, aunque en este caso el padre y los hijos de cada nodo se identifican mediante punteros. Si existe el elemento correspondiente (el padre o el hijo), el puntero apuntará a un nodo del árbol; en caso contrario el puntero se hará nulo:

```
NODO es registro compuesto por
        DATO de tipo T                      ** información asociada al
nodo
        HIJO_IZQUIERDO puntero a NODO  ** puntero al hijo izquierdo
        HIJO_DERECHO puntero a NODO    ** puntero al hijo derecho
        PADRE puntero a NODO           ** puntero al padre
fin-registro
```

En este caso se definirá un puntero que apuntará permanentemente a la raíz del árbol, a partir de la cual se podrá acceder a todos sus nodos:

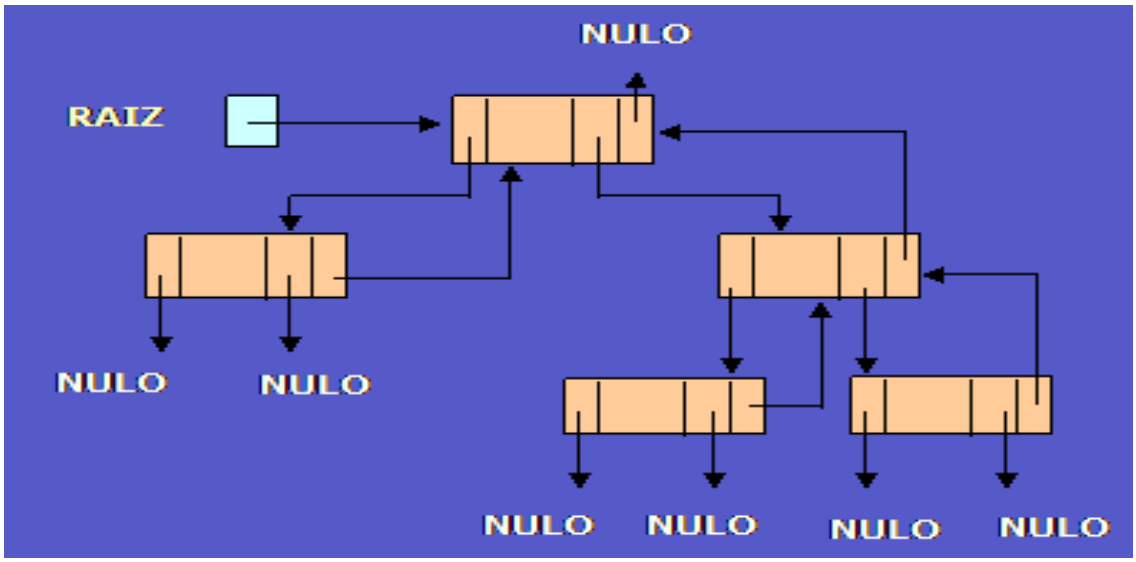
**RAIZ puntero a NODO**

Este puntero se hará inicialmente nulo para indicar que el árbol está vacío:

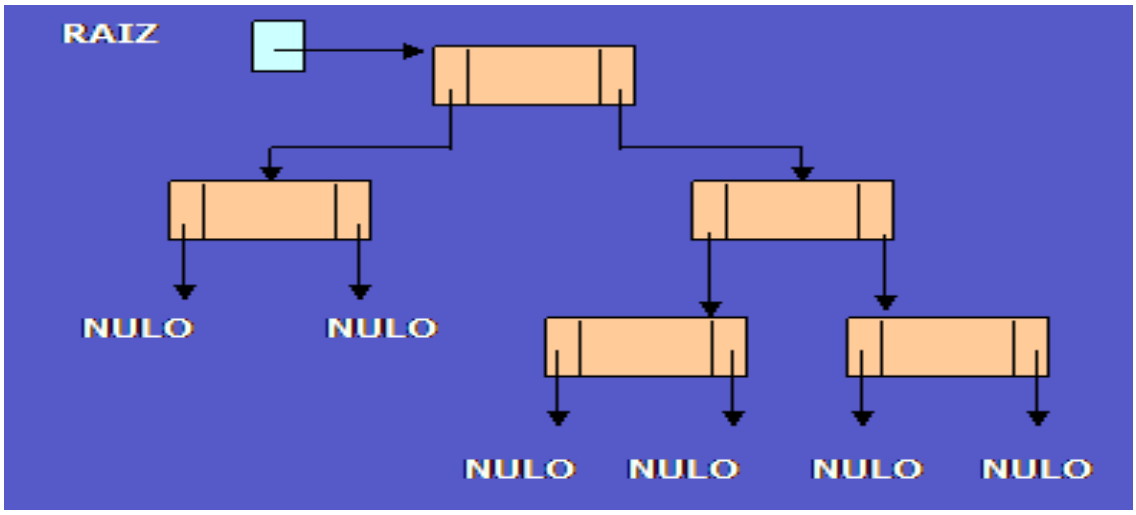
**RAIZ ← NULO**

EJEMPLO:

El esquema del árbol del ejemplo anterior, representado mediante nodos enlazados, se muestra a continuación. Se consideran dos posibilidades: que exista o que no exista el campo PADRE:




Representación de árbol mediante nodos enlazados incluyendo un campo padre




Representación de árbol mediante nodos enlazados sin incluir un campo al padre

En el resto del tema simplificaremos esta representación gráfica, sustituyendo el rectángulo que representa a un nodo de la estructura enlazada por un pequeño círculo del que salen los enlaces a los hijos y al padre, si existe este enlace. Los enlaces nulos no se representan.

## *4.2.4 Operaciones con Árboles*

### *Operaciones en la representación vectorial*

**Creación de un árbol vacío**

Para crear el árbol, en primer lugar se define la estructura que representa sus nodos:

```
NODO es registro compuesto por
        DATO de tipo T                  ** información asociada al nodo
        HIJO_IZQUIERDO entero    ** posición del hijo izquierdo
        HIJO_DERECHO entero      ** posición del hijo derecho
        PADRE entero                    ** posición del padre
fin-registro
```

A continuación, se define el vector que almacenará el árbol, con capacidad para N nodos como máximo:

```
ARBOL(N) de NODO
```

se define el contador de nodos y se hace cero:

```
N_NODOS entero
N_NODOS ← 0
```

**Comprobación de árbol vacío**

El árbol estará vacío si el contador de elementos es cero.

```
MÓDULO Arbol_vacio  devuelve lógico
DATOS
        PARÁMETROS
                Recibe NUM_NODOS entero
INICIO
        Si (NUM_NODOS = 0) entonces
                DEVOLVER Verdadero
        sino
                DEVOLVER Falso
        fin-si
FIN
```

**Comprobación de árbol lleno**

El árbol estará lleno si el contador de elementos es igual al tama□o del vector que aloja el árbol.

```
MÓDULO Arbol_lleno  devuelve lógico
DATOS
        PARÁMETROS
                Recibe NUM_NODOS entero
                Recibe N entero
```

```
INICIO
        Si (NUM_NODOS = N) entonces
                DEVOLVER Verdadero
        sino
                DEVOLVER Falso
        fin-si
FIN
```

### Inserción de un nodo

Para insertar un nuevo nodo en el árbol, su información asociada se almacenará en la primera posición libre al final del vector, siempre que dicho vector no esté lleno. Además, si el nuevo nodo no es la raíz del árbol, se debe modificar la información almacenada en el padre, de modo que pase a hacer referencia a su nuevo hijo.

El módulo de inserción recibe como parámetros el vector que implementa el árbol (**ARBOL**), el número de nodos que contiene actualmente el árbol (**NUM_NODOS**), el número de posiciones del vector (**N**), el índice del elemento del vector que contiene la información referente al padre del nuevo nodo, o -1 si el nuevo nodo va a ser la raíz del árbol (**I_P**), la información a almacenar en el campo de datos del nuevo nodo (**NUEVO_DATO**), y un parámetro que indica si el nuevo nodo debe ser hijo derecho (si **DERECHO** es Verdadero) o izquierdo (si es Falso) de su padre. El módulo retorna un valor lógico informando del éxito o fracaso de la operación de inserción:

```
MÓDULO Insertar_nodo  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ARBOL(N) de tipo NODO
                Transforma NUM_NODOS entera
                Recibe N entera
                Recibe I_P entera
                Recibe NUEVO_DATO de tipo T
                Recibe DERECHO de tipo lógico
INICIO
        Si ( NUM_NODOS = N ) entonces
                ESCRIBIR "No hay espacio para más nodos"
                DEVOLVER Falso
        fin-si

        Si (I_P ≠ -1) entonces
                Si (DERECHO = Verdadero) entonces
                    Si (ARBOL(I_P).HIJO_DERECHO = -1) entonces
                        ARBOL(I_P).HIJO_DERECHO ← NUM_NODOS+1
                    sino
                        ESCRIBIR "Ya existe el hijo derecho"
                        DEVOLVER Falso
```

```
                fin-si
            sino
                Si (ARBOL(I_P).HIJO_IZQUIERDO = -1) entonces
                    ARBOL(I_P).HIJO_IZQUIERDO ← NUM_NODOS+1
                sino
                    ESCRIBIR "Ya existe el hijo izquierdo"
                    DEVOLVER Falso
                fin-si
            fin-si
        fin-si

        NUM_NODOS ← NUM_NODOS +1

        ARBOL(NUM_NODOS).HIJO_IZQUIERDO ← -1
        ARBOL(NUM_NODOS).HIJO_DERECHO ← -1
        ARBOL(NUM_NODOS).DATO ← NUEVO_DATO
        ARBOL(NUM_NODOS).PADRE ← I_P

        DEVOLVER Verdadero
    FIN
```

**Inserción de un nuevo nodo - representación vectorial**

En primer lugar se comprueba si hay espacio en el vector para un nuevo elemento. Si lo hay y el nuevo nodo no es la raíz del árbol, se comprueba que no exista para el padre un hijo en la misma posición que va a ocupar el nuevo (izquierda o derecha). De existir dicho hijo, el proceso concluye sin completar la inserción. De no existir, se modifica la información almacenada en el padre sobre el hijo correspondiente. A continuación, se incrementa el número de elementos almacenados en el árbol y se almacena la información sobre el nuevo nodo, que pasará a ocupar la primera posición libre dentro del vector que contiene los nodos del árbol. Si los nodos del árbol no almacenan información sobre el padre de un nodo, se eliminará del módulo la última asignación incluida.

### Eliminación de nodos

Para mantener consistente la información del árbol, y simplificar las operaciones de eliminación, sólo permitiremos eliminar nodos hoja. Como tales nodos no tienen hijos, la eliminación no requerirá determinar un nuevo padre para ningún nodo.

También es sencillo eliminar la raíz o todo el árbol, provocando ambas operaciones que el árbol quede vacío.

#### Eliminación de una hoja

Al eliminar una hoja del árbol se reduce en uno el número de nodos del árbol. Al eliminar un nodo, además de descartar la información referente al mismo dentro del vector **ARBOL**, se debe modificar la información referente a su padre, de forma que se especifique que el padre ha dejado de tener el hijo que se ha eliminado.

Para mantener la información de los nodos del árbol en las primeras **NUM_NODOS** posiciones del vector **ARBOL**, puede ser necesario realizar desplazamientos de los elementos que permanecen en el vector. Si el nodo a eliminar ocupa la posición **R** dentro del vector **ARBOL**, se deben desplazar una posición hacia el comienzo del vector todos los datos comprendidos entre las posiciones **R**+1 y **NUM_NODOS**. Hay que tener en cuenta que al desplazar un nodo a la izquierda se debe actualizar la información de su padre y la de sus posibles hijos, de forma que hagan referencia correcta a la nueva ubicación del nodo dentro del vector.

Se podría plantear otra posibilidad para eliminar del vector la información del nodo. Para evitar los desplazamientos de información dentro del vector, podemos copiar la información del último elemento en la posición del que se va a eliminar. En este caso ya no garantizamos que la información de un nodo aparezca en el vector en una posición posterior a la de su padre.

```
MÓDULO Eliminar_hoja
DATOS
        PARÁMETROS
                Transforma ARBOL(N) de tipo NODO
                Recibe I_HOJA entera
                Transforma NUM_NODOS entera
        VARIABLES
                PADRE entera
INICIO
        PADRE ← Padre(ARBOL, NUM_NODOS, I_HOJA)

        ARBOL(I_HOJA) ← ARBOL(NUM_NODOS)

        Si (ARBOL(PADRE).HIJO_IZQUIERDO = I_HOJA) entonces
                ARBOL(PADRE).HIJO_IZQUIERDO ← -1
        sino
                ARBOL(PADRE).HIJO_DERECHO ← -1
        fin-si

        PADRE ← Padre(ARBOL, NUM_NODOS, NUM_NODOS)

        Si (ARBOL(PADRE).HIJO_IZQUIERDO = NUM_NODOS) entonces
                ARBOL(PADRE).HIJO_IZQUIERDO ← I_HOJA
        sino
                ARBOL(PADRE).HIJO_DERECHO ← I_HOJA
        fin-si

        Si (ARBOL(I_HOJA).HIJO_IZQUIERDO ≠ -1) entonces
          ARBOL(ARBOL(I_HOJA).HIJO_IZQUIERDO).PADRE ← I_HOJA
        fin-si
```

```
        Si (ARBOL(I_HOJA).HIJO_DERECHO ≠ -1) entonces
          ARBOL(ARBOL(I_HOJA).HIJO_ DERECHO).PADRE ← I_HOJA
        fin-si

        NUM_NODOS ← NUM_NODOS -1
FIN
```

**Eliminación de un elemento del vector - implementación vectorial**

Se copia la información del último nodo almacenado sobre el elemento del vector que almacena el nodo a eliminar. Para completar la eliminación correctamente, se deben actualizar los padres de la hoja que se elimina y del nodo cuya información se ha desplazado. El padre de la hoja que desaparece del árbol pasará a almacenar -1 en la posición del hijo representado por dicha hoja. El padre del nodo desplazado debe hacer referencia a la nueva ubicación de su hijo. Además, si existe el campo **PADRE** será necesario actualizar la información de los posibles hijos del nodo desplazado, para que identifiquen correctamente la nueva posición de su padre. Esta última actualización se hace necesaria, aunque el nodo eliminado sea una hoja, ya que sucesivas eliminaciones nos llevarán a que el nodo movido no sea necesariamente una hoja.

Para completar la eliminación, se reduce en uno el contador de nodos del árbol.

El módulo **Padre** retorna la posición del padre del nodo almacenado en la posición del vector indicada como tercer argumento de la llamada. Si los nodos almacenan información sobre el padre, simplemente se consultará dicho campo; en caso contrario, será necesario recorrer el árbol hasta encontrar un nodo que tenga como hijo (izquierdo o derecho) al indicado. Esta última operación se basa en la de recorrido del árbol, que se trata más adelante.

**Eliminación de la raíz**

Dado que para insertar un nodo en el árbol, debe existir ya su padre, el primer nodo insertado será la raíz del árbol. Por tanto, dicho nodo ocupará la primera posición del vector **ARBOL**.

Si se desea eliminar la raíz, eliminaremos todo el árbol, siguiendo el procedimiento descrito en el apartado posterior.

**Eliminación del árbol**

La operación se reduce a poner a cero la variable que determina el número de nodos del árbol.

```
MÓDULO Eliminar_arbol
DATOS
        PARÁMETROS
                Transforma NUM_NODOS entera
INICIO
        NUM_NODOS ← 0
FIN
```

### Modificación de un nodo del árbol

Permite modificar el campo de datos asociado al nodo. También se podrían modificar los campos que referencian a los hijos o al padre, de existir éste último, pero ello requiere comprobar que la estructura resultante sigue siendo un árbol válido.

```
MÓDULO Modificar_nodo
DATOS
        PARÁMETROS
                Transforma ARBOL(N) de tipo  NODO
                Recibe POSICION entero
                Recibe NUEVO_VALOR de tipo T
INICIO
        ARBOL(POSICION).DATO ← NUEVO_VALOR
FIN
```

## *Operaciones en la representación basada en el uso de memoria dinámica*

### Creación de un árbol vacío

Para crear el árbol, en primer lugar se define la estructura que representa sus nodos:

```
NODO es registro compuesto por
        DATO de tipo T                          ** información asociada al
nodo
        HIJO_IZQUIERDO puntero a NODO  ** puntero al hijo izquierdo
        HIJO_DERECHO puntero a NODO    ** puntero al hijo derecho
        PADRE puntero a NODO                    ** puntero al padre
fin-registro
```

A continuación, se define el puntero que dará acceso al árbol:

```
RAIZ puntero a NODO
```

y se hace nulo, indicando que el árbol se crea vacío:

```
RAIZ ← NULO
```

### Comprobación de árbol vacío

El árbol estará vacío si el puntero a la estructura es nulo.

```
MÓDULO Arbol_vacio  devuelve lógico
DATOS
        PARÁMETROS
                Recibe RAIZ puntero a NODO
INICIO
        Si (RAIZ = NULO) entonces
                DEVOLVER Verdadero
        sino
```

```
                DEVOLVER Falso
        fin-si
FIN
```

### Inserción de un nodo

El módulo que realiza la inserción toma como parámetros el puntero a la raíz del árbol, un puntero al padre del nuevo nodo (será NULO si el nuevo nodo es la raíz del árbol), la información que contendrá el nuevo nodo en el campo de datos, y una indicación de si debe ser hijo derecho (Verdadero) o hijo izquierdo (Falso) del padre. El módulo retorna Verdadero si la inserción se completa con éxito y Falso en caso contrario.

```
MÓDULO Insertar_hijo  devuelve lógico
DATOS
        PARÁMETROS
                Transforma RAIZ puntero a NODO
                Recibe PADRE puntero a NODO
                Recibe NUEVO_DATO de tipo T
                Recibe DERECHA de tipo lógico
        VARIABLES
                P_HIJO puntero a NODO
INICIO
        Si ( (  (PADRE ≠ NULO) Y (DERECHA = Verdadero) Y
                (PADRE↑.HIJO_DERECHO ≠ NULO)  )    O
             (  (PADRE ≠ NULO) Y (DERECHA = Falso) Y
                (PADRE↑.HIJO_IZQUIERDO ≠ NULO)  )     O
             (  (PADRE = NULO) Y (RAIZ ≠ NULO)  )  ) entonces
                        ESCRIBIR "Ya existe un nodo en la posición
indicada"
                        DEVOLVER Falso
        fin-si

        P_HIJO ← RESERVAR_MEMORIA(TAMAÑO(NODO))

        Si (P_HIJO = NULO) entonces
                DEVOLVER Falso  ** inserción anulada
        fin-si

        P_HIJO↑.HIJO_IZQUIERDO ← NULO
        P_HIJO↑.HIJO_DERECHO ← NULO
        P_HIJO↑.DATO ← NUEVO_DATO

        Si (PADRE = NULO) entonces
                RAIZ ← P_HIJO
        sino
                Si (DERECHA = Verdadero) entonces
```

```
                PADRE↑.HIJO_DERECHO ← P_HIJO
            sino
                PADRE↑.HIJO_IZQUIERDO ← P_HIJO
            fin-si
        fin-si

        DEVOLVER Verdadero   ** inserción completada
    FIN
```

En primer lugar se comprueba que no exista ya un nodo en la posición del árbol en la que se debe colocar el nuevo. El nuevo nodo puede ocupar tres lugares:

- la raíz del árbol: la inserción será posible si el árbol está vacío.
- hijo derecho de su padre: la inserción será posible si se ha especificado un padre que aún no tenga hijo derecho.
- hijo izquierdo de su padre: la inserción será posible si se ha especificado un padre que aún no tenga hijo izquierdo.

Si la inserción es posible, se reserva espacio para el nuevo nodo del árbol. Si la solicitud de memoria se ha completado con éxito, se almacena en el nuevo nodo la información correspondiente al campo de datos (contenida en **NUEVO_DATO**), y se hacen nulos los punteros a los hijos.

Si el nuevo nodo va a ser la raíz del árbol, se hace que el puntero a la estructura pase a apuntar a dicho nodo. En caso contrario, se enlaza el padre del nodo con su nuevo hijo, considerando el campo de enlace que corresponda a la posición que debe ocupar.

### Eliminación de nodos

La representación del árbol basada en el uso de memoria dinámica hace sencillo eliminar cualquier nodo del árbol. Se pueden eliminar tres tipos de nodos, realizando un tratamiento diferente en cada caso:

- una *hoja*: simplemente se hace nulo el campo de enlace desde el padre a esta hoja y se libera la memoria que ocupa la hoja.
- la *raíz*: se eliminará el árbol completo.
- un nudo: se eliminará el subárbol para el que dicho nodo es raíz.

#### Eliminación de una hoja

Para eliminar una hoja del árbol, apuntada por el puntero **P**, se hace nulo el puntero del padre que apuntaba al nodo que se desea eliminar y se libera la memoria asociada al nodo.

```
MÓDULO Eliminar_hoja
DATOS
        PARÁMETROS
```

```
                Recibe P puntero a NODO
                Recibe RAIZ puntero a NODO
        VARIABLES
                PADRE puntero a NODO
INICIO
        PADRE ← Padre(P, RAIZ)

        Si (PADRE↑.HIJO_IZQUIERDO = P) entonces
                PADRE↑.HIJO_IZQUIERDO ← NULO
        sino
                PADRE↑.HIJO_DERECHO ← NULO
        fin-si

        LIBERAR(P)
FIN
```

**Eliminación del árbol**

Para eliminar el árbol se utiliza un procedimiento recursivo que en primer lugar elimina los subárboles y, por último, elimina la raíz. Al concluir el módulo el puntero a la raíz del árbol se habrá hecho nulo.

```
MÓDULO Eliminar_arbol
DATOS
        PARÁMETROS
                Transforma RAIZ puntero a NODO
INICIO
        Si (RAIZ ≠ NULO) entonces
                Eliminar_arbol (RAIZ↑.HIJO_IZQUIERDO)
                Eliminar_arbol (RAIZ↑.HIJO_DERECHO)
                LIBERAR(RAIZ)
                RAIZ ← NULO
        fin-si
FIN
```

**Modificación de un nodo del árbol**

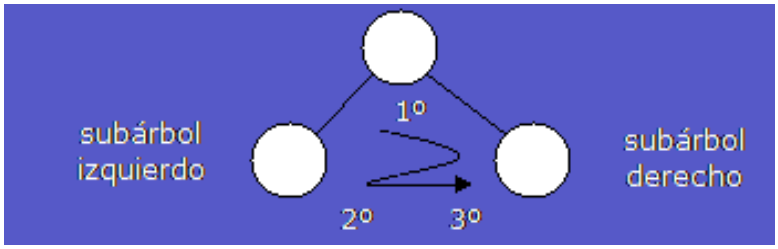
Permite modificar el campo de datos asociado al nodo. También se podrían modificar los campos de enlace, pero ello requiere comprobar que la estructura resultante sigue siendo un árbol válido.

```
MÓDULO Modificar_nodo
DATOS
        PARÁMETROS
                Recibe P puntero a NODO
                Recibe NUEVO_VALOR de tipo T
 INICIO
        P↑.DATO ← NUEVO_VALOR
FIN
```

### *4.2.5 Recorrido del árbol*

El recorrido de un árbol consiste en procesar cada nodo una vez para aplicar algún tratamiento (visualizar el contenido, consultarlo, modificarlo,).

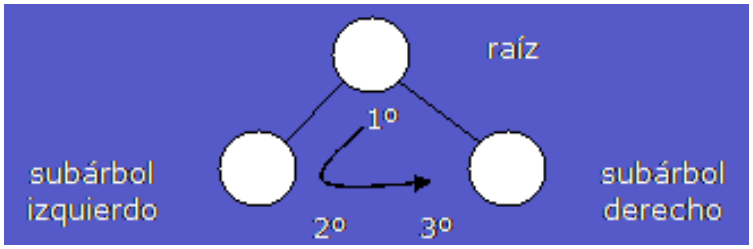
Se puede aplicar un **recorrido en anchura** o un **recorrido en profundidad**.
El **recorrido en anchura** parte de la raíz del árbol y visita los nodos por niveles. Este recorrido utiliza una cola como estructura auxiliar, para mantener una referencia a los subárboles izquierdo y derecho de cada nodo procesado.




Esquema de los pasos del recorrido en anchura de un árbol binario

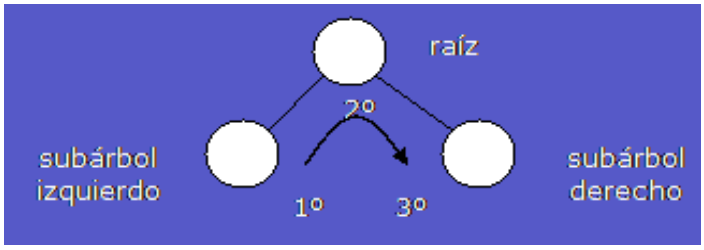
El **recorrido en profundidad** visita cada nodo y sus descendientes. Existen 3 variantes: el **recorrido en orden**, en **orden previo** y en **orden posterior**. En cualquiera de ellos, si el árbol no está vacío se recorrerá la raíz y los dos subárboles, residiendo la diferencia en el orden en que se procesan estos 3 elementos.

En el recorrido en **preorden** u **orden previo** en primer lugar se procesa la raíz del árbol, luego el subárbol izquierdo y, por último, el subárbol derecho.



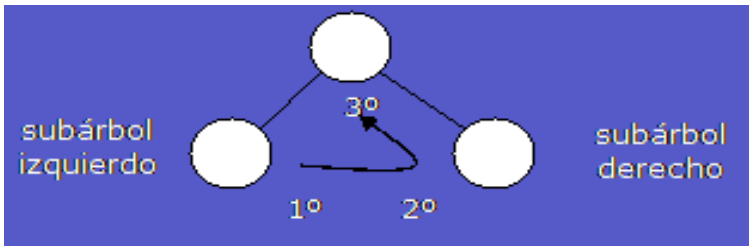

Esquema de los pasos del recorrido en orden previo de un árbol binario

En el recorrido en **orden** u **orden simétrico** se procesa primero el subárbol izquierdo, luego la raíz y, por último, el subárbol derecho.




Esquema de los pasos del recorrido en orden de un árbol binario

En el recorrido en **posorden** u **orden posterior** primero se recorre el subárbol izquierdo, luego el derecho y, por último, se aplica el tratamiento al nodo raíz.




Esquema de los pasos del recorrido en orden posterior de un árbol binario

## *Recorrido de un árbol representado mediante un vector*

### Recorrido en anchura

El árbol se recorre por niveles, comenzando por la raíz y avanzando hacia las hojas.

El módulo propuesto toma como parámetros el vector que almacena la información de los nodos del árbol y el índice de la posición que ocupa el árbol o subárbol que se va a recorrer.

La operación se apoya sobre una cola de enteros, que representan posiciones de nodos dentro del vector. En este caso se considera una cola lineal e implementada mediante el uso de memoria dinámica. Las operaciones de encolado, desencolado y comprobación de cola vacía se describen en el tema que describe las colas.

```
MODULO Recorrido_anchura
DATOS
        PARÁMETROS
                Recibe ARBOL(N) de tipo NODO
                Recibe RAIZ entero
        VARIABLES
                E_C puntero a NODO_COLA  ** punt. a la entrada de la cola
                S_C puntero a NODO_COLA  ** punt. a la salida de la cola
                P entero
INICIO
        Si (RAIZ = -1) entonces
                ACABAR
        fin-si

        E_C ← NULO
        S_C ← NULO
        Encolar(RAIZ, E_C, S_C )

        Mientras (Cola_vacia(E_C) = Falso) hacer
                Desencolar(P, E_C, S_C)
                TRATAR (ARBOL(P).DATO)
```

```
            Si (ARBOL(P).HIJO_IZQUIERDO ≠  -1) entonces
                  Encolar(ARBOL(P).HIJO_IZQUIERDO, E_C, S_C)
            fin-si

            Si (ARBOL(P).HIJO_DERECHO ≠  -1) entonces
                  Encolar(ARBOL(P).HIJO_DERECHO, E_C, S_C)
            fin-si
      fin-mientras
FIN
```

**Recorrido en orden**

Se recorre el subárbol izquierdo, se trata la raíz y, por último se recorre el subárbol derecho.

```
MODULO Inorden
DATOS
      PARÁMETROS
            Recibe ARBOL(N) de tipo NODO
            Recibe I_R entero
INICIO
      Si  (I_R ≠ -1)  entonces
            Inorden(ARBOL, ARBOL(I_R).HIJO_IZQUIERDO)
            TRATAR ( ARBOL(I_R) )
            Inorden(ARBOL, ARBOL(I_R).HIJO_DERECHO)
      fin-si
FIN
```

**Recorrido en orden previo**

Se trata la raíz y, a continuación, los subárboles izquierdo y derecho.

```
MODULO Preorden
DATOS
      PARÁMETROS
            Recibe ARBOL(N) de tipo NODO
            Recibe I_R entero
INICIO
      Si  (I_R ≠ -1)  entonces
            TRATAR ( ARBOL(I_R) )
            Preorden(ARBOL, ARBOL(I_R).HIJO_IZQUIERDO)
            Preorden(ARBOL, ARBOL(I_R).HIJO_DERECHO)
      fin-si
FIN
```

**Recorrido en orden posterior**

Se recorre el subárbol izquierdo, luego el derecho y, por último, se trata la raíz del árbol.

```
MODULO Posorden
DATOS
        PARÁMETROS
                Recibe ARBOL(N) de tipo NODO
                Recibe I_R entero
INICIO
        Si  (I_R ≠ -1)  entonces
                Posorden(ARBOL, ARBOL(I_R).HIJO_IZQUIERDO)
                Posorden(ARBOL, ARBOL(I_R).HIJO_DERECHO)
                TRATAR ( ARBOL(I_R) )
        fin-si
FIN
```

## *Recorrido de un árbol representado usando memoria dinámica*

**Recorrido en anchura**

El árbol se recorre por niveles, comenzando por la raíz y avanzando hacia las hojas.

La operación se apoya sobre una cola de punteros a nodos. En este caso se considera una cola lineal e implementada mediante el uso de memoria dinámica. Las operaciones de encolado, desencolado y comprobación de cola vacía se describen en el tema que describe las colas.

```
MODULO Recorrido_anchura
DATOS
        PARÁMETROS
                Recibe RAIZ puntero a NODO
        VARIABLES
                E_C puntero a NODO_COLA  ** punt. a la entrada de la cola
                S_C puntero a NODO_COLA  ** punt. a la salida de la cola
                P puntero a NODO
INICIO
        Si (RAIZ = NULO) entonces
                ACABAR
        fin-si


        E_C ← NULO
        S_C ← NULO
        Encolar(RAIZ, E_C, S_C)

        Mientras (Cola_vacia(E_C) = Falso) hacer
```

```
                Desencolar(P, E_C, S_C)
                TRATAR (P↑.DATO)
                Encolar(P↑.HIJO_IZQUIERDO, E_C, S_C)
                Encolar(P↑.HIJO_DERECHO, E_C, S_C)
        fin-mientras
FIN
```

**Recorrido en orden**

Se recorre el subárbol izquierdo, se trata la raíz y, por último se recorre el subárbol derecho.

```
MODULO Inorden
DATOS
        PARÁMETROS
                Recibe RAIZ puntero a NODO
INICIO
        Si  (RAIZ ≠ NULO)  entonces
                Inorden(RAIZ↑.HIJO_IZQUIERDO)
                TRATAR (RAIZ)
                Inorden(RAIZ↑.HIJO_DERECHO)
        fin-si
FIN
```

**Recorrido en orden previo**

Se trata la raíz y, a continuación, los subárboles izquierdo y derecho.

```
MODULO Preorden
DATOS
        PARÁMETROS
                Recibe RAIZ puntero a NODO
INICIO
        Si  (RAIZ ≠ NULO)  entonces
                TRATAR (RAIZ)
                Preorden(RAIZ↑.HIJO_IZQUIERDO)
                Preorden(RAIZ↑.HIJO_DERECHO)
        fin-si
FIN
```

**Recorrido en orden posterior**

Se recorre el subárbol izquierdo, luego el derecho y, por último, se trata la raíz del árbol:

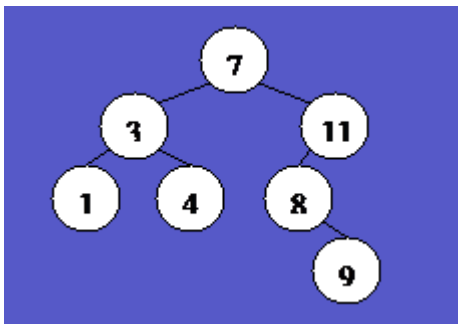
```
MODULO Posorden
DATOS
        PARÁMETROS
```

```
                Recibe RAIZ puntero a NODO
INICIO
        Si (RAIZ ≠ NULO)  entonces
                Posorden(RAIZ↑.HIJO_IZQUIERDO)
                Posorden(RAIZ↑.HIJO_DERECHO)
                TRATAR (RAIZ)
        fin-si
FIN
```

### *4.2.6 Árboles Binarios de Búsqueda*

Un árbol binario de búsqueda es un árbol binario que puede estar vacío y que, si no lo está, cumple las siguientes propiedades:

- Todos los nodos están identificados por una clave única dentro del árbol.
- Las claves de los nodos del subárbol izquierdo son menores que la clave del nodo raíz.
- Las claves de los nodos del subárbol derecho son mayores que la clave del nodo raíz.
- Los subárboles izquierdo y derecho son por si mismos árboles binarios de búsqueda.




**Ejemplo de árbol binario de búsqueda con claves enteras**

Para seguir utilizando una estructura de nodo similar a la propuesta en apartados anteriores, supondremos que la clave que identifica cada nodo es un campo contenido en la información general del nodo.

```
T es registro compuesto por
        CLAVE de tipo T1

        ** otros posibles campos de datos
fin-registro
```

Las características de este árbol hacen que las operaciones de inserción, eliminación y búsqueda sean diferentes de las generales sobre árboles binarios.

## *Construcción de un Árbol Binario de Búsqueda*

Las operaciones de inserción y eliminación de nodos del árbol deben realizarse de modo que se mantenga la estructura del mismo (que siga siendo un árbol binario de búsqueda). Para poder insertar un nuevo nodo, su clave no debe aparecen todavía en el árbol. Al eliminar un nodo del árbol será necesario comprobar si los nodos que permanecen en el mismo siguen definiendo un árbol binario de búsqueda, reorganizándolos si es necesario.

**Inserción de un nodo en un ABB**

Para realizar una inserción, en primer lugar se busca la clave del nuevo nodo dentro del árbol. Si la búsqueda no tiene éxito, se insertará el nuevo nodo en el punto donde se ha detenido dicha búsqueda, que será el punto del árbol donde, de existir, debería estar ubicado el nodo con la clave del nuevo. El nodo a insertar siempre será un nodo hoja, lo que facilitará la inserción. Si la búsqueda tuvo éxito, significa que no se puede insertar el nuevo nodo, pues su clave ya aparece en el árbol.

A continuación se muestra el algoritmo de inserción para un árbol binario de búsqueda representado mediante memoria dinámica:

```
MÓDULO Insertar  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ARBOL puntero a NODO   ** puntero a la raíz del
árbol
                Recibe NUEVO_DATO de tipo T          ** infor. del nuevo
nodo
        VARIABLES
                P, Q punteros a NODO
                ENCONTRADO lógico
INICIO
        ** búsqueda de la clave en el árbol
        Q ← NULO
        P ← ARBOL
        ENCONTRADO ← Falso

        Mientras ( (P ≠ NULO) Y (ENCONTRADO = Falso) ) hacer
                Q ← P
                Si (NUEVO_DATO.CLAVE = P↑.DATO.CLAVE) entonces
                        ENCONTRADO ← Verdadero
                sino
                        Si (NUEVO_DATO.CLAVE < P↑.DATO.CLAVE) entonces
                                P ← P↑.HIJO_IZQUIERDO
                        sino
                                P ← P↑.HIJO_DERECHO
                        fin-si
        fin-mientras
```

```
        ** inserción, si la búsqueda no tuvo éxito
        Si (ENCONTRADO = Falso) entonces

                Si (Q = NULO) entonces     ** se inserta la raíz
                    DEVOLVER  Insertar_hijo(RAIZ,  NULO, NUEVO_DATO,
Falso)

                sino
                    ** se inserta un hijo izquierdo
                    Si (NUEVO_DATO.CLAVE < Q↑.DATO.CLAVE) entonces
                        DEVOLVER  Insertar_hijo(RAIZ, Q, NUEVO_DATO,
Falso)

                    ** se inserta un hijo derecho
                    sino
                        DEVOLVER  Insertar_hijo(RAIZ, Q, NUEVO_DATO,
Verdadero)
                fin-si
            fin-si
        fin-si

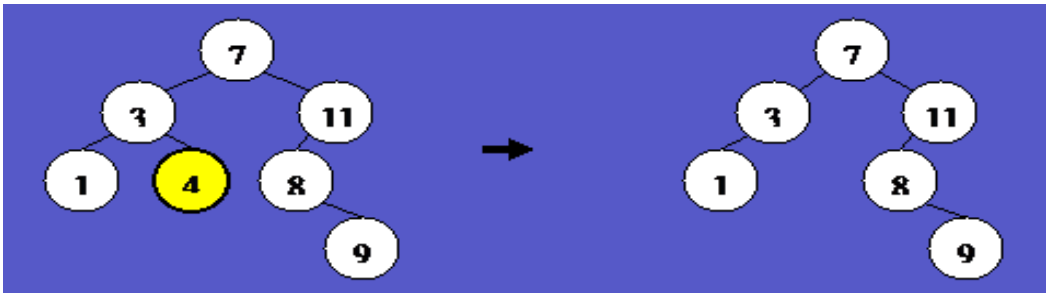
        DEVOLVER Falso   ** inserción no completada
FIN
```

Para simplificar el algoritmo, el módulo anterior llama al módulo Insertar_hijo ya descrito, pasando como argumentos un puntero a la raíz del árbol, un puntero al padre del nuevo nodo (NULO si el nuevo nodo va a ser la raíz del árbol), la información asociada al nuevo nodo y un valor que determina si el nuevo nodo será hijo derecho o izquierdo del padre.

**Eliminación de un nodo en un ABB**

Una vez encontrado en el árbol el nodo que contiene la clave que se desea eliminar, se procede a su eliminación, redefiniendo además las conexiones necesarias con sus hijos o con su padre, si los tiene. En este sentido se pueden plantear los siguientes casos:

- Se debe eliminar un ***nodo sin hijos***: si el nodo tiene padre, dicho padre dejará de referenciar al hijo que desaparece; si el nodo a eliminar es la raíz del árbol (no tiene padre), dicho árbol quedará vacío.

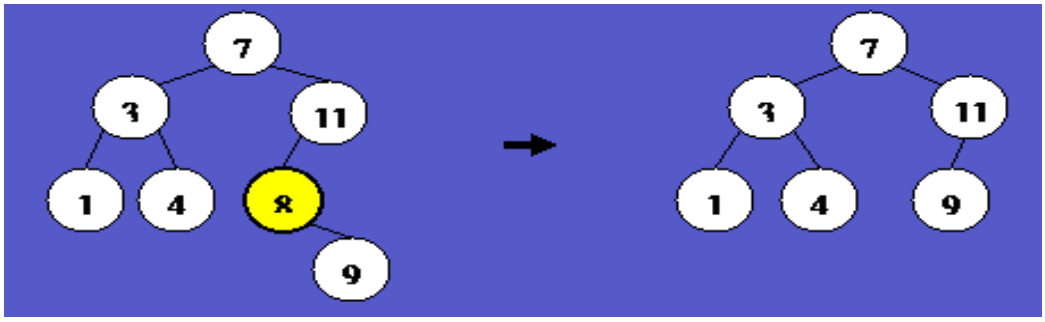



**Eliminación de un nodo hoja de un ABB**

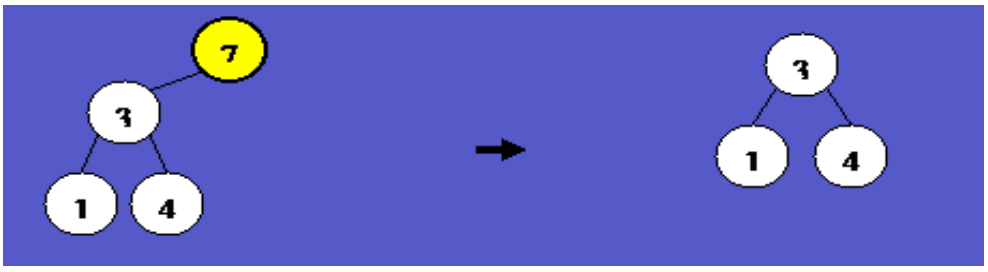



**Eliminación de la raíz de un ABB que no contiene más nodos**

- Se debe eliminar un ***nodo que sólo tiene un hijo***: se conecta el padre de dicho nodo con su único hijo; si no existe el padre, el hijo se convierte en nueva raíz del árbol.

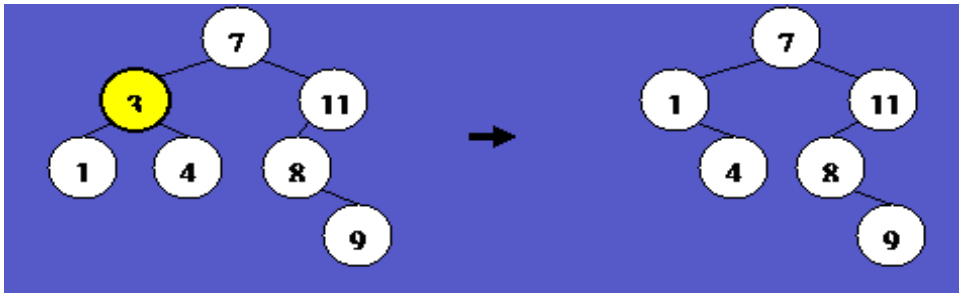



**Eliminación de un nodo con un solo hijo de ABB - caso de un nudo**

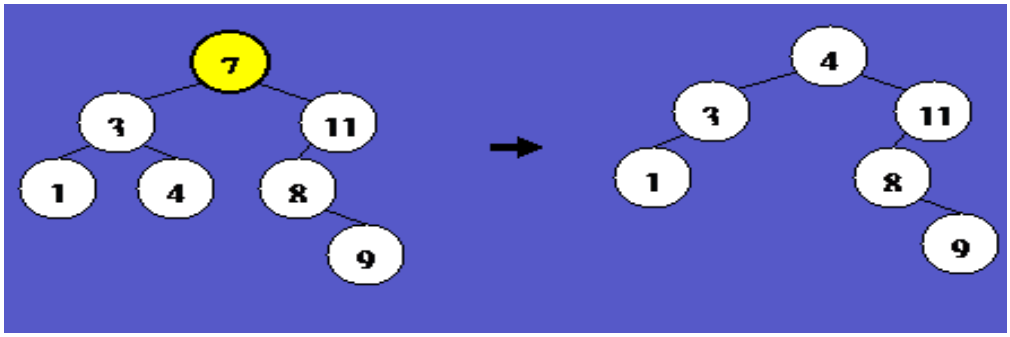



**Eliminación de un nodo con un solo hijo de un ABB - caso de la raíz**

- Se debe eliminar un ***nodo que tiene dos hijos***: en este caso se busca en el subárbol izquierdo del nodo (que contiene claves menores que la que se va a eliminar) la mayor de las claves que contiene. Para ello se toma como punto de partida de la búsqueda la raíz del subárbol izquierdo y, dentro de dicho subárbol se recorren las conexiones a los hijos derechos hasta llegar a un nodo sin hijo derecho, que contendrá la mayor de las claves menores que estamos buscando. En ese momento se copia sobre el nodo que se va a eliminar el valor del nodo localizado. Si dicho nodo era una hoja, se elimina; antes de proceder a esta eliminación, se elimina la conexión a la hoja desde su padre. Si el nodo localizado no era una hoja, tendrá un solo hijo (pues sabemos que no tiene hijo derecho). En este caso, ese hijo del nodo que se va a eliminar se conecta con el padre del nodo que se va a eliminar: como hijo izquierdo, si elimino la raíz del subárbol izquierdo analizado, y como hijo derecho si no se trata de la raíz.




**Eliminación de un nodo con dos hijos de un ABB - caso de un nudo**




**Eliminación de un nodo con dos hijos de un ABB - caso de la raíz**

A continuación se muestra el algoritmo de eliminación de un nodo para un árbol binario de búsqueda representado mediante memoria dinámica:

```
MÓDULO Eliminar  devuelve lógico
DATOS
        PARÁMETROS
                Transforma ARBOL puntero a NODO
                Recibe X de tipo T1
        VARIABLES
                P puntero a NODO          ** puntero al nodo que se va a
eliminar
                PADRE puntero a NODO  ** puntero al padre de dicho nodo
                R, S punteros a NODO
                ENCONTRADO lógico
INICIO
        ** búsqueda de la clave en el árbol
        PADRE ← NULO
        P ← ARBOL
        ENCONTRADO ← Falso

        Mientras ( (P ≠ NULO) Y (ENCONTRADO = Falso) ) hacer
                Si (X = P↑.DATO.CLAVE) entonces
                        ENCONTRADO ← Verdadero
                sino
                        PADRE ← P

                         Si (X < P↑.DATO.CLAVE) entonces
```

```
                    P ← P↑.HIJO_IZQUIERDO
                sino
                    P ← P↑.HIJO_DERECHO
                fin-si
        fin-si
fin-mientras

** si la búsqueda no tuvo éxito, se acaba
Si (ENCONTRADO = Falso) entonces
        DEVOLVER Falso
fin-si

** si se debe eliminar un nodo sin hijos
Si (  (P↑.HIJO_IZQUIERDO = NULO)   Y
      (P↑.HIJO_DERECHO = NULO)  )  entonces

        ** si se elimina la raíz de un árbol con un solo nodo
        Si (PADRE = NULO) entonces
            ARBOL ← NULO

        ** si se elimina una hoja
        sino
            ** si la hoja es hijo izquierdo de su padre
            Si ( P = PADRE↑.HIJO_IZQUIERDO) entonces
                PADRE↑.HIJO_IZQUIERDO ← NULO

            ** si la hoja es hijo derecho de su padre
            sino
                PADRE↑.HIJO_DERECHO ← NULO
            fin-si
        fin-si

        LIBERAR (P)
fin-si

** si el nodo a eliminar sólo tiene hijo derecho
Si (  (P↑.HIJO_IZQUIERDO = NULO)   Y
      (P↑.HIJO_DERECHO ≠ NULO)  )  entonces

        ** si se elimina la raíz de un árbol con más de un nodo
        Si (PADRE = NULO) entonces
            ARBOL ← P↑.HIJO_DERECHO

        ** si se elimina un nudo
        sino
```

```
                ** si el nudo es hijo izquierdo de su padre
                Si ( P = PADRE↑.HIJO_IZQUIERDO) entonces
                    PADRE↑.HIJO_IZQUIERDO ← P↑.HIJO_DERECHO

                ** si es hijo derecho de su padre
                sino
                    PADRE↑.HIJO_DERECHO ← P↑.HIJO_DERECHO
                fin-si
            fin-si

            LIBERAR(P)
        fin-si

        ** si el nodo a eliminar sólo tiene hijo izquierdo
        Si (  (P↑.HIJO_IZQUIERDO ≠ NULO)   Y
              (P↑.HIJO_DERECHO = NULO)  )  entonces

            ** si se elimina la raíz de un árbol con más de un nodo
            Si (PADRE = NULO) entonces
                ARBOL ← P↑.HIJO_IZQUIERDO

            ** si se elimina un nudo
            sino
                Si ( P = PADRE↑.HIJO_IZQUIERDO) entonces
                    PADRE↑.HIJO_IZQUIERDO ← P↑.HIJO_IZQUIERDO
                sino
                    PADRE↑.HIJO_DERECHO ← P↑.HIJO_IZQUIERDO
                fin-si
            fin-si

            LIBERAR(P)
        fin-si

        ** si se debe eliminar un nodo con dos hijos
        Si (  (P↑.HIJO_IZQUIERDO ≠ NULO)   Y
              (P↑.HIJO_DERECHO ≠ NULO)  )  entonces
            R ← P
            S ← P↑.HIJO_IZQUIERDO

            Mientras (S↑.HIJO_DERECHO ≠ NULO) hacer
                    R ← S
                    S ← S↑.HIJO_DERECHO
            fin-mientras

            Si (S↑.HIJO_IZQUIERDO ≠ NULO) entonces
```

```
                    Si (R = P) entonces
                      R↑.HIJO_IZQUIERDO ← S↑.HIJO_IZQUIERDO

                    sino

                      R↑.HIJO_DERECHO ← S↑.HIJO_IZQUIERDO

                    fin-si

                sino
                    Si (R = P) entonces

                      R↑.HIJO_IZQUIERDO ← NULO

                    sino

                      R↑.HIJO_DERECHO ← NULO

                    fin-si

                fin-si

                P↑.DATO ← S↑.DATO
                LIBERAR(S)
            fin-si

            DEVOLVER Verdadero
        FIN
```

## *Búsquedas en un Árbol Binario de Búsqueda*

Dado que los nodos de un árbol binario de búsqueda están ordenados por la clave, la búsqueda de una clave no requiere recorrer todo el árbol.

Supongamos que se desea buscar un elemento con clave **X**. Si la raíz del árbol contiene la clave buscada, la operación concluirá con éxito; si su clave es menor que **X**, la búsqueda debe continuar por el subárbol derecho ya que, de existir un nodo en el árbol con clave **X**, estará en dicho subárbol; si la clave del nodo es mayor que **X**, la búsqueda deberá continuar por el subárbol izquierdo. El proceso continuará hasta que se encuentre un nodo con clave igual a **X** o se llegue a un subárbol vacío. En este último caso, significa que no existe ningún nodo con clave **X** en el árbol.

A continuación se muestra el algoritmo de búsqueda sobre un árbol representado mediante memoria dinámica. Se incluye un algoritmo recursivo y otro no recursivo. Ambos toman como parámetros el puntero que da acceso al árbol sobre el que se va a realizar la búsqueda y el valor de la clave que se debe buscar. Retornan un puntero al nodo que contiene la clave **X** en caso de éxito, y un puntero nulo si no se ha encontrado ningún nodo con la clave especificada:

```
MÓDULO Buscar_rec  devuelve puntero a NODO
DATOS
        PARÁMETROS
                Recibe ARBOL puntero a NODO
                Recibe X de tipo T1                     ** valor a buscar
INICIO
        Si (ARBOL = NULO) entonces
                DEVOLVER NULO
        sino
                Si (X = ARBOL↑.DATO.CLAVE) entonces
                   DEVOLVER ARBOL
                sino
                   Si (X < ARBOL↑DATO.CLAVE) entonces
                     DEVOLVER Buscar_rec(ARBOL↑.HIJO_IZQUIERDO, X)
                   sino
                     DEVOLVER Buscar_rec(ARBOL↑.HIJO_DERECHO, X)
                   fin-si
                fin-si
        fin-si
FIN
```

**Búsqueda en un ABB - procedimiento recursivo**

```
MÓDULO Buscar_no_recursivo  devuelve puntero a NODO
DATOS
        PARÁMETROS
                Recibe ARBOL puntero a NODO
                Recibe X de tipo T1             ** valor a buscar
        VARIABLE
                P puntero a NODO
                ENCONTRADO lógico       **indicador de nodo encontrado
INICIO
        P ← ARBOL
        ENCONTRADO ← Falso

        Mientras (  (P ≠ NULO) Y (ENCONTRADO = Falso)  ) hacer
                Si (P↑.DATO.CLAVE = X) entonces
                        ENCONTRADO ← Verdadero
                sino
                        Si (X < P↑.DATO.CLAVE) entonces
                                P ← P↑.HIJO_IZQUIERDO
                        sino
                                P ← P↑.HIJO_DERECHO
```

```
                    fin-si
            fin-mientras

            DEVOLVER P
FIN
```

**Búsqueda en un ABB - procedimiento no recursivo**

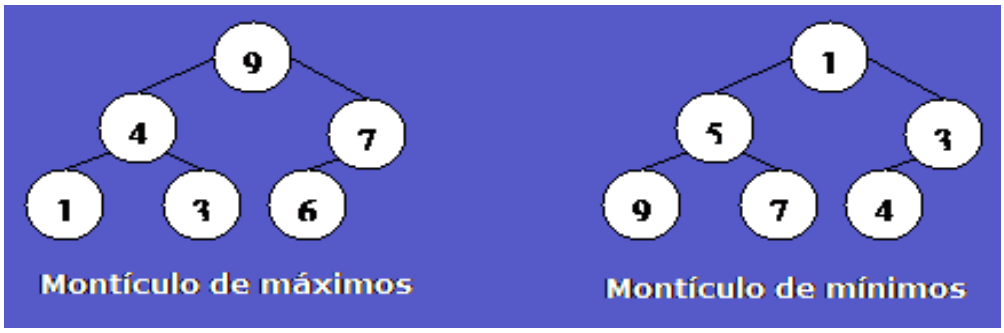
### *4.2.7 Montículos binarios (heaps)*

Los montículos (heaps) son un tipo especial de árboles binarios completos.

Un **árbol binario completo** es un árbol binario de profundidad **k** en el que cualquier nodo a un nivel menor que **k**-2 tienen dos hijos y además todo nodo **i** con un descendiente derecho en el nivel **k**-1 tiene un hijo izquierdo y cada descendiente izquierdo del nodo **i** es una hoja en el nivel **k**-1 o tiene dos hijos.

Existen dos tipos de montículos:

- Un **montículo de máximos** es un árbol binario completo en el cual el valor de la clave de cada nodo es mayor o igual que las claves de sus nodos hijos, si los tiene.
- Un **montículo de mínimos** es un árbol binario completo en el cual el valor de la clave de cada nodo es menor o igual que las claves de sus nodos hijos, si los tiene.




**Ejemplos de montículo con claves enteras**

La clave contenida en el nodo raíz de un montículo de máximos es la mayor clave del árbol, pero esto no quiere decir que sea la única con ese valor. En los montículos de mínimos la raíz contiene la menor clave del árbol, que puede no ser única.

Dado que los montículos son árboles binarios completos, se pueden representar de forma eficiente mediante un vector. Los nodos del árbol se almacenan en el vector comenzando por la raíz y avanzando por niveles hasta las hojas, considerando en cada nivel los elementos de izquierda a derecha según aparecen en la representación gráfica. Con ello, sabemos que:

- el padre del nodo I, ocupa la posición I div 2, si I > 1. (Si I ≤ 1, el nodo no tiene padre)

- el hijo izquierdo del nodo I, ocupa la posición 2I, si 2I ≤ N. (Si 2I > N, el nodo no tiene hijo izquierdo)

- el hijo derecho del nodo I, ocupa la posición 2I+1, si 2I+1 ≤ N. (Si 2I+1 > N, el nodo no tiene hijo derecho)

Representaremos el árbol mediante un vector de tamaño **N**, siendo **N** el número máximo de nodos que puede incluir el árbol. La variable **NUM_NODOS** indicará el número real de nodos que contiene el árbol en cada momento.

```
NUM_NODOS entero
ARBOL(N) de tipo NODO

NODO es registro compuesto por
        DATO de tipo T              ** información asociada al nodo
fin-registro
```

Supondremos que la información de cada nodo del árbol se almacena en un registro similar al descrito en el apartado referente a árboles binarios de búsqueda:

```
T es registro compuesto por
        CLAVE de tipo T1

        ** otros posibles campos de datos
fin-registro
```

## *Construcción de un montículo*

Las operaciones de inserción y eliminación de nodos del árbol deben realizarse de modo que se mantenga la estructura del mismo (que siga siendo un montículo), restaurando el montículo si es necesario.

### Restauración de un montículo

La propiedad que caracteriza a un montículo puede ser restaurada eficazmente tras cualquier modificación de un nodo (cambio de su valor, inserción o eliminación). Supongamos que estamos trabajando con un montículo de máximos (cualquier comentario sobre este tipo de montículos se puede extender fácilmente a los montículos de mínimos), en cuyo caso la clave de cualquier nodo debe ser mayor o igual que las claves de sus hijos. Supongamos ahora que se modifica la clave de un nodo. Dicha modificación puede suponer o un aumento o un decremento en el valor de la clave.

Si se incrementa la clave del nodo, es posible que la propiedad del montículo no se cumpla con respecto a sus nodos antecesores, pero no respecto a sus hijos. Esto hace necesario ir ascendiendo por el árbol y comprobar si el nodo modificado es mayor que su padre; si es así, intercambiarlos y repetir el proceso en el nivel superior, hasta el momento en que ya no sea necesario realizar ningún intercambio o se haya llegado al nodo raíz. En cualquier caso, tras este proceso se habrá restaurado la propiedad del montículo.

Si se reduce la clave del nodo, el problema consistirá en comprobar la condición de montículo respecto a sus descendientes. Si la clave del nodo es mayor que las claves de sus hijos, la condición se sigue cumpliendo; en caso contrario, hay que intercambiar el nodo con el hijo que tenga la clave máxima y repetir el proceso tomando como nuevo

punto de partida a dicho hijo. Las comprobaciones proseguirán por sucesivos niveles hasta que no se realice ningún intercambio, o hasta llegar a una hoja.

Restauración de un montículo de máximos

**Inserción de un nodo**

Para insertar un nuevo nodo en el montículo se coloca en la posición que permite obtener un nuevo árbol completo y, a continuación, se comprueba si es necesario restaurar el montículo.

En la siguiente figura se muestra con una línea punteada la posición que debe ocupar el siguiente nodo que se inserte en el árbol, independientemente del valor de éste, para que el árbol siga siendo completo:




**Posición del siguiente nodo que se insertará en el montículo**

Como el nuevo nodo se inserta como una hoja, para mantener la propiedad del montículo bastará con aplicar el algoritmo **Restaurar_arriba** a partir del nodo insertado (ver Restauración de un montículo de máximos).

Por tanto, la operación de inserción consiste en insertar el nodo en la primera posición libre del vector que representa el árbol (siempre que el árbol no esté lleno) y, si el árbol ya no es un montículo válido, restablecer la condición de montículo.

El módulo que realiza la inserción retorna Falso si no se puede insertar el nuevo nodo, porque el vector está lleno:

```
MÓDULO Insertar   devuelve lógico
DATOS
        PARÁMETROS
                Transforma ARBOL(N) de tipo T    ** vector que representa
el árbol
                Recibe NUEVO_DATO de tipo T      ** inform. nuevo nodo
                Transforma NUM_NODOS entero   ** num. nodos del árbol
                Recibe N entero                  ** tam. del vector
INICIO
        Si (NUM_NODOS < N) entonces
                NUM_NODOS ← NUM_NODOS + 1
                ARBOL(NUM_NODOS) ← NUEVO_DATO
                Restaurar_arriba(ARBOL, NUM_NODOS)
                DEVOLVER Verdadero
        sino
```

```
                Escribir "No hay espacio para un nuevo nodo"
                DEVOLVER Falso
        fin-si
FIN
```

**Inserción de un nuevo elemento en un montículo de máximos**

### Eliminación de un nodo

Al eliminar un nodo del árbol se debe obtener un árbol completo, por lo que el nodo que debe desaparecer realmente del árbol es el último. Además, al eliminar un nodo del árbol se deben reorganizar los hijos de éste y los descendientes de los mismos, de forma que se siga cumpliendo la condición de montículo.




**Posición del siguiente nodo que se eliminará de un montículo**

Para conseguirlo, intercambiaremos la información del nodo a borrar y la del último nodo del montículo, que es el que realmente debe desaparecer de la estructura para seguir teniendo un árbol completo. Tras dicho intercambio se comprueba si es necesario restablecer el montículo. Se deberá aplicar el módulo Restaurar_arriba (si la nueva clave del nodo I es mayor que la previa) o Restaurar_abajo ( si es menor) para restaurar esa propiedad (ver Restauración de un montículo de máximos).

```
MÓDULO Eliminar_nodo
DATOS
        PARÁMETROS
                Transforma ARBOL(N) de tipo T    ** vector que representa
el árbol
                Transforma NUM_NODOS entero  ** num. nodos del árbol
                Recibe I entero                  ** posición eliminación
        VARIABLES
                D de tipo T
INICIO
        D ← ARBOL(I)

        ARBOL(I) ← ARBOL(NUM_NODOS)
        NUM_NODOS ← NUM_NODOS - 1

        Si (ARBOL(I).DATO.CLAVE ≠ D.DATO.CLAVE) entonces
                Si (ARBOL(I).DATO.CLAVE > D.DATO.CLAVE) entonces
                        Restaurar_arriba(ARBOL, I)
                sino
```

```
                    Restaurar_abajo(ARBOL, NUM_NODOS, I)
            fin-si
        fin-si
    FIN
```

**Eliminación de un elemento de un montículo de máximos que ocupa la posición I**

## *Algoritmo Heapsort*

El algoritmo heapsort permite ordenar un vector trabajando con un montículo.

Dado un vector de elementos, el primer paso consiste en construir un montículo a partir de los datos a ordenar (de máximos si se desea una ordenación ascendente y de mínimos si se desea ordenación descendente). Aquí supondremos que deseamos realizar ordenación ascendente. Una vez formado el montículo, la raíz contiene el mayor de los elementos (aunque puede haber otros con el mismo valor). Por tanto, se intercambia éste con el último elemento del vector, con lo que se habrá colocado en la última posición el valor más grande. A continuación, se considera que el vector (y, por tanto, el árbol) tiene un elemento menos (el que se acaba de llevar al final), y se comprueba si el montículo obtenido tras el intercambio es válido. Si el montículo no es válido, se reorganizan sus nodos hasta obtener un montículo de máximos válido. Una vez restablecido el montículo, se aplica la misma operación, intercambiando la raíz del árbol y su último elemento, y restableciendo después el montículo resultante, que tendrá un elemento menos que el anterior. Si se repite este proceso hasta que se tenga un montículo con un único elemento, se conseguirá ordenar el vector.

```
MÓDULO Heapsort
DATOS
    PARÁMETROS
        Transforma ARBOL() de tipo T
        Recibe N entero                ** elementos del vector
    VARIABLES
        I entero
INICIO
    Para I desde 2 hasta N hacer
        Restaurar_arriba(ARBOL, I)
    fin-para

    I ← N

    Mientras (I > 1) hacer
        Intercambiar(ARBOL(1), ARBOL(I))
        I ← I - 1
        Restaurar_abajo(ARBOL, N, 1)
    fin-mientras
FIN
```